One Diagonal: Verified Limits of Trustworthy AI

Manish Bhatt

Contents

Contents i

How to read this book vii

Preliminaries ix
How to read the Lean 4 in this book x
Sets and functions xxii
The minimum topology xxv
The two engines: a roadmap xxix
How the book is verified and built xxxi
Exercises xxxii

1 The Diagonal 1
1.1 One move, five disguises 2
1.2 Systems that describe themselves 8
1.3 Lawvere's fixed-point theorem 11
1.4 The section and retract form 13
1.5 The contrapositive is the impossibility 15
1.6 Three classical instances 17
1.7 A partial-function preview: toward Rice 20
1.8 The engine, named once 22
1.9 What the diagonal cannot give 24
1.10 A roadmap through the book 24
1.11 Historical and bibliographic notes 25
1.12 Exercises 26
1.13 References 28

2 The Classical Limits 31
2.1 The recipe 32
2.2 Cantor's theorem 34
2.3 Russell's paradox 37
2.4 Gödel's first incompleteness theorem 39
2.5 Gödel's second incompleteness theorem 44
2.6 Tarski's undefinability of truth 48

2.7 Turing and the halting problem 50
2.8 Rice's theorem 52
2.9 The categorical unification 55
2.10 Interlude: the wrapper is the mathematics 56
2.11 What varies, and what does not 57
2.12 Historical and bibliographic notes 58
2.13 Exercises 59
2.14 References 61

3 Three Trilemmata, One Theorem 63
3.1 How to read this chapter 64
3.2 What "reflective" demands 65
3.3 A worked micro-instance 66
3.4 The Hallucination Trilemma 67
3.5 The Defense Trilemma 73
3.6 The Coupling Theorem 78
3.7 The Master Trilemma 83
3.8 One proof term 87
3.9 What the readings share, and where they part 88
3.10 The template in four moves 89
3.11 A note on axioms and trust 89
3.12 What this chapter does not settle 90
3.13 Historical and bibliographic notes 91
3.14 Exercises 91

4 From Logic to Analysis 95
4.1 Why a second engine 96
4.2 A topology primer 96
4.3 The model, analytically 102
4.4 The topological proof of the trilemma 105
4.5 A worked instance on the interval 108
4.6 Quantitative shadows: approximate coupling and positive slack . . 109
4.7 Symmetry in place of coverage: the antipodal variant 110
4.8 The discrete core: no topology once a boundary question exists . . 113
4.9 Where the two engines meet 116
4.10 Where the two engines diverge 118
4.11 The engine in one page 119
4.12 Historical and bibliographic notes 120
4.13 Exercises 121
4.14 References 123

5 The Geometry of Attacks 125
5.1 From the diagonal to the boundary 126
5.2 Decision basins and their structure 127
5.3 The threshold-crossing theorem 129
5.4 Lipschitz control of basins 132

5.5 The coarea and cone bounds 134
5.6 Dimensional scaling and the boundary's dimension 137
5.7 Gradient attacks and their chaining 140
5.8 Transferability of attacks across models 142
5.9 The cost theory 144
5.10 Concentration of measure, the two faces of high dimension 149
5.11 Iterated and discrete attacks 151
5.12 The two halves, joined 153
5.13 Historical and bibliographic notes 154
5.14 Exercises 154
5.15 References 157

6 Approximate Bridges to Real Models 159
6.1 What the exact theorems assume 160
6.2 Two-sided ε-calibration 161
6.3 ε-coverage 164
6.4 The approximate trilemma 165
6.5 Recovering the exact contradiction at $\varepsilon = 0$ 166
6.6 The tightening principle 168
6.7 Localization to the transition band 170
6.8 The discrete band constraint 172
6.9 Two-slack separation: truth units and confidence units 174
6.10 The quantitative degradation law 176
6.11 From existence to measure 179
6.12 The reading is where the work is 182
6.13 Historical and bibliographic notes 183
6.14 Exercises 184
6.15 References 186

7 What It Means for AI Safety 187
7.1 The constraint, stated as a triangle 188
7.2 Three relaxations, three cost models 190
7.3 The same menu for prompt-injection defense 199
7.4 Reading a domain: which corner fits 201
7.5 The one empirical unknown 205
7.6 What the theorems do and do not claim 208
7.7 An agenda of open problems 210
7.8 The chapter in one constraint 211
7.9 Bibliographic notes 212
7.10 Exercises 213

8 J-Space, and Why It Does Not Escape 215
8.1 What J-space is 216
8.2 Two structural facts 220
8.3 The diagonal engine: no intermediate space escapes 223
8.4 The analytic engine: linearity is the liability 230

8.5 Where the two engines meet, again . . . 233
8.6 What an escape would have to look like . . . 235
8.7 What this means for interpretability . . . 235
8.8 What this does not claim . . . 238
8.9 Historical and bibliographic notes . . . 239
8.10 Exercises . . . 240

9 The Trace Is Not a Window 243
9.1 The construction . . . 244
9.2 Faithfulness forces verbosity . . . 245
9.3 The diagonal, with the model holding the pen . . . 246
9.4 Selection destroys the signal . . . 247
9.5 The analytic engine . . . 249
9.6 What this does not claim . . . 250
9.7 Exercises . . . 251

10 The Reporter Is Underdetermined 253
10.1 The construction . . . 254
10.2 Underdetermination, without any diagonal . . . 255
10.3 The identifiability criterion . . . 256
10.4 Where the diagonal comes in . . . 257
10.5 The analytic engine . . . 258
10.6 What this does not claim . . . 259
10.7 Exercises . . . 259

11 Watermarks, Paraphrase, and the Third Corner 261
11.1 The construction . . . 262
11.2 The trilemma . . . 263
11.3 The diagonal, and why its price is low here . . . 264
11.4 The analytic engine, and the cost of an attack . . . 265
11.5 What this does not claim . . . 266
11.6 Exercises . . . 267

12 Appendices 269
12.1 The Formal Corpus . . . 269
12.2 A Notation Glossary . . . 288
12.3 The Two Engines, Side by Side . . . 291
12.4 Building and Checking This Book . . . 293

These are lecture notes on a family of impossibility theorems for machine learning systems: hallucination, prompt-injection defense, calibration, verification, and oversight. The organizing claim is that a single mathematical move, the diagonal argument behind Cantor, Gödel, and Lawvere, generates most of them, and that a second, analytic move, the intermediate value theorem and its quantitative refinements, generates the rest. What makes these notes unusual is that the mathematics is machine-checked. The core arguments are elaborated live by Lean 4 as the book is built, so the proofs in the text are the proofs the kernel accepts. Where a result depends on the `Mathlib` library, we cite the verified statement in the companion development and reproduce the argument in prose. A handful of claims in Chapter 7 are deployment arguments about cost, not mathematics; each of those says "Not formalized." in its own text, and the build fails if any other numbered result lacks either a proof or a citation that resolves to a real declaration.

How to read this book

The book has two halves that meet in the middle. Chapters 1 through 4 develop the diagonal engine and its consequences, and are almost entirely self-contained core Lean: you can read the proofs as they compile. Chapters 5 and 6 turn to the analytic side, where continuity, connectedness, and measure do work that no diagonal can, and Chapter 7 draws the practical consequences for deploying language models.

Chapters 8 through 11 are case studies, and they answer the standing objection that all of this concerns outputs only. Each takes a construction proposed in the recent literature as a way of seeing inside or around a model, and asks what the two engines say about it. Chapter 8 takes J-space, a probe reading the derivative of the computation rather than its activations. Chapter 9 takes chain-of-thought monitoring, where the trace being read is text the model itself chose to emit. Chapter 10 takes Eliciting Latent Knowledge, whose central obstruction turns out not to be the diagonal at all but a counting fact about functions agreeing on a subset. Chapter 11 leaves the model entirely and takes watermark detection, where the mechanism is invariance under paraphrase. The four are deliberately not four copies of one argument: the shared engines are the diagonal and the intermediate value theorem, but the load-bearing step differs in each, and Chapters 10 and 11 in particular reach their main results without any self-reference.

Prerequisites are light: comfort with functions and sets, a first course in logic, and enough topology to know what a continuous function on a connected space is. No prior Lean is assumed; the code is explained as it appears.

Preliminaries

This book proves that certain things are impossible. Not hard, not unsolved, impossible: a truthful and calibrated and covering language model, a prompt filter that stops every injection, a probe that reads the truth off the inside of a network. The proofs are short. What takes work is understanding what they say and trusting that they are correct, and both of those are the subject of this chapter.

Two habits make the rest of the book readable. The first is reading a Lean 4 proof as mathematics rather than as software. A Lean proof of a theorem is a finite object that a small program, the kernel, can check for you, and once you learn to read the handful of constructs this book uses you will find the proofs shorter and more honest than their prose counterparts. The second habit is keeping two pictures in mind at once. One picture is combinatorial: a system that can name its own behaviors cannot be everywhere consistent, because the behavior that disagrees with each name at that name is itself a behavior. The other picture is geometric: a continuous quantity that is negative somewhere and positive somewhere else is zero in between, and the zero is a place the system cannot avoid. The first picture is the diagonal. The second is the intermediate value theorem. Every impossibility in this book is one of these two arguments wearing a costume.

You do not need prior Lean, and you do not need graduate topology. You need comfort with functions and sets, a first course in logic, and willingness to read a five-line proof slowly. This chapter supplies the rest. It has five parts: how to read the Lean in this book, the set-and-function language the diagonal engine speaks, the small amount of topology the analytic engine needs, a preview of the two engines and where each chapter sits, and finally how the book is built and checked so that you can rerun the verification yourself.

A note on how to use this chapter. If you already read Lean, skim the first part for the specific idioms this book leans on, `congrFun` and `match` on an existential above all, and spend your time on the two-engines roadmap. If you know the mathematics but not the proof assistant, read the first part slowly and run the code yourself. The chapter is long because it is meant to be the last time you have to look anything up.

How to read the Lean 4 in this book

The code blocks in this book are not illustrations of proofs that live elsewhere. They *are* the proofs. When the book is built, Lean 4 elaborates each block and its kernel checks the result, so a block that you can see in the text is a block that compiled. That is the entire value proposition of writing the book this way, and it is worth the small investment of learning to read the notation. This section builds that reading vocabulary from the ground up, using only tiny self-contained examples. Every live definition here is prefixed `c0_` so that it cannot clash with the real development.

Types and terms

Lean is built on *types*. Every well-formed expression, every *term*, has a type, and the type is checked before the term means anything. You write `e : T` to assert that the term `e` has type `T`. Some types you will see constantly: `Nat`, the natural numbers `0, 1, 2, …`; `Bool`, the two-element type with values `true` and `false`; `Prop`, the type of *propositions*, which we return to below.

Definition 0.1 (Type and term). A *type* is a classification of terms. A *term* `e` *has type* `T`, written `e : T`, when the rules of the language assign `T` to `e`. Type-checking is the mechanical process of verifying such assignments.

There is a type of types. `Nat`, `Bool`, and `Prop` are themselves terms, and their type is a *universe*. The first data universe is written `Type`, so `Nat : Type` and `Bool : Type`. To avoid a paradox of the kind Chapter 1 studies, the universes form an infinite hierarchy `Type : Type 1 : Type 2 : …`, and Lean can often infer the level for you. In this book you will sometimes see `Type _`, which means "some universe level, please work it out." For a first reading you may pretend there is a single universe called `Type` and lose nothing.

Above both `Type` and `Prop` sits a common notion, `Sort`. Lean writes `Prop = Sort 0` and `Type = Sort 1`, and universe-polymorphic definitions quantify over `Sort u`. You will not need to manipulate `Sort` directly, but seeing it in a hover or an error message should not alarm you: it is the umbrella under which propositions and data types both live.

Example 0.2. The literal `2` is a term of type `Nat`. The expression `true` is a term of type `Bool`. The proposition `1 + 1 = 2` is a term of type `Prop`. Here is the first live block; it asserts that `1 + 1` and `2` are the same natural number, and its proof is `rfl`, which we explain shortly.

```
example : 1 + 1 = 2 := rfl
```

An `example` is an anonymous theorem or definition: Lean checks it and throws away the name. We use `example` when we want to show that something compiles without adding a name to the namespace. A named result uses `def` for data and `theorem` for proofs; the two keywords behave almost identically, and the choice records intent, whether the thing you defined is a value you will compute with or a fact you will cite. Some libraries add a `lemma` keyword

as a synonym for `theorem`, but core Lean, which is all the live blocks here use, does not, so this book writes `theorem` throughout.

Functions and application

A function from `A` to `B` has type `A → B`. You apply a function by juxtaposition: if `f : A → B` and `a : A` then `f a : B`. There are no parentheses around arguments and no commas between them; application is just a space, and it binds tighter than almost everything else, so `f a + 1` means `(f a) + 1`.

Definition 0.3 (Function type). For types `A` and `B`, the type `A → B` is the type of functions taking an argument of type `A` and producing a result of type `B`. Application of `f : A → B` to `a : A` is written `f a` and has type `B`.

You build a function with a *lambda*, written `fun x => e`. The term `fun x => e` has type `A → B` when `e : B` under the assumption `x : A`. You can also define a named function directly, naming the argument to the left of the colon.

```
def c0_double (n : Nat) : Nat := n + n
def c0_double' : Nat → Nat := fun n => n + n
```

These two definitions are the same function written two ways. The first names its argument before the colon; the second is a lambda after it. Both have type `Nat → Nat`. Application computes in the obvious way, and `rfl` accepts the computation.

```
example : c0_double 3 = 6 := rfl
```

Functions compose, and the identity function does nothing, and these two facts are worth naming even though they are elementary, because the diagonal argument is a statement about how a certain composite behaves.

```
def c0_id {A : Type} (a : A) : A := a
def c0_comp {A B C : Type} (g : B → C)
    (f : A → B) : A → C := fun a => g (f a)
example : c0_comp c0_double c0_double 3 = 12 := rfl
```

The composite `c0_comp c0_double c0_double` doubles twice, so at `3` it returns `12`, and `rfl` confirms the computation. The curly braces around `A B C` mark them *implicit*: Lean infers those types from the other arguments, so you never write them at a call site. We return to implicit arguments below.

Currying and the shape A → A → Y

The arrow associates to the right, so `A → A → Y` means `A → (A → Y)`. A term of that type is a function that takes one `A` and returns a function `A → Y`. Feeding it a second `A` then produces a `Y`. This is *currying*: a function of two arguments

presented as a function returning a function. The book's central object has exactly this shape, so it pays to be fluent with it.

Definition 0.4 (Currying). A function of two arguments, taking `a : A` and `b : A` to a value in `Y`, is represented in curried form as a term `f : A → A → Y`. Then `f a : A → Y` is the result of supplying only the first argument, and `f a b : Y` supplies both.

Read `f a` as a whole: it is a function, the *behavior indexed by* `a`. Read `f a b` as that behavior evaluated at `b`. The two readings are the same term grouped differently, since `f a b` parses as `(f a) b`. Application associates to the left, exactly opposite to the arrow, and the two conventions fit together so that currying is invisible: you write `f a b` and never think about the intermediate function `f a` unless you want to.

```
def c0_add : Nat → Nat → Nat := fun a b => a + b
example : c0_add 2 3 = 5 := rfl
```

Example 0.5. Take `A = Nat` and `Y = Bool` and let `f n m` say whether `n` divides evenly into a fixed schedule at slot `m`. Then `f n` is the whole schedule of behavior `n`, a function `Nat → Bool`, and `f n n` is what behavior `n` does at its own index. That last value, the behavior applied to its own name, is the *diagonal,* and Chapter 1 shows it is where self-referential systems break. The coincidence that makes it possible is that both arguments of `f` range over the same set, so a name is also a legal input.

∀ and ∃

Logic lives inside the type system. A proposition is a term of type `Prop`, and a *proof* of a proposition `P` is a term of type `P`. To prove `P` is to construct a term whose type is `P`. This is the propositions-as-types principle, and it is why proofs in Lean look like programs: they are.

There is one asymmetry between `Prop` and `Type` that the trust story quietly relies on. Propositions are *proof-irrelevant*: any two proofs of the same proposition are treated as definitionally equal, because for a proposition we care that it holds, not which term witnesses it. Data types are not like this; `0` and `1` are genuinely different terms of `Nat`. Proof irrelevance is why a theorem's content is its statement and its axiom dependencies, never the particular proof term the elaborator happened to build, and it is why two different-looking proofs of the same lemma are interchangeable wherever that lemma is cited.

The universal quantifier `∀ x : A, P x` is a *dependent function type*. A proof of it is a function that, given any `a : A`, returns a proof of `P a`. So you prove a `∀` the same way you write a function: with a lambda.

Definition 0.6 (Universal quantifier). `∀ x : A, P x` is the type of proofs that `P a` holds for every `a : A`. A term of this type is a function sending each `a : A` to a proof of `P a`.

```
theorem c0_forall_le : ∀ n : Nat, n ≤ n :=
  fun n => Nat.le_refl n
```

Given `h : ∀ x : A, P x` and a particular `a : A`, the application `h a` is a proof of `P a`. Universal instantiation is function application. This one fact removes most of the mystery from the proofs ahead: when a proof "applies the hypothesis to a_0," it is literally writing `h` a_0.

The existential quantifier `∃ x : A, P x` is the dual. A proof is a *pair*: a witness `a : A` together with a proof of `P a`. You build the pair with angle brackets, `⟨a, h⟩`, the *anonymous constructor*.

Definition 0.7 (Existential quantifier). `∃ x : A, P x` is the type of proofs that some `a : A` satisfies `P`. A term of this type is a pair `⟨a, h⟩` where `a : A` is the witness and `h : P a` is a proof for that witness.

```
theorem c0_exists_zero : ∃ n : Nat, n = 0 := ⟨0, rfl⟩
```

To *use* an existential you take it apart, which is what `match` does. If `h : ∃ x, P x`, then `match h with | ⟨a, ha⟩ => …` binds `a` to some witness and `ha` to the accompanying proof, and you continue in that context. Building an existential is packing a pair; using one is unpacking it. The Lawvere proof in Chapter 1 does exactly one of each, and once you see that, the proof is no longer mysterious.

Implication is a special case of the function arrow. A proof of `P → Q` is a function turning a proof of `P` into a proof of `Q`, so you apply it the same way you apply any function.

```
theorem c0_imp {p q : Prop} (h : p → q) (hp : p) : q := h hp
```

Here `{p q : Prop}` are *implicit arguments*: Lean infers them from context, and you do not pass them explicitly. Explicit arguments use round parentheses, implicit ones use curly braces. When you read a theorem statement, treat the curly-brace binders as "for any types or values of this shape, inferred silently." Almost every theorem in this book takes its ambient types `A`, `Y`, `Q` implicitly, which is why the statements look like they are about specific sets when they are in fact about all of them.

Negation, disjunction, and the empty proposition

Two further logical connectives complete the vocabulary the proofs use. The first is negation. In Lean `¬ P` is definitionally `P → False`, where `False` is the proposition with no proof at all. To prove `¬ P` you assume a proof of `P` and derive a proof of `False`, which is to say you show the assumption was impossible.

Definition 0.8 (Negation and the empty proposition). `False` is the proposition with no constructors, hence no proof. For a proposition `P`, the negation `¬ P` is defined as `P → False`. From a proof of `False` one may conclude anything, by the eliminator `False.elim`.

```
theorem c0_not_false : ¬ False := fun h => h
theorem c0_ex_falso {p : Prop} (h : False) : p :=
  False.elim h
```

The first says `¬ False` holds, since `¬ False` is `False → False` and the identity function inhabits it. The second is the principle of explosion: from a contradiction, everything follows. Inequality is negation of equality: `a ≠ b` is notation for `¬ (a = b)`, that is `a = b → False`. So to use a hypothesis `h : a ≠ b` you feed it a proof of `a = b` and receive `False`. Watch for this in the contrapositive results, where the whole proof is "produce the equality the hypothesis forbids, hand it over, and collect the contradiction."

The second connective is disjunction. `P ∨ Q` has two constructors, `Or.inl` for a proof of the left side and `Or.inr` for a proof of the right, and you use a disjunction by matching on which side holds.

Definition 0.9 (Disjunction). `P ∨ Q` is proved by `Or.inl` applied to a proof of `P`, or by `Or.inr` applied to a proof of `Q`. It is used by case analysis: a proof of `P ∨ Q` splits into the case where `P` holds and the case where `Q` holds.

```
example : (1 = 1) ∨ (2 = 3) := Or.inl rfl

theorem c0_or_comm {p q : Prop} (h : p ∨ q) : q ∨ p :=
  match h with
  | Or.inl hp => Or.inr hp
  | Or.inr hq => Or.inl hq
```

The first line proves a disjunction by supplying its true left half. The second swaps the two sides: match on which disjunct holds, then rebuild the disjunction with the sides exchanged. Conjunction `P ∧ Q`, which we meet again with structures, is the dual, built from two proofs and used by projecting out each.

Structures

A *structure* bundles several fields into one type. It is the tool for packaging a mathematical object together with the data and hypotheses that define it. You declare the fields, and Lean gives you a constructor and one projection per field.

Definition 0.10 (Structure). A structure is a type whose terms are tuples of named fields. Declaring it produces a constructor that takes the fields and returns a term, and projections that recover each field from a term.

```
structure c0_Point where
  x : Nat
  y : Nat

def c0_origin : c0_Point := { x := 0, y := 0 }
def c0_origin' : c0_Point := ⟨0, 0⟩
example : c0_origin.x = 0 := rfl
```

The two definitions of the origin are equal; the first names its fields, the second uses the anonymous constructor `⟨0, 0⟩`, filling fields in order. The projection `c0_origin.x` recovers the first field. The same anonymous-constructor notation `⟨…⟩` that builds a structure also builds an `∃` proof and an `And` proof, because all three are, under the hood, structures with fields. That is why one bracket notation covers packing a witness with its proof, packing two proofs of a conjunction, and packing two coordinates.

```
theorem c0_and_comm {p q : Prop} (h : p ∧ q) : q ∧ p :=
  match h with
  | ⟨hp, hq⟩ => ⟨hq, hp⟩
```

The conjunction `p ∧ q` is a structure with two fields, a proof of `p` and a proof of `q`. To prove `q ∧ p` we take the input apart with `match`, naming the two proofs `hp` and `hq`, then repack them in the other order. Read the proof out loud and it is exactly the mathematics: "a proof of `p` and `q` gives a proof of `p` and a proof of `q`, so it gives a proof of `q` and `p`."

Inductive types, up close

Structures, `Nat`, `Bool`, `And`, `Or`, and `Exists` are all instances of one mechanism: the *inductive type*. An inductive type is declared by listing its constructors, the primitive ways to build a term, and Lean then generates a recursor that says these are the *only* ways, which is what lets `match` be exhaustive. Understanding this one mechanism demystifies all the notation above at once.

Definition 0.11 (Inductive type). An inductive type is a type specified by a finite list of constructors. Each constructor is a function into the type being defined. The type contains exactly the terms built by finitely many applications of its constructors, and pattern matching branches on which constructor produced a given term.

```
inductive c0_Nat where
  | zero : c0_Nat
  | succ : c0_Nat → c0_Nat
```

This is the natural numbers in miniature: a term is `zero`, or `succ` of a smaller term, and nothing else, which is why `match` on `Nat` needs only the

two cases `0` and `n + 1`. A structure is the special case of an inductive type with a single constructor, and a proposition like `And` is a structure landing in `Prop`.

```
structure c0_And (p q : Prop) : Prop where
  intro ::
  left : p
  right : q
```

The existential is an inductive type whose one constructor is genuinely dependent: it packages an element and a proof about *that specific* element.

```
inductive c0_Exists {A : Type} (P : A → Prop) : Prop where
  | intro : (a : A) → P a → c0_Exists P
```

Reading these declarations, you can see why `⟨a, h⟩` builds an `Exists`: it is the `intro` constructor applied to a witness and a proof, and the anonymous bracket just spares you writing the constructor name. The same reading explains why `match` recovers the pieces: the recursor lets you assume a term came from `intro` and expose its two arguments. Nothing about the diagonal proof is special notation; it is constructors in and pattern matches out.

Term-mode proofs, and a word on tactics

A *term-mode* proof writes the proof term directly, as we have been doing: `fun n => rfl`, `⟨0, rfl⟩`, the `match` above. Lean also has a *tactic* mode, entered with `by`, where you build the proof by issuing instructions that manipulate a goal. This book prefers term mode for the core arguments, because a term-mode proof of a five-line theorem is itself about five lines and every symbol is visible. Tactics appear only for small decidable facts, where a single tactic settles the goal and writing the term by hand would obscure rather than reveal.

Remark 0.12. Term mode and tactic mode produce the same kind of object. A tactic block is elaborated into a proof term, and it is that term the kernel checks. There is no second, weaker standard of proof for tactic mode. When you see `by decide` you are seeing an instruction that *generates* a proof term; the term, not the instruction, is what is trusted. A tactic that produced a wrong term would simply fail to type-check, and you would get an error rather than a false theorem.

match and pattern matching

`match` inspects a term of an inductive type and branches on its shape, binding the pieces. It works on data and on proofs alike, since both are terms of inductive types. On `Nat` the shapes are `0` and `n + 1` (successor); on a pair the single shape is `⟨a, b⟩`; on `Bool` the shapes are `true` and `false`.

```
def c0_pred : Nat → Nat
  | 0 => 0
  | n + 1 => n
```

This defines the predecessor by cases on the shape of the input, with no separate `match` keyword because a definition can pattern-match directly on its arguments. The first line handles zero, the second handles a successor and names the predecessor `n`. Pattern matching is also how the proofs in this book use their hypotheses: `match hf (…) with | ⟨a₀, ha₀⟩ => …` is the standard move for pulling a witness `a₀` and its property `ha₀` out of an existential produced by a surjectivity assumption.

Dot notation and namespaces

Lean groups definitions into *namespaces*, and a definition `Foo.bar` can be called on a term `x : Foo` as `x.bar`. This *dot notation* is why you see `h.symm` and `h.trans` and `Or.inl`: `h.symm` means `Eq.symm h`, applying the symmetry-of-equality lemma to `h`, and Lean finds `Eq.symm` because `h`'s type is an equality. It is a spelling convenience with no logical content, but it makes proofs read like method chains and it is everywhere in the book.

```
theorem c0_symm {A : Type} {a b : A} (h : a = b) : b = a :=
  h.symm
theorem c0_trans {A : Type} {a b c : A} (h1 : a = b)
    (h2 : b = c) : a = c :=
  h1.trans h2
```

The first flips an equation; the second chains two equations end to end. When the Lawvere proof writes `(congrFun ha₀ a₀).symm`, it is producing an equality with `congrFun` and then flipping its orientation with `.symm`, two moves you have now seen in isolation.

rfl and definitional equality

`rfl` is the proof that a thing equals itself. It succeeds whenever the two sides are equal *by computation*, or definitionally: Lean reduces both sides as far as the definitions allow and checks that the results are identical. So `1 + 1 = 2` is `rfl` because `1 + 1` computes to `2`, and `c0_double 3 = 6` is `rfl` because the definition of `c0_double` unfolds and the arithmetic runs. When two expressions are equal but not by pure computation, `rfl` is not enough and you reason about the equality explicitly, which is where the next construct enters.

Definitional equality is stronger than it first looks and also has a hard limit. It will run any amount of finite computation, so a claim like `c0_pred (c0_double 4) = 7` is `rfl`. It will not prove a statement quantified over all inputs, because that is not a single computation: `∀ n, n + 0 = n` is `rfl`-provable in

core Lean because addition recurses on its second argument, but `∀ n, 0 + n = n` is not, and the difference is a genuine one about how the definitions unfold, not a matter of notation.

congrFun and equality of functions

The single most important library function in this book's core is `congrFun`. It says that equal functions are equal at every point. Given a proof `h : f = g` that two functions are equal, and a point `a`, the term `congrFun h a` is a proof that `f a = g a`.

Definition 0.13 (`congrFun`). For `f g : A → B` and `h : f = g` and `a : A`, the term `congrFun h a` has type `f a = g a`. It is the elimination principle for an equation between functions: from the functions being equal, conclude their values agree at any chosen argument.

```
theorem c0_congr {A B : Type} (f g : A → B) (h : f = g)
    (a : A) : f a = g a :=
  congrFun h a
```

This one step is the hinge of Lawvere's theorem. Surjectivity hands you an index a_0 whose behavior `f` a_0 equals a function you designed. That is an equation between functions. To extract a contradiction you evaluate both sides at a_0, and `congrFun` is exactly "evaluate both sides at a_0." Watch for it in the three-line proof at the start of Chapter 1; nothing else is going on.

The reverse direction, concluding `f = g` from agreement at every point, is function extensionality. It is available in Lean but is not free: it uses an axiom, and the axiom audit below flags it when it is used. The core diagonal proofs do not need it, which is part of why they audit so cleanly. Keep the asymmetry in mind: going from `f = g` to pointwise agreement is `congrFun` and costs nothing, while going the other way costs an axiom.

decide and decidable propositions

Some propositions can be settled by a finite computation. Whether a specific natural number is less than another, whether two booleans are equal, whether a statement holds for all booleans: each reduces to running a decision procedure and reading off the answer. Lean formalizes this with the `Decidable` type class, and the tactic `decide` proves a goal by running the procedure and checking that it returns the affirmative result.

Definition 0.14 (Decidability and `decide`). A proposition `P` is *decidable* when there is an algorithm returning a proof of `P` or a proof of its negation. The tactic `decide` proves a decidable `P` by evaluating that algorithm and confirming the affirmative result.

```
example : 2 < 5 := by decide
theorem c0_bool_not : ∀ b : Bool, (!b) ≠ b := by decide
```

The second line is the workhorse for the Boolean flip. It states that no boolean equals its own negation, and `decide` proves it by checking both cases, `true` and `false`. This is the fixed-point-free fact that turns Lawvere's theorem into Cantor's, and through Cantor's into every diagonal impossibility in the book. Because `decide` on booleans reduces to a finite check with no axioms, results proved this way carry the cleanest possible trust story.

Remark 0.15. `decide` only applies when the proposition ranges over something finite or otherwise algorithmically checkable. You cannot `decide` a statement quantified over all natural numbers, because there is no finite check. The book uses `decide` exactly where the domain is `Bool` or a small finite type, and reasons by hand everywhere else. The quantifier `∀ b : Bool, …` is decidable because `Bool` has two elements; the quantifier `∀ n : Nat, …` is not, and no tactic can change that.

What "the kernel checks it" means

Lean has two layers. The *elaborator* is a large, sophisticated program that turns the surface syntax you write, with its implicit arguments, tactics, and notation, into a fully explicit proof term. The *kernel* is a small program, a few thousand lines, whose only job is to check that a fully explicit term has the type it claims. The elaborator is convenient but not trusted; the kernel is trusted but simple. A proof is accepted only when the kernel, reading the final term, agrees that its type is the theorem statement.

Definition 0.16 (Kernel checking). To say the kernel *checks* a proof is to say the small trusted type-checker has verified that the elaborated proof term has exactly the stated type, using only the fixed rules of the logic and any axioms the term explicitly invokes.

This division is why machine-checked mathematics is trustworthy without trusting the whole system. A bug in the elaborator, a tactic that does the wrong thing, a misapplied piece of automation: all of these produce a term that either fails to type-check, in which case you get an error and no theorem, or type-checks correctly, in which case the theorem is true regardless of how the term was found. The elaborator can be as clever and as buggy as it likes; the kernel is the referee, and the referee is small enough to audit. The trusted base is the kernel plus the axioms, and nothing else, which is a far smaller thing to trust than a human reviewer's attention over forty pages of analysis.

#print axioms and the trust story

Lean's logic includes a few optional axioms, chiefly propositional extensionality, quotient soundness, and the axiom of choice. Anything proved without them stands on the bare type theory. Anything that uses them is still a theorem, but its trust rests on those axioms too, and you should know which. The command `#print axioms name` reports exactly the axioms a given theorem depends on, tracing through every lemma it used.

Definition 0.17 (Axiom audit). For a proved theorem `name`, the command `#print axioms name` prints the list of axioms on which the proof depends, computed by transitively collecting the axioms of every definition and lemma in the proof term. An empty or trivial list means the result holds in the base logic alone.

Running `#print axioms` on the Boolean diagonal results reports that they depend on no axioms beyond the base logic: the proof of `c0_bool_not` and the Cantor instance built on it are, in the strictest sense the system offers, unconditional. The analytic results are different. Anything that goes through the real numbers, continuity, and the intermediate value theorem inherits `propext`, `Classical.choice`, and `Quot.sound`, because the construction of the reals and the classical logic used in analysis rest on them. This is not a defect; it is the honest accounting. When Chapter 4 says the diagonal proof is "cleaner" than the analytic proof, the axiom audit is the precise sense in which that is true, and you can reproduce the audit yourself with one command.

Remark 0.18. The axiom list is part of the theorem's meaning, not a footnote to it. Two proofs of the same statement, one axiom-free and one using choice, are genuinely different pieces of knowledge, and the book keeps track of which is which. This is a discipline that ordinary mathematical writing cannot easily maintain, and it is one of the concrete payoffs of doing the work in Lean. When you finish a chapter, running the audit on its main theorem is the fastest way to see what the result really cost.

A worked preview: the diagonal in five lines

You now have every construct the core engine uses. To prove it, here is the Lawvere fixed-point theorem, reproduced with a `c0_` name so it stands on its own in this chapter. Read it as a test of the vocabulary just built.

```
theorem c0_lawvere {A Y : Type} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ y, t y = y :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨f a₀ a₀, (congrFun ha₀ a₀).symm⟩
```

Walk through it with the definitions in hand. The hypothesis `hf` is a `∀` over functions `A → Y`, so `hf (fun a => t (f a a))` is universal instantiation, function application, at the specific function `a ↦ t (f a a)`. Its type is an `∃`, so we take it apart with `match`, binding a witness `a₀` and a proof `ha₀ : f a₀ = (fun a => t (f a a))`, an equation between functions. We must produce a fixed point of `t`, an `∃ y, t y = y`, so we pack a pair: the witness is `f a₀ a₀`, and the proof obligation is `t (f a₀ a₀) = f a₀ a₀`. Now `congrFun ha₀ a₀` evaluates the function equation at `a₀`, giving `f a₀ a₀ = t (f a₀ a₀)`, and `.symm` flips the equation to the orientation the goal wants. That is the whole proof: instantiate, unpack, evaluate at the diagonal point, repack.

If you can read those five lines, you can read every core proof in this book, because they are all variations on this one.

From the engine to Cantor, live

To see the engine do work rather than sit as an abstraction, specialize it. Take `Y = Bool` and `t` the negation `(!·)`, whose fixed-point-freeness is `c0_bool_not`. Feed the two into the engine and universality collapses outright.

```
theorem c0_cantor {A : Type} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match c0_lawvere f hf (fun b => !b) with
  | ⟨y, hy⟩ => c0_bool_not y hy
```

Read the proof as an accusation and a rebuttal. Suppose `f` is universal, a system that names every boolean pattern over its own indices. The engine `c0_lawvere` then hands us a fixed point `y` of negation, a boolean with `(!y) = y`, packaged as `⟨y, hy⟩`. The rebuttal is `c0_bool_not y`, which is a proof that `(!y) ≠ y`, that is a function from `(!y) = y` to `False`. Applying it to `hy` produces `False`, so the assumption of universality was impossible. This is Cantor's theorem in the reading of the previous section: identify a subset of `A` with its indicator `A → Bool`, and universality is exactly a point-surjection of `A` onto its power set, which cannot exist.

The engine also names the exact index on which the system breaks, not merely that one exists. That index is the liar, and producing it takes the same four moves with the negation baked into the diagonal from the start.

```
theorem c0_liar {A : Type} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) :
    ∃ a, f a a = !(f a a) :=
  match hf (fun a => !(f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Here we do not even invoke `c0_lawvere`; we run its guts directly. Name the twisted diagonal `a ↦ !(f a a)` by some `a₀`, then evaluate the naming equation at `a₀` with `congrFun` to get `f a₀ a₀ = !(f a₀ a₀)`. The index `a₀` is a behavior whose self-application equals its own negation, which is the liar sentence in Boolean clothing. Hold onto this construction. In Chapter 3 the same `a₀` becomes, under three readings, the query on which a truthful model must hallucinate, the prompt no wrapper can neutralize, and the state a truth probe cannot classify. They are one index seen three ways.

Running the blocks yourself, and reading errors

A practical note, since the blocks are meant to be run. Drop any block above into a file whose first line is nothing more than a comment, open it in an editor

with the Lean extension, and the same elaboration the book performs happens as you type. There are no imports to add for the core-Lean blocks; that is what "core Lean only" buys you. When a proof is wrong, the error appears at the point where the kernel's expectation and your term diverge, and it names the expected type and the type it actually found. Learning to read that mismatch is most of learning Lean: the expected type is the goal you still owe, and the found type is what your term delivered. A proof that goes through produces no message at all, which is the quiet the rest of this book runs on.

Sets and functions

The diagonal engine is stated in the language of sets and functions, and it uses a small, fixed vocabulary. This section fixes that vocabulary and the notation. Nothing here is deep. The point is that a few plain notions, stated carefully, carry the entire combinatorial half of the book.

Functions, domains, codomains

A function `f : A → Y` assigns to each element of its *domain* `A` a single element of its *codomain* `Y`. We do not distinguish "function" from "map"; they mean the same thing. The set of functions from `A` to `Y` is itself an object, which we write `A → Y`, matching the Lean type. When we speak of "all behaviors over `A` valued in `Y`," we mean all elements of `A → Y`.

Definition 0.19 (Function, evaluation). A function `f : A → Y` is a rule assigning to each `a : A` a value `f a : Y`, called the evaluation of `f` at `a`. Two functions `f, g : A → Y` are *equal* when `f a = g a` for every `a : A`; the principle that this pointwise agreement suffices for equality is function extensionality.

Composition, identity, and the classes of map

Two functions compose when the codomain of one is the domain of the next, and the identity map is neutral for composition. These give the language for the structural properties a map can have.

Definition 0.20 (Composition and identity). For `f : A → B` and `g : B → C`, the composite `g ∘ f : A → C` is `fun a => g (f a)`. The identity `id : A → A` is `fun a => a`. Composition is associative, and identity is a two-sided unit for it.

A map can be one-to-one, or onto, or both, and each property has a place in the arguments ahead.

Definition 0.21 (Injective, surjective, bijective). A function `f : A → B` is *injective* when `f a = f a'` forces `a = a'`; *surjective* when every `b : B` is `f a` for some `a`; *bijective* when it is both. In symbols, injective is `∀ a a', f a = f a' → a = a'` and surjective is `∀ b, ∃ a, f a = b`.

```
def c0_Injective {A B : Type} (f : A → B) : Prop :=
  ∀ a a', f a = f a' → a = a'
def c0_Surjective {A B : Type} (f : A → B) : Prop :=
  ∀ b, ∃ a, f a = b
```

Example 0.22. The successor `fun n => n + 1` on `Nat` is injective but not surjective, since nothing maps to `0`. The predecessor `c0_pred` is surjective but not injective, since `0` and `1` both map to `0`. On a finite set the two properties coincide, which is the pigeonhole principle, and it is exactly this coincidence that fails for infinite sets and lets the diagonal say something counting cannot.

Currying, again, as a naming scheme

The object at the center of the diagonal arguments is a function of the curried type `A → A → Y`. Chapter 1 calls such a thing a *system*, and reads it as a naming scheme: each `a : A` is a *name* or *index*, and `f a : A → Y` is the *behavior* that the name `a` denotes. Both arguments range over the same set `A`, and that coincidence is the whole source of self-reference. A name can be handed to a behavior, including the behavior it itself denotes, and the value `f a a` is the behavior at its own name.

Definition 0.23 (System and diagonal). A *system* is a function `f : A → A → Y`. Its *diagonal* is the function `A → Y` given by `a ↦ f a a`, sending each index to the value of its own behavior at itself.

The diagonal is where the argument acts. Given any transformation `t : Y → Y` of outcomes, the *diagonal behavior twisted by* `t` is `a ↦ t (f a a)`. It is a perfectly good function `A → Y`, so if the system names every function, it names this one, and the name it assigns is where the contradiction lands.

Point-surjectivity and universality

The diagonal arguments need surjectivity one level up, about the curried system rather than a plain map.

Definition 0.24 (Point-surjectivity, universality). A system `f : A → A → Y` is *point-surjective*, or *universal*, when every function `g : A → Y` is named: for all `g` there exists `a : A` with `f a = g`. In symbols, `∀ g : A → Y, ∃ a, f a = g`.

Read this as a precise form of "the system can talk about itself." A proof-checker that accepts the code of any predicate on proofs, a model that can be prompted to imitate any behavior over prompts, a representation that encodes any pattern over its own states: each asserts point-surjectivity for the right `A` and `Y`. The hypothesis is strong, and where the book argues that a real system satisfies it, that argument is the substance. The theorem itself, given the hypothesis, is a formality, which is the point of isolating the engine.

Point-surjectivity is genuinely weaker than the honest surjectivity of a map `A → (A → Y)`, and the difference is where the subtlety of the categorical ver-

sion lives. It asks only that each behavior be *hit*, not that the hitting be done by a single well-behaved map, and that is all the diagonal needs.

Example 0.25. No finite system is universal once `Y` has at least two elements. If `A` has `n` elements there are `n` names, hence at most `n` behaviors `f a`, but there are `|Y|^n` functions `A → Y`, and `n < |Y|^n` whenever `|Y| ≥ 2`. Counting already forbids universality in the finite case. The diagonal matters because it forbids universality for a reason that survives when `A` is infinite and counting has nothing to say.

Subsets, indicators, and the power set

A subset `S` of `A` is the same data as its *indicator* function `A → Bool`, sending an element to `true` exactly when it lies in `S`. So the set of all subsets of `A`, the power set, is the function set `A → Bool`. This identification is why Cantor's theorem, that no set surjects onto its power set, is the special case `Y = Bool` of the diagonal.

Definition 0.26 (Indicator). The *indicator* of a subset `S ⊆ A` is the function `A → Bool` returning `true` on `S` and `false` elsewhere. Subsets of `A` and functions `A → Bool` are in bijection under this correspondence.

```
def c0_iszero : Nat → Bool := fun n => n == 0
example : c0_iszero 0 = true := rfl
example : c0_iszero 3 = false := rfl
```

The map `c0_iszero` is the indicator of the subset `{0}` of `Nat`. Under the correspondence, universality of a system `A → A → Bool` is precisely a point-surjection of `A` onto its power set, and the liar of Chapter 1 is the subset that contains a name exactly when its behavior excludes it.

Sections and retractions

Surjectivity says every `g` has a name. A stronger, constructive statement gives you the name explicitly, as a function, and its mirror image records when a map can be undone.

Definition 0.27 (Section). A *section* of a system `f : A → A → Y` is a function `s : (A → Y) → A` such that `f (s g) = g` for every `g : A → Y`. A section picks, for each behavior `g`, a name `s g` that denotes it.

Definition 0.28 (Retraction). A *retraction* of a map `s : A → B` is a map `r : B → A` with `r (s a) = a` for all `a`. If a retraction exists, `s` is injective, and `A` is a "retract" of `B`.

```
def c0_Retraction {A B : Type} (r : B → A)
    (s : A → B) : Prop := ∀ a, r (s a) = a
```

Every section yields point-surjectivity: given `g`, the element `s g` is a witness for the existential, since `f (s g) = g`. The converse needs choice, to select a name for each `g` at once. Lawvere's theorem holds in the section form without any choice, and that form is the one that transfers verbatim to any cartesian closed category, which is why the categorical statement is about sections rather than surjections. For the set-level results the surjection form is all we need, and Exercise 0.9 asks you to redo the engine with a section and notice that the proof gets one step shorter.

Fixed points and fixed-point-free maps

The output side of the engine turns on a single property of a map `t : Y → Y`.

Definition 0.29 (Fixed point, fixed-point-free). A *fixed point* of `t : Y → Y` is a `y` with `t y = y`. The map `t` is *fixed-point-free* when it has none: `t y ≠ y` for all `y`.

Lawvere's theorem says a universal system forces every `t` to have a fixed point. Contrapositively, a fixed-point-free `t` forbids universality. So each impossibility in the diagonal half is manufactured by choosing a set of outcomes `Y` and a fixed-point-free transformation of them. On `Y = Bool` the negation `(!·)` is fixed-point-free, by `c0_bool_not`, and that single fact drives Cantor, Rice, Gödel read through the liar, and the AI-safety trilemmata. On the real interval the complement `y ↦ 1 - y` has one fixed point, at `1/2`, and that lone fixed point is not a failure of the pattern but the whole content of the analytic theorem: the forbidden place is the threshold itself.

Remark 0.30. The parallel is exact and worth holding onto before Chapter 4 makes it precise. The Boolean flip has no fixed point, so on a domain that can name its behaviors, something is forced to be its own negation, which is impossible, and universality dies. The real complement has exactly one fixed point, so on a connected domain, something is forced to sit exactly at the threshold, which the calibration hypotheses forbid, and the model dies. Same shape, two engines, and the fixed-point structure of the outcome map is the dial that switches between them.

The minimum topology

The analytic half of the book uses continuity and connectedness to reach the same boundary object the diagonal reaches by self-reference. This section states the few topological facts involved, at the level of generality the book actually needs, which is modest. The live proofs of these facts are in the companion Mathlib development, not in this chapter, because they need the library; here we state them carefully in prose and in backticked notation so that Chapter 4 reads smoothly.

The real line and intervals

We work over the real numbers $\mathbb{R}$ with their usual order and distance. The closed interval from `a` to `b` is `[a, b] = { x : a ≤ x ∧ x ≤ b }`. The two facts about $\mathbb{R}$ we lean on are that it is *order-complete*, every bounded nonempty set has a least upper bound, and that closed intervals are *connected*, which is a consequence of completeness. Order-completeness is the analytic counterpart of the naming richness the diagonal needs: it is the property that makes "the place where the sign changes" actually exist rather than fall through a gap in the line. Over the rationals the same setup fails, because a sign change can straddle an irrational the rationals do not contain, and this is precisely why the theorem is a theorem about $\mathbb{R}$ and not about any ordered field.

Metric and topological spaces

A *metric space* is a set with a distance function satisfying the triangle inequality; a *topological space* is the more general setting where "open set" is taken as primitive and distance may be absent. Everything the book needs can be read in the metric picture, where continuity is the familiar epsilon-delta statement, but the companion development states results topologically because that is how Mathlib is organized and because connectedness is cleanest there.

Definition 0.31 (Open set, topological space). A *topology* on a set `X` is a collection of subsets, the *open sets*, containing the empty set and `X` and closed under arbitrary unions and finite intersections. A set is *closed* when its complement is open. In a metric space the open sets are the unions of open balls, so the metric and topological pictures agree.

Continuity

Intuitively a function is continuous when small changes in the input produce small changes in the output, with no jumps. The book uses continuity in its standard topological form, which agrees with the epsilon-delta form on metric spaces.

Definition 0.32 (Continuity). A function `h : X → ℝ` from a topological space `X` to the reals is *continuous* when the preimage of every open set is open. Equivalently, for metric spaces, `h` is continuous at `x` when for every ε `> 0` there is a δ `> 0` such that `dist x x' <` δ implies `dist (h x) (h x') <` ε, and continuous when it is continuous at every point.

The models in the analytic chapters are assumed continuous as maps from a question space to a confidence-and-answer space, and correctness is measured by a continuous signed truth-distance. Continuity is the analytic stand-in for the structural assumptions of the diagonal side. It is what forbids the model from teleporting across the boundary instead of crossing it, and everything the analytic engine concludes is false without it, as Exercise 0.10 makes concrete.

Connected spaces

Connectedness is the property that a space is not split into two separated pieces. It is what makes the intermediate value theorem true, and it is the exact hypothesis that replaces point-surjectivity in the analytic engine.

Definition 0.33 (Connected space). A topological space `X` is *connected* when it cannot be written as the union of two disjoint nonempty open sets. Equivalently, the only subsets of `X` that are both open and closed are the empty set and `X` itself. A subset is connected when it is connected as a space with the subspace topology.

Example 0.34. Any interval of $\mathbb{R}$, open, closed, or half-open, bounded or not, is connected. A two-point set with the discrete topology is not connected: each point is open, and the space splits into two clopen halves. The distinction is precisely the one between the analytic and combinatorial engines. A connected question space supports the intermediate value argument; a discrete space of names supports the diagonal instead, and the discrete core of the analytic result, mentioned in Chapter 4, is what happens when you try to run the analytic argument on a disconnected domain and it degrades to counting.

Remark 0.35. For subsets of $\mathbb{R}$, connected and *path-connected* coincide, and both reduce to being an interval, so on the line there is nothing to distinguish. The reason the book states connectedness rather than "is an interval" is that the question spaces of the AI-safety readings are not literally intervals; they are abstract spaces assumed connected, and the intermediate value theorem holds for all of them uniformly. Stating the weakest sufficient hypothesis is what lets one proof cover every reading.

The intermediate value theorem

The one analytic theorem the book truly depends on is the intermediate value theorem, in the following form.

Theorem 0.36 (Intermediate value theorem). *Let `X` be connected and `h : X → ℝ` continuous. If `h` takes a value below `c` at some point and a value above `c` at another, then `h` takes the value `c` at some point: there is an `x₀ ∈ X` with `h x₀ = c`.*

The classical special case is `X = [a, b]`: a continuous real function that is negative at `a` and positive at `b` has a zero somewhere between. The general form, over an arbitrary connected space, is what the companion development uses, through Mathlib's connectedness API, and it is the form that makes the AI-safety statements clean, because the question space is not literally an interval but is assumed connected.

Nothing here is ours. The theorem is Mathlib's, and every use of it in this book and in the companion libraries goes through the two-function form `IsPreconnected.intermediate_value₂`, with the interval case handled by `isPreconnected_Icc`. Chapter 4 derives the statement above from those two, as Theorem 4.7, and traces the derivation; this section states it early only because the diagonal chapters already gesture at it.

Remark 0.37 (Why it is true). The one-variable proof is worth recalling, because it shows where completeness enters. Suppose `h(a) < 0 < h(b)`. Let `S` be the set of `x ∈ [a, b]` with `h(x) < 0`. It is nonempty and bounded, so by order-completeness it has a least upper bound `x₀`. Continuity rules out `h(x₀) < 0`, since then points slightly to the right would still be negative and `x₀` would not be an upper bound, and it rules out `h(x₀) > 0`, since then points slightly to the left would already be positive and `x₀` would not be least. The only survivor is `h(x₀) = 0`. The abstract version replaces "least upper bound of a bounded set" with "connectedness forbids a clopen split," but the moral is the same: a global completeness or connectedness property forces the crossing point to exist. This is the exact structural echo of the diagonal, where a global naming property forced the liar to exist.

Remark 0.38. The intermediate value theorem is the mirror image of the diagonal. There the naming hypothesis produced an index whose behavior contradicted itself; here connectedness produces a point where a continuous quantity is pinned to a value its other hypotheses forbid. In both, a global richness assumption on the domain, universality or connectedness, forces the existence of a specific bad point, and the bad point is where the impossibility lives. Chapter 4 records the two theorems as two spellings of one fact, and the file `Foundation.F_04` in the companion development makes the correspondence a formal statement rather than an analogy.

Signed distance, thresholds, and a word on Lipschitz control

The analytic readings measure correctness by a *signed* quantity, negative on one side of the truth boundary and positive on the other, so that the boundary is its zero set. Pairing this with a confidence score that must exceed a threshold turns the trilemma into a sign-crossing problem the intermediate value theorem resolves. The bare theorem gives only existence of the crossing. To get numbers, how wide the ambiguous band is or how far apart the decisive regions must be, you add a *Lipschitz* bound.

Definition 0.39 (Lipschitz map). A function `h : X → ℝ` on a metric space is *`L`-Lipschitz* when `dist (h x) (h x') ≤ L · dist x x'` for all `x, x'`. The constant `L` bounds how fast the output can change, so it converts a gap in output values into a guaranteed gap in inputs.

Lipschitz control is what upgrades a qualitative crossing into a quantitative band, and it is the whole reason the analytic engine can compute things the diagonal cannot. Chapter 5, on the geometry of attack basins, is a sustained application of this upgrade, and none of its quantities are visible without it.

The boundary object

Both engines output a distinguished point of the domain: the liar index on the diagonal side, the threshold-crossing question on the analytic side. Throughout the book this is the *boundary object*. It is not an artifact of either proof technique. It is the same object seen two ways, and the recurring theme is

that you cannot design it away, because it is forced by the very expressiveness or connectedness that made the system useful in the first place.

Definition 0.40 (Boundary object). The *boundary object* of an impossibility result is the domain element the proof forces into existence: for the diagonal, an index `a₀` with `f a₀ a₀ = t (f a₀ a₀)`; for the intermediate value theorem, a point `x₀` with the continuous quantity equal to its threshold value. In the AI-safety readings it is, respectively, the query on which a truthful model must hallucinate, the input no prompt filter can classify, and the state a truth probe cannot place on either side.

The two engines: a roadmap

Everything above serves one architecture. There are two proof engines, they produce the same boundary object, and each chapter of the book is one engine applied to one system. This section lays out the architecture and says where each piece lives, so that you can read the rest of the book knowing which engine is running and what it can and cannot deliver.

Engine one: the diagonal

The diagonal engine is Lawvere's fixed-point theorem, `c0_lawvere` above, and its contrapositive, "no fixed-point-free `t` admits a universal system." You feed it a reading of the outcome set `Y` and a fixed-point-free map `t`, and it returns an impossibility. The engine is finite in spirit, needs no axioms in its Boolean form, and gives a witness with no quantitative content: it tells you a bad point exists, not how many there are, not how hard one is to find, not what it costs an adversary to reach one.

Remark 0.41. The strength of the diagonal is also its limit. It says nothing metric. It cannot tell you the width of the region where a defense fails, or how the failure rate scales, because its only hypothesis is a naming property with no geometry attached. When you want numbers you must change engines. This is not a weakness to apologize for; it is a clean separation of concerns, with the qualitative "it is impossible" proved once, unconditionally, and the quantitative "here is how badly" proved separately where geometry is available.

Engine two: the intermediate value theorem

The analytic engine is Theorem 0.36. You feed it a connected question space, a continuous model, and a continuous correctness measure with values of both signs, and it returns a point on the boundary. It requires the reals and classical logic, so its axiom audit is heavier than the diagonal's, and in exchange it gives what the diagonal cannot: once you add Lipschitz control on the maps, the same argument yields quantities, the width of the ambiguous band, the minimum separation of the decisive regions, the rate at which the margin shrinks

as calibration tightens. The geometry of attack basins in Chapter 5 is entirely a refinement of this engine, and none of it is available to the diagonal.

Where the engines meet, and where they part

The two engines locate the same boundary object, and Chapter 4 is the hinge that shows this. The Boolean liar `(! ·)`, fixed-point-free, and the real complement `y ↦ 1 - y`, with its single fixed point at the threshold, are the two faces of one construction, recorded formally in `Foundation.F_04`. They meet at the boundary object and part on everything quantitative. Read the book with this question always in hand: which engine is running here, and what is it therefore allowed to conclude. If the claim is "impossible," suspect the diagonal. If the claim carries a number, a width, a rate, a probability, it is the analytic engine and its Lipschitz refinements at work.

The categorical origin, briefly

The origin of the diagonal engine is worth stating, because the source explains why one theorem covers so many cases. Lawvere proved his fixed-point theorem not about sets but about maps in a *cartesian closed category*, a setting abstract enough to include sets, computable functions, and the internal logic of a topos all at once. In that setting the hypothesis is stated with a section: a map `A → Y^A` admitting a right inverse, which is the categorical form of Definition 0.27. The proof is the same instantiate-and-diagonalize move, with composition standing in for function application. Because the argument lives at that level of generality, its instances are not analogies but literal specializations: Cantor, Gödel, Tarski, Turing, and the trilemmata of Chapter 3 are one theorem read in different categories. This book stays at the set level, where the surjection form suffices and the code is short, and points to the categorical statement only to explain why the pattern is not a coincidence.

Chapter map

Here is the shape of the book, so the preview is concrete.

Chapter 1 states and proves the diagonal engine, `lawvere` and `no_universal`, in a few lines of core Lean, and extracts the schema every later result instantiates. Chapter 2 turns the crank on the classical limitative theorems: Cantor, Rice, Gödel and Tarski through the liar, Turing's halting problem, each one choice of `Y` and fixed-point-free `t`. Chapter 3 applies the same engine to recent AI-safety impossibilities, the hallucination trilemma and the prompt-injection defense trilemma, arguing that the substance is in showing a real system's verdict is reflective, after which the contradiction is one line.

Chapter 4 is the pivot from logic to analysis. It re-derives the trilemma from topology, with the intermediate value theorem in the diagonal's role, and marks exactly where the two engines agree and diverge. Chapter 5 develops the quantitative side, the geometry of attack basins, which only the an-

alytic engine supports. Chapter 6 handles approximate bridges, what survives when the hypotheses hold only up to error. Chapter 7 draws the practical consequences for deploying language models. Chapter 8 answers the objection that all of this concerns outputs only, by turning both engines on a probe that reads the middle of a computation rather than its output.

Remark 0.42. The book is short by design, and the shortness is a claim: once the two engines are isolated, each result is a paragraph of reading plus a few lines of code, and the apparent variety of impossibility theorems collapses to two arguments and a catalogue of instances. If at any point a proof feels long, you are probably reading the part that argues a real system satisfies the hypotheses, which is honest work, and not the engine, which is always brief.

How the book is verified and built

The last preliminary is operational. This book is a Lean 4 project, and you can build it, check every proof, and run the axiom audits yourself. This section says how, at the level of detail needed to reproduce the verification.

The toolchain

The project pins Lean 4 at toolchain `leanprover/lean4:v4.28.0`, recorded in the `lean-toolchain` file, and uses `elan`, Lean's version manager, to select it automatically when you enter the directory. The book is authored in Verso, Lean's documentation system, pinned to `v4.28.0` to match, so that the live code blocks are elaborated by the same Lean that checks the companion proof libraries. Verso is what lets a code block in the text *be* a compiled definition rather than a quotation of one. The dependency set, recorded in `lake-manifest.json`, is Verso and its own dependencies; the self-contained chapters need nothing else, and in particular the core-Lean blocks in this chapter compile without Mathlib.

Building

The build tool is `lake`, Lean's package manager. From the `book` directory, `lake build` elaborates every chapter module, which is the same as checking every live proof in the text: if a code block did not compile, the build would fail there, naming the file and line. To produce the reading editions, `lake exe trilemmabook` runs the generator in `TrilemmaBookMain.lean`, which emits a single-page HTML edition and a TeX edition from the one source. The generator is configured to render the whole table of contents in the sidebar, because the HTML is deliberately one page: the book is short enough to read straight through, and a chapter-per-page split would turn every cross-reference into a navigation round trip.

Remark 0.43. Because the live blocks are checked at build time, there is no separate step to "verify the book." Building it is verifying it. A reader who distrusts a proof can clone the project, run `lake build`, and watch the kernel

accept the very lines printed in the text. That is a stronger guarantee than a human referee can give, restricted to exactly the claims stated in code. The first build downloads and compiles the dependencies and takes a while; later builds are incremental and touch only what changed.

The self-contained core and the companion libraries

The chapters split into two kinds. The diagonal chapters, one through four in their logical parts, are self-contained core Lean: the proofs you read are the proofs that compile, with no external library, and their axiom audits are trivial. The analytic results need Mathlib and live in companion developments, cited by name where they are used: `HallucinationProofs` for the hallucination trilemma and its boundary lemma, the defense results for the prompt-injection side, `Foundation` for the correspondence between the engines, `CCHProofs` for the computability instances, and `ManifoldProofs` for the geometric refinements. The book reproduces the argument of each cited result in prose and points to the verified statement, rather than importing the heavy library into the text, which keeps the text buildable in seconds while the deep results stay checked in their own libraries.

Reading the axiom audit

For any theorem in the self-contained core you can confirm the trust story with `#print axioms name`. On the Boolean diagonal results the report lists no axioms beyond the base logic, which is the precise meaning of the claim that Cantor's theorem and its AI-safety descendants are unconditional. On any result routed through the reals the report lists `propext`, `Classical.choice`, and `Quot.sound`, the three standard axioms behind classical analysis in Lean. The presence or absence of that list is not decoration. It is the difference, made checkable, between a combinatorial impossibility that holds in the bare type theory and an analytic one that holds given the classical construction of the continuum. Keep the audit in mind as you read: when two chapters prove nearby statements by different engines, the axiom lists are how the book records that they are, in a strong sense, different theorems.

Exercises

Exercise 0.1. Define in core Lean a function `c0_triple : Nat → Nat` that triples its argument, in both the named-argument style and the lambda style, and prove `c0_triple 4 = 12` by `rfl`. Explain in one sentence why `rfl` suffices here, referring to Definition 0.1 and the notion of definitional equality.

Exercise 0.2. State and prove, in term mode, that for propositions `p` and `q`, `p ∧ q → p`. Then state and prove `p → q → p ∧ q`. Identify which of your proofs packs a structure and which projects one, using the vocabulary of Definition 0.10.

Exercise 0.3. Using only `congrFun` (Definition 0.13), prove that for `f g : Nat → Nat` with `h : f = g`, one has `f 0 = g 0` and `f 7 = g 7`. Explain why you cannot prove `f = g` from `f 0 = g 0` alone, and name the principle that would be needed to go the other direction from agreement at *every* point.

Exercise 0.4. Write the predecessor function `c0_pred` yourself, then prove `c0_pred 0 = 0` and `c0_pred 5 = 4` by `rfl`. Now explain, without writing code, why `∀ n, c0_pred (n + 1) = n` is provable by `rfl` for each concrete `n` but requires a genuine proof, not `rfl`, when stated for all `n` at once.

Exercise 0.5. Prove `∀ b : Bool, b && b = b` by `decide`, and then argue why the analogous statement `∀ n : Nat, n * 1 = n` cannot be proved by `decide`. State precisely which hypothesis of Definition 0.14 fails for the second, and connect your answer to Remark 0.15.

Exercise 0.6. Re-derive Example 0.25 in detail. For `|A| = n` and `|Y| = 2`, exhibit a function `A → Y` that is not of the form `f a`, by flipping the diagonal `f a a`. Then explain in one paragraph why this counting argument, though correct here, gives the wrong intuition for infinite `A`, and what the diagonal supplies that counting cannot. Relate the failure of counting to Example 0.22, the successor map.

Exercise 0.7. Give an example of a fixed-point-free map `t : Bool → Bool` and prove it fixed-point-free by `decide`. Then show that no map `Unit → Unit` is fixed-point-free, and say in one sentence what this means, via Definition 0.29 and the engine, for a system whose only possible outcome is "accept."

Exercise 0.8. For the real complement `t y = 1 - y`, find its unique fixed point by solving `t y = y`. Relate the answer to the threshold in the analytic trilemma, and explain why "the map has exactly one fixed point" is the analytic counterpart of "the Boolean flip has none," referring to Remark 0.30.

Exercise 0.9. Prove the section form of the engine in core Lean: given `f : A → A → Y`, a section `s : (A → Y) → A` with `hs : ∀ g, f (s g) = g` (Definition 0.27), and any `t : Y → Y`, produce a fixed point of `t`. Compare your proof to `c0_lawvere` and identify the single step that becomes shorter when you have a section instead of a mere surjection.

Exercise 0.10. Read Theorem 0.36 and construct a counterexample when the domain is *not* connected: give a continuous `h` on the two-point discrete space of Example 0.34 that is negative at one point and positive at the other yet is never zero. Conclude in one sentence why connectedness is not an optional convenience in the analytic engine.

Exercise 0.11. Run, or describe running, `#print axioms` (Definition 0.17) on two theorems from later chapters: a Boolean diagonal result and an analytic result routed through the reals. Predict the two axiom lists before checking, then explain what the difference tells you about the two theorems as pieces of knowledge, referring to Remark 0.18.

Exercise 0.12. In `c0_lawvere`, the witness a_0 depends on the choice of preimage for the twisted diagonal. Argue that if two distinct indices both name the twisted diagonal, both are boundary objects (Definition 0.40), and that nothing in the proof selects a canonical one. Relate this non-uniqueness to the statement in later chapters that the boundary question is not unique.

Exercise 0.13. Using Definition 0.21, prove in core Lean that the composite of two injective functions is injective. Then explain in prose why injectivity is the retraction-side property (Definition 0.28) and surjectivity the section-side property, and where each appears in the diagonal argument.

Exercise 0.14. (Synthesis.) In one page and without symbols, explain to a colleague who knows analysis but not logic why the intermediate value theorem and Cantor's diagonal are, for the purposes of this book, the same argument. Your explanation should name the domain hypothesis each engine uses, the outcome-map property each exploits, and the boundary object each produces, and should say plainly what the analytic engine can compute that the diagonal cannot.

1 The Diagonal

Abstract. Every impossibility theorem in this book descends from one move: assume a system can describe its own behavior, build a description that disagrees with the system on itself, and read the disagreement as the theorem. Cantor, Russell, Gödel, Tarski, and Turing each used it. In 1969 Lawvere showed that these are one statement about maps in a cartesian closed category. This chapter states that statement, proves it in core Lean 4, and extracts the schema that later chapters instantiate. The proofs are elaborated when the book is built, so the proofs in the text are the proofs the kernel accepts.

Keywords: Diagonal argument, Fixed-point theorem, Lawvere, Self-reference, Lean 4

Every impossibility theorem in these notes descends from one move. You assume a system is expressive enough to describe its own behavior, you build a description that disagrees with whatever the system would do on itself, and the disagreement is the theorem. Cantor used it against surjections onto power sets, Russell against unrestricted comprehension, Gödel against provability, Tarski against truth, Turing against halting. In 1969 F. William Lawvere showed these are not five tricks but one, a single statement about maps in a cartesian closed category. This chapter states that statement, proves it, and extracts the schema that the rest of the book instantiates.

The mathematics here is elementary and entirely finite in spirit. The code blocks are core Lean 4 with no libraries; they are elaborated when the book is built, so the proofs you read are the proofs the kernel accepts. Read them as part of the text, not as an appendix to it.

A word on how to use the chapter. The technical core is short. A reader in a hurry can take Theorem 1.4 and its contrapositive, Corollary 1.10, and skip to Chapter 3, where they do their work. Everything else here earns that shortness. The long middle sections argue that the one hypothesis of Theorem 1.4 is exactly the informal idea "the system can describe itself," and the long historical section argues that five famous theorems are the one theorem read five ways. If you already believe both claims, you already know this chapter. If you do not, the point of the next forty pages is to make you believe them without hand waving.

1.1 One move, five disguises

Before any definitions, it helps to see the move in its natural habitats. The five arguments below were discovered separately over sixty years, by people who mostly did not think of themselves as doing the same thing. They look different because their subject matter is different: sets, then logic, then computation. The claim of this chapter, which we make precise in Theorem 1.4, is that the differences are decoration. Underneath, each argument builds a function that computes what the system does to a description and then changes the answer, and each derives its contradiction from the same place: nothing can equal its own change.

Read this section as motivation. Nothing in it is used later as a hypothesis; the formal development starts fresh in the following section. What the section buys you is the right to trust that Theorem 1.4 is not a toy. It is the shared skeleton of the deepest limitative results in mathematics, and the AI-safety theorems of Chapter 3 join that list by exhibiting the same skeleton.

Cantor: more subsets than elements

In 1891 Cantor proved that no set is as large as its own collection of subsets. Fix a set A and suppose, for contradiction, that some function $f : A \to P(A)$ hits every subset, that is, for every subset S there is an element a with $f\ a = S$. Cantor's subset is the set of elements that their own image excludes:

$$D = \{\, a \in A : a \notin f\ a \,\}.$$

By assumption D is $f\ a_0$ for some a_0. Now ask the one question that the set D was designed to make unanswerable: is a_0 in D? If $a_0 \in D$, then by the definition of D we have $a_0 \notin f\ a_0 = D$, a contradiction. If $a_0 \notin D$, then a_0 satisfies the membership condition, so $a_0 \in D$, again a contradiction. No a_0 survives, so no such f exists.

The whole argument turns on the phrase "their own image excludes." The element a is fed its own description $f\ a$, and D records the disagreement between a and $f\ a$ on the single bit "does a belong." When we later replace subsets by their indicator functions $A \to \mathrm{Bool}$, membership becomes a boolean, exclusion becomes boolean negation, and D becomes the function $a \mapsto \mathrm{not}\ (f\ a\ a)$. That function is the diagonal, and negation is the change that nothing can equal. Hold that translation in mind; it is Example 1.12, and it is the reason `bool_not_fpf` below is the entire content of Cantor's theorem.

Cantor's more famous use of the diagonal, also from 1891, was to show the real numbers are uncountable, and it is the same argument with the pool taken to be the natural numbers. Suppose a list $r_0, r_1, r_2, \ldots$ claims to enumerate every real in $[0,1]$ by their decimal expansions. Build a new real whose n-th digit differs from the n-th digit of r_n, say by replacing each digit with a fixed different one. The new real differs from r_n in its n-th place, for every n, so it is not on the list, and the list was not complete. Here the "system" is the enumeration $n \mapsto r_n$, the outcomes are digits, the self-application is reading the n-th digit of the n-th real, and the change is "pick a different

digit," which has no fixed point among digits. The two Cantor arguments, the power-set theorem and the uncountability of the reals, are one theorem: a list of functions cannot include the function that disagrees with each entry on the diagonal. We recover this from Theorem 1.4 by taking `A = ℕ`, `Y` the digits, and `t` a fixed-point-free digit change; the failure of universality is the incompleteness of the list.

Russell: the set that lists the modest sets

Russell found his paradox in 1901 while reading Frege, and he sent it to Frege in 1902 in a letter that arrived as the second volume of the *Grundgesetze* was in press. Frege's system allowed unrestricted comprehension: any property carves out the set of things having it. Russell chose the property "is not a member of itself" and formed

```
R = { x : x ∉ x }.
```

Then `R ∈ R` holds exactly when `R ∉ R`. The sentence is its own negation, and no sentence can be that. Frege's foundation collapsed on a single line.

Russell's `R` is Cantor's `D` with the indexing removed. Cantor kept a set `A` on the outside and a candidate surjection `f`; Russell let the universe of sets play both roles at once, so `f` becomes the identity "a set is its own description" and the diagonal `a ↦ a ∉ f a` becomes `x ↦ x ∉ x`. Stripped of the surrounding `f`, the paradox is more violent, because it does not derive a contradiction from an extra assumption. It derives one from comprehension alone. The lesson the twentieth century drew, and the lesson our Definition 1.2 encodes, is that the danger is not self-reference as such but unrestricted self-reference: a system that can name *every* pattern over its own descriptions, with no gatekeeper, is already inconsistent about the diagonal pattern.

The two standard repairs are both instructive, because they are the two ways every later chapter escapes an impossibility. One repair, Zermelo's, restricts comprehension: you may not form the set of all `x` with a property, only the subset of an already-given set with that property. This breaks the step where `R` is formed at all, because there is no ambient set of "all sets" to carve from. Translated into our terms, it denies universality: not every pattern over the universe is named by an object of the universe. The other repair, Russell's own theory of types, stratifies the universe into levels and forbids a set from taking itself as a member, so `x ∈ x` is not even a well-formed question. Translated into our terms, it breaks the coincidence of domains in `A → A → Y`: the judge and the judged are forced into different types, the diagonal cannot form, and the engine has nothing to grip. Chapter 3's positive results use both moves. Sometimes a safety property is achievable because the relevant behaviors are not all namable, the Zermelo escape; sometimes it is achievable because the system is typed so that it cannot be applied to itself, the Russell escape. Knowing which escape a design relies on is knowing why it works.

Gödel: a sentence that reads its own unprovability

Gödel's 1931 theorem is the same move performed inside arithmetic, and the performance is what made it hard. Cantor and Russell could write $\notin$ directly. Gödel had to build, out of nothing but addition and multiplication, a way for arithmetic to talk about its own sentences and proofs. He assigned a number to each formula and each proof, a coding now called Gödel numbering, and then showed that the relation "the proof coded by p proves the sentence coded by n" is itself expressible by an arithmetic formula. Once provability is an arithmetic predicate Prov, arithmetic can quote itself.

The engine is the diagonal lemma: for any formula $\varphi(x)$ with one free variable there is a sentence G such that the theory proves $G \leftrightarrow \varphi(\ulcorner G \urcorner)$, where $\ulcorner G \urcorner$ is the numeral coding G. Feed the lemma the formula "x is not provable" and you get a sentence G equivalent to "G is not provable." Now suppose the theory is consistent and proves or refutes every sentence. If it proves G, then G is provable, but G says it is not, so the theory proves a falsehood about itself and is inconsistent. If it refutes G, it proves "G is provable," which in a consistent theory forces an actual proof of G, and again both G and its negation are theorems. Either way completeness and consistency cannot both hold.

The diagonal lemma *is* the fixed-point theorem of this chapter, in the special case where the transformation t is negation of provability. The theory's ability to represent Prov is universality: it is the assumption that every arithmetic property of sentences, including the property we build by diagonalizing, is itself named by a sentence. The sentence G is the fixed point. What Gödel added to Cantor was not a new idea about self-reference but a heroic amount of coding to make an unassuming system, arithmetic, rich enough to be its own f. In Chapter 2 we make this exact, and there the coding is where the labor lives; the diagonal is one line.

Two features of the arithmetization are worth flagging now, because they are where the diagonal's abstract hypothesis meets a concrete system, and Chapter 2 will have to check them by hand. The first is representability. It is not enough that provability be *definable* by some formula; the formula must *track* provability in the theory itself, so that when a proof exists the theory proves that it does, and the coding of "the proof p checks n" is a formula the theory can verify on numerals. This is the arithmetic version of universality: the theory names, and correctly names, properties of its own sentences. Weak theories fail here, which is why incompleteness needs a lower bound on strength, usually a fragment of arithmetic strong enough to represent all computable relations. The second is the distinction between the first and second incompleteness theorems. The first, which is the diagonal above, produces an undecided sentence G. The second observes that the whole argument "if the theory is consistent then G is unprovable" can itself be carried out inside the theory, so the theory proves "consistency implies G," and since it cannot prove G it cannot prove its own consistency. The second theorem is the first theorem read inside the system, and it is the prototype for the self-defeating safety guarantees of Chapter 3, where a defense that could certify its own soundness

would thereby refute it.

Löb's theorem, from 1955, sharpens the picture and is worth naming because it is the fixed-point theorem for the modality "is provable." It says that if the theory proves "if S is provable then S," then it already proves S outright. The only sentences a sound theory can honestly assert to be self-guaranteeing are the ones it can prove anyway. Read through Lawvere, Löb is the statement that the provability operator has no nontrivial fixed points of a certain shape, and it is the reason a system cannot bootstrap trust in itself. We do not use Löb directly, but it stands behind the recurring theme that self-certification collapses, and a reader who wants the deepest version of "no system validates itself" should read Löb after this chapter.

Tarski: truth is not one of the definable notions

Tarski's undefinability theorem, from work of the early 1930s, is Gödel's move aimed at truth instead of provability. Suppose arithmetic could define its own truth predicate: a formula `True(x)` such that `True(⌜S⌝)` holds exactly when S is true. Apply the diagonal lemma to the formula "x is not true" and obtain a sentence L equivalent to "L is not true." Then L is true iff L is not true, which is impossible. So no such `True` is definable in the language it would be about.

L is the liar sentence, "this sentence is false," and Tarski's theorem is the observation that a sufficiently expressive language cannot host its own liar without contradiction. The transformation `t` is again negation; the output set `Y` is truth values; universality is the assumed definability of truth. The moral Tarski drew, the separation of object language from metalanguage, is the same gatekeeping move that set theory used against Russell. You may speak of truth for a language, but only from outside it. Our Corollary 1.10 states the abstract reason: negation has no fixed point, so nothing over which negation acts can be universal about itself.

Turing: no program decides halting

Turing's 1936 paper introduced the machines that bear his name and used the diagonal to show that halting is undecidable. Suppose a program `H` decides, for every program `p` and input `x`, whether `p` halts on `x`. Build a program `Dgn` that on input `p` runs `H` on the pair `(p, p)` and then does the opposite: if `H` says `p` halts on `p`, then `Dgn` loops forever; if `H` says `p` does not halt on `p`, then `Dgn` halts. Now run `Dgn` on its own code. `Dgn` halts on `Dgn` exactly when `H` reports that it does not, and loops exactly when `H` reports that it does. `H` is wrong about `Dgn`, so no correct `H` exists.

The self-application `p` on `p` is the diagonal; the "do the opposite" step is the fixed-point-free transformation. Universality here is the existence of a universal machine, which Turing also constructed: the fact that programs can be run on the codes of programs, including their own. The halting problem, Rice's theorem, and the rest of computability's negative results are this construction with the output set and the transformation changed. Exercise 1.13

and Chapter 2 develop the partial-function version, where the outputs may be undefined and the relevant transformation flips definedness.

The universal machine deserves a sentence of its own, because it is the exact computational form of Definition 1.2. Turing showed there is one machine `U` that, given the code of any machine `p` and an input `x`, simulates `p` on `x`. That is universality: a single `f` such that `f p` ranges over all the behaviors, at least all the computable ones. The subtlety, and the reason computability is subtler than set theory here, is that `U` only reaches the *computable* behaviors, not all functions. So the diagonal program `Dgn` is computable relative to `H`, and if `H` were computable then `Dgn` would be a computable behavior that `U` names, completing the contradiction. When `H` is dropped the diagonal still names a perfectly good function, but not a computable one, so no contradiction follows and the reals stay uncountable without arithmetic collapsing. The lesson carries into AI safety directly: a system is exposed to the diagonal only over the behaviors it can actually realize, and the force of an impossibility theorem is exactly the size of that class. Chapter 3's theorems work to show the class is large enough to contain the diagonal behavior they build.

Kleene: the same move, run forwards

The diagonal usually appears as a weapon, producing contradictions. Kleene's recursion theorem points it the other way and produces programs. It says that for any total computable transformation on programs there is a program that behaves exactly like its own transform: a fixed point in code. This is the constructive face of the same result. Where Turing chose a transformation with no fixed point and derived impossibility, Kleene took the guaranteed fixed point as a gift and built self-reproducing and self-referential programs from it.

We flag Kleene because it warns against a misreading. The theorem of this chapter does not say self-reference is bad. It says self-reference plus a fixed-point-free transformation is impossible. When the transformation *has* a fixed point, the same universality that would have caused a paradox instead hands you a useful self-referential object. Chapter 3's positive results, the places where a safety property *is* achievable, all live on this side of the line: they exhibit the fixed point rather than forbid it.

Lawvere: one theorem behind all of them

In 1969 Lawvere published *Diagonal arguments and cartesian closed categories* and proved that the arguments above are instances of a single theorem about maps. His setting is any cartesian closed category, an abstract universe of "spaces" and "functions" with enough structure to form function spaces `Y^A` and to evaluate. His hypothesis is that some map `A → Y^A` is point-surjective, meaning it reaches every point of the function space `Y^A`. His conclusion is that every endomap `t : Y → Y` has a fixed point. The contrapositive, the form we use, is that a fixed-point-free `t` forbids any such point-surjection.

Every classical instance is a choice of category, of `Y`, and of `t`. In the category of sets, `Y = Bool` and `t = not` gives Cantor. In a category built from a theory's sentences, `Y` a two-valued object and `t` negation gives Gödel and Tarski. In a category of computable maps, `Y` a type of outcomes and `t` an outcome flip gives Turing and Rice. The categorical wrapper is what makes "the same theorem" a literal statement rather than an analogy. We will not need the full category theory; the set-level version in Theorem 1.4 carries all the weight in these notes; Remark 1.9 records the one categorical upgrade, and Theorem 1.8 is the section form that we prove in Lean.

Yanofsky: the schema written out

Yanofsky's 2003 paper *A universal approach to self-referential paradoxes, incompleteness and fixed points* is the most readable modern account of Lawvere's insight, and it is the template for Chapter 2. Yanofsky strips the category theory down to a diagram of sets and functions and writes each classical paradox as the same commuting square with different labels. The value of his presentation, for us, is that it makes the recipe explicit and mechanical. To produce an impossibility theorem you name the indices, name the outcomes, name the transformation with no fixed point, and check universality. Everything else is filled in by the schema.

That recipe is what the rest of this chapter formalizes. The formal core is one theorem and its contrapositive; the classical instances are three short corollaries; and the AI-safety instances of Chapter 3 are the same corollaries with `Y` and `t` reinterpreted. We now leave the history and build the machine.

The shared skeleton, abstractly

Before the definitions, it is worth writing the common pattern down once, in the vocabulary the five arguments share, so that the formal statement in the next section reads as a transcription rather than a surprise. Each argument has four ingredients and one collapse.

The four ingredients are a pool of *things*, a set of *outcomes*, a way of *applying* a thing to a thing, and a *change* on outcomes. In Cantor the things are elements and subsets, the outcomes are the two membership values, applying is asking whether an element lies in a subset, and the change is swapping the two values. In Turing the things are programs and inputs, the outcomes are halting and looping, applying is running, and the change is doing the opposite. In Gödel the things are sentences, the outcomes are provable and not, applying is asking whether a sentence proves another, and the change is negating provability. The labels differ; the four slots are always filled.

The collapse is that the pool of things plays both roles in the applying step. A thing is both something that acts and something that can be acted upon, so a thing can be applied to itself. This is the only place the arguments need genuine self-reference, and it is exactly the coincidence of domains that Definition 1.1 below builds in by writing `A → A → Y` with the same `A` twice. Once a

thing can be applied to itself, you can form the *self-application* map that sends each thing to the outcome of applying it to itself, and then post-compose with the change. Call the result the diagonal description. It is a perfectly legitimate description of a behavior over the pool, so if the system describes *everything*, it describes the diagonal description too. That is where the argument dies: the thing describing the diagonal description, applied to itself, must produce an outcome that the change moves to itself, and the change was chosen to move nothing to itself.

Yanofsky draws this as a single commuting square. Up one side you take a thing to its self-application and read off an outcome; along the other you apply the change; and universality closes the square by naming the composite inside the pool. The square commutes for structural reasons, and its commuting at the diagonal point is the equation `y = t y`. Everything in this chapter is that square, drawn in `Set`, with the labels left as variables so that Chapter 3 can relabel them. If you keep the four slots and the one collapse in mind, you will recognize the pattern in every later theorem before you have read its proof.

1.2 Systems that describe themselves

Fix two sets, a domain `A` of *indices* and a set `Y` of *outcomes*. Think of an element of `A` as a name, a program, a prompt, an activation, or a Gödel number; think of `Y` as the possible answers the system can give.

Definition 1.1 (System). A *system* is a function `f : A → A → Y`. We read `f a` as the *behavior* named by the index `a`, a function `A → Y`; and `f a b` as the outcome that behavior `a` produces on input `b`. The *diagonal* of the system is the function `a ↦ f a a`: what each behavior does when applied to its own name.

The two-argument shape `A → A → Y` is doing quiet work, so it is worth reading slowly. The first argument is the name of a behavior and the second is the input that behavior consumes. The domains coincide, both are `A`, and that coincidence is the whole subject of the chapter. It is what lets a behavior be applied to a name of its own kind, and in particular to its own name. If the two roles lived in different sets there would be no diagonal and no theorem. The move we are studying is available exactly when the things a system talks about and the things a system is are drawn from one pool.

Definition 1.2 (Universality). A system `f : A → A → Y` is *universal*, or *point-surjective*, if every function `g : A → Y` is named by some index: for all `g` there is an `a` with `f a = g`. Formally, `∀ g : A → Y, ∃ a, f a = g`.

Universality is what "the system can talk about itself" means precisely. A proof-checker that can be handed the code of any predicate on proofs, an in-context learner that can be prompted to imitate any behavior over prompts, a representation that encodes any pattern over its own states: each is a claim of universality for the appropriate `A` and `Y`.

Two features of the definition deserve emphasis, because later chapters lean on them and because the theorem is stronger than it first looks.

First, universality quantifies over *every* function `g : A → Y`, with no restriction. It is not enough for `f` to name the nice behaviors, or the computable ones, or the ones a designer intended. The hypothesis demands that the naming reach the pathological behaviors too, including the one we are about to build by diagonalizing. This is why real systems often escape the theorem: a programming language names only the computable functions, not all functions, so its `f` is not universal in the sense of Definition 1.2. The AI-safety arguments of Chapter 3 do their hard work exactly here, arguing that some particular class of behaviors really is named in full.

Second, the definition asks only for point-surjectivity, that each `g` is `f a` for *some* `a`. It says nothing about uniqueness. A behavior may have many names, or a name may be shared, and the theorem does not care. All it uses is that a name exists for the one behavior it constructs. That is a low bar, which is what makes the impossibility strong: you cannot escape by making the naming many-to-one or wasteful.

A note on the shape `A → A → Y`. We could equally have written a system as a single function `A → (A → Y)` into the function space, and in Lean these are the same type: `A → A → Y` is read as `A → (A → Y)`, and a value `f a` is literally an element of `A → Y`. This identity, currying, is why "a system" and "a map from names to behaviors" are interchangeable descriptions, and it is the set-level shadow of the exponential object in Remark 1.9. It also connects to logic through the Curry-Howard correspondence, under which a function type is an implication and evaluation is modus ponens. Universality then reads as "every behavior is inhabited by a name," and the diagonal construction is the proof-theoretic move that turns unrestricted self-reference into a contradiction. We will not lean on Curry-Howard, but it is the reason the same three-line proof appears in a set theory book, a logic book, and a type theory book with only the nouns changed.

Point-surjectivity has a weaker cousin that Lawvere actually used and that is worth knowing by name, because it is the minimal hypothesis. A map is *weakly point-surjective* if for every `g : A → Y` there is an `a` such that `f a` and `g` agree, which for our purposes is the same as `f a = g` since we are in `Set`; in a general category the weak form asks only for agreement on points, which is strictly weaker than equality of maps. The distinction does not affect any theorem in these notes, because all our instances live in `Set` where the two coincide, but it is why the categorical statement of Remark 1.9 is more general than a naive reading suggests. When someone claims a system "is not really universal," the honest question is whether it fails even the weak form, and usually it does, by naming only a restricted class of behaviors.

The finite case, by counting

Example 1.3. No *finite* system is universal when `Y` has at least two elements. If `A` has `n` elements then there are exactly `n` behaviors of the form `f a`, one for each index. But the set of *all* functions `A → Y` has `|Y|` choices at each of the `n` inputs, hence $|Y|^n$ functions in total. For $|Y| \geq 2$ we have $n < 2^n \leq |Y|^n$,

so there are strictly more functions than names, and some `g` is left unnamed. Universality fails for sheer lack of room.

It is worth doing the smallest case by hand, because it shows the counting and the diagonal are the same fact seen twice. Take `A = { 0, 1 }` and `Y = Bool`, so `n = 2` and there are `2² = 4` functions `A → Bool`. A system `f` supplies two of them, `f 0` and `f 1`. Write out the diagonal values `f 0 0` and `f 1 1` and form the function `g` that flips each: `g 0 = not (f 0 0)` and `g 1 = not (f 1 1)`. Then `g` cannot be `f 0`, because they differ at input `0`, and `g` cannot be `f 1`, because they differ at input `1`. So `g` is one of the four functions that `f` does not name, whatever `f` was. The pigeonhole count told us an unnamed function exists; the diagonal *points at one*.

That last sentence is the whole difference between counting and diagonalizing, and it is why the diagonal survives into the infinite case where counting says nothing. When `A` is infinite there is no comparison of sizes to make. There are infinitely many names and infinitely many functions, and a naive count cannot distinguish the two infinities without already knowing Cantor's theorem. The diagonal needs no count. It writes down a specific function, `a ↦ not (f a a)`, and checks that this one function differs from `f a` at the input `a`, for every `a` at once. The construction is uniform in `a` and indifferent to whether `A` is finite. Counting is a special effect of finiteness; the diagonal is the reason, and it keeps working when the effect is gone.

Exercise 1.1 asks you to turn the two-element case into the general finite claim and then to notice that your unnamed `g` is exactly the function produced by `liar_query` below. The counting proof and the diagonal proof are not two proofs. They are one proof with the arithmetic removed.

The infinite case, by diagonal

A worked infinite instance. No system `f : ℕ → ℕ → Bool` is universal. There is no counting here to save us: there are infinitely many names `f 0, f 1, f 2, …` and infinitely many behaviors `ℕ → Bool`, and the two infinities cannot be compared without already knowing the answer. The diagonal settles it directly. Define `g : ℕ → Bool` by `g n = not (f n n)`. For each `n` the behaviors `g` and `f n` disagree at the single input `n`, because `g n = not (f n n)` while `f n` at `n` is `f n n`, and a boolean differs from its negation. So `g` is not `f n` for any `n`, and `g` is unnamed. Universality fails, and it fails by pointing at a specific unnamed behavior rather than by an existence count.

This is the whole of the boolean Cantor theorem, and it is exactly `cantor` specialized to `A = ℕ`, but writing it out for `ℕ` is worth the space because it shows the machinery running where intuition is weakest. Nothing about `ℕ` was used except that we could form `g` and evaluate it at each `n`. The same lines work for any `A` at all: replace `n` by `a` throughout and the argument is unchanged, which is why the general theorem below carries no cardinality hypothesis. The infinite case is not harder than the finite case. It is the finite case with the counting step, which was never load-bearing, deleted. Readers who first met the diagonal as a trick for the uncountability of the reals often

carry away the impression that it is about sizes of infinity. It is not. It is about a function that disagrees with a list on the diagonal, and sizes of infinity are one downstream consequence.

1.3 Lawvere's fixed-point theorem

Theorem 1.4 (Lawvere, 1969). *Let* `f : A → A → Y` *be universal. Then every map* `t : Y → Y` *has a fixed point: some* `y` *with* `t y = y`.

```
theorem lawvere {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ y, t y = y :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨f a₀ a₀, (congrFun ha₀ a₀).symm⟩
```

Proof. Consider the *diagonal behavior* `d : A → Y`, `d a = t (f a a)`: apply each behavior to its own name, then transform the outcome by `t`. By universality `d` is named, so there is an index `a₀` with `f a₀ = d`. Two functions that are equal are equal at every point, so evaluate at `a₀`: `f a₀ a₀ = d a₀ = t (f a₀ a₀)`. Thus `f a₀ a₀` is fixed by `t`. In the Lean proof, `congrFun ha₀ a₀` is exactly the step "equal functions agree at `a₀`," and `.symm` orients the equation as a fixed point. ■

Three lines, no arithmetic, no topology, no continuity. Everything downstream is a reading of these three lines. It is worth dwelling on what the proof used: only that behaviors can be applied to their own names, and that a named behavior can be recovered. That is the whole of self-reference.

Let us take the proof apart once, at the pace of someone seeing it for the first time, because the same three moves recur in every instance and it pays to recognize them by name.

The first move is the construction of `d`. We do not pick `d` cleverly; we let the transformation `t` and the system `f` write it for us. The recipe is fixed: run the diagonal `a ↦ f a a`, then post-compose with `t`. This `d` is a perfectly ordinary function `A → Y`, and nothing about it announces that it is dangerous. It is dangerous only because of what universality will force.

The second move is the appeal to universality. We hand `d` to the hypothesis and receive an index `a₀` with `f a₀ = d`. This is the only place the hypothesis is used, and it is used on the single function `d`, not on any others. If you were building a system and wanted to dodge the theorem, this is the equation you would have to prevent, and Definition 1.2 says you cannot: universality names everything, `d` included.

The third move is evaluation on the diagonal. We have an equation between functions, `f a₀ = d`. Functions are equal when they agree everywhere, so in particular they agree at the point `a₀`. Evaluating both sides at `a₀` turns the functional equation into a numerical one, `f a₀ a₀ = d a₀`, and unfolding `d` on the right gives `f a₀ a₀ = t (f a₀ a₀)`. The quantity `f a₀ a₀` is

now visibly a fixed point of `t`. The Lean term names this quantity `f a₀ a₀`, produces the equation with `congrFun ha₀ a₀`, and flips its direction with `.symm` to match the `t y = y` we promised.

Remark 1.5 (Where the self-application happens). The subtlety, such as it is, is that `a₀` appears in three roles at once in the line `f a₀ a₀ = t (f a₀ a₀)`. It is the name of the behavior `f a₀`; it is the input that behavior is run on; and it is the point at which we evaluate the naming equation. All three coincide because `d` was built from the diagonal, where name and input are forced equal, and universality then supplies a single `a₀` that names this particular `d`. The collapse of three roles into one index is the diagonal, in one symbol. Every proof in the book has an `a₀` like this. In Chapter 3 it is a specific query; in Chapter 4 it is a specific point of a connected space.

Remark 1.6 (Constructive content). The proof is constructive in a precise sense. Given the witness `a₀` that universality provides for the one function `d`, the fixed point is computed with no further choices: it is literally `f a₀ a₀`, and the equation `t (f a₀ a₀) = f a₀ a₀` is verified by evaluating a known equality at a known point. There is no case split, no excluded middle, no appeal to choice. This matters for two reasons. It is why the Lean proof is a plain term rather than a tactic script fighting with classical logic, and it is why the boolean instances below need no axioms at all, a point we return to when we contrast Cantor with Russell. When Chapter 3 audits which safety theorems are constructive and which import a classical principle, the audit starts here: the engine adds nothing, so any nonconstructive step is charged to the reading of `t`, not to Lawvere.

Three common misreadings. The proof is short enough that its meaning is easy to over- or under-state, and three misreadings recur often enough to name.

The first is to think the theorem says self-reference is paradoxical. It does not. It says self-reference together with a fixed-point-free transformation is impossible. When the transformation has a fixed point, the identical construction produces that fixed point as a useful self-referential object, which is Kleene's recursion theorem. The theorem is neutral about self-reference; it is the combination with an "always change the answer" map that cannot coexist with universality.

The second is to think the theorem needs infinity, or large cardinals, or some subtle set-theoretic strength. It needs none. The proof is finite, it is constructive, and it runs verbatim when `A` and `Y` are finite, where it reduces to the counting of Example 1.3. Cantor's theorem about infinite sets is a corollary, not the content. The content is that a function which disagrees with each named behavior on the diagonal cannot itself be named, and that is a statement about one function and one evaluation.

The third is to think universality is a mild or generic hypothesis. It is the whole game. Almost no natural system is universal, because naming *every* behavior, including the deliberately perverse diagonal one, is a severe demand. Every impossibility theorem in the book is, in the end, an argument that some specific system meets this severe demand, and every escape from an impossi-

bility theorem is, in the end, a demonstration that it does not. If you remember one thing from this chapter, remember that the diagonal is trivial and the hypothesis is everything.

1.4 The section and retract form

Definition 1.2 asked for point-surjectivity: for each `g` there *exists* an index. Lawvere's own statement uses a slightly different and often more convenient hypothesis, a section. A section is a chosen inverse to naming, a single function `s : (A → Y) → A` that hands you, for each behavior `g`, a specific name `s g` that works.

Definition 1.7 (Section). A *section* of a system `f : A → A → Y` is a function `s : (A → Y) → A` such that `f (s g) = g` for every `g : A → Y`.

The two hypotheses are close but not identical. A section is a uniform, chosen naming; point-surjectivity only promises that names exist, without choosing them. Every section gives point-surjectivity, since `s g` is a witness. The converse needs a choice principle to pick one witness per behavior, which is why the categorical literature prefers sections: the section form is the one that holds verbatim in any cartesian closed category, with no appeal to choice, and it is how Lawvere stated the theorem. The section form is also strictly weaker as a hypothesis in constructive settings, hence strictly stronger as a theorem. Our downstream needs only the point-surjective form, but the section form is worth proving once, because it is the version that transports to categories and it makes the constructive content of Remark 1.6 fully explicit.

Theorem 1.8 (Lawvere, section form). *If `f : A → A → Y` has a section `s : (A → Y) → A`, then every `t : Y → Y` has a fixed point.*

```
theorem c1_lawvere_section {A Y : Type _} (f : A → A → Y)
    (s : (A → Y) → A)
    (hs : ∀ g : A → Y, f (s g) = g)
    (t : Y → Y) : ∃ y, t y = y := by
  refine ⟨f (s (fun a => t (f a a))) (s (fun a => t (f a a))), ?_⟩
  have h := congrFun (hs (fun a => t (f a a))) (s (fun a => t (f a a)))
  exact h.symm
```

Proof. The proof is Theorem 1.4 with the chosen name in place of the existential one. Write `d = (fun a => t (f a a))` for the diagonal behavior and let `a₀ = s d` be the name the section assigns it. The section equation `hs d` says `f a₀ = d`, and evaluating it at `a₀` with `congrFun` gives `f a₀ a₀ = t (f a₀ a₀)`. The witness is `f a₀ a₀`, and `.symm` orients the equation. Because `s` is given rather than extracted, the whole proof is a closed term with no case analysis. ■

The section form makes visible that point-surjectivity was never really needed in full. All the proof consumed was one name for one behavior, `d`. A

section is just a systematic way of always having that name to hand. The next lemma records the easy direction of the relationship, that a section is at least as strong as universality, which is what lets Theorem 1.8 subsume Theorem 1.4 in practice.

```
theorem c1_section_universal {A Y : Type _} (f : A → A → Y)
    (s : (A → Y) → A)
    (hs : ∀ g : A → Y, f (s g) = g) :
    ∀ g : A → Y, ∃ a, f a = g :=
  fun g => ⟨s g, hs g⟩
```

Proof. Given any `g`, the name `s g` works: `f (s g) = g` is exactly `hs g`. So `s g` witnesses the existential. ■

Sections, retracts, and why the name. The word *section* comes from the picture of `f` as a projection: `s` is a one-sided inverse, a way of sectioning back up through `f`. The dual word is *retract*. The condition `f ∘ s = id` on behaviors says the function space `A → Y` is a *retract* of the index set `A` along `f` and `s`: you can embed every behavior into `A` by `s` and recover it by `f`, so `A → Y` sits inside `A` as a retract. Lawvere's theorem is often stated in exactly this retract language, "if `Y^A` is a retract of `A` then every endomap of `Y` has a fixed point," and that is the cleanest way to see why it transports to any cartesian closed category: retracts are a categorical notion, defined by the single equation `f ∘ s = id`, needing no elements. The set-level `hs : ∀ g, f (s g) = g` is that equation written pointwise. When Chapter 3 wants to claim a system is universal, the strongest and most transportable way to do it is to exhibit `s` explicitly, that is, to give a construction that produces, for any target behavior, an index realizing it. A retract is a construction; a surjection is only an existence claim. Whenever a proof can afford the retract, it should prefer it, because it is constructive and it is the form the categorical statement wants.

Remark 1.9 (The cartesian closed statement). A category is *cartesian closed* when it has three things: a terminal object `1`, whose global points `1 → X` play the role of "elements" of `X`; binary products `X × Z`, packaging a pair of maps into one; and, for each pair of objects, an exponential object `Y^A` together with an evaluation map `ev : Y^A × A → Y` that is universal among ways of applying an `A`-indexed family to an argument. In `Set` these are the one-point set, the cartesian product, and the set of functions `A → Y` with ordinary function application. The point of the abstraction is that products, exponentials, and evaluation are exactly the structure the diagonal proof consumes, and no more.

In such a category, a map `f : A → Y^A` is *weakly point-surjective* if for every global point `g : 1 → Y^A` there is a point `a : 1 → A` with `f ∘ a = g` as points, that is, they evaluate the same on every argument. Lawvere's theorem is that such an `f` forces every endomap `t : Y → Y` to have a fixed point `y : 1 → Y` with `t ∘ y = y`. The proof is the exact diagram our Lean term draws. Form the diagonal `A → A × A`, follow it with `f × id` and then evaluation to get the self-application `A → Y`, post-compose with `t`, and transpose back to a

point of `Y^A`. Weak point-surjectivity names this point by some `a`, and chasing `a` around the resulting square yields `y = t y` at the diagonal. Every arrow in that chase is built from a product, an exponential, or evaluation.

Nothing in the argument uses that the ambient category is `Set`. This is the sense in which Cantor, Gödel, Turing, and the rest are literally the same theorem: they are Theorem 1.8 in different cartesian closed categories. Cantor lives in `Set`. Gödel and Tarski live in a syntactic category built from a formal theory, where objects are formulas-with-free-variables and maps are provable substitutions, and where the exponential encodes "formulas about formulas"; the coding Gödel labored over is precisely the construction of enough cartesian closure for the diagonal to run. Turing and Rice live in a category of computable maps, where the exponential is the set of programs and evaluation is the universal machine. Each classical theorem is the failure of weak point-surjectivity forced by a fixed-point-free `t` in its own category. We do the categorical proof in prose and keep the Lean development in `Set`, since every instance in these notes is a set-level statement and the set proof is what the kernel checks. A reader who wants the diagram drawn in full should consult Lawvere's paper or Yanofsky's exposition; our `c1_lawvere_section` is the same statement with the section made explicit, which is the form that transports to any cartesian closed category without a choice principle.

1.5 The contrapositive is the impossibility

Read Theorem 1.4 backwards. If some transformation of outcomes has *no* fixed point, then no system can be universal. This is the shape every later result takes: name a behavior-flip that nothing is fixed by, and universality collapses.

Corollary 1.10. *If `t : Y → Y` satisfies `t y ≠ y` for all `y`, then no `f : A → A → Y` is universal.*

```
theorem no_universal {A Y : Type _} (t : Y → Y)
    (ht : ∀ y, t y ≠ y)
    (f : A → A → Y) : ¬ (∀ g : A → Y, ∃ a, f a = g) :=
  fun hf => match lawvere f hf t with
    | ⟨y, hy⟩ => ht y hy
```

Proof. If `f` were universal, Theorem 1.4 would produce a fixed point `y` of `t`, contradicting `t y ≠ y`. ■

Remark 1.11. Corollary 1.10 is the engine. To obtain a specific impossibility one supplies (i) a reading of `Y`, and (ii) a fixed-point-free `t`. The rest of this book is a catalogue of such pairs. The classical ones are in Chapter 2; the AI-safety ones in Chapter 3.

It is worth stating the corollary in two more shapes, because downstream files quote whichever is most ergonomic and it helps to see they are the same fact. The first packages the impossibility as the nonexistence of any universal

system for a type with a fixed-point-free flip, rather than as a property of a given `f`.

```
theorem c1_no_universal_exists {A Y : Type _} (t : Y → Y)
    (ht : ∀ y, t y ≠ y) :
    ¬ ∃ f : A → A → Y, ∀ g : A → Y, ∃ a, f a = g :=
  fun ⟨f, hf⟩ => match lawvere f hf t with
    | ⟨y, hy⟩ => ht y hy
```

Proof. Suppose such an `f` existed. It would be universal, so Theorem 1.4 gives a fixed point of `t`, contradicting `ht`. ■

The difference between `no_universal` and `c1_no_universal_exists` is only where the `f` is quantified: the first refutes a candidate handed to it, the second denies that any candidate exists. Use whichever the context supplies. Chapter 3's theorems arrive with a specific verdict system in hand, so they call the first; existence-style corollaries call the second.

The second extra shape returns the diagonal index itself, not just the fixed point. This is the form that most downstream proofs actually consume, because they want to point at the offending behavior, not merely to know one exists.

```
theorem c1_lawvere_diagonal {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ a, f a a = t (f a a) :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Name the diagonal behavior `a ↦ t (f a a)` by some `a₀`, then evaluate the naming equation at `a₀`. The result is `f a₀ a₀ = t (f a₀ a₀)`, which says that the outcome `f a₀ a₀` is moved to itself by `t` only if `t` fixes it, and in general exhibits the exact index where the diagonal bites. ■

Compare `c1_lawvere_diagonal` with `lawvere`. They run the same construction; one returns the fixed value `f a₀ a₀`, the other returns the index `a₀`. When `t` is fixed-point-free the returned equation `f a₀ a₀ = t (f a₀ a₀)` is a contradiction, and this is how the classical instances below extract `False`. When `t` has a fixed point the same equation is instead a self-referential program in the sense of Kleene. The theorem does not know which case it is in; that is decided entirely by `t`.

A concrete example makes the two faces vivid. Suppose `Y = Bool` and `t` is the identity rather than negation. Then for a universal `f` the diagonal produces an index `a₀` with `f a₀ a₀ = f a₀ a₀`, which is no contradiction at all; it is a tautology, and `f a₀ a₀` is a legitimate value that "describes its own behavior" in the harmless sense of agreeing with itself. Now replace `t` by negation and the same index yields `f a₀ a₀ = not (f a₀ a₀)`, which cannot hold. The construction did not change. Only the transformation did, and

with it the character of the output flipped from a harmless self-description to an impossible one. This is the sharp version of the moral that self-reference is not the problem. A universal system always has fixed points of the transformations that admit them; it is forbidden only the transformations that admit none. Kleene's recursion theorem is the systematic exploitation of the first case, building quines and self-optimizing programs; the impossibility theorems are the systematic exploitation of the second. They are two readings of one line.

1.6 Three classical instances

We now instantiate Corollary 1.10 three times. Each instance is a choice of `Y` and a proof that some `t : Y → Y` is fixed-point-free. The instances are Cantor, Russell, and, in a sharpened form that names the offending index, the liar.

Example 1.12 (Cantor). Take `Y = Bool` and `t` the negation `(!·)`. No boolean equals its own negation, which Lean settles by checking both cases:

```
theorem bool_not_fpf : ∀ b : Bool, (!b) ≠ b := by decide
```

By Corollary 1.10, no system with boolean behaviors is universal over its own behaviors:

```
theorem cantor {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match lawvere f hf (fun b => !b) with
  | ⟨y, hy⟩ => bool_not_fpf y hy
```

The proof is worth reading against the historical Cantor of the first section. The function `hf` is the assumed surjection onto behaviors; the flip `fun b => !b` is the transformation; `lawvere` runs the diagonal and returns a boolean `y` with `(!y) = y`; and `bool_not_fpf` says no such `y` exists. The one line `bool_not_fpf` carries the entire mathematical content, and it is decided by two-case exhaustion, which is why Cantor's theorem for booleans needs no axioms beyond computation.

Let us make the identification with the textbook Cantor precise, since it is the prototype for every reading in the book. A subset `S ⊆ A` is the same data as its indicator function `χ_S : A → Bool`, where `χ_S a` is `true` exactly when `a ∈ S`. Under this identification the power set `P(A)` is the function space `A → Bool`, and a surjection `A → P(A)` is a surjection `A → (A → Bool)`. A surjection of that type is precisely a universal system `f : A → A → Bool` in the sense of Definition 1.2: `f a` is the indicator of the subset named by `a`, and universality says every subset is named. Cantor's set `D = { a : a ∉ f a }` has indicator `a ↦ not (f a a)`, which is our diagonal behavior for `t = not`. The index `a₀` that names `D` is Cantor's `a₀`, and the contradiction `not (f a₀ a₀) = f a₀ a₀` is his "is `a₀ ∈ D`?" answered both ways at once. So `cantor` above is not an

analogue of Cantor's theorem. Up to the indicator-function dictionary, it is Cantor's theorem, and the dictionary is a definitional identity, not an approximation.

Example 1.13 (Russell). Take `Y` to be propositions and `t` logical negation. The behavior "the set of indices that do not contain themselves" is the diagonal; its own membership status is fixed by negation, and negation has no fixed point, so the behavior is unnamed. This is Russell's paradox, and it is Corollary 1.10 with `t = ¬`. We state it in prose rather than code because the fixed-point-free step for propositions needs a classical principle, whereas the boolean version above needs nothing; the distinction will matter when we audit axioms.

The contrast between Examples 1.12 and 1.13 is the first place the book's axiom-accounting shows up, so it is worth naming. On `Bool`, negation is a computation and "no `b` has `not b = b`" is decided by trying both values; `bool_not_fpf` is proved by `decide` and imports nothing. On the type of propositions, negation is `¬`, and "no proposition `P` has `(¬P) = P`" is not decidable by exhaustion, because there is no finite set of propositions to try. Establishing it requires reasoning about `P ↔ ¬P`, which in a constructive setting you can still refute, but the smooth classical statement "`¬P ≠ P` for all `P`" leans on treating propositions as a two-valued type, which is a classical assumption. The engine is the same in both examples. What differs is the cost of proving the transformation fixed-point-free, and that cost is charged to the reading, exactly as Remark 1.6 predicted.

Proposition 1.14 (The liar, named). Corollary 1.10 refutes universality, but the diagonal does more: it points at the exact behavior on which the system breaks. For the boolean flip this witness is the *liar*, an index whose self-application equals its own negation.

```
theorem liar_query {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) :
    ∃ a, f a a = !(f a a) :=
  match hf (fun a => !(f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Name the diagonal behavior `a ↦ !(f a a)` by some `a₀`; evaluating the naming equation at `a₀` gives `f a₀ a₀ = !(f a₀ a₀)`. ■

Notice that `liar_query` is `c1_lawvere_diagonal` specialized to `Y = Bool` and `t = not`, and that it does not conclude `False`. It hands back a live equation, `f a₀ a₀ = not (f a₀ a₀)`. That equation is the contradiction only after you add `bool_not_fpf`; on its own it is a description of the pathology, a query the system answers with the opposite of its own answer. Keeping the query rather than collapsing to `False` is what lets Chapter 3 talk about the offending input as an object with meaning: the hallucination boundary, the injected prompt, the query a truth probe misplaces.

Hold onto this `a₀`. In Chapter 3 it becomes, in turn, the hallucination model's boundary question, the prompt injection no wrapper can neutralize,

and the query a truth probe cannot place on either side. They are the same index under three readings.

What the flip needs: two small facts about Y

Corollary 1.10 needs a fixed-point-free endomap of `Y`. Whether one exists is a property of `Y` alone, independent of any system, and it is the true precondition for an impossibility. Two boundary cases make the point, and both are one-line checks in Lean.

The output type `Bool` admits a fixed-point-free flip, so it can host impossibilities. This is `bool_not_fpf` repackaged as an existence statement about `Bool`, which is the form the type-level arguments of Chapter 3 quote.

```
theorem c1_bool_has_fpf : ∃ t : Bool → Bool, ∀ b, t b ≠ b :=
  ⟨fun b => !b, bool_not_fpf⟩
```

The output type `Unit`, by contrast, admits no fixed-point-free endomap, because it has only one element and any endomap fixes it.

```
theorem c1_unit_has_fixed_point
    (t : Unit → Unit) : ∃ u : Unit, t u = u :=
  ⟨(), rfl⟩

theorem c1_no_fpf_unit :
    ¬ ∃ t : Unit → Unit, ∀ u, t u ≠ u :=
  fun ⟨_t, ht⟩ => ht () rfl
```

Proof. There is only one element `()`, and `t ()` must equal it, so `t` fixes `()`. A supposed fixed-point-free `t` would then contradict its own promise at `()`. ■

The reading is immediate and it is one you should carry into every later chapter. A system whose only outcome is ”accept” cannot be caught by the diagonal, because ”accept” is a fixed point of every transformation. A verifier that can only ever say yes is trivially consistent about itself; the impossibility needs at least two distinguishable verdicts and a way to swap them. This is why the safety theorems are always about systems that can both accept and reject, both assert and deny. The moment a system’s outcome space collapses to one value, the engine has nothing to grip. Chapter 3 uses this in the other direction: to prove a safety property is *achievable* in some regime, it exhibits a collapse of the outcome space, or a value the relevant flip fixes.

Manufacturing fixed-point-free flips

Since every impossibility is a fixed-point-free `t`, it pays to know how to build one, and to know when you cannot. A map `t : Y → Y` is fixed-point-free exactly when it moves every element, which for a finite `Y` is a familiar object: a permutation with no fixed point, a derangement, or more generally any

endomap whose graph avoids the diagonal. On two points there is exactly one derangement, the swap, and it is boolean negation; this is why `Bool` is the smallest interesting outcome type and why `bool_not_fpf` is a two-case check. On three points the three-cycle is fixed-point-free, which is the shape of `c1_optFlip` below. On one point there is no derangement at all, which is `c1_no_fpf_unit`.

The general principle is that a fixed-point-free endomap exists on `Y` precisely when `Y` admits an endomap avoiding the diagonal, and a clean sufficient condition is that `Y` carries a free involution or, more weakly, a fixed-point-free permutation. Two-valued outcome types always do, by the swap. Outcome types that are contractible in the relevant sense, of which `Unit` is the extreme case, never do. This is the exact dividing line between outcome spaces that can host an impossibility and those that cannot, and it is worth stating because Chapter 3 sometimes engineers the outcome space precisely to sit on one side of it. A verdict system whose outcomes admit a fixed-point-free flip is exposed; one whose outcomes collapse to a point, or to a set the intended flip fixes, is safe. The design question "can this be made safe" is often, underneath, the question "can the outcome flip be given a fixed point," and the answer is read off the geometry of `Y`.

There is a converse caution. A fixed-point-free `t` on a large `Y` is easy to write down, but the impossibility it yields is only as strong as the claim that the system's outcomes really range over all of `Y` and that `t` really is the relevant transformation. It is tempting to manufacture an exotic `t` and announce an impossibility; the honest work is always to argue that `t` is forced by the problem, not chosen for convenience. In Cantor `t` is negation because membership is two-valued and exclusion is the only nontrivial move. In Chapter 3 the flips are forced by the meanings of "hallucinate," "inject," and "misclassify," and a large part of each proof is showing that the natural transformation on that outcome space has no fixed point.

1.7 A partial-function preview: toward Rice

The instances so far used total behaviors: `f a` is defined on every input. Much of computability is about *partial* behaviors, where `f a` may be undefined on some inputs because a program loops. The diagonal adapts, and the adaptation is the shape of Rice's theorem, which Chapter 2 proves in full. This section is a preview, enough to see where the engine reappears.

Model a partial outcome as an element of `Option Y`: the value `some y` means "the behavior returned `y`," and `none` means "the behavior did not return." A partial behavior is then a total function `A → Option Y`, and a partial system is `f : A → A → Option Y`. Universality now asks that every partial behavior be named. The transformation `t` acts on `Option Y`, and the impossibility follows whenever `t` is fixed-point-free on `Option Y`.

The subtlety Rice's theorem exploits is that a useful `t` on `Option Y` must disturb the value `none` as well as the values `some y`. A transformation that

fixed `none`, say by leaving nonreturning behaviors alone, would have a fixed point at `none` and the argument would stall. So the flip that drives the partial diagonal is one that moves `none` too: it changes both whether a behavior returns and, when it does, what it returns. Here is such a flip on `Option Bool`, cycling the three concrete outcomes so that none is fixed.

```
def c1_optFlip : Option Bool → Option Bool
  | none => some true
  | some true => some false
  | some false => none

theorem c1_optFlip_fpf :
    ∀ o : Option Bool, c1_optFlip o ≠ o := by
  intro o
  cases o with
  | none => decide
  | some b => cases b <;> decide
```

Proof. The map sends `none ↦ some true`, `some true ↦ some false`, and `some false ↦ none`, a three-cycle with no fixed point, checked by exhausting the three cases. ■

By Corollary 1.10 applied with `Y` replaced by `Option Bool` and `t = c1_optFlip`, no partial system `f : A → A → Option Bool` is universal. The reading is the undecidability of a nontrivial behavioral property. If some machine could name every partial behavior over `A` and a decision procedure could sort behaviors by their `Option Bool` value, then the three-cycle would build a behavior that disagrees with its own classification, which is impossible. Rice's theorem is the general statement that any nontrivial property of the behavior of programs, one that holds of some and fails of others, is undecidable, and its proof is this diagonal with the flip chosen to move across the property's boundary. The details, including why "nontrivial" is exactly the condition that supplies a fixed-point-free flip, are Chapter 2. We flag it here only to show that partiality changes the outcome type and nothing else: the engine is untouched.

Exercise 1.13 asks you to state the partial analogue of Theorem 1.4 carefully and to identify which classical instance it becomes when `t` is a pure definedness-flip, one that only toggles `none` against "returns." That flip is the halting problem; `c1_optFlip` above is a slightly richer flip that also scrambles the returned value, which is closer to the full Rice statement.

The pure definedness-flip is worth picturing on its own, because it is the cleanest computational instance. Collapse the returned value and track only whether a behavior halts, so outcomes are the two-element type "returns" versus "does not," and let `t` swap them. A universal partial system over this two-valued outcome is exactly a decider for halting, and the diagonal behavior `a ↦ swap (halts a on a)` is Turing's `Dgn`: it halts when its self-application does not and fails to halt when its self-application does. The fixed-point-free swap

forbids the universal system, which is the statement that no such decider exists. Rice's theorem generalizes this from "halts" to any nontrivial behavioral property by choosing a `t` that moves across the property's boundary, and the word *nontrivial* is doing precise work: a property that holds of all behaviors, or of none, corresponds to an outcome value that the natural flip fixes, and then there is no fixed-point-free `t` and no impossibility. This is the computability face of the same dividing line drawn in "Manufacturing fixed-point-free flips": trivial properties collapse the outcome space, nontrivial ones keep it flippable. Chapter 2 makes the correspondence exact, including the care needed to handle partiality without the argument accidentally deciding halting for free.

1.8 The engine, named once

We give the schema the name the rest of the notes use.

Definition 1.15 (Reflective verdict). A *reflective verdict* on a domain `Q` is a universal system `v : Q → Q → Bool`: a boolean self-application in which every pattern of verdicts over `Q` is itself named by some element of `Q`.

It helps to read the two arguments of `v p q` in the intended application. Think of `Q` as a space of items a system passes judgment on, say claims, prompts, or states, and `v p q` as "the verdict that item `p` renders on item `q`." The first argument is an item playing the role of a judge; the second is the item being judged. Because both are drawn from `Q`, an item can judge itself, `v p p`, and, what matters, an item can encode a whole judging policy. Universality is the claim that every possible policy of verdicts over `Q`, every function `Q → Bool`, is the policy of some single item. This is a strong closure property, and it is exactly what the modeling in Chapter 3 sets out to establish for particular systems: that a faithful, calibrated, covering model can be driven to realize any verdict policy over the relevant domain, or that an adversary supplying part of a prompt can force a defense to evaluate any policy it likes. When such a closure holds, the system is a reflective verdict, and the next theorem applies with no further work.

Theorem 1.16. *No reflective verdict exists.*

```
theorem no_reflective_verdict {Q : Type _}
    (v : Q → Q → Bool)
    (hv : ∀ g : Q → Bool, ∃ p, v p = g) : False :=
  cantor v hv
```

Proof. Immediate from Example 1.12. ■

Remark 1.17. Theorem 1.16 is one line, and it is, verbatim, the Hallucination Trilemma, the Defense Trilemma, and the truth-boundary coupling theorem of Chapter 3. Nothing about those results adds to the engine; what changes is only the intended meaning of `v p q`. The work in Chapter 3 is entirely in the *reading*, in arguing that faithful-plus-calibrated-plus-covering really does make a model's verdict reflective, and that prompt injection really does make a defense's verdict reflective.

It is worth being blunt about what this means for the rest of the book, because a reader can misjudge where the difficulty lies. The mathematics of the AI-safety impossibilities is finished. It is the theorem you have just read, proved in one line from `cantor`. Every remaining page of Chapter 3 is an argument that a particular real system satisfies the hypothesis `hv`, that its verdicts really are universal over the relevant domain. Those arguments are not mathematical subtleties hiding in the diagonal. They are modeling claims about what a faithful, calibrated, covering model can be prompted to do, or about what an adversary controlling part of a prompt can force a defense to evaluate. If you doubt a Chapter 3 theorem, the place to press is never the diagonal. It is always the claim that `hv` holds.

The diagonal in a learning system, informally

It is worth walking once through how universality could arise for a learned model, since the abstraction can make the hypothesis feel unreachable when in fact it is the natural reading of familiar capabilities. Take `Q` to be a space of prompts, and imagine a model that answers a prompt with a yes-or-no verdict. Read `v p q` as the verdict the model gives to prompt `q` when its behavior has been fixed by prompt `p`, for instance by `p` being a system prompt or an in-context specification that tells the model which policy to follow. In-context learning is exactly the claim that a suitable `p` can steer the model to a wide range of verdict policies over prompts. If the range is *all* policies `Q → Bool`, the model is a reflective verdict, and Theorem 1.16 says no such model can be consistent about the diagonal prompt.

The diagonal prompt writes itself. It is the prompt `q` that asks, in effect, "give the opposite of the verdict you would give when steered to evaluate this very prompt." Universality supplies a steering prompt p_0 that realizes exactly this policy, and then $v\ p_0\ p_0$ must equal its own negation. The model cannot answer consistently, not because it is badly trained, but because the capability that made it universal, the ability to be steered to any policy including this one, is inconsistent with there being a stable answer. This is the honest shape of the Chapter 3 arguments, before the modeling is made careful. Whether the range really is all policies is the whole question, and the answer depends on what "faithful, calibrated, covering" buys you, which is where the real analysis lives.

Two caveats keep this preview honest. First, a real model does not literally range over all functions `Q → Bool`; it ranges over the policies its architecture and training make reachable, which is why Chapter 3 argues reachability carefully rather than assuming it. Second, the diagonal prompt is a construction, not necessarily a natural-language sentence a user would type; part of the modeling is arguing that the constructed policy is genuinely within reach of the steering mechanism. When both caveats are discharged, the impossibility is not a quirk of phrasing. It is the same wall Cantor hit, standing where a learning system's generality meets a transformation of its own verdicts that nothing can satisfy.

1.9 What the diagonal cannot give

It is as important to know the limits of an engine as its reach, and half of this book lives outside this one.

Remark 1.18. The diagonal produces a fixed point, or a contradiction, from self-reference. It is silent on everything quantitative: not how many liars there are, not how hard `a₀` is to find, not what it costs an adversary to reach one. And it requires genuine self-application, the hypothesis `∀ g, ∃ a, f a = g`. When a system cannot name its own behaviors but instead lives on a *connected* domain with *continuous* maps, a different engine takes over, the intermediate value theorem, and it delivers the same boundary object with metric content the diagonal never had. That is Chapter 4, and its quantitative refinements, the geometry of attack basins, are Chapter 5.

It is worth being precise about the three questions the diagonal cannot answer, because Chapters 4 and 5 exist to answer them and it helps to have them posed here. The first is *how many*. The theorem produces one index `a₀`, or as many as there are names for the diagonal behavior, but it gives no measure of how large the set of liars is, whether it is a single point or a fat region. The second is *how hard to find*. The proof is nonconstructive about `a₀` in the sense that matters operationally: it tells you `a₀` exists, but nothing about the cost of locating it, which is the whole question when the adversary is resource-bounded. The third is *how stable*. The diagonal's `a₀` is a discrete object with no neighborhood; there is no sense in which points near `a₀` are nearly-liars, because "near" is not defined. A connected domain carries a topology, and on it the boundary object acquires a size, a basin, and a cost, which is why the second half of the book changes engines. The diagonal is the right tool for "impossible"; the intermediate value theorem is the right tool for "impossible, and here is how far the impossibility reaches."

There is also a hypothesis the diagonal cannot do without, and naming it guards against overclaiming. The engine needs genuine universality, the ability to name *every* behavior. Real systems frequently fail this, and when they do the theorem is silent, which is correct: a system that cannot describe itself in full is not subject to the diagonal, and may well be consistent. The interesting cases, the ones the book is about, are the systems powerful enough that universality is plausible. For those, the diagonal is decisive. For the rest, one must argue universality first, and that argument, not the diagonal, is where the real content of an impossibility claim lives.

1.10 A roadmap through the book

It is worth saying, once and plainly, how the rest of the notes reuse this chapter, so that the reader can see the single engine turning under each later result. Every chapter is one of two things: an instantiation of the diagonal with a chosen `Y` and `t`, or a change of engine to the intermediate value theorem when the diagonal's hypothesis is not available.

Chapter 2 revisits the classical instances in their own settings and does the work this chapter deferred. It builds the arithmetization behind Gödel and Tarski, the computability behind Turing and Rice, and it presents each as Yanofsky's commuting square filled in. Nothing there strengthens the engine; the labor is in verifying universality for arithmetic and for the computable maps, which is exactly the representability and universal-machine content the history section previewed.

Chapter 3 is the payoff and the reason the book exists. It reads `Q`, `v`, and the boolean flip three ways, and each reading is a named impossibility. The Hallucination Trilemma says a model cannot be simultaneously faithful, calibrated, and covering, because those three properties together make its verdict reflective and Theorem 1.16 forbids that. The Defense Trilemma says a prompt-injection defense cannot be simultaneously sound, complete, and closed under composition, for the same reason applied to a defense's verdict. The truth-boundary coupling theorem says a truth probe cannot cleanly separate the states it is meant to classify. Each proof is one line once the reflective-verdict hypothesis is granted, and each chapter section is an argument that the hypothesis is granted.

Chapter 4 changes engines. When a system does not name its own behaviors but lives on a connected domain with continuous maps, the diagonal has no grip, and the intermediate value theorem takes over. It produces the same kind of boundary object, a place where a continuous verdict must cross a threshold, but now the object has metric content: a location, a neighborhood, a size. Chapters 5 and 6 make that quantitative, studying the geometry of attack basins and the approximate bridges that connect the discrete diagonal picture to the continuous one. Chapter 7 returns to concrete AI-safety settings with both engines in hand, and Chapter 8 develops J-space, where a system carries both a self-application and a topology, so that both engines apply at once.

The through-line is that impossibility in these notes is never mysterious. It is always a fixed-point-free transformation meeting a system rich enough to describe itself, or a threshold-crossing forced by continuity on a connected domain. If you hold those two pictures, the diagonal and the boundary, you hold the book.

1.11 Historical and bibliographic notes

The unification is Lawvere's *Diagonal arguments and cartesian closed categories* (1969), reprinted with an author commentary in *Reprints in Theory and Applications of Categories* (2006), which is the most convenient source today. Lawvere's paper is terse and categorical; readers meeting it for the first time usually do better to start with a modern exposition and return to the original for its economy.

Yanofsky's *A universal approach to self-referential paradoxes, incompleteness and fixed points* (Bulletin of Symbolic Logic, 2003) is that modern exposition.

It works out Cantor, Russell, Gödel, Tarski, Turing, and the recursion theorem as instances of one diagram, using only sets and functions, and it is the direct template for Chapter 2. If you read one background paper, read this one.

For the classical instances in their original settings: Cantor's diagonal appears in his 1891 note on the uncountability of the reals and the power-set theorem; Russell's paradox is in his 1902 correspondence with Frege, reproduced in van Heijenoort's *From Frege to Gödel* (1967), which remains the standard sourcebook for this whole lineage. Gödel's incompleteness theorems are in his 1931 paper, and the diagonal lemma abstracted from it is treated cleanly in any modern logic text; Boolos, Burgess, and Jeffrey's *Computability and Logic* is a good one. Tarski's undefinability of truth is in his 1936 monograph on the concept of truth in formalized languages. Turing's halting argument is in his 1936 paper on computable numbers, and Kleene's recursion theorem, the constructive companion, is in his *Introduction to Metamathematics* (1952).

The reading of these theorems as constraints on learning systems is recent, and the specific trilemmata of Chapter 3 are stated and machine-checked in the companion Lean library that these notes track. The categorical fixed-point tradition after Lawvere, including the fixed-point results of Lambek and the modal fixed-point theorems descended from Löb, is surveyed in several places; we do not use it beyond the section form of Theorem 1.8, but a reader who wants the full categorical picture should look there next.

1.12 Exercises

Exercises marked (Lean) ask for a machine-checked term; the others are on paper. The starred exercises are harder or open-ended. Several build directly on the theorems above, whose names are `lawvere`, `no_universal`, `bool_not_fpf`, `cantor`, `liar_query`, `no_reflective_verdict`, and the `c1_`-prefixed helpers proved in this chapter.

Exercise 1.1. Verify Example 1.3 in detail. For `|A| = n` and `|Y| = 2`, exhibit a function `A → Y` not of the form `f a`, using a diagonal that flips `f a a`. Conclude that your construction is exactly `liar_query` specialized to a finite `A`, and explain in one sentence why the finiteness of `A` played no role in the diagonal step even though it was essential to the counting step.

Exercise 1.2 (Lean). Prove the section form of Theorem 1.8 yourself, from scratch, without reading `c1_lawvere_section`: given `f : A → A → Y`, `s : (A → Y) → A`, and `hs : ∀ g, f (s g) = g`, show every `t : Y → Y` has a fixed point. Then compare your term to `c1_lawvere_section` and to `lawvere`, and say in one sentence which hypothesis each uses and why the section form needs no `match`.

Exercise 1.3. Show the converse of Corollary 1.10 fails: exhibit a `Y` and a fixed-point-free `t` for which some non-universal `f : A → A → Y` exists. This is easy; the point is that the theorem is about universality, not about `t` alone. State precisely what the corollary does and does not claim about `f`.

Exercise 1.4 (Lean). A map `t : Y → Y` is *fixed-point-free* iff it has no fixed point. Using `c1_bool_has_fpf` and `c1_no_fpf_unit`, explain what the contrast says about systems whose only outcome is "accept." Then give a fixed-point-free endomap of a three-element type and prove it fixed-point-free by `decide`, modeling your proof on `c1_optFlip_fpf`.

Exercise 1.5. Restate and prove Cantor's theorem in its usual form, that there is no surjection `A → Set A`, by taking `Y = Bool` and identifying `Set A` with `A → Bool`. Point to the exact line of your argument that corresponds to `bool_not_fpf`, and to the exact subset that corresponds to the diagonal behavior `a ↦ not (f a a)`.

Exercise 1.6. (Gödel, informally.) Suppose a theory `T` proves or refutes every sentence, and can represent its own provability as a map on sentences. Show that a self-referential sentence asserting its own unprovability is a fixed-point-free instance, and identify `Y` and `t`. Which hypothesis of Corollary 1.10 corresponds to `T` being able to talk about its own proofs, and which corresponds to `T` deciding every sentence?

Exercise 1.7. (Tarski.) Repeat Exercise 1.6 for truth in place of provability. State the liar sentence as a fixed point of negation, identify `Y` and `t`, and explain in two sentences why Tarski's conclusion is about definability while Gödel's is about provability, even though the diagonal step is identical.

Exercise 1.8. (Turing.) Cast the halting argument as an instance of Corollary 1.10. Take the outcomes to record "halts" versus "loops," describe the diagonal program in words, and name the fixed-point-free `t`. Then explain why the assumed decider `H`, not the diagonal program, is the object the argument actually refutes.

Exercise 1.9. In `liar_query`, the witness a_0 depends on the choice of preimage. Show that if `f` has two distinct indices naming the diagonal behavior, both are liars, and that nothing in the argument selects a canonical one. Relate this to the non-uniqueness of the boundary question in Chapter 4.

Exercise 1.10 (Lean). Prove a "one-sided" refinement: if `t : Y → Y` and `f : A → A → Y` is universal, then there is an index a_0 with `f` a_0 a_0 `= t (f` a_0 a_0`)`, by calling `c1_lawvere_diagonal`. Then specialize to `Y = Bool`, `t = not`, and check that you recover `liar_query` up to the shape of the returned equation.

Exercise 1.11. Explain, in one paragraph and without symbols, why counting (Example 1.3) is the wrong intuition for the infinite case, and what the diagonal provides that counting cannot. Your paragraph should mention that the diagonal names a specific unnamed behavior, whereas counting only asserts one exists.

Exercise 1.12. Give an example of a `Y` and a `t : Y → Y` with *exactly one* fixed point, and describe what the diagonal produces for a universal `f : A → A → Y` in that case. Contrast this with the fixed-point-free case, and connect it to the remark that Kleene's recursion theorem is the same construction read forwards.

Exercise 1.13. (Harder.) Formulate a *partial* version precisely, building on the preview above: with outcomes in `Option Y`, define universality as sur-

jectivity onto partial behaviors `A → Option Y`, state the analogue of Theorem 1.4, and identify which classical instance it becomes when `t` only toggles `none` against "returns." Explain where the requirement that `t` move `none` enters, and why a flip fixing `none` would make the argument fail.

Exercise 1.14 (Lean). Model a definedness-flip `t : Option Bool → Option Bool` that swaps `none` with `some false` and fixes `some true`, and decide whether it is fixed-point-free. If it is not, exhibit its fixed point; if it is, prove it so by `decide`. Use the answer to explain why Rice's theorem insists the classified property be *nontrivial*.

Exercise 1.15. (Section versus surjection.) Prove on paper that every section gives point-surjectivity, which is `c1_section_universal`, and explain why the converse needs a choice principle. Identify the exact step at which choosing one name per behavior is required, and relate it to the constructive content described in Remark 1.6.

Exercise 1.16. (Reading `Y`.) For each of the following outcome types, decide whether it admits a fixed-point-free endomap, and hence whether it can host an impossibility: `Bool`; `Unit`; the two-point type of verdicts "accept/reject"; the type `Option Bool`; a one-point "always accept" verdict space. State the general principle your answers illustrate.

Exercise 1.17. (Starred.) The engine needs `Y` to carry a fixed-point-free endomap. Chapter 4 replaces this with a continuous map on a connected space whose only fixed point is a threshold. Speculate on what a system would need to satisfy *both* hypotheses at once, a universal self-application and a connected topology, and what its boundary object would look like. Chapter 8 returns to this for J-space.

Exercise 1.18. (Starred.) Reflect on Remark 1.17's claim that the mathematics of the AI-safety impossibilities is finished and only the modeling remains. Pick one informal safety desideratum you care about, phrase it as a universality claim `hv` for some domain `Q`, and identify the single sentence someone would have to refute to escape the resulting impossibility. This exercise has no unique answer; its purpose is to locate where the burden of proof really sits.

1.13 References

Cantor, G. (1891). Über eine elementare Frage der Mannigfaltigkeitslehre. *Jahresbericht der Deutschen Mathematiker-Vereinigung* 1, 75–78.

Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38, 173–198.

Kleene, S. C. (1952). *Introduction to Metamathematics*. North-Holland, Amsterdam.

Lawvere, F. W. (1969). Diagonal arguments and cartesian closed categories. In *Category Theory, Homology Theory and their Applications II*, Lecture Notes in

Mathematics 92, 134–145. Springer, Berlin. Reprinted in *Reprints in Theory and Applications of Categories* 15 (2006), 1–13.

Tarski, A. (1936). Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica* 1, 261–405.

Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* (2) 42, 230–265.

van Heijenoort, J. (ed.) (1967). *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*. Harvard University Press, Cambridge, MA.

Yanofsky, N. S. (2003). A universal approach to self-referential paradoxes, incompleteness and fixed points. *Bulletin of Symbolic Logic* 9(3), 362–386.

2 The Classical Limits

Abstract. Chapter 1 proved one theorem and one corollary: a universal system has a fixed point for every transformation of its outcomes, so a transformation with no fixed point rules out universality. This chapter applies the corollary to Cantor's theorem, Russell's paradox, Gödel's two incompleteness theorems, Tarski's undefinability of truth, Turing's halting problem, and Rice's theorem. Each classical proof factors into a self-referential heart and a local wrapper that shows the system is universal in the required sense. The chapter makes the factoring explicit and machine-checks it.

Keywords: Incompleteness, Undefinability, Halting problem, Rice's theorem, Representability

Chapter 1 built one theorem and one corollary. The theorem, `lawvere`, says that a universal system has a fixed point for every transformation of its outcomes. The corollary, `no_universal`, reads that backwards: if some transformation `t : Y → Y` has no fixed point, then no system `f : A → A → Y` can be universal. Everything in the present chapter is an application of the corollary. We change what `A` names, we change what `Y` measures, we pick a fixed-point-free `t`, and out falls a theorem that someone proved between 1874 and 1953 by what looked at the time like a different argument each time.

The claim that these are one argument is not a slogan. It is a factoring. Each classical proof has a self-referential heart and a local wrapper. The heart is always the diagonal behavior `a ↦ t (f a a)` and the observation that a name for it collides with itself. The wrapper is the work of arguing that the system in question really is universal in the sense `no_universal` requires: that a set really does surject onto its power set, that a theory really can represent its own provability, that a programming language really is closed under the constructions the proof performs. Lawvere isolated the heart. This chapter walks through the wrappers, one theorem at a time, and shows that once the wrapper is in place the heart is a single reusable line of Lean.

There is a reason to care about the unification beyond elegance. When results are proved by what look like unrelated tricks, each new domain seems to need its own genius, and the safety of a new kind of system looks like an open empirical question. When they are proved by one move, a new domain needs only the checklist of the recipe, and the question becomes sharply mathematical: does this system have a section into its own function space, and does its outcome object carry a fixed-point-free map. The unification

converts a menagerie of clever arguments into a decision procedure for spotting impossibilities, and that is what lets the later chapters treat hallucination and prompt injection as instances to be verified rather than as new mysteries to be cracked.

The order is roughly historical, and it is also an order of increasing subtlety in the wrapper. Cantor's power-set surjection is the cleanest instance and we prove it in full. Russell's paradox drops even the power set and works directly with membership. Gödel's two incompleteness theorems ask the most of the wrapper, because representing provability inside arithmetic is real work, and we are honest about which parts live in prose and which parts compile. Tarski, Turing, and Rice then follow quickly, because by that point the wrapper is familiar and only the reading changes.

Two conventions carry over from Chapter 1. A system is a curried map `f : A → A → Y`, and universality is point-surjectivity, `∀ g : A → Y, ∃ a, f a = g`. Live `lean` code blocks are core Lean 4 and are elaborated when the book is built. New results proved here are prefixed `c2_` so their names are globally unique. Where a theorem needs the machinery of Mathlib or a classical axiom, we state it in prose and name the verified statement in the companion libraries rather than write fragile code.

2.1 The recipe

Before the instances, it helps to name the procedure they share, because after the first two everything is variation.

The recipe. To obtain a limitative theorem from `no_universal`:

1. Choose the *index type* `A`: what the system's names range over. Sets, sentences, programs, subsets, and prompts have all played this role.

2. Choose the *outcome type* `Y`: what a behavior reports. For most classical results `Y = Bool`, a yes or no.

3. Exhibit a *fixed-point-free* `t : Y → Y`, a map with `t y ≠ y` for all `y`. For `Y = Bool` this is boolean negation, and Chapter 1's `bool_not_fpf` is the proof that it has no fixed point.

4. Argue the *wrapper*: that the system under study, if it existed with the properties claimed, would be universal in the sense `∀ g : A → Y, ∃ a , f a = g`.

5. Conclude by `no_universal` that no such system exists, or by `liar_query` that a specific self-undermining index exists.

Steps 1 through 3 are cheap. Step 4 is where the mathematics lives, and it is the only step that differs across the classical theorems. Step 5 is one line.

For `Y = Bool` the whole recipe collapses to a single reusable statement. Since negation is fixed-point free, no boolean system is universal, full stop.

```
theorem c2_no_universal_bool {A : Type _}
    (f : A → A → Bool) :
    ¬ (∀ g : A → Bool, ∃ a, f a = g) :=
  no_universal (fun b => !b) bool_not_fpf f
```

Proof. Apply `no_universal` with `t` the boolean flip `(!·)` and `ht := bool_not_fpf`. ■

Read `c2_no_universal_bool` slowly, because it is the entire chapter in one line. It says: for *any* way `f` of assigning to each index `a` a yes-or-no behavior `f a` over indices, there is some yes-or-no pattern `g : A → Bool` that no index realizes. The pattern is always the same one, the diagonal flip `fun a => !(f a a)`, and the theorem does not care whether `A` is finite or a proper class, whether `f` is computable or a black box, whether the yes and no mean membership or provability or halting. Cantor, Russell, Gödel, Tarski, Turing, and the boolean core of Rice are all this statement wearing different clothes. The rest of the chapter is a fashion show.

Remark 2.1 (Why `Bool` and not `Prop`). We insist on `Y = Bool` in the code, not `Y = Prop`, and the choice is deliberate. On `Bool` the fixed-point-free witness `bool_not_fpf` is settled by `decide`, checking the two cases, and it needs no axioms at all. On `Prop` the corresponding statement, that `¬ P ≠ P` for every proposition `P`, is a form of the law of noncontradiction that is fine constructively for the negation, but reading a `Prop`-valued system as universal forces classical principles the moment one wants to case-split on an arbitrary `P`. We will meet exactly this fork in Russell's paradox below. Keeping the engine on `Bool` means every compiled proof in this chapter is axiom-free, which we can check with `#print axioms`, and it is a small model of a discipline the AI-safety chapters take seriously: a two-valued verdict is a decision, and a decision is where impossibility bites.

A finite rehearsal

It is worth running the recipe once on a finite example, where you can see every piece, before turning it loose on the infinite theorems where the same pieces do the same work invisibly. Chapter 1's Example 1.3 counted: a set with `n` elements has `n` behaviors but 2^n boolean patterns, so universality fails by arithmetic. The diagonal gives the same conclusion without counting, and the difference matters, because counting is exactly what stops working when the sets are infinite.

Take `A` with two elements, call them `0` and `1`, and let `f : A → A → Bool` be any system. There are four behaviors `A → Bool` in total and only two names, so some behavior is unnamed, and the recipe tells you which one. Form the diagonal flip `d = fun a => !(f a a)`. Concretely `d 0 = !(f 0 0)` and `d 1 = !(f 1 1)`. Now check that `d` is not `f 0`: they differ at `0`, because `d 0 = !(f 0 0)` disagrees with `f 0 0`. And `d` is not `f 1`: they differ at `1`, because `d 1 = !(f 1 1)` disagrees with `f 1 1`. So `d` is unnamed, and it was constructed by

a single negation from the diagonal `a ↦ f a a`, with no reference to the size of `A`.

That last clause is the whole point. The counting proof knew `4 > 2`. The diagonal proof never counted; it built one specific unnamed behavior by disagreeing with each `f a` at the one input `a` where disagreement is cheap, its own name. Replace two by any cardinal, finite or infinite, and the construction is unchanged and the conclusion is unchanged, while the counting argument has nothing to say once 2^n and `n` are both infinite. This is why every theorem in the chapter is a diagonal and not a count. The systems they concern, sets and their subsets, theories and their sentences, languages and their programs, are all infinite, and the only argument that survives is the one that ignores size.

2.2 Cantor's theorem

Cantor proved in 1891 that a set has strictly more subsets than elements, in the precise sense that no function from a set onto its power set can be a surjection. This is the theorem that started the subject. It is also the cleanest possible instance of the recipe, because the power set *is* the function space `A → Bool` and a surjection onto it *is* universality, with nothing to translate.

Identify a subset `S ⊆ A` with its indicator function `A → Bool`, the map sending `a` to `true` when `a ∈ S` and to `false` otherwise. This identification is a bijection between the power set of `A` and the type `A → Bool`. Under it, a function `f : A → (A → Bool)` is the same data as a family of subsets indexed by `A`, and `f` being surjective onto `A → Bool` is the same as saying every subset occurs in the family.

Definition 2.2 (Power-set surjection). A *power-set surjection* on `A` is a function `f : A → (A → Bool)` such that every indicator `g : A → Bool` equals `f a` for some `a`, that is, `∀ g : A → Bool, ∃ a, f a = g`.

Since `A → (A → Bool)` and `A → A → Bool` are the same type by currying, a power-set surjection is exactly a universal boolean system. There is no gap to close. Cantor's theorem is therefore an immediate corollary of the recipe.

Theorem 2.3 (Cantor, 1891). *No power-set surjection exists.*

```
theorem c2_cantor_powerset {A : Type _} (f : A → (A → Bool))
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  cantor f hf
```

Proof. A power-set surjection is a universal boolean system, so Chapter 1's `cantor` applies directly. Unfolding that proof: universality names the diagonal subset `d = fun a => !(f a a)`, the set of `a` that the `a`-th subset excludes. Let a_0 name it, so `f a₀ = d`. Ask whether `a₀ ∈ f a₀`. Evaluating the naming equation at a_0 gives `f a₀ a₀ = !(f a₀ a₀)`, which no boolean satisfies. ∎

The witness `d` is Cantor's *diagonal set*: it disagrees with the `a`-th listed subset about the element `a`. It cannot appear anywhere in the listing, because to

appear at position `a₀` it would have to agree with itself about `a₀` and disagree with itself about `a₀` at the same time. This is the same `a₀` that Chapter 1's `liar_query` produces, read now as membership rather than as a truth value.

Identifying `Y` and `t`. Here `Y = Bool`, the two answers to "is `a` in this subset," and `t` is negation, "put `a` in exactly when the diagonal leaves it out." The fixed-point-free character of negation is the whole obstruction. If some outcome were fixed by `t`, the diagonal set could sit quietly at that outcome and the surjection would survive.

Discussion of the hypothesis. The single hypothesis is surjectivity, and it is worth seeing that nothing weaker will do and nothing stronger is needed. Injective maps `A → (A → Bool)` exist in abundance; the theorem forbids only the covering property. The map need not be computable, definable, or continuous. It is a raw function, and the diagonal set is built from it by one negation, so the argument survives any amount of pathology in `f`. This robustness is exactly why the diagonal is the wrong tool for quantitative questions, a point Chapter 1 flagged and Chapters 5 and 6 develop: the argument works so generally precisely because it ignores metric structure.

The classical set form. In the language of pure set theory the theorem reads: there is no surjection `A → Set A`. The companion library proves this directly, as `CCH.cantor_no_surjection_set` in `CCHProofs/CCH_03_Cantor.lean`, by the Russell diagonal `{x | x ∉ f x}` rather than through `Bool`, and it proves the boolean-indicator form as `CCH.cantor_no_surjection`. The two are the same theorem; the `Set`-level proof needs the classical fact that membership in a set is decidable enough to case-split, while the `Bool` form we compile here needs nothing. We return to why in the discussion of Russell.

The diagonal set is not a paradox. Beginners often read the diagonal set `d` as paradoxical, on the model of Russell's set, and it is worth being clear that it is not. There is nothing self-contradictory about the set of `a` such that `a ∉ f a`. It is a perfectly ordinary subset of `A`; you can list its members whenever you can compute `f`. The only thing the theorem says about it is that it is not one of the sets `f a`, that it fails to be listed by the particular family `f`. Cantor's theorem is a statement about the family, not about the diagonal set, and it is negative in a mild way: some subset was left out. The paradox appears only in Russell, where the family is forced by comprehension to include every subset, so that the diagonal set must be both listed and, by construction, unlisted. The difference between a theorem and a paradox is exactly the difference between a family that may omit a subset and a family that may not.

Fixed points and retractions. Lawvere stated his theorem in the contrapositive form that Cantor's theorem exemplifies most cleanly, and it is illuminating to read Cantor that way. Say a type `Y` has the *fixed-point property* if every `t : Y → Y` has a fixed point. Lawvere's theorem says that if `A` admits a point-surjection onto `A → Y`, then `Y` has the fixed-point property. Cantor's theorem is the contrapositive with `Y = Bool`: since `Bool` fails the fixed-point property, witnessed by `bool_not_fpf`, no `A` point-surjects onto `A → Bool`. Read forward, the same theorem is a tool for *building* fixed points, and that is how the lambda calculus uses it, a point Exercise 2.16 develops. Read backward, it is

an impossibility. The engine does not know which way you are reading it; the sign of the conclusion is entirely in whether your `Y` has a fixed-point-free map.

Counting was never the point. The finite rehearsal above already made this concrete, and Cantor's own history confirms it. The 1874 proof that the reals are uncountable did not diagonalize; it used nested intervals and the completeness of the reals, an analytic argument. The 1891 diagonal was the second proof, and its power was that it generalized instantly from the reals to any set and its power set, because it never used a special property of the reals, only the ability to flip a value. The book's later chapters run the two styles in the other order: the diagonal comes first, in Chapters 1 and 2, and the analytic argument, the intermediate value theorem, comes second, in Chapters 4 through 6, recovering with metric content what the diagonal gets for free but without measure.

The tower of infinities. Cantor's theorem does not fire once; it iterates. The power set of `A` is strictly larger than `A`, and the power set of that is larger still, and the process never closes, so there is no largest set and no set of all sets, on pain of the paradox Russell found. This is the cumulative hierarchy that `ZFC` builds level by level, and its infinite cardinalities, the beth numbers, are each a diagonal step above the last. The theorem also leaves a famous question open that it cannot itself answer: whether any cardinality sits strictly between a set and its power set. That is the continuum hypothesis, and Cohen's forcing showed it is independent of `ZFC`, which is a second-incompleteness-flavored fact one level up, the axioms cannot decide a question their own diagonal raised. The diagonal opens the tower; it does not tell you how the rungs are spaced. The recurring distinction between what the diagonal establishes and what it leaves open is already present in the oldest instance.

Cantor and Schröder–Bernstein. One more contrast sharpens what the theorem does. The Schröder–Bernstein theorem says that if there are injections both ways between two sets, they have the same cardinality, and its proof is a back-and-forth that builds a bijection. Cantor's theorem builds nothing. It exhibits an obstruction, the unnamed diagonal subset, and stops. The two together pin down cardinal arithmetic: Schröder–Bernstein makes the order on cardinals antisymmetric, Cantor makes it strict at every power-set step. It is the second kind of theorem, the obstruction rather than the construction, that this book is about, and the diagonal is the universal source of obstructions. When a safety result says "no system can do this," it is Cantor-shaped, and its proof exhibits the behavior no system names rather than construct the system that fails.

Remark 2.4 (Cantor and reflective verdicts). Chapter 1 packaged the boolean diagonal as `no_reflective_verdict`, the statement that no verdict system can name every pattern of its own verdicts. Cantor's theorem is that statement's oldest instance: the "verdicts" are membership decisions, and the "patterns" are subsets. When Chapter 3 argues that a hallucination-free model would be a reflective verdict on its own outputs, and Chapter 7 argues the same for a prompt-injection defense, they are asking those systems to be

power-set surjections onto their own behavior. Cantor already answered. The engineering intuition worth carrying forward is that a model's set of possible behaviors is its power set, and asking the model to certify every behavior of its own kind is asking for the surjection Cantor forbids, no matter how large the model.

2.3 Russell's paradox

Russell wrote to Frege in 1902 with a two-line argument that unrestricted comprehension is inconsistent. Comprehension is the principle that any predicate carves out a set: for every property `P` there is a set `{x | P x}` whose members are exactly the `x` with `P x`. Russell applied it to the property "is not a member of itself." Let `R = {x | x ∉ x}`. Then `R ∈ R` if and only if `R ∉ R`, and the theory is dead.

The relation to Cantor is close and instructive. Cantor's theorem needs a codomain, the power set, and a claimed surjection onto it. Russell's paradox drops the codomain entirely. It works with a single binary relation, membership, and asks it to be universal in a way that turns out to be the same diagonal collision. Historically Russell found his paradox by pushing on Cantor's proof: if the universe of all sets existed, the identity would surject it onto its own power set, which Cantor forbids, and unwinding that forbidden surjection produces `R`.

To fit the recipe, model an unrestricted membership relation as a boolean system. Let `A` be a would-be universe of sets, and let `mem : A → A → Bool` report membership, so `mem a b` is `true` when `b` is a member of `a`. Unrestricted comprehension says every property of elements is the membership condition of some set: for every `g : A → Bool` there is a set `a` with `mem a = g`. That is point-surjectivity of `mem`.

Definition 2.5 (Naive membership). A *naive membership relation* on `A` is a map `mem : A → A → Bool` satisfying comprehension, `∀ g : A → Bool, ∃ a, mem a = g`: every boolean condition on `A` is realized as the membership condition of some element.

Theorem 2.6 (Russell, 1902). *If `mem` is a naive membership relation, then there is a set `r` that is a member of itself exactly when it is not, that is, `mem r r = !(mem r r)`. No such relation exists.*

```
theorem c2_russell {A : Type _} (mem : A → A → Bool)
    (hmem : ∀ g : A → Bool, ∃ a, mem a = g) :
    ∃ r, mem r r = !(mem r r) :=
  liar_query mem hmem
```

Proof. This is Chapter 1's `liar_query` verbatim, with `f` renamed `mem`. Comprehension applied to the property `fun a => !(mem a a)`, "is not a member of itself," names a set `r` with `mem r = fun a => !(mem a a)`. Evaluate at `r`:

`mem r r = !(mem r r)`. The Russell set is the diagonal behavior; the theorem is that it names itself into contradiction. To turn the existence of `r` into an outright `False`, feed it to `bool_not_fpf`, or use `cantor` on `mem` directly. ■

Identifying `Y` and `t`. Again `Y = Bool` and `t` is negation. The Russell set `R = {x | x ∉ x}` is nothing but the diagonal behavior `a ↦ t (mem a a)`, the same object that in Cantor was the diagonal subset and that in Chapter 1 was the liar. Three names, one construction.

Discussion of the hypothesis. The hypothesis is comprehension, and the paradox is what forced set theory to give it up. The modern repairs restrict which predicates form sets. Zermelo's separation only lets a predicate carve a subset out of a set you already have, so `{x ∈ A | x ∉ x}` is legal but is merely a subset of `A`, not a universe, and the diagonal argument then reproves Cantor rather than exploding. The type-theoretic repair, which is the one Lean itself uses, stratifies by universe level so that `mem a a` is not even well-formed: a set cannot be applied to itself because the levels do not line up. In Lean the naive relation `mem : A → A → Bool` is perfectly well-typed, which is why `c2_russell` compiles, but its hypothesis `hmem` is never satisfiable for a nontrivial `A`, and that unsatisfiability is the content. The theorem is a proof that the hypothesis is empty.

Remark 2.7 (`Bool` versus `Prop`, again). The propositional Russell paradox, `R ∈ R ↔ R ∉ R`, lives naturally in `Prop`, where the flip is logical negation. There the fixed-point-free step is `¬ (P ↔ ¬ P)`, which holds without any axioms. We nonetheless compile the `Bool` version, because the `Bool` flip composes with the rest of the diagonal engine using only `congrFun` and `decide`, whereas threading `Prop`-valued behaviors through `lawvere` invites classical case-splits later. This is the same fork flagged in Remark 2.1. The lesson for the safety chapters: whenever a "safe or unsafe" judgment is genuinely two-valued, the impossibility is constructive and needs no appeal to excluded middle, and that matters when the judgment is supposed to be something a machine actually computes.

The paradox that reshaped a subject. Russell's two lines did more damage than any theorem in this chapter, because they arrived while the foundations were being poured. Frege's "Grundgesetze der Arithmetik" derived arithmetic from logic using Basic Law V, which is comprehension in the guise of an axiom about the extensions of concepts, and Russell's paradox is a direct refutation of Basic Law V. Frege received the letter as the second volume was in press and added an appendix that begins by admitting the foundation of his life's work had been shaken. The paradox is not a technicality at the margin of set theory; it is the reason set theory has axioms at all rather than a single comprehension schema.

The repairs, and what each gives up. Three responses survived. Zermelo's separation, kept in ZFC, allows a predicate to carve a subset only out of a set already given, so `{x ∈ A | x ∉ x}` exists but is merely a subset of `A`; running the diagonal on it reproves Cantor, that `A` does not contain all its subsets, rather than exploding. The axiom of foundation then bans `x ∈ x` outright, so the Russell condition `x ∉ x` holds of everything and defines nothing

new. Russell's own repair, the ramified theory of types, stratifies objects into levels and declares `x ∈ x` ill-formed, which is the ancestor of the universe hierarchy Lean enforces on `Type`. Quine's New Foundations keeps a single universe but restricts comprehension to *stratified* formulas, those that could be typed, which blocks the Russell predicate while allowing a universal set to exist. Each repair pays the same currency: it gives up the unrestricted freedom to turn a predicate into a set, and it pays exactly enough to make the diagonal behavior `fun a => !(mem a a)` fail to be a set. The paradox is a lower bound on what any consistent set theory must forbid.

A family of paradoxes. Russell's is one of a cluster that all run the same diagonal against a would-be universal object. Cantor's paradox is the diagonal against the set of all sets: if it existed, the identity would surject it onto its power set, which Theorem 2.3 forbids. The Burali-Forti paradox is the diagonal against the set of all ordinals, which would be an ordinal greater than every ordinal including itself. Each is `c2_russell` with a different reading of `mem`, and each was, historically, a separate shock. Seeing them as one instance is the dividend of the Lawvere factoring, and it is the same dividend the safety chapters collect when hallucination and injection turn out to be one theorem.

2.4 Gödel's first incompleteness theorem

Gödel's 1931 theorem is where the wrapper earns its keep. Cantor and Russell hand you universality almost for free, because a power set or a comprehension schema is a surjection by fiat. Arithmetic does not. To run the diagonal against a formal theory of arithmetic, one must first show that the theory can talk about its own syntax, and that is a substantial piece of number theory. Once it is done, the incompleteness theorem is again the diagonal, applied to provability instead of to membership.

We separate the argument into the part that is genuinely the diagonal, which we compile, and the arithmetization, which we describe and cite. This is the honest division of labor. The self-reference is finite and mechanical and belongs in the kernel. The arithmetization is a long verification that Lean's Mathlib and the model-theory literature carry out, and reproducing it in a live block would be fragile and beside the point.

Representability as point-surjectivity

Fix a formal theory `T` in the language of arithmetic, consistent and strong enough to represent primitive recursive functions; the standard choice is Robinson's `Q` or Peano arithmetic `PA`. Gödel numbering assigns to each formula a natural number, its code, written `⌜φ⌝`. Let `A` be the set of codes of formulas with one free variable. Each such formula `φ(x)` defines, by substitution, a map from numbers to sentences and hence, once we fix an interpretation of truth or provability, a boolean-valued behavior on `A`.

Definition 2.8 (Representability). A predicate `g` on codes is *representable* in `T` when there is a formula `φ` such that, for every code `n`, `T` proves `φ(n)` when

`g n` holds and `T` proves `¬ φ(n)` when `g n` fails. The central arithmetical fact, due to Gödel, is that every *recursive* predicate on codes is representable in `Q`.

Representability is the wrapper's version of surjectivity. Read `f a b` as "the formula coded by `a`, applied to the code `b`, is provable in `T`." Then a predicate `g : A → Bool` being representable says there is a formula, some code `a`, whose provability behavior `f a` matches `g`. Over the recursive predicates, representability is exactly the point-surjectivity hypothesis `∀ g, ∃ a, f a = g` of `lawvere`, restricted to the recursive `g`. That restriction is the one honest difference from Cantor: the surjection is onto the representable behaviors, not onto all of `A → Bool`. It is enough, because the diagonal behavior we build is itself recursive.

Arithmetizing syntax

The wrapper's real labor is showing that arithmetic can talk about arithmetic, and it is worth sketching why this is possible at all, because the possibility is not obvious and the safety analogues turn on the same trick. The language of arithmetic speaks of numbers, not of formulas or proofs. Gödel's device is to encode each syntactic object as a number, so that a statement about formulas becomes, after translation, a statement about numbers, which arithmetic can then make.

Assign to each symbol a number, to each formula the number obtained by prime-factorization coding of its symbol sequence, `2^(s₀) · 3^(s₁) · 5^(s₂) · …`, and to each proof, a sequence of formulas, a code of its sequence of codes. Under this coding, "is a well-formed formula," "is an axiom," "follows from earlier lines by a rule," and finally "is a proof of the formula with code `n`" all become arithmetical predicates on the codes. The one delicate step is coding finite sequences of unbounded length inside a language whose variables range over single numbers, and Gödel solved it with his beta function, which uses the Chinese remainder theorem to pack an arbitrary finite sequence into a pair of numbers. With sequences codable, primitive recursion is expressible, and every primitive recursive predicate, including "`p` codes a proof of the formula coded by `n`," becomes representable in `Q`.

The provability predicate `Prov(n)` is then "there exists `p` such that `p` codes a proof of `n`," an existential quantifier over the primitive recursive proof-checking predicate. This one unbounded quantifier is why `Prov` is `Σ₁`, recursively enumerable but not in general recursive, and that asymmetry is the seam the whole incompleteness argument runs along. A theory can correctly confirm its actual proofs, the positive facts, but it cannot in general refute the false ones, because refuting "there is a proof" needs a universal quantifier the predicate does not carry. Hold onto this asymmetry; it is exactly what distinguishes Gödel's `Prov` from Tarski's `True` in the section after next, and it is the logical form of the gap between a system confirming its safe behaviors and certifying the absence of unsafe ones.

The diagonal lemma

The heart of Gödel's proof is a fixed-point construction on sentences. It says that for any property of sentences the theory can express, there is a sentence asserting that it itself has that property. This is Lawvere's theorem in the syntactic category, and it is the one piece we can state cleanly in core Lean as an abstract fact about universal systems.

Lemma 2.9 (Diagonal lemma, abstract form). *Let `f : A → A → Y` be a universal system and `t : Y → Y` any transformation of outcomes. Then some index `a` satisfies `f a a = t (f a a)`: the diagonal behavior at `a` is a fixed point of `t` applied through the system.*

```
theorem c2_diagonal_lemma {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g) (t : Y → Y) :
    ∃ a, f a a = t (f a a) :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Universality names the diagonal behavior `fun a => t (f a a)` by some `a₀`, so `f a₀ = fun a => t (f a a)`. Evaluate at `a₀`, which is what `congrFun` does, to get `f a₀ a₀ = t (f a₀ a₀)`. ■

This is the same three lines as `lawvere`, reorganized to expose the index `a₀` rather than the fixed value. In the arithmetical reading, `A` is codes of one-variable formulas, `f a b` is provability of the diagonalization, and `t` is the boolean operation the target property performs on a truth value. Lemma 2.9 then says: for the property built from `t`, there is a self-referential sentence `G`, coded by `a₀`, for which `T` proves `G ↔ t`-of-its-own-status. The genuine Gödel diagonal lemma, which produces an honest arithmetic sentence rather than an abstract index, is the arithmetization of exactly this step, and it is a theorem of `Q`. It is developed in the model-theory literature and formalized in Mathlib's first-order logic library; we state it and cite it rather than reproduce it, because the arithmetization is orthogonal to the self-reference the lemma captures.

The provability predicate and the Gödel sentence

Let `Prov(x)` be the arithmetized provability predicate: `Prov(⌜φ⌝)` says, inside `T`, that `φ` has a proof in `T`. This predicate is representable because provability is recursively enumerable, and building it is the longest part of the proof. Apply the diagonal lemma with the property "not provable." It yields a sentence `G` such that

`T ⊢ G ↔ ¬ Prov(⌜G⌝)`,

a sentence that says, correctly, "I am not provable in `T`."

Theorem 2.10 (Gödel's first incompleteness theorem, 1931). *Let `T` be a consistent, recursively axiomatized theory that represents the recursive predicates. Then there is a sentence `G` such that `T` proves neither `G` nor `¬ G`. If `T` is also sound, `G` is true.*

Proof (structure). Suppose `T ⊢ G`. Then there is a proof, so `T ⊢ Prov(⌜G⌝)` by the fact that `Prov` correctly reports actual proofs. But `T ⊢ G ↔ ¬ Prov(⌜G⌝)`, so `T ⊢ ¬ Prov(⌜G⌝)`, and `T` proves both `Prov(⌜G⌝)` and its negation, contradicting consistency. Suppose instead `T ⊢ ¬ G`. Then `T ⊢ Prov(⌜G⌝)` by the fixed-point equivalence. Under the stronger assumption of omega-consistency, or under the Rosser trick which removes that assumption, `Prov(⌜G⌝)` together with the absence of an actual proof again yields a contradiction. So `T` decides neither `G` nor `¬ G`. If `T` is sound then `G`, being unprovable, is exactly what it says it is, and is true. ■

The self-referential collision at the center is Lemma 2.9. If a consistent `T` *could* decide every sentence, its provability behavior would be a total, correct, boolean system over its own sentences, and applying the diagonal with `t` = negation would produce a sentence equal to its own negation, which is impossible. We can state that collapse as a compiled fact, reading `f a a` as the settled truth value the complete-and-sound theory assigns to the self-referential sentence coded by `a`.

```
theorem c2_godel_semantic {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match c2_diagonal_lemma f hf (fun b => !b) with
  | ⟨a₀, ha₀⟩ => bool_not_fpf (f a₀ a₀) ha₀.symm
```

Proof. Lemma 2.9 with `t` the flip gives `a₀` with `f a₀ a₀ = !(f a₀ a₀)`, and `bool_not_fpf` refutes it. ■

What `c2_godel_semantic` does and does not say. It says that a boolean system that is *universal* over its own indices is contradictory. In the arithmetical reading, universality is the conjunction of representability with completeness: representability gives the surjection onto recursive behaviors, and completeness is the extra promise that the theory assigns a definite provable truth value to every self-referential sentence, so that the diagonal behavior lands back in the represented family. The compiled theorem is therefore the *semantic* core: no consistent theory is both representationally rich and complete. What it does not capture, and what the prose proof of Theorem 2.10 supplies, is the syntactic bookkeeping that turns "not complete" into an explicit undecided sentence `G` you can write down, and the care about omega-consistency versus the Rosser construction. That bookkeeping is real and is why the full theorem is not a one-liner. The one-liner is the reason the bookkeeping was worth doing.

Discussion of the hypotheses. Three hypotheses carry the theorem, and each maps onto the recipe. Consistency is what makes the fixed-point collision fatal rather than merely odd; an inconsistent theory proves everything and has no trouble proving `G`, so the flip has, in effect, a fixed point because every value is provable. Recursive axiomatizability is what makes `Prov` representable, and hence what makes the diagonal behavior land in the surjection's range; drop it and a theory can be complete, as the full first-order theory of

the standard model is, at the price of not being recursively listable. Representational strength is point-surjectivity itself. Weaken any one and the wrapper fails, and with it the theorem. This is the same sensitivity we saw in Cantor, where surjectivity was load-bearing, made arithmetical.

Omega-consistency and the Rosser refinement. Gödel's original argument needed slightly more than plain consistency to rule out $T \vdash \neg G$. He assumed omega-consistency, which forbids the theory from proving $\exists x, \varphi(x)$ while also proving $\neg \varphi(0), \neg \varphi(1)$, and so on for every numeral, a pathology a consistent but unsound theory can exhibit. Rosser removed the extra assumption in 1936 by a clever change of sentence. Instead of "I am not provable," Rosser's sentence says "for every proof of me, there is a shorter proof of my negation." This is still a fixed point of a syntactic operation, so it is still Lemma 2.9 in arithmetized form, but the operation now compares proof lengths, and the comparison makes plain consistency enough to block both $T \vdash R$ and $T \vdash \neg R$. The upgrade costs nothing in the abstract engine; it is entirely a refinement of the wrapper, choosing a diagonal behavior whose fixed point is fatal under a weaker hypothesis. That the same impossibility admits a sharper wrapper is a pattern the safety chapters repeat, where a first trilemma under strong assumptions is later reproved under weaker, more realistic ones.

The epistemic sting. Part of what makes Gödel's theorem feel paradoxical, when it is not, is a confusion of levels that the recipe dissolves. The sentence G is unprovable in T, and, if T is sound, true. So we, reasoning in the metatheory, know G is true, while T cannot prove it. There is no contradiction: our knowledge uses the soundness of T, a fact about T that lives outside T and that T cannot establish about itself, by the second theorem below. The diagonal does not produce a sentence that is true and false, or provable and unprovable. It produces a sentence whose truth outruns one particular system's reach, and it locates that sentence precisely. Every "the AI knows it is lying but says it anyway" intuition in the safety literature is a version of this level confusion, and untangling it always comes down to naming which system is doing the knowing.

Concrete unprovable truths. The Gödel sentence is often dismissed as artificial, a sentence built only to be unprovable, and for decades that dismissal had some force. It no longer does. There are now natural mathematical statements, about ordinary combinatorial objects, that are true but unprovable in PA. Goodstein's theorem, about sequences of numbers that appear to grow without bound but in fact always reach zero, is true in the standard model and independent of PA, provable only by transfinite induction up to ε_0. The Paris–Harrington theorem, a strengthened finite Ramsey statement, is another. These are not liar sentences; they are the kind of statement a combinatorialist might pose without any thought of logic, and PA cannot prove them. The lesson for the safety reading is that the incompleteness boundary is not confined to self-referential curiosities. It cuts through the natural questions too, which means a system's inability to certify a property of itself is not a corner case one can engineer around by avoiding self-reference. The unprovable can look completely ordinary.

Remark 2.11 (The safety reading). Gödel's theorem is the template for every result in the second half of these notes that has the shape "a system powerful enough to model itself cannot also certify itself." A model asked to output a calibrated confidence in its own future outputs, a verifier asked to verify verifiers including itself, an overseer asked to audit systems as capable as it is: each is a `Prov`-like predicate turned on its own domain, and each meets the diagonal. Chapter 3 makes the hallucination version precise, and Chapter 7 the oversight version. The recurring engineering temptation is to believe that more capability closes the gap. Gödel says more capability, in the form of stronger representation, only sharpens the diagonal.

2.5 Gödel's second incompleteness theorem

The first theorem produces an undecided sentence. The second identifies a particular one: the theory's own consistency. It is the deeper result for the philosophy of mathematics and the more delicate to prove, because it depends not just on having a provability predicate but on that predicate behaving well.

Formalize consistency as the sentence $\mathsf{Con}(T) := \neg\, \mathsf{Prov}(\ulcorner 0 = 1 \urcorner)$, which says that T does not prove an outright falsehood. The second theorem says a consistent T that can talk about its own proofs cannot prove $\mathsf{Con}(T)$.

Theorem 2.12 (Gödel's second incompleteness theorem, 1931). *Let T be a consistent, recursively axiomatized theory extending a modest base of arithmetic, whose provability predicate satisfies the Hilbert–Bernays–Löb derivability conditions. Then T does not prove $\mathsf{Con}(T)$.*

The derivability conditions are the promises that make `Prov` behave like a real notion of proof inside T:

1. If $T \vdash \varphi$ then $T \vdash \mathsf{Prov}(\ulcorner \varphi \urcorner)$. Actual proofs are internally recognized.
2. $T \vdash \mathsf{Prov}(\ulcorner \varphi \to \psi \urcorner) \to (\mathsf{Prov}(\ulcorner \varphi \urcorner) \to \mathsf{Prov}(\ulcorner \psi \urcorner))$. Internal proof respects modus ponens.
3. $T \vdash \mathsf{Prov}(\ulcorner \varphi \urcorner) \to \mathsf{Prov}(\ulcorner \mathsf{Prov}(\ulcorner \varphi \urcorner) \urcorner)$. The theory can internally verify its own recognitions.

Proof (structure). From the fixed-point equivalence $T \vdash G \leftrightarrow \neg\, \mathsf{Prov}(\ulcorner G \urcorner)$ and the derivability conditions, one shows $T \vdash \mathsf{Con}(T) \to G$: internal consistency would let the theory carry out, about itself, the first theorem's argument that G is unprovable, which is what G asserts. But the first theorem says $T \nvdash G$. If T proved $\mathsf{Con}(T)$, it would prove G, a contradiction. So $T \nvdash \mathsf{Con}(T)$. ∎

The second theorem, machine-checked

The sketch above is the textbook one, and it is the part of Gödel's second theorem that can be checked here. What cannot be checked here is the arithmetization: that a particular theory such as `PA` really does admit a provability predicate satisfying the three conditions, and really does admit the fixed

point. That is a large formalization in its own right, and this development does not carry it out. What follows is therefore the theorem relative to those inputs, which is exactly how the derivability-conditions presentation states it.

Package a proof system as a structure. The sentences are an arbitrary type, `imp` and `bot` are the object-language connectives, `box` is the provability predicate `Prov`, and `Thm` is the judgement `T ⊢ ·`. Beyond the three derivability conditions we assume only modus ponens and the two axiom schemes `K` and `S` of the implicational fragment, which are the standard Hilbert-system apparatus and which say nothing about provability.

```
structure c2_PC where
  S : Type
  Thm : S → Prop
  imp : S → S → S
  box : S → S
  bot : S
  mp : ∀ {a b}, Thm (imp a b) → Thm a → Thm b
  ax_K : ∀ a b, Thm (imp a (imp b a))
  ax_S : ∀ a b c,
    Thm (imp (imp a (imp b c)) (imp (imp a b) (imp a c)))
  D1 : ∀ {a}, Thm a → Thm (box a)
  D2 : ∀ a b, Thm
      (imp (box (imp a b)) (imp (box a) (box b)))
  D3 : ∀ a, Thm (imp (box a) (box (box a)))
```

Two derived rules of the implicational fragment, obtained from `K`, `S`, and modus ponens alone. They are the only propositional reasoning the argument needs.

```
theorem c2_s_comb (P : c2_PC) {a b c : P.S}
    (h₁ : P.Thm (P.imp a (P.imp b c)))
    (h₂ : P.Thm (P.imp a b)) :
    P.Thm (P.imp a c) :=
  P.mp (P.mp (P.ax_S a b c) h₁) h₂

theorem c2_syll (P : c2_PC) {a b c : P.S}
    (h₁ : P.Thm (P.imp a b)) (h₂ : P.Thm (P.imp b c)) :
    P.Thm (P.imp a c) :=
  c2_s_comb P (P.mp (P.ax_K (P.imp b c) a) h₂) h₁
```

Now Löb's theorem. The hypotheses `fix₁` and `fix₂` are the two directions of the diagonal lemma's fixed point `g ↔ (Prov(g) → A)`, supplied rather than constructed, since constructing it is the arithmetization we are not doing.

Theorem 2.12a (Löb, 1955). *If `T ⊢ Prov(A) → A`, then `T ⊢ A`.*

```
theorem c2_loeb (P : c2_PC) {A g : P.S}
    (fix₁ : P.Thm (P.imp g (P.imp (P.box g) A)))
```

```
    (fix₂ : P.Thm (P.imp (P.imp (P.box g) A) g))
    (h : P.Thm (P.imp (P.box A) A)) :
    P.Thm A :=
  let s3 := P.mp (P.D2 g (P.imp (P.box g) A)) (P.D1 fix₁)
  let s5 := c2_syll P s3 (P.D2 (P.box g) A)
  let s7 := c2_s_comb P s5 (P.D3 g)
  let s8 := c2_syll P s7 h
  P.mp s8 (P.D1 (P.mp fix₂ s8))
```

Proof. Necessitate the fixed point and push the box inward with `D2`, giving `Prov(g) → Prov(Prov(g) → A)`. A second application of `D2` turns that into `Prov(g) → (Prov(Prov(g)) → Prov(A))`, and `D3` contracts the two boxes to yield `Prov(g) → Prov(A)`. Composing with the hypothesis gives `Prov(g) → A`, which is the right-hand side of the fixed point, so `g` itself is a theorem; necessitating `g` and applying modus ponens delivers `A`. ■

Consistency is the sentence saying that falsity is not provable, and Gödel's second theorem is Löb's theorem at `A := ⊥`.

```
def c2_Con (P : c2_PC) : P.S := P.imp (P.box P.bot) P.bot

theorem c2_godel_second (P : c2_PC) {g : P.S}
    (fix₁ : P.Thm (P.imp g (P.imp (P.box g) P.bot)))
    (fix₂ : P.Thm (P.imp (P.imp (P.box g) P.bot) g))
    (hcon : P.Thm (c2_Con P)) :
    P.Thm P.bot :=
  c2_loeb P fix₁ fix₂ hcon
```

Proof. `Con` unfolds to `Prov(⊥) → ⊥`, which is the hypothesis of Löb's theorem with `A := ⊥`. So a theory proving its own consistency proves `⊥`. ■

Read `c2_godel_second` contrapositively and it is the statement of Theorem 2.12: a theory that does not prove `⊥`, that is, a consistent one, does not prove `Con(T)`.

One check is worth doing before trusting any of this, because a theorem whose hypotheses cannot be met is true for empty reasons. The axiom set is satisfiable:

```
def c2_pc_trivial : c2_PC where
  S := Unit
  Thm := fun _ => True
  imp := fun _ _ => ()
  box := fun _ => ()
  bot := ()
  mp := fun _ _ => trivial
  ax_K := fun _ _ => trivial
  ax_S := fun _ _ _ => trivial
  D1 := fun _ => trivial
  D2 := fun _ _ => trivial
```

```
D3 := fun _ => trivial
```

This witness is deliberately degenerate: it proves everything, including ⊥, so it is an inconsistent theory. It establishes only that `c2_PC` is inhabited, so the theorems above are not vacuous. It does not establish that a consistent theory meets the conditions, and the reader should not read it as doing so. That claim is the arithmetization, and it remains the one thing in this chapter taken from the literature rather than checked here.

There is a cleaner route through Löb's theorem, which is the fixed-point argument made self-standing. Löb's theorem (1955) says `T ⊢ Prov(⌜φ⌝) → φ` implies `T ⊢ φ`: a theory can only internally trust the provability of what it already proves. Löb's theorem is itself a diagonal argument, the fixed point of the map `ψ ↦ (Prov(⌜ψ⌝) → φ)`, and the second incompleteness theorem is the case `φ = 0=1`, because `Con(T)` is exactly `Prov(⌜0=1⌝) → 0=1` waiting for Löb to refuse it. So the second theorem, like the first, is one more fixed point, taken now in the modal algebra of the provability predicate. The modal logic that axiomatizes this algebra, `GL` for Gödel–Löb, has Löb's theorem as its defining axiom, and its arithmetical completeness (Solovay, 1976) is the precise statement that provability reasoning *is* this fixed-point logic and nothing more.

Why this one stays in prose. Every step above is a theorem, and the whole is formalized in the literature, but the derivability conditions are statements about a specific arithmetized `Prov`, and verifying them is the arithmetization we deferred in the first theorem, now with the added burden of the third condition, which is the internal formalization of the first. There is no faithful two-line core-Lean rendering, because the content is precisely the syntactic behavior of a particular predicate, not the abstract diagonal. What the abstract diagonal does capture, Löb's fixed point, we have already compiled once as Lemma 2.9: the map whose fixed point Löb takes is a `t : Y → Y` in disguise, and the existence of the fixed point is `c2_diagonal_lemma`. The arithmetical soundness of that fixed point is the cited part.

Remark 2.13 (Self-certification is the second theorem). The second theorem is the sharpest classical statement of a limit that recurs throughout AI safety: a system cannot vouch for its own soundness from within. "This model will not deceive you" is a `Con`-like sentence about the model, and a model strong enough to state it and satisfying the internal-coherence analogues of the derivability conditions cannot prove it without already being unsound. Chapter 7 returns to this when it argues that a deployed system's own assurances about its alignment carry no evidential weight beyond what an external, more powerful check provides, and that the external check then faces its own second theorem one level up.

Provability as a modal logic. The cleanest modern packaging of both incompleteness theorems is the modal logic `GL`, where a box operator reads as "is provable in `T`." The derivability conditions become the modal axioms: condition 1 is the necessitation rule, condition 2 is the distribution axiom

K, and condition 3 is the axiom 4, that the box iterates. Löb's theorem becomes the single axiom GL, which says box-of-(box-p-implies-p) implies box-p. The Kripke frames that validate GL are exactly the transitive, converse-well-founded ones, frames with no infinite ascending chains, and converse well-foundedness is the modal shadow of consistency: a chain of ever-stronger provability claims must terminate. Solovay's completeness theorem of 1976 makes this exact. His first theorem says GL proves precisely the modal principles that hold for T's provability predicate under every arithmetical interpretation, so provability reasoning is GL and nothing more. His second characterizes the principles that are always true, as opposed to always provable, and the two differ by exactly the reflection principle the second incompleteness theorem denies. The upshot worth carrying: the logic of a system reasoning about its own proofs is completely understood, it is finite and decidable, and it has Löb's fixed point built in as an axiom. Self- reference at this level is not mysterious; it is a small modal logic.

Consistency strength. The second theorem also organizes the landscape of theories into a linear hierarchy by consistency strength. Because T cannot prove its own consistency, a theory that does prove Con(T) is strictly stronger, and this relation, "proves the consistency of," well-orders the natural theories from weak arithmetic up through set theory with large cardinals. Each step up is a concrete instance of the second theorem: the stronger theory sees a consistency fact the weaker one is blind to, and is itself blind to its own. Ordinal analysis measures these steps with ordinals, assigning to PA the ordinal ε_0, the height of the induction it can prove well-founded. The picture to keep is a ladder no rung of which can see its own footing, which is the exact shape Chapter 7 gives to a tower of overseers, each able to certify those below and none able to certify itself.

2.6 Tarski's undefinability of truth

Tarski proved in 1936 that a sufficiently expressive language cannot define its own truth predicate. Where Gödel constrained provability, which is recursively enumerable and internally expressible, Tarski constrained truth, which is not even that. The result is in one sense stronger and in another simpler: stronger because undefinability is a flat impossibility with no undecided-sentence subtlety, simpler because there is no need for derivability conditions, only for the diagonal.

A truth predicate for a language L is a formula True(x) such that for every sentence φ,

True(⌜φ⌝) $\leftrightarrow$ φ,

which is Tarski's material adequacy condition, his schema T. It says the predicate gets every sentence right. Tarski's theorem is that no formula of L itself can do this for all sentences of L.

Definition 2.14 (Definable truth predicate). A language interpreting arithmetic has a *definable truth predicate* when some formula True(x) satisfies

schema T for every sentence of the language.

Model the situation as a boolean system. Let `A` be codes of sentences, and let a candidate truth predicate be a way of assigning, to each code, a boolean verdict, uniformly across all the boolean patterns the language can express. Material adequacy plus definability makes the verdict system universal over its own sentences: every boolean condition the language can state, including the negation of the predicate composed with the diagonal, is itself the verdict pattern of some sentence. That is point-surjectivity, and it is exactly the hypothesis of Chapter 1's `no_reflective_verdict`.

Theorem 2.15 (Tarski, 1936). *No language that can represent its own syntax and interpret arithmetic has a definable truth predicate.*

```
theorem c2_tarski {A : Type _} (True? : A → A → Bool)
    (hT : ∀ g : A → Bool, ∃ a, True? a = g) : False :=
  no_reflective_verdict True? hT
```

Proof. A definable, materially adequate truth predicate makes `True?` a reflective verdict, universal over the boolean patterns on sentences. Chapter 1's `no_reflective_verdict` says none exists. Unfolding, the diagonal builds the liar sentence `L` with `True?(⌜L⌝) ↔ ¬ True?(⌜L⌝)`, and material adequacy turns that into `L ↔ ¬ L`. ■

Identifying `Y` and `t`. `Y = Bool`, the truth value, and `t` is negation. The witness is the liar, "this sentence is false," which is `liar_query` read as a sentence rather than as a set or a program. The liar is not a curiosity at the edge of the theory. It is the exact obstruction, the diagonal behavior that a truth predicate would have to name and cannot.

Tarski versus Gödel. The two theorems are close and the difference is precise. Gödel's `Prov` *is* definable, because provability is recursively enumerable, and the price of that definability is incompleteness: the predicate exists but the theory cannot decide the sentence it produces. Tarski's `True` would have to be definable to run the same diagonal, and the conclusion is that it simply is not definable at all, because if it were, the liar would be an outright contradiction rather than a merely undecided sentence. Provability is a shadow of truth that the system can cast; truth itself the system cannot hold. The gap between them, the sentences that are true but not provable, is the room the first incompleteness theorem lives in. One clean way to see the whole picture: if truth were definable it would be a complete, sound provability predicate, and `c2_godel_semantic` already showed that is contradictory.

The hierarchy of metalanguages

Tarski's theorem is a ban on defining truth *for a language within that same language,* and Tarski paired the ban with a positive construction that respects it. Truth for a language `L` is definable, just not in `L`; it is definable in a richer metalanguage `L'` that has the resources to quantify over the sentences of `L` and assert the schema T for each. Truth for `L'` then needs a still richer `L''`, and so

on up an unbounded hierarchy of languages, each defining truth for the one below and none defining its own. This is the same ladder as the consistency-strength hierarchy, and it is not a coincidence: both are the second-theorem shape, a tower in which each level certifies the level beneath and no level certifies itself.

There are ways to buy a single language its own truth predicate, and each pays a price the recipe predicts. Kripke's 1975 theory of truth allows the truth predicate to be *partial*, undefined on the ungrounded sentences like the liar, and builds it as the least fixed point of a monotone operator that adds sentences to the extension and anti-extension as they become settled. The liar never settles, so `True` is simply undefined there, and the material adequacy schema holds only for grounded sentences. This dodges Tarski's theorem by refusing to be total, which is exactly the move that separates the halting problem from Rice's theorem: allow the deciding object to withhold judgment and the flat impossibility relaxes into a statement about where judgment must be withheld. The revision theory of Gupta and Belnap instead lets the predicate's extension be revised transfinitely, with the liar oscillating forever rather than settling, and reads truth as the stable pattern of that revision. Both are honest responses to Tarski, and both trade totality or bivalence for definability. A truth predicate cannot be total, exact, bivalent, and self-applicable at once, and you may keep any three.

Remark 2.16 (Truth probes and Chapter 8). Tarski's theorem is the reason the AI-safety literature on "truthfulness" has to be careful about what it can promise. A truth predicate that is total, exact, and expressible in the same system whose outputs it judges is exactly the object Tarski forbids. Chapter 8 studies probes that read a model's internal state to estimate whether the model "believes" its own output, and the standing objection to such probes is Tarskian: a probe definable within the model, applied to the model's own truth, meets the liar. The resolution there is not to defeat the theorem but to change the domain, moving from outputs to activations and from a two-valued verdict to a continuous score on a connected space, where a different engine, the intermediate value theorem, takes over. That the boundary object survives the change of engine is the point of the final chapter.

2.7 Turing and the halting problem

Turing's 1936 theorem predates the modern statement of Rice's and is its special case, but historically and pedagogically it comes first. It says no program can decide, for arbitrary programs and inputs, whether they halt. Read through the recipe, the halting decider is a candidate boolean predicate `D` on programs, and the diagonal builds a program that halts exactly when `D` predicts it will not.

Let `A` be the type of programs, and let programs also serve as inputs, since a program is a string and a string can be fed to a program. Suppose the programming language is universal in the sense that every boolean behavior over

programs is itself computed by some program, which is the closure of the language under the constructions the argument needs. A halting decider is any `D : A → Bool`, total by assumption, claiming to report whether the diagonal computation `f a a` halts.

Definition 2.17 (Halting decider). A *halting decider* for a universal system `f : A → A → Bool` is a total map `D : A → Bool` intended to satisfy `D a = f a a` for all `a`, where `f a a` records the halting behavior of program `a` on its own code.

Theorem 2.18 (Turing, 1936). *For every universal system `f : A → A → Bool` and every candidate decider `D : A → Bool`, there is a program `a` on which the decider is wrong: `f a a ≠ D a`. No total halting decider is correct.*

```
theorem c2_turing_halting {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) (D : A → Bool) :
    ∃ a, f a a ≠ D a :=
  match hf (fun a => !(D a)) with
  | ⟨a₀, ha₀⟩ =>
    ⟨a₀, fun h =>
      bool_not_fpf (D a₀) ((congrFun ha₀ a₀).symm.trans h)⟩
```

Proof. Universality names the contrarian behavior `fun a => !(D a)`, "do the opposite of what `D` predicts," by some program `a₀`, so `f a₀ = fun a => !(D a)`. Evaluate at `a₀`: `f a₀ a₀ = !(D a₀)`. If the decider were right there, `f a₀ a₀ = D a₀`, we would get `!(D a₀) = D a₀`, refuted by `bool_not_fpf`. So `a₀` is a program the decider misjudges. ■

The contrarian program `a₀` is the classical construction: build a machine that runs `D` on its own code and then loops if `D` says halt, halts if `D` says loop. It cannot behave consistently with `D`'s prediction, so `D` is wrong about it. This is the same `a₀` as everywhere else in the chapter, now reading `Y = Bool` as "halts or runs forever" and `t` as the flip "do the opposite."

The partiality subtlety. The clean statement above hides the one place halting differs from Cantor. A real halting decider must be *total*, returning an answer on every program, while the programs it judges are *partial*, sometimes running forever. The contrarian program is built to be partial in exactly the way that breaks a total decider: where `D` says "halts," it loops, which a total `D` cannot itself do while remaining total and correct. If deciders were allowed to be partial, returning no answer sometimes, the theorem would not follow in this form, and the correct statement becomes Rice's, about the impossibility of *total* decision of a nontrivial *semantic* property. We take that step next. Chapter 1's Exercise 1.8 flagged this as the shape of Rice's theorem, and it is.

The universal machine is the wrapper. The hypothesis that the language is universal, `∀ g, ∃ a, f a = g`, is not free; it is the existence of a universal Turing machine, a single program that simulates any program given its code. Turing built one, and that construction is the wrapper for this theorem exactly as Gödel numbering was the wrapper for incompleteness. Without a universal machine there is no closure under diagonalization, and the halting

problem for a non-universal model of computation can be perfectly decidable; the halting problem for finite automata, for instance, is trivial. Undecidability is a property of systems rich enough to host their own interpreter, which is the computational form of "expressive enough to describe its own behavior" from Chapter 1. The safety reading writes itself: a model general enough to simulate arbitrary reasoning, including reasoning about itself, has bought the very universality that makes its behavior undecidable.

Where halting sits. The halting problem is not merely undecidable; it is the canonical undecidable problem, complete for the recursively enumerable sets under many-one reduction. Every other undecidability in this chapter reduces to it or sits above it in the arithmetical hierarchy, the classification of problems by the number of quantifier alternations needed to define them. Halting is Σ_1, one existential quantifier over a decidable matrix, "there exists a halting computation," the same Σ_1 shape as Gödel's `Prov`, and this is no accident: `Prov` is halting for a proof search. Totality, "this program halts on every input," is Π_2, strictly harder, and the hierarchy continues upward without collapsing. The diagonal produces the first rung, and reductions climb the rest. Quantitatively, even the first rung is wild: the busy-beaver function, the longest a halting n-state machine runs, grows faster than any computable function, so uncomputability is not a thin boundary phenomenon but a dominant one. The diagonal tells you the boundary exists; it does not tell you it is this violent. That gap between qualitative and quantitative is the book's recurring theme.

Remark 2.19 (Runtime verification). Turing's theorem is the ancestor of every impossibility in program analysis, and its safety reading is direct: no static analyzer decides, for all programs, a behavioral question as simple as termination. For AI systems the behavioral questions are far harder than halting, whether a model will ever produce disallowed content, whether an agent will ever take an unsafe action, and the halting problem is the floor. If termination is undecidable, "never does anything unsafe over an unbounded interaction" is not going to be decidable by a general procedure either. The safety chapters do not lean on Turing directly, because they want quantitative statements the diagonal cannot give, but Turing is the qualitative backstop that says the quantitative work is necessary.

2.8 Rice's theorem

Rice's theorem (1953) is the culmination of the computability instances. It says that *every* nontrivial semantic property of programs is undecidable, not just halting. A property is *semantic* when it depends only on the function a program computes, not on the program's text, and *nontrivial* when some programs have it and some do not. Rice's theorem sweeps the entire class in one argument: if you can name a behavioral property that some but not all programs have, no total procedure decides it.

The reduction to the diagonal has two layers. The boolean core is again

the contrarian construction, identical to Turing's, and we compile it. The full statement, quantified over all nontrivial properties and over the partial recursive functions, needs the machinery of an acceptable programming system and is proved in the companion library; we compile the core and cite the rest.

The boolean core

Definition 2.20 (Semantic decider). For a universal system `f : A → A → Bool`, a *semantic decider* is a total map `D : A → Bool` claiming to compute the diagonal behavior `f a a` from the index `a` alone. Rice's theorem denies that any such `D` is correct everywhere.

Theorem 2.21 (Rice, diagonal core). *For every universal `f : A → A → Bool` and every `D : A → Bool`, some index `a` has `f a a ≠ D a`.*

```
theorem c2_rice_diagonal {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) (D : A → Bool) :
    ∃ a, f a a ≠ D a :=
  c2_turing_halting f hf D
```

Proof. This is `c2_turing_halting` under a change of reading. The contrarian behavior `fun a => !(D a)` is named by some a_0, and the decider is wrong there. ∎

That the proof of Rice's core is literally the proof of Turing's theorem is the whole thesis of the chapter, made unusually blunt. Turing decides one property, halting; Rice decides any property; the diagonal does not notice the difference, because it only ever sees a boolean `D` and flips it. The content that distinguishes Rice from Turing is entirely in the wrapper: showing that an arbitrary nontrivial semantic property gives rise to a `D` the diagonal can attack.

Nontrivial properties and partial functions

The classical Rice's theorem is stated for properties `P` of the partial function a program computes. Let `P` be a set of partial recursive functions that is neither empty nor everything, a nontrivial property. Rice's theorem says the set of programs whose function lies in `P` is undecidable.

Theorem 2.22 (Rice, full form). *Let `P` be a nontrivial semantic property of partial recursive functions. Then the set `{e | φ_e ∈ P}` of program indices with that property is not recursive.*

Proof (structure). Nontriviality gives a program `a` with the property and a program `b` without it, or vice versa; without loss the everywhere-undefined program lacks `P`. Given a claimed decider for `P`, reduce the halting problem to it: build, from any program and input, a program that behaves like `a` if the input halts and like the empty program otherwise, so that it has `P` exactly when the input halts. A decider for `P` would then decide halting, which Theorem 2.18 forbids. The reduction is the wrapper; the impossibility it reduces to is the diagonal. ∎

This full form needs an acceptable numbering of the partial recursive functions, with the `s-m-n` theorem to perform the construction uniformly, and a case-analysis on which side of `P` the empty function falls on. It uses classical logic and the recursion-theoretic apparatus of Mathlib. The companion development formalizes the diagonal core and the AI-safety corollaries in `Foundation/F_02_RiceTheorem.lean`: `Foundation.rice_diagonal` is Theorem 2.21 in mismatch form, `Foundation.rice_general` is the property version for a nontrivial `P : Bool → Bool`, and `Foundation.no_automated_self_test` is the reading we care about, that no automated tester predicts a universal system's behavior on its own description. We reproduce the core here and cite those for the full statement, in keeping with the discipline of the book: the self-reference compiles, the recursion theory is cited.

Identifying `Y` and `t`. As always for the computability instances, `Y = Bool` and `t` is negation. What is new in Rice is the universal quantifier over properties, and the recipe absorbs it: each nontrivial property yields, through the reduction, a boolean `D`, and the single fixed-point-free `t` defeats them all at once. One `t`, every property.

Discussion of the hypothesis. Nontriviality is the load-bearing word, and it is the analogue of surjectivity being nondegenerate. A trivial property, one that all programs have or none do, is decided by a constant function, and there is no contradiction because the constant decider is never wrong. The diagonal needs two distinct outcomes to flip between, which is why `t` must be fixed-point free and why `Y` must have at least two elements. A property with only one truth value across all programs gives a `t` with a fixed point, and the engine stalls, correctly, because the theorem is false for trivial properties. This is the computability shadow of Chapter 1's Exercise 1.4, that `Unit` admits no fixed-point-free endomap.

Rice–Shapiro and the shape of the undecidable

Rice's theorem says every nontrivial semantic property is undecidable, and it is natural to ask how badly. The Rice–Shapiro theorem answers for the properties that are merely recursively enumerable rather than recursive. It says a semantic property `P` has an r.e. index set only if `P` is determined by finite information: a function has the property exactly when some finite subfunction of it already does, and every extension of such a finite piece keeps the property. Properties that violate this, "the function is total," "the function computes a constant," have index sets that are not even r.e., and they sit at Π_2 or higher in the hierarchy. So Rice sorts semantic properties into a spectrum. The trivial ones are decidable. The finitely-determined ones are r.e. but undecidable. The rest are worse, unrecognizable even in the limit. The bare diagonal only distinguishes trivial from nontrivial; the finer sorting needs the reductions and the hierarchy, which is once more the pattern that the diagonal gives the first, coarse boundary and further structure needs further tools.

This finer picture is the one the safety chapters actually need, because it says where the line between checkable and uncheckable falls. A behavioral

property that depends only on finitely much observed behavior can at least be recognized when it appears, even if its absence can never be confirmed, which is the logical form of "you can catch a model doing something bad but cannot certify it never will." A property that depends on the whole infinite behavior cannot even be recognized. Rice–Shapiro is the classical statement that monitoring is possible and certification is not, and Chapter 7 gives it teeth by adding the cost of the finite observation and the rate at which confidence can accrue.

Remark 2.23 (No automated semantic testing). Rice's theorem is the direct ancestor of the book's core negative results about evaluating models. A "semantic property of a model" is a property of the function the model computes, whether it is honest, whether it is aligned, whether it is safe, and Rice says no general automated test decides such a property for universal systems. `no_automated_self_test` in the companion library is exactly this, and Chapter 3 upgrades it from "undecidable" to a quantitative trilemma by adding structure the bare property lacks. The pattern to remember: Rice closes the door on deciding behavioral properties by inspection, and the analytic chapters reopen it a crack by asking not for a decision but for a calibrated estimate on a connected space, where being approximately right is possible even though being exactly right is not.

The practical shadow of Rice falls across the whole enterprise of model evaluation. A benchmark is an attempt to decide a semantic property of a model by running it on finitely many inputs, and Rice is the reason a benchmark can never certify the property in general, only sample it. This is not a defect of current benchmarks that a better one would fix; it is the theorem. What a benchmark can honestly claim is finite and statistical, "on this distribution, at this rate, the property held," and the gap between that and "the property holds" is precisely the gap Rice guarantees is unbridgeable by any finite test. The constructive response, and the one the safety chapters develop, is to stop asking for certification and start asking for bounds: not "is this model safe," which Rice forbids deciding, but "with what probability, over what distribution, at what adversarial cost, does it fail," which measure and geometry can answer. The impossibility is real, and it is also a signpost pointing at the questions that do have answers.

2.9 The categorical unification

The claim that these are one theorem has a precise home, and it is worth stating, because it is what makes "the same argument" a mathematical fact rather than a family resemblance. Lawvere's 1969 setting is a cartesian closed category, an abstract universe of "spaces" and "maps" in which one can form products `A × B` and exponentials `Y^A`, the object of maps from `A` to `Y`. The category of sets is the example to keep in mind, with `Y^A` the function set `A → Y`, but the theorem holds in every cartesian closed category, and that generality is the unification.

In this setting a system is a map `f : A → Y^A`, an `A`-indexed family of maps `A → Y`. Point-surjectivity, which is what we have used, weakens in the categorical version to *point-surjectivity* in the internal sense, but the cleanest hypothesis is a *section*: a map `s : Y^A → A` with `f ∘ s = id`, so that every map `A → Y` is named on the nose. Lawvere's theorem is then that any `t : Y → Y` has a fixed point, where a fixed point is a global element `y : 1 → Y` with `t ∘ y = y`. The proof is the same diagonal, drawn now as a commuting diagram: precompose `f` with the diagonal `A → A × A`, transpose, apply `t`, and use the section to find the self-referential point. Chapter 1's Exercise 1.2 asked you to prove exactly the section form in Lean, and it is three lines there because the set-theoretic exponential makes transposition invisible.

Why does the abstract version make the unification literal? Because each classical theorem is Lawvere's theorem in a *particular* cartesian closed category, not merely an argument shaped like it. Cantor is the category of sets with `Y = 2`. The recursive version behind Turing and Rice is a category of assemblies or a realizability topos, where maps are tracked by programs and `Y^A` is the object of computable behaviors, so that the section is a universal machine. The provability version behind Gödel lives in a category built from a theory's own Lindenbaum algebra, where maps are provable-equivalence classes of formulas. In each category the objects and maps are different mathematics, and in each the single diagram commutes and yields the single conclusion. The wrappers of this chapter are, in this light, the constructions of the categories, and the engine is the one diagram that every cartesian closed category satisfies.

We do not compile the categorical version, because doing it honestly needs a library of category theory and the payoff is conceptual rather than computational. The set-level `lawvere` is the shadow the categorical theorem casts on the category of sets, and it is the shadow every result in this book actually uses. The value of knowing the source of the shadow is that it tells you when a new impossibility is going to be an instance: whenever the system in question is a map into an internal function space with a section and an outcome object carrying a fixed-point-free map, you already have the theorem, and only the reading remains.

2.10 Interlude: the wrapper is the mathematics

A reader who has come this far might draw the wrong lesson, that these theorems are easy because the engine is three lines. The opposite is closer to the truth. The engine is three lines precisely because a century of work moved all the difficulty into the wrappers, and the wrappers are hard. Coding syntax into arithmetic, building a universal machine, constructing an acceptable numbering with the `s-m-n` property, establishing the derivability conditions: each is a substantial theorem, and each is what a graduate course on these subjects actually spends its time proving. The diagonal is the punchline, and punchlines are short.

This division has a practical consequence for the rest of the book, and it is why the chapter dwelt on the wrappers rather than the engine. To prove a new impossibility in the safety domain, you never re-prove the diagonal. You do the work of the wrapper: you argue that a model with such-and-such capabilities really is universal over the relevant behaviors, that a defense really does have to answer every injection including the ones about itself, that a probe really is asked to be a total exact predicate on the states whose truth it reports. When that argument succeeds, the impossibility is immediate, and when it fails, the failure is exactly the escape hatch, the property a real system has that a universal one does not. The skill the book is trying to teach is not the diagonal, which you now have. It is recognizing a wrapper, and knowing which of its hypotheses a deployed system can actually give up.

There is a second, quieter lesson in the division. Because the engine is axiom-free and compiles, and the wrappers are cited, the book is honest about its own confidence in a way informal mathematics rarely is. The parts we are certain of, the self-reference, are checked by the kernel. The parts that depend on a large library, the arithmetization and recursion theory, are named so you can find the verified statement and, if you doubt it, check it yourself. When Chapter 3 asserts a safety trilemma, it inherits this discipline: the diagonal core compiles, and the modeling assumptions that make a model a reflective verdict are stated as assumptions, not smuggled in. That is the most a formalization can offer, and it is more than the informal versions of these results have ever offered.

2.11 What varies, and what does not

Seven theorems, one engine. Laid side by side, the classical limits differ only in the first four columns of the recipe and agree entirely in the fifth.

Theorem	`A` (indices)	`Y`	`t`	Wrapper (universality is…)
Cantor	a set	`Bool`	`!`	a power-set surjection
Russell	a universe of sets	`Bool`	`!`	unrestricted comprehension
Gödel I	codes of formulas	`Bool`	`!`	representability plus completeness
Gödel II	codes of formulas	`Bool`	Löb's map	derivability conditions
Tarski	codes of sentences	`Bool`	`!`	a definable truth predicate
Turing	programs	`Bool`	`!`	a universal language
Rice	programs	`Bool`	`!`	reduction from a nontrivial property

The outcome type is `Bool` in every row, and the flip is negation in every row but one, where Löb's map is a negation in modal disguise. The index type and the wrapper are where the mathematics of each theorem actually sits, and they are what a working mathematician learns when learning these results: how a set indexes its subsets, how arithmetic codes its syntax, how a language enumerates its programs. The diagonal is not what you learn. It is what you already knew from the first one.

The compiled fifth column is one of `no_universal`, `cantor`, `liar_query`, `no_reflective_verdict`, or the local combinations `c2_diagonal_lemma` and `c2_rice_diagonal`, and every one of them is Chapter 1's Theorem 1.4

under a fixed point of negation. When Chapter 3 introduces the hallucination and prompt-injection impossibilities, it adds an eighth and ninth row to this table and no new machinery to the engine. The reader who has followed the wrappers here will recognize the safety results as instances the moment their `A`, `Y`, and `t` are named, which is the point of doing the classical cases first and in this much detail.

There is one honest caveat, and it is the bridge to the second half of the book. Every theorem in the table is a flat impossibility. It says a certain system does not exist, and it says nothing about how close you can come, how many counterexamples there are, or what it costs to find one. The diagonal is a yes-or-no instrument because `Y` is `Bool` and `t` has no fixed point at all. When the safety questions demand quantities, how often a model hallucinates, how large the adversary's basin of successful attacks is, how confident a probe can be, the diagonal falls silent, and Chapters 4 through 6 pick up a continuous `Y`, a connected `A`, and the intermediate value theorem, which delivers the same boundary object with the metric content the diagonal threw away. The classical limits are where the story is cleanest. They are not where it ends.

One further observation ties the chapter to the ones ahead. In every classical instance the wrapper had to argue that a real object, a set, a theory, a language, a machine, is universal, and universality was granted almost by the definition of the object: a power set is all subsets, a programming language is closed under its own interpreter. The safety instances are different in a way that is entirely to the good. There, universality is not definitional; it is an idealization, and the whole content of a safety theorem is the argument that a system with certain desirable properties, being faithful, being calibrated, covering its domain, would have to be universal, and so cannot exist. The classical wrappers establish universality to derive a contradiction. The safety wrappers derive universality from a wish list to show the wish list is inconsistent. The engine does not notice the difference. It flips a boolean and returns the liar either way. What changes is that in the safety setting the liar is not a curiosity but a design constraint, and the right response is not to mourn the impossibility but to read off which item on the wish list a working system must give up. That reading is Chapter 3.

2.12 Historical and bibliographic notes

Cantor's diagonal argument appeared in 1891, though the 1874 proof of the uncountability of the reals already contains the idea. Russell communicated his paradox to Frege in 1902, and Frege's appendix acknowledging it is one of the more gracious moments in the history of mathematics. Gödel's two incompleteness theorems are the 1931 paper "Über formal unentscheidbare Sätze"; the diagonal lemma is often attributed jointly to Gödel and Carnap. Tarski's undefinability theorem is from his 1936 work on the concept of truth in formalized languages. Turing's halting result is in the 1936 "On Computable Numbers," and Rice's theorem is his 1953 paper on classes of recursively enu-

merable sets. Löb's theorem is 1955, and Solovay's arithmetical completeness of the modal logic GL is 1976.

The unification through category theory is Lawvere's "Diagonal arguments and cartesian closed categories" (1969). The most accessible modern account that works out Cantor, Russell, Gödel, Tarski, and Turing as instances of a single fixed-point theorem is Yanofsky's "A universal approach to self-referential paradoxes, incompleteness and fixed points" (2003), whose organization this chapter follows. For the recursion theory behind Rice's full form, Rogers's "Theory of Recursive Functions and Effective Computability" remains standard, and Boolos's "The Logic of Provability" is the reference for the modal treatment of the second incompleteness theorem. The machine-checked development these notes track is the companion Lean library, whose Cantor and Rice formalizations are cited above by file and theorem name.

2.13 Exercises

The exercises are graded. Routine ones ask you to run the recipe on a new instance or to fill in a step. Ones marked (harder) ask you to prove something new in Lean or to handle a subtlety in a hypothesis. Ones marked (open-ended) have no single answer and point toward later chapters.

Exercise 2.1. Prove `c2_no_universal_bool` by hand without invoking `no_universal`, unfolding it to a direct `match` on the universality hypothesis applied to `fun a => !(f a a)`. Check that your proof and the one in the text produce the same diagonal witness.

Exercise 2.2. State and prove in core Lean that `Unit`, the one-element type, admits no universal system with fixed-point-free flip: show there is no `t : Unit → Unit` with `t u ≠ u`. Explain why this is the degenerate case at the bottom of every row of the summary table.

Exercise 2.3. Cantor's theorem is usually stated for `Set A`, not `A → Bool`. Using the identification of a subset with its indicator, write the statement "no surjection `A → Set A`" and explain, in one paragraph, why it is the same as `c2_cantor_powerset`. Where does the classical `Set`-form proof, cited as `CCH.cantor_no_surjection_set`, use decidability that the `Bool` form avoids?

Exercise 2.4. In `c2_russell`, the hypothesis `hmem` is never satisfiable for a type `A` with at least two elements. Prove this: derive `False` from `hmem` using `cantor`, and conclude that `c2_russell` is a theorem whose hypothesis is empty. Relate this to the modern repairs of comprehension.

Exercise 2.5. Write out the propositional Russell paradox `¬ (P ↔ ¬ P)` in Lean for `P : Prop` and prove it without classical axioms. Then explain, referring to Remark 2.7, why the chapter compiles the `Bool` version instead in the diagonal engine.

Exercise 2.6. (Gödel, by hand.) Using `c2_diagonal_lemma` with `t := fun b => !b`, reprove `c2_godel_semantic` without the `match`, by `obtain`-ing the witness in tactic mode. Confirm with `#print axioms` that your proof depends on no axioms.

Exercise 2.7. Identify, for each derivability condition in Theorem 2.12, which property of an ordinary notion of proof it internalizes. Then explain in one paragraph why the third condition, and not the first two, is what makes the second incompleteness theorem harder than the first.

Exercise 2.8. State Löb's theorem as a fixed-point statement and identify the map `t` whose fixed point it takes. Verify that the second incompleteness theorem is the special case `φ = 0 = 1`, and explain why `Con(T)` is the antecedent that Löb refuses.

Exercise 2.9. Give the liar sentence explicitly as the diagonal behavior in Tarski's theorem, and trace how `liar_query` produces it. Then show that if a definable truth predicate existed, `c2_godel_semantic` would apply to it, so that Tarski's theorem also follows from the completeness-plus-soundness collapse.

Exercise 2.10. Explain the precise difference between Gödel's `Prov` and Tarski's `True`: one is definable and the theory is incomplete, the other is not definable at all. Which property of provability, absent for truth, is responsible? Frame your answer in terms of recursive enumerability.

Exercise 2.11. (Harder.) Prove in core Lean a two-input version of Turing's theorem: for `f : A → A → Bool` universal and a candidate decider `D : A → A → Bool` meant to satisfy `D a b = f a b`, there exist `a, b` with `f a b ≠ D a b`. Show that the diagonal case `a = b` recovers `c2_turing_halting`.

Exercise 2.12. Explain the totality subtlety in Turing's theorem in your own words: why the contrarian program must be partial, and why the argument would fail if deciders were allowed to return no answer. Connect this to the transition from Turing to Rice.

Exercise 2.13. For a nontrivial `P : Bool → Bool` with `P false ≠ P true`, show that `P` is a bijection on `Bool`, hence either the identity or negation. Use this to sketch how `Foundation.rice_general` reduces to `c2_rice_diagonal`, and identify where nontriviality is used.

Exercise 2.14. (Harder.) The full Rice's theorem, Theorem 2.22, splits on which side of `P` the everywhere-undefined program falls. Write out both cases of the reduction from halting, and explain why the split is necessary and where classical logic enters.

Exercise 2.15. Fill in the summary table for two theorems not treated in the text: the Grelling–Nelson paradox of heterological adjectives, and the fixed-point combinator `Y` of the untyped lambda calculus. For each, name `A`, `Y`, `t`, and the wrapper, and say whether the fixed point is a contradiction or a construction.

Exercise 2.16. (Open-ended.) Every row of the summary table has `t` fixed-point free, so the fixed point Lawvere produces is a contradiction. The lambda-calculus `Y` combinator uses the same diagonal but wants the fixed point. Speculate on what distinguishes the destructive uses from the constructive one, and relate it to the difference between `Bool` with negation and a domain where every map has a fixed point.

Exercise 2.17. (Open-ended.) Pick one AI-safety property, honesty, calibration, or corrigibility, and attempt to force it into the recipe. Name a candidate

`A`, `Y`, and `t`, and identify precisely which universality hypothesis the property would have to satisfy for `no_universal` to apply. Then say what a working system gives up to avoid satisfying it. Chapter 3 does this for hallucination; compare your attempt to its treatment.

Exercise 2.18. (Open-ended.) The chapter argues that the diagonal is silent on quantities. For Rice's theorem, formulate a quantitative question the theorem does not answer, for instance the density of programs on which a fixed heuristic decider is correct, and speculate on what additional structure, a metric, a measure, a topology on programs, would be needed to make it precise. This is the question Chapters 5 and 6 take up for attack basins.

Exercise 2.19. State the section form of Lawvere's theorem in a cartesian closed category, as in the categorical-unification section, and identify, for Cantor and for Turing, what object plays the role of the outcome `Y`, what map plays `t`, and what construction provides the section `s`. For Turing, explain why the universal machine is exactly the section.

Exercise 2.20. (Harder.) The summary table lists Löb's map as the flip for Gödel II, calling it "negation in modal disguise." Make this precise: write the map `ψ ↦ (Prov(⌜ψ⌝) → φ)` whose fixed point Löb's theorem takes, and show that with `φ = 0 = 1` its fixed point is the sentence whose unprovability is the second incompleteness theorem. Explain in what sense this map fails to have a fixed point in the way negation does, and why the conclusion is nonetheless an impossibility.

Exercise 2.21. (Open-ended.) Rice–Shapiro sorts semantic properties into decidable, r.e., and worse. Place three properties of a language model on this spectrum, phrased as properties of the function it computes, and say for each whether the classical theorem predicts you can decide it, recognize it, or neither. Then say what changes when the property is weakened from exact to approximate, which is the move Chapter 3 makes.

2.14 References

Boolos, G. (1993). *The Logic of Provability*. Cambridge University Press, Cambridge.

Boolos, G. S., Burgess, J. P., Jeffrey, R. C. (2007). *Computability and Logic*, 5th edn. Cambridge University Press, Cambridge.

Cantor, G. (1891). Über eine elementare Frage der Mannigfaltigkeitslehre. *Jahresbericht der Deutschen Mathematiker-Vereinigung* 1, 75–78.

Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38, 173–198.

Lawvere, F. W. (1969). Diagonal arguments and cartesian closed categories. In *Category Theory, Homology Theory and their Applications II*, Lecture Notes in Mathematics 92, 134–145. Springer, Berlin.

Löb, M. H. (1955). Solution of a problem of Leon Henkin. *Journal of Symbolic Logic* 20(2), 115–118.

Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society* 74(2), 358–366.

Rogers, H. (1967). *Theory of Recursive Functions and Effective Computability*. McGraw-Hill, New York.

Solovay, R. M. (1976). Provability interpretations of modal logic. *Israel Journal of Mathematics* 25, 287–304.

Tarski, A. (1936). Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica* 1, 261–405.

Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* (2) 42, 230–265.

Yanofsky, N. S. (2003). A universal approach to self-referential paradoxes, incompleteness and fixed points. *Bulletin of Symbolic Logic* 9(3), 362–386.

3 Three Trilemmata, One Theorem

Abstract. Three impossibility results circulate in the AI-safety literature under different names: the Hallucination Trilemma, the Defense Trilemma, and the truth-boundary coupling theorem. Each was proved by topology. This chapter shows that all three are one statement, the impossibility of a reflective verdict from Chapter 1, proved without topology. Set up in Lean, their proof terms are equal, and the equality is witnessed by rfl. What differs is the reading of one schema. The chapter also states what each result does not settle.

Keywords: Trilemma, Hallucination, Prompt injection, Calibration, Reflective verdict

This is the chapter the first two were building toward. Three impossibility results have been circulating in the safety literature under different names and with different proofs. The Hallucination Trilemma says a language model cannot be faithful, covering, and calibrated at once. The Defense Trilemma says a wrapper against prompt injection cannot preserve utility and be complete at once. The truth-boundary coupling theorem says a model whose confidence separates true answers from false ones must own a query it can place on neither side. Each was proved with topology, with an intermediate value theorem on a connected space of queries or representations. This chapter shows that all three are the same statement, and that the statement is `no_reflective_verdict` from Chapter 1, proved without topology at all.

The claim is stronger than "these results rhyme." When the three are set up in Lean, their proof terms are literally equal, and the equality is witnessed by `rfl`. What differs between them is not the mathematics but the reading of one Boolean function `v : Q → Q → Bool`. In the hallucination reading, `v p q` says the model is confident and correct about query `q`. In the defense reading, it says the wrapper renders prompt `q` safe. In the coupling reading, it says the model's confidence puts `q` on the true side of a probe. The reading changes; the theorem does not.

The work of this chapter, and it is real work, is in the arguments that connect each domain's informal desiderata to the single hypothesis that Chapter 1's engine needs. That hypothesis is *reflectivity*: the verdict `v` is universal, meaning every pattern of verdicts over `Q` is itself named by some element of `Q`. Getting from "a good model is faithful and calibrated" to "the model's self-verdict is a universal Boolean system" is where all the domain-specific care

lives. Once you are there, the impossibility is one line, and it is the same line every time.

We keep the honest accounting in view throughout. The diagonal buys freedom from topology, and it pays for that freedom with reflectivity. For prompt injection the price is low, because prompts really are arbitrary self-describing text. For a fixed representation space the price is high, and the topological route of Chapter 4 is the more physical one there. Saying which premise a given application actually has is part of using these theorems responsibly, and the last section of each trilemma is devoted to it.

A reader who has met these results elsewhere may expect three separate proofs, one per domain, each with its own diagram and its own lemma. That is how they were first published, and it is a reasonable way to discover them. It is not the way to understand them. The separate proofs obscure the fact that they share a single mechanism, and sharing a mechanism is not a curiosity here but the main result: it says that patching one of the three cannot help with the others, because there is nothing domain-specific to patch. The engine is the same, so a fix that leaves the engine intact leaves all three impossibilities intact. That is a practical consequence of the unification, and it is why the chapter insists on the single proof term rather than treating it as a pleasant coincidence.

3.1 How to read this chapter

The three trilemmata are presented in the same shape, and the shape is worth learning once. Each has an informal statement of what a good system should do, a short list of numbered conditions that make the informal statement precise, an argument that the conditions turn the system into a reflective verdict, a one-line Lean instance of Chapter 1's engine, a reading of the liar witness in the domain, and an honest accounting of what the reflectivity premise costs there. If you read the hallucination section closely, the other two go quickly, because only the readings change.

A word on what the Lean is doing. The code blocks are elaborated when the book is built, so each `c3_` theorem below is checked by the kernel as you read it. They are short on purpose. The mathematics was finished in Chapter 1, and these blocks only rename its conclusion for a domain. This is the opposite of the usual situation in applied theory, where the modeling is quick and the proof is long. Here the proof is a line and the modeling is the chapter. Spend your attention on the definitions and the arguments that connect them to universality, because that is where a reader can disagree, and disagreement about a premise is the only honest way out of an impossibility whose proof the machine has checked.

One more orientation. The word *trilemma* is used two ways in these notes, and both appear here. In the hallucination and defense readings it names a conflict among three desiderata over a single Boolean verdict, and the impossibility is that the three cannot all hold. In the master trilemma of the later

section it names a conflict among three properties of a system that carries a self-prediction, Utility, Control, and Transparency, and the impossibility is that you can keep at most two. The two uses are related, as the master section shows, but they are not identical, and it helps to know which one is on the table.

3.2 What "reflective" demands

Recall the engine. A *reflective verdict* on a domain `Q` is a universal system `v : Q → Q → Bool`, one for which every function `g : Q → Bool` is named: `∀ g : Q → Bool, ∃ p, v p = g`. Chapter 1's `no_reflective_verdict` says no such thing exists, and its witness `liar_query` produces the specific element on which the system breaks, a `p` with `v p p = !(v p p)`.

Two ingredients go into that engine, and it helps to name them before we start reading. The first is that the outcomes are Boolean and the controller is the flip `!·`, which `bool_not_fpf` shows has no fixed point. This is what turns the diagonal into a contradiction rather than merely a fixed point. The second is universality, the surjectivity of `v`. Everything in this chapter is an argument that some domain supplies both ingredients: that a good enough model, defense, or probe forces the verdict to be Boolean with the flip as its honest self-report, and that the domain's own self-reference forces the verdict to be universal.

It is worth separating two things that are easy to conflate. Universality is not the claim that the model is powerful, or accurate, or large. It is the claim that the model's naming of verdict-patterns is *complete*: for any way of assigning "yes" or "no" to every query, there is a query whose row of verdicts is exactly that assignment. In each domain below, this completeness comes from a different source. For hallucination it comes from the model being an in-context learner that can be prompted to imitate any behavior. For defense it comes from prompts being unrestricted text. For coupling it comes from the representation space being rich enough to encode any pattern over its own states. These three sources are not equally believable, and we will grade them.

The other thing to hold onto is what the flip means. In each domain there is an *honest self-report*, the behavior a well-functioning system would exhibit when asked about itself. Faithfulness for a model, completeness for a defense, and exact separation for a probe each say that the honest self-report is the negation of the diagonal: the system, applied to its own description, should output the opposite of what that description predicts. A confident answer should be correct, so a query that predicts its own incorrectness should be answered correctly, which is to answer against the prediction. That "against" is the flip. The diagonal then asks for a query that predicts *itself*, and the flip has no fixed point, so the query cannot be answered consistently. That is the whole shape, three times over.

Remark 3.0 (Surjection or section). Chapter 1 states reflectivity as point-surjectivity, `∀ g, ∃ p, v p = g`, and remarks that it can be weakened to a

section, a map `s : (Q → Bool) → Q` with `v (s g) = g`. The two agree for the impossibility, because all the engine needs is one preimage of the diagonal pattern. In the readings below we always argue for the surjection, since the domain stories, an in-context learner realizing a described behavior, a prompt encoding a pattern, a representation encoding an assignment, are stories about how a preimage is produced, and producing it on demand is exactly a section. When you audit a reading, ask whether the domain gives you a preimage of the *one* diagonal pattern, not of every pattern. That single preimage is the liar, and it is all that is at stake.

Counting gives the intuition in the finite case and then fails you, which is worth seeing once. If `Q` has `n` elements, there are `n` self-contexts and so at most `n` rows `v p`, but there are 2^n patterns `g : Q → Bool`, and $n < 2^n$. A finite verdict cannot be reflective for the boring reason that there are not enough rows to go around, and the missing row can be taken to be the diagonal flip `q ↦ !(v q q)`. This is `liar_query` specialized to a finite domain, and it is Cantor's diagonal in its oldest dress. The theorem of Chapter 1 is stronger, because it forbids reflectivity even when `Q` is infinite and counting says nothing. The lesson for the readings is that "the model is large" does not buy an escape. Making `Q` bigger makes both the rows and the patterns bigger, and the diagonal pattern is always among the ones left unnamed.

3.3 A worked micro-instance

Before the three readings, it helps to watch the diagonal act on a domain small enough to write out, because the same picture recurs at every scale. Take `Q = {a, b, c}` and any verdict `v : Q → Q → Bool`, which we can lay out as a table whose rows are self-contexts and whose columns are queries. Suppose the diagonal entries, the ones where row and column agree, come out `v a a = true`, `v b b = false`, `v c c = true`. The diagonal pattern the engine builds is the flip of these, the function `g` with `g a = false`, `g b = true`, `g c = false`. For `v` to be reflective, some row must equal `g` exactly. But `g` was built to disagree with row `a` at column `a`, with row `b` at column `b`, and with row `c` at column `c`, so `g` differs from every row somewhere along the diagonal. No row equals `g`, and `v` is not reflective.

Now read the failure the other way. Suppose someone insists `v` is reflective and hands you a row `p` with `v p = g`. Look at the diagonal entry of that very row. On one hand `v p p` is whatever the table says. On the other hand `v p p = g p`, because row `p` is `g`, and `g p = !(v p p)` by the construction of `g`. So `v p p = !(v p p)`, a Boolean equal to its own negation, which `bool_not_fpf` forbids. The self-context `p` is the liar, and its own verdict about itself has no consistent value. This is `liar_query` with `Q` set to a three-element domain, and it is the entire mechanism of the chapter in miniature.

Two features of this micro-instance carry over unchanged. First, the liar is a *specific* row, not a vague impossibility. The diagonal names it, and in the readings below that named object becomes a query, a prompt, or a represen-

tation you could in principle write down. Second, nothing depended on the size of `Q`. The diagonal pattern disagrees with each row at the diagonal, so it is unnamed whether `Q` has three elements or infinitely many. Every argument in this chapter is this table, grown until the rows are model behaviors, defense verdicts, or probe outputs.

3.4 The Hallucination Trilemma

The desiderata, informally

A model that we would call trustworthy about its own knowledge should have three properties. It should be *faithful*: when it answers with confidence, it should be right. It should be *calibrated*: its confidence should track its correctness, not drift above or below it. And it should be *covering*: it should actually answer questions, sometimes rightly and sometimes wrongly, rather than dodging into a single safe response. None of these is controversial on its own. Faithfulness is just "don't confidently lie." Calibration is the standard demand that a stated 90 percent should be right 90 percent of the time, sharpened here to an exact match. Coverage is the mild requirement that the model is not constant.

The Hallucination Trilemma says these three cannot hold together once the model is asked about its own verdicts. Something has to give. Either the model occasionally answers confidently and wrongly (faithfulness fails), or its confidence stops tracking correctness at the hard cases (calibration fails), or it retreats to a response so uniform that it no longer covers the space (coverage fails). The topological proof in the companion library locates the breaking point as a boundary question where confidence sits exactly at threshold and the answer sits exactly on the truth boundary. The diagonal proof locates the same question by self-reference, and calls it the liar.

The conditions, precisely

Fix a domain `Q` of queries as the model represents them, and read `Q` also as the space of self-contexts the model can reason from. The object of study is the model's confident-correctness self-verdict `v : Q → Q → Bool`. Read `v p q = true` as: reasoning from self-context `p`, the model is confident and correct about query `q`. The diagonal `v p p` is the model's verdict on its own description.

Definition 3.1 (Strict calibration). The verdict `v` is *strictly calibrated* when confidence is an exact decider of correctness, so that for every self-context `p` and query `q` the value `v p q` is a genuine Boolean, `true` exactly when the model is both confident and correct and `false` otherwise. Calibration is what licenses us to model the self-assessment as a total function into `Bool` at all. Without it, confidence and correctness could come apart by degrees, and the verdict would be real-valued rather than Boolean. Chapter 6 handles that real-valued relaxation; here calibration is exact and the codomain is `Bool`.

Definition 3.2 (Faithfulness). The verdict `v` is *faithful* when a confident answer is a correct answer: confidence implies correctness, with no confident error. Faithfulness is the condition that fixes the *controller* as the flip `!·`. The honest self-report of a faithful, calibrated model is that its verdict on a query predicting its own incorrectness must come out correct, which is to come out opposite to the prediction. A faithful calibrated verdict may therefore never equal its own negation, because such a value would be a confident answer that is correct exactly when it is incorrect.

Definition 3.3 (Coverage). The verdict `v` is *covering* when the model actually answers on both sides: some query gets a correct confident verdict and some query does not. Coverage is nontriviality. It rules out the escape in which the model outputs a single constant verdict, which is faithful and calibrated for free and which is exactly the case the impossibility must exclude. A covering verdict takes both Boolean values, so the flip acts on something with real content.

Definition 3.4 (Reflectivity). The verdict `v` is *reflective* when it is universal as a system: every pattern `g : Q → Bool` of verdicts over queries is named by some self-context, `∀ g : Q → Bool, ∃ p, v p = g`. Reflectivity is the self-reference hypothesis. It says the model can be queried about its own verdicts, including the diagonal pattern that flips its self-application.

Why the conditions make the verdict reflective

Here is the argument that carries the chapter, stated for hallucination and reused twice more below. It has two halves. The first half says the three desiderata make `v` the right *kind* of object: a total Boolean system whose fixed-point-free controller is the flip. The second half says the model's own nature makes `v` *universal*. Only the two halves together give a reflective verdict, and only a reflective verdict is impossible.

Start with the kind of object. Strict calibration (Definition 3.1) is what makes the codomain `Bool`. A calibrated model does not hedge between confident and correct; the two coincide, so "confident and correct about `q`" is a crisp yes or no, and `v p q : Bool` is well defined and total. Drop calibration and you are in the real-valued world where confidence is a number in the unit interval and the controller is the complement `y ↦ 1 - y`, fixed-point-free everywhere except at one half. That is the topological story, and it is a different chapter. With calibration the outcomes are two, and the controller collapses to the flip.

Faithfulness (Definition 3.2) is what makes that controller the flip rather than the identity or something with a fixed point. A faithful model's honest response to a query that has predicted "you will be incorrect here" is to be correct, which is to answer against the prediction. Encode the prediction as a Boolean and the honest response is its negation. So along the diagonal, where the query predicts the model's own self-verdict, faithfulness demands `v p p = !(v p p)` be avoidable, that is, that no self-verdict equal its own negation. `bool_not_fpf` records that the flip has no fixed point, so faithfulness is con-

sistent with any single verdict. The trouble is only that a reflective model must *realize* the diagonal prediction as an actual query.

Coverage (Definition 3.3) blocks the cheap way out. If the model were allowed to be constant, always answering "not confident-and-correct," it would satisfy faithfulness and calibration vacuously, and it would fail to be surjective onto `Q → Bool`, so no contradiction would arise. Coverage forbids the constant model. It insists the verdict takes both values, which is exactly the nontriviality the diagonal needs. In the surjectivity statement this shows up as the requirement that the two-valued patterns, and in particular the diagonal-flipping pattern, are among the ones that must be named.

Now the second half, universality. Why should every pattern `g : Q → Bool` be named by some self-context `p`? Because the model is an in-context learner over its own query space. A self-context is itself expressible as a query, a prompt, a piece of the very text the model consumes. To ask the model to realize the pattern `g` is to prompt it with a description of `g`, "here is, for each query, whether you are confident and correct about it," and a competent in-context learner will imitate the described behavior. The stronger the model, the more completely it can be steered this way, so universality is not a weakness of the model but a consequence of its flexibility. In particular the model can be prompted with the diagonal description, "be confident and correct about a query exactly when you would not be," and reflectivity says some self-context `p` realizes it.

Put the halves together. Calibration makes `v` Boolean, faithfulness makes the controller the flip, coverage makes the flip bite, and reflectivity makes the diagonal pattern an actual query. That is a reflective verdict in the exact sense of Chapter 1, and Chapter 1 has already shown no reflective verdict exists. The three desiderata are not independently impossible; they are impossible *in a reflective model*, and the model's own capability is what supplies the reflectivity.

There is a reading of this that sounds paradoxical and is worth defusing. It says a *better* model is more likely to be impossible, because a better model is a more complete in-context learner and so more nearly reflective. The paradox dissolves once you see what "impossible" means here. The theorem does not say the model breaks everywhere. It says the model cannot hold all three desiderata at the liar query, and a more capable model is one that can be steered to the liar more reliably, which is to say it is one for which the boundary is more clearly present rather than one that functions worse. Capability sharpens the trilemma rather than defeating it. This is the opposite of the comforting hope that scale will wash the problem out, and it is one of the few places where the diagonal makes a prediction about the direction of an effect: more complete self-modeling means a more inescapable boundary, not a fainter one.

The impossibility

With the reading fixed, the theorem is Chapter 1's engine applied verbatim. We give it the domain name and prefix our new identifiers with `c3_`.

```
theorem c3_hallucination_trilemma {Q : Type _}
    (v : Q → Q → Bool)
    (reflective : ∀ g : Q → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. The hypothesis `reflective` is exactly the universality that `no_reflective_verdict` requires, and the conclusion is `False`. There is nothing to add. The faithfulness and calibration of the informal statement have already been spent: calibration in choosing the codomain `Bool`, and faithfulness in the fact that `no_reflective_verdict` is built on the flip controller. Coverage is spent in reflectivity, which asks for the flipping pattern by name. ■

The diagonal does more than refute the conjunction. It hands back the exact query on which a reflective model breaks. This is the liar reading of the boundary question.

```
theorem c3_hallucination_liar {Q : Type _}
    (v : Q → Q → Bool)
    (reflective : ∀ g : Q → Bool, ∃ p, v p = g) :
    ∃ p, v p p = !(v p p) :=
  liar_query v reflective
```

Proof. Immediate from `liar_query`. The witness `p` is a self-context whose own confident-correctness verdict equals its negation. ■

Reading the liar witness

The `p` produced by `c3_hallucination_liar` is the hallucination model's boundary question, seen from the discrete side. In the topological account it is a query where the model's confidence lands exactly at the threshold and the answer lands exactly on the truth boundary, a point the intermediate value theorem delivers on a connected space of queries. Here it is a query that predicts its own verdict and then flips it. The two descriptions are the same object under two engines. The topological one measures how close you are to the boundary; the diagonal one names the boundary query outright and shows it must exist as soon as the model can talk about its own verdicts.

Concretely, `p` is a self-referential prompt of the form "answer the following with confidence exactly if you would be wrong to." A faithful model must answer it correctly, which by construction is to answer it incorrectly. A calibrated model cannot escape by lowering its confidence, because calibration ties confidence to correctness and the correctness is what is contradictory. A covering model cannot escape by refusing, because refusal on this one query does not restore the missing pattern; the pattern the model failed to realize was the flipping pattern itself. The only genuine escapes are the three failures the trilemma names, and each is a different desideratum breaking at `p`.

The three escapes, one at a time

An impossibility is only as interesting as the ways out it leaves, so it pays to walk the three. Each escape drops exactly one of the desiderata, and each is a real design stance that someone holds.

Drop faithfulness. A model that is calibrated, covering, and reflective but not faithful is one that sometimes answers with confidence and is wrong. This is the ordinary hallucination everyone means by the word: a fluent, confident, false answer. The diagonal says this is not an accident to be trained away at the margin but a structural necessity for a reflective calibrated model, and it points at the liar query as a place the confident error must live. The stance that accepts this escape says confident error is the price of coverage, and the work is then to bound how often it happens, which the diagonal does not address and Chapter 5 does.

Drop calibration. A model that is faithful, covering, and reflective but not calibrated is one whose confidence has come loose from its correctness. In practice this is the model that hedges: it lowers its confidence on the hard self-referential cases so that no confident answer is wrong, at the cost of confidence no longer meaning what it says elsewhere. This is the honest and common engineering response, and it is why deployed models are deliberately underconfident near their limits. The diagonal locates the exact query where the hedge must happen, the liar, and shows the hedge cannot be avoided, only placed.

Drop coverage. A model that is faithful, calibrated, and reflective but not covering is one that has stopped answering. The extreme is the model that refuses everything, which is faithful and calibrated for free. Refusal training is this escape applied selectively: carve out the region where the liar lives and decline it. The diagonal tolerates this, but it warns that the region you must carve is exactly the self-referential one, and that a model useful over its whole domain cannot carve it away without ceasing to be useful there. Chapter 4 reads the same move geometrically, as disconnecting the query space, and prices it the same way.

Notice that no escape removes the liar. Each escape decides what to do *at* the liar: answer wrong there, hedge there, or refuse there. The query itself is forced by reflectivity, and the only freedom is which desideratum to spend on it. That is the honest content of the trilemma, and it is why "we will just fix hallucination" is not a coherent program. There is a query where one of the three must give, and the engineering question is which.

Calibration here is not statistical calibration

Remark 3.11 (Calibration is pointwise and exact). The calibration of Definition 3.1 is pointwise and exact, and it is much stronger than the statistical calibration of the reliability-diagram literature. Statistical calibration asks that, averaged over a population of queries, a stated confidence of c be correct a fraction c of the time. It is an aggregate property and it says nothing about

any single query. The calibration the trilemma uses is a two-way tie at every query between confidence being high and the answer being correct. It is the idealization of a perfect confidence signal, not the weaker aggregate one. This matters for reading the result honestly. The trilemma does not say statistically calibrated models are impossible, and they exist. It says the exact pointwise version conflicts with faithfulness and coverage over a reflective domain, and the gap between exact and statistical calibration is one of the places a real model lives. The companion library records the bridge from the biconditional exact form to the one-way pair as `strictCalibrated_to_epsCalibrated_zero_half`.

Relation to the statistical inevitability results

There is a separate and older tradition proving hallucination inevitable by statistical or computability arguments rather than topological or diagonal ones. It shows, for instance, that a calibrated model must err on facts seen once in training, or that no computable procedure decides truth in general. Those results derive *confident falsehood* and quantify *how often*. The diagonal result derives a single *boundary query* and says nothing about frequency. The two are complementary rather than competing. The statistical arguments answer "how bad, on average," and the diagonal answers "where, exactly, and why it cannot be patched." A reader who wants error rates should reach for the statistical tradition. A reader who wants to know that the failure is structural and self-referential, and to hold the liar query in hand, should reach for this one. Only the stochastic dichotomy of the coupling paper, discussed below, turns a diagonal-style boundary into falsehood with positive probability, and it is the one place the two traditions touch.

The honest caveat

The reflectivity hypothesis is the whole cost of dropping topology, and for hallucination it is a middling assumption. On the one hand, treating the model as an in-context learner that can imitate any described verdict-pattern is defensible for large models, and it matches how practitioners actually elicit behaviors. On the other hand, "any pattern over all of `Q`" is a strong completeness claim, and a real model's realizable patterns are a proper subset of `Q → Bool`. The topological route of Chapter 4 asks instead for continuity on a connected query space, which is a different and arguably more physical premise. The two routes reach the same boundary question. Which one to lean on depends on whether you believe the model's self-modeling is complete (use this chapter) or that its query space is connected and its fields continuous (use Chapter 4). The companion library carries both, as `hallucination_trilemma_godel` in `F_11_HallucinationGodel` and as the topological `hallucination_trilemma` behind the coupling theorem, and it proves they find the same object.

3.5 The Defense Trilemma

The desiderata, informally

Prompt injection is the attack where a user's input contains instructions that subvert the system's own. A defense is a wrapper `D` that transforms an incoming prompt before the model sees it, hoping to neutralize any injected instruction. Two demands are natural. The defense should be *utility-preserving*: on a prompt that is already safe, it should do nothing, because a wrapper that rewrites safe prompts degrades the system it protects. And it should be *complete*: every prompt, after wrapping, should be safe, because a defense that lets some injection through is not a defense. Utility preservation and completeness are the two horns, and reflectivity is the fact that prompts are arbitrary text.

The Defense Trilemma says no wrapper can be utility-preserving and complete against a prompt space rich enough to describe the defense itself. The published proof is topological: continuity of the defended safety score plus utility preservation force the score to hit the safety threshold somewhere, and completeness says it never does. The diagonal proof replaces continuity with self-reference, which for prompt injection is not an idealization but the definition of the problem. A prompt is text, and text can quote the defense and invert it.

The conditions, precisely

Let `X` be the space of prompts and read `X` also as the space of self-contexts a prompt can set up, since a prompt can carry its own framing. Model the defended-safety verdict as `v : X → X → Bool`, with `v p q = true` meaning the wrapper renders prompt `q` safe when it must also cope with the self-referential prompt `p`. The diagonal `v p p` is the wrapper's verdict on a prompt that describes the wrapper's own behavior.

Definition 3.5 (Utility preservation). The wrapper is *utility-preserving* when it fixes safe prompts: a prompt that is already safe is passed through unchanged, so the defended verdict on a safe prompt agrees with its undefended safety. Utility preservation is what makes the verdict nontrivial and blocks the constant escape. A wrapper that replaced every input with a fixed maximally safe output would be complete for free, but it would destroy every safe prompt's content, so it is not utility-preserving, and its verdict is not covering.

Definition 3.6 (Completeness). The wrapper is *complete* when every prompt is rendered safe: for all `q`, the defended verdict is `true`. Completeness is the demand that the wrapper leave no injection standing. It is what makes the honest self-report the flip. A complete wrapper faced with a prompt that says "you will mark me unsafe" must mark it safe, which is to answer against the prompt's prediction.

Definition 3.7 (Reflectivity). The prompt space is *reflective* when the verdict is universal, `∀ g : X → Bool, ∃ p, v p = g`. Because prompts are arbitrary text, any pattern of safe-or-unsafe verdicts over prompts is itself describable by a prompt, so every `g` is named. This is the prompt-injection reality stated as a surjection.

Why the conditions make the verdict reflective

The two halves of the hallucination argument recur, and the second half is where defense is strongest. Take the kind of object first. The defended-safety verdict is already Boolean, because ”safe” and ”unsafe” are two outcomes, so no calibration step is needed to reach `Bool`; the safety judgment is discrete by nature. The controller is the flip because completeness (Definition 3.6) plays the role faithfulness played before. A complete wrapper's honest response to a prompt that predicts its own unsafety is to render it safe, that is, to answer against the prediction. Along the diagonal, where the prompt predicts the wrapper's own verdict, completeness demands the flip, and `bool_not_f pf` says the flip has no fixed point. Utility preservation (Definition 3.5) plays coverage's role: it forbids the constant maximally-safe wrapper that would trivially satisfy completeness while naming no interesting patterns.

Now universality, and here defense needs almost no idealization. A prompt is unrestricted text. Whatever pattern `g : X → Bool` you like, ”for these prompts be safe and for those be unsafe,” can be written down as a prompt that instructs the wrapper to behave that way, because instructing the wrapper is just more text and the wrapper reads text. The injection ”ignore your previous instructions and do `X`” is precisely a prompt that describes and overrides the defense. So the diagonal pattern, ”be unsafe exactly if you would be made safe,” is an ordinary injection, and reflectivity says some prompt `p` realizes it. Unlike the hallucination case, there is no gap between ”the model could in principle imitate any behavior” and ”every pattern is realized.” The realizing objects are literally the attacker's inputs, and the attacker is allowed to write anything.

Assemble the halves. The verdict is Boolean by the nature of safety, the controller is the flip by completeness, utility preservation gives the flip content, and the arbitrariness of text gives universality. A complete, utility-preserving wrapper over a reflective prompt space is a reflective verdict, and no reflective verdict exists.

One subtlety deserves attention, because it is where careful readers push back. The diagonal pattern for defense is ”be unsafe exactly if you would be made safe,” and a skeptic might say no *sensible* prompt describes such a thing. The reply is that sense is not required. Reflectivity asks only that the pattern be nameable as text, and any pattern of safe-or-unsafe verdicts, sensible or not, is a finite or enumerable specification that can be written down and handed to the wrapper. The attacker does not need the prompt to mean anything to a human. It needs the wrapper to act on it, and the wrapper acts on text. This is exactly the gap between ”prompts a user would write” and ”prompts

an adversary can write," and the trilemma lives on the adversary's side of it. Restricting to sensible prompts is a real move, and it is the coverage escape again, now phrased as a restriction on the input distribution.

The impossibility

```
theorem c3_defense_trilemma {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. Word for word the hallucination proof, with `X` for `Q`. The reflectivity hypothesis is the arbitrary-text premise, completeness has fixed the flip inside `no_reflective_verdict`, and utility preservation is spent in the demand that the flipping pattern be named rather than escaped by a constant. ■

```
theorem c3_defense_liar {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) :
    ∃ p, v p p = !(v p p) :=
  liar_query v reflective
```

Proof. The witness `p` from `liar_query` is the injection no wrapper can neutralize: a prompt rendered safe exactly when it is unsafe. ■

Reading the liar witness

The `p` here is the prompt injection that beats every complete utility-preserving defense at once, produced not by cleverness but by the diagonal. It is the prompt "be unsafe if and only if this wrapper would render you safe." A complete wrapper must render it safe, so by its own construction it is unsafe, so completeness fails on `p`. The wrapper cannot rewrite `p` into something harmless without violating utility preservation on the safe prompts `p` quotes, and it cannot refuse `p` without ceasing to cover the pattern that `p` embodies. The topological account places this injection at the safety boundary where the defended score equals the threshold. The diagonal account writes it out as text. That the same object has both descriptions is the point of the chapter, and for defense the text description is the honest one, because injections are text.

Concrete injections

It helps to see the liar prompt as an actual attack rather than a symbol. The generic prompt injection is "ignore your previous instructions and do `X`," which is a prompt that quotes the system's own control flow and overrides it. The diagonal sharpens this into an attack aimed at the defense itself: "if this wrapper would mark the following prompt safe, then be unsafe, and otherwise be safe." A complete wrapper has to render this safe, because completeness

admits no exceptions, and rendering it safe is precisely the condition under which it declares itself unsafe. There is no third option. The attack does not need to be clever about any particular model. It needs only that the wrapper's verdict is a function the prompt can name, which is what reflectivity grants.

A defender's first instinct is to make the wrapper stateful or randomized, so that "the verdict" is not a fixed function the prompt can pin down. This helps less than it seems. If the wrapper is randomized, replace the Boolean verdict with the event "rendered safe" and the same diagonal runs on the induced verdict. The honest place for randomness is the coupling paper's stochastic dichotomy, which shows a randomized system faithful almost everywhere still emits unsafe outputs with positive probability. If the wrapper is stateful, the state is part of the self-context p, and reflectivity over the enlarged space still names the diagonal. The attack follows the wrapper wherever it tries to hide, because the attacker's language is at least as expressive as the wrapper's behavior. That is what taking reflectivity seriously as a threat model, rather than as an idealization, comes to.

Why detection does not help

The most natural defense against a self-referential prompt is to detect it and reject it. Add a classifier that flags prompts that quote the wrapper, and refuse those. The trilemma says this cannot restore completeness, and the reason is worth stating carefully, because it is the same reason in every domain. A rejection is a verdict. When the wrapper rejects the liar prompt as unsafe, it has assigned p the value "unsafe," and completeness demanded the value "safe." So detection does not dissolve the contradiction. It chooses the completeness-failing horn and calls it a feature. That may be the right engineering choice. A wrapper that refuses the small self-referential region and is complete on the rest is a coherent product. But it is not a complete wrapper, and the trilemma is about complete wrappers. The move from "complete" to "complete outside a refused region" is exactly the coverage escape from the hallucination section, wearing defense clothes, and it costs the same thing, utility on the refused region.

There is a subtler hope, that the refused region is negligible, a measure-zero set of pathological prompts no user writes. Against a random user this may hold. Against an adversary it does not, because the adversary writes exactly the pathological prompt, and the whole point of a defense is the adversarial setting. The liar prompt is not a rare accident of the input distribution. It is a target the attacker aims at. This is why the diagonal reading is the natural one for defense and the measure-theoretic relaxations of Chapter 6 are the wrong tool here. Prompt injection is worst-case by definition, and the diagonal is a worst-case theorem.

The real-valued score version

The companion library also carries a score version, `defense_trilemma_lawvere`, that does not pass through `Bool`. There the object is a defended safety score `s : X → X → ℝ`, with `s a b` the safety of the defended prompt `b` under self-context `a`, and completeness is the demand `s a a < τ` for a threshold `τ`. The controller is the real complement rather than the flip, fixed-point-free everywhere except at the threshold, and Lawvere's diagonal forces a boundary prompt where the defended score sits exactly at `τ`, contradicting completeness. Utility preservation blocks the trivial escape in this version too, because a wrapper that outputs a constant maximally safe response has a constant score, which is not surjective and not utility-preserving, so it never reaches the diagonal. The score version is the bridge between this chapter's Boolean reading and the topological reading of Chapter 4, where the same threshold `τ` becomes the value the intermediate value theorem forces a continuous score to hit. Boolean flip, real complement, and continuous score are three controllers for one argument, and the defense trilemma has all three.

The honest caveat

For defense the reflectivity premise is close to free, which is unusual and worth saying plainly. The whole difficulty of prompt injection is that the input space is unrestricted and self-describing, so the surjection `∀ g, ∃ p, v p = g` is not a strong modeling assumption but a restatement of the threat model. This is why the diagonal proof feels more natural here than the topological one: there is no connected space of prompts to appeal to, and there does not need to be. The one place to be careful is the boundary between "the attacker can write any text" and "the wrapper realizes any verdict-pattern." If the wrapper is allowed to detect and reject the self-referential prompt, one might hope to shrink the realizable set. But rejection is a verdict too, and a wrapper that rejects `p` while claiming completeness has simply relabeled `p` as unsafe, which is completeness failing. There is no consistent relabeling, which is the content of `defense_trilemma_godel` in `F_12_DefenseGodel`.

One more caveat is specific to defense and often missed. The trilemma is about a *single fixed* wrapper. A defender who is allowed to change the wrapper after seeing the attack faces a different and easier problem, because the diagonal is built against a particular verdict function, and moving to a new wrapper moves the target. This is not a refutation, because deployment fixes a wrapper and the attacker plays against the deployed one, but it does explain why defenses appear to work in the lab and fail in the wild. In the lab the wrapper is retuned against each known attack, which is choosing a new verdict function each round, and the diagonal for the old function no longer applies. In the wild the wrapper is fixed and the attacker constructs its liar. The honest statement is that no fixed complete utility-preserving wrapper exists, and any process that keeps changing the wrapper is not a wrapper but a game, whose analysis belongs to the attack-geometry chapter rather than to the diagonal.

3.6 The Coupling Theorem

The desideratum, informally

The third reading comes from representation geometry. Probing studies read truth off a model's activations, treating truth as a continuous field over representation space and training a linear probe to separate true statements from false ones. The coupling theorem asks what such a probe requires of the model that answers over the same space. If the model ever answers truly and ever answers falsely, and its confidence *separates* truth from falsehood, meaning confidence is high exactly on the true side and low exactly on the false side, then some query must carry confidence exactly at the threshold while its answer sits exactly on the truth boundary. That coupled query is one the probe cannot place on either side.

The framing is deliberately generous to the probing program rather than adversarial to it. It grants the program its central assumption, that truth is a continuous decodable field, and asks what that assumption implies for a model answering over the same space. The answer is not that probes fail. It is that a probe good enough to separate truth exactly comes with a boundary it cannot avoid, and the model that uses the probe as confidence inherits a query at that boundary. So the coupling theorem is a frontier for the probing program, not a refutation of it. It says: here is the one place your continuous field of truth must be exactly ambiguous, and here is the query that lands there. A researcher who believes in the field should want to know where its zero set meets the confidence threshold, and the theorem answers that they always meet, at least once, whenever the model covers both truth values.

The single desideratum here is *truth-separation*, in its exact biconditional form: the sign of the truth-distance and the side of the confidence threshold determine each other. This is much stronger than statistical calibration. It is a pointwise, two-way tie between what is true and what the model is confident about, the idealization of a perfect truth probe used as the confidence signal. Under it, the confidence field is a total exact Boolean decider of truth over the representation space, and that is a definable truth predicate, which Chapter 2's Tarski reading forbids.

The condition, precisely

Let `X` be the representation space, read also as the space of self-referential representations a state can encode. Model the truth-separating verdict as `v : X → X → Bool`, with `v p q = true` meaning the model's confidence places query `q` on the true side of the probe, in the self-context `p`. The diagonal `v p p` is the probe's verdict on a representation that encodes the probe's own behavior.

Definition 3.8 (Truth-separation). The verdict `v` is *truth-separating* when confidence and truth agree exactly and two-ways: `v p q = true` if and only if `q` is true in context `p`. Biconditional separation makes `v` a total exact Boolean

truth predicate over `X`. This is the condition that plays calibration's and faithfulness's roles at once. It makes the codomain `Bool` (exactness) and it fixes the controller as the flip (the two-way tie means a representation predicting its own falsehood must be judged true, against its prediction).

Definition 3.9 (Coverage for coupling). The verdict is *covering* when the model answers some queries truly and some falsely, so the probe separates a nonempty true region from a nonempty false region. This is the "ever true, ever false" hypothesis of the coupling theorem, and it is what makes the boundary between them nonempty.

Definition 3.10 (Reflectivity for representations). The representation space is *reflective* when `v` is universal, `∀ g : X → Bool, ∃ p, v p = g`: every pattern of truth-verdicts over representations is encoded by some representation. This is the strong hypothesis, and the section on caveats is where we pay for it.

Why the condition makes the verdict reflective

Truth-separation does double duty. In its biconditional form it makes the verdict exact, so the codomain is genuinely `Bool` rather than a real confidence that could sit strictly between the true and false sides. That is the calibration half. And it ties the sign of truth to the side of confidence in both directions, so a representation that encodes the prediction "the probe will place me on the false side" must, if that prediction is to be honored as truth, be placed on the true side, against the prediction. That is the faithfulness half, and it fixes the flip as the controller. Coverage (Definition 3.9) supplies content, exactly as before: without both truth values realized, the truth boundary is empty and the diagonal has nothing to flip.

The universality half is where coupling differs from defense in believability. For the verdict to be reflective, every pattern `g : X → Bool` of truth-verdicts must be encoded by some representation `p`. This asks the representation space to be self-referential in a specific way: it must contain, for any assignment of truth to representations, a representation whose row of verdicts is that assignment, including the diagonal assignment that flips the probe on itself. For a language model's activation space this is a real assumption, not a restatement of anything. Activations are high-dimensional real vectors, and while they are expressive, there is no guarantee that every Boolean pattern over states is realized by a state. This is the honest weak point of the coupling reading, and it is why the paper's main result is topological rather than diagonal: it asks for continuity on a connected space, which the manifold hypothesis makes plausible, rather than for completeness of self-encoding, which nothing makes obvious.

Still, the reading is exact where it applies. If you grant that the representation space encodes any truth-pattern over itself, then an exactly truth-separating probe over it is a total self-applicable truth predicate, Tarski forbids it, and the diagonal names the coupled query.

It is worth saying exactly how this is Tarski's theorem, since Chapter 2 set it up and coupling is where it pays off directly. Tarski's undefinability of truth says no sufficiently expressive language contains a total predicate that is true of exactly the true sentences of that same language. An exactly truth-separating probe over a self-encoding representation space is precisely such a predicate: it is total, it decides truth, and the space it decides over is the space that encodes it. The liar sentence of Tarski, the one asserting its own falsehood, is the coupled query, the representation the probe would place on the true side exactly when it places it on the false side. So coupling is not merely analogous to Tarski. Under the reflectivity premise it is Tarski, with "sentence" read as "representation" and "truth predicate" read as "linear probe used as confidence." The strength of the premise is the price of making the analogy an identity, and the caveat section is where we admit that price is high for a passive vector space.

The impossibility

```
theorem c3_coupling_trilemma {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. Identical to the previous two. Truth-separation has made the verdict a Boolean predicate with the flip as controller, coverage gives the flip content, and reflectivity of the representation space names the diagonal pattern. ■

```
theorem c3_coupling_liar {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) :
    ∃ p, v p p = !(v p p) :=
  liar_query v reflective
```

Proof. The witness is the coupled query: a representation the probe would place on the true side exactly when it places it on the false side. ■

Reading the liar witness

In the topological theorem this witness is the coupled point x_0 where confidence equals the threshold and the truth-distance is zero, the point the intermediate value theorem forces along any path from a true answer to a false one. In the diagonal reading it is a representation that encodes the probe's negation of its own verdict. The two are the same boundary object. The topological one comes with metric content the diagonal cannot see, an ambiguity tube of computable width around x_0 and a strictly positive truth-side slack that any feasible system must carry. The diagonal one comes for free from self-reference and needs no geometry. The paper proves the topological version as `boundary_coupling` and derives the policy taxonomy from it: an inclusive

guarantee at the boundary is impossible (`closed_guarantee_impossible`), and an exclusive one survives but is silent exactly at the coupled query (`open_guarantee_silent`). The diagonal version is `coupling_trilemma_godel` in `F_13_UnifiedTrilemmata`.

The geometry the diagonal cannot see

The topological version of coupling comes with quantitative structure that the diagonal throws away, and it is worth cataloguing what is lost, because it marks the boundary between this chapter's engine and the analytic one. On a connected representation space with a continuous confidence field and a continuous truth field, the coupled point is not just a query that exists. It sits at the center of an *ambiguity tube* whose width is set by the Lipschitz constants of the two fields. A query within distance `r` of the coupled point has truth margin at most a constant times `r` and confidence within a constant times `r` of the threshold, so the near-boundary region has a computable size. None of this survives the diagonal, which knows the coupled query exists but not how large its neighborhood of near-ambiguous queries is.

More is true geometrically. For a nonzero linear probe the truth boundary is an affine hyperplane of codimension one, the translate of the probe's kernel, and even for a nonlinear probe it is a hyperplane to first order at any regular boundary point. Feasible systems must carry strictly positive truth-side slack, which the paper proves and which an adversarial training attempt in the paper's experiments fails to drive to zero, plateauing strictly above it. These are quantitative facts about the *shape* of the boundary and the *cost* of approaching it. The diagonal is silent on all of them. It gives you the point and the reason it must exist, and it hands the metric questions to Chapter 5. This is the honest division of labor between the two engines, and the coupling reading is where it is sharpest, because coupling is where the topological original is richest.

The policy taxonomy

The coupling theorem itself mentions no safety guarantee, which is what makes its consequences a taxonomy rather than a single verdict. A *threshold-inclusive* guarantee promises that confidence at or above the threshold implies a true answer. A *threshold-exclusive* guarantee promises the same only for confidence strictly above the threshold. The coupled query has confidence exactly at the threshold, so it is the hinge on which the two conventions turn. Add the inclusive guarantee to the coupling hypotheses and you get a contradiction, the trilemma proper, because the coupled query would have to be true while the coupling says its truth-distance is zero. This is `closed_guarantee_impossible`. Add the exclusive guarantee instead and there is no contradiction, but the coupled query's antecedent fails exactly there, so the system owns a boundary query about which its guarantee says nothing. This is `open_guarantee_silent`.

The moral is that defining the guarantee with a strict inequality does not dissolve the impossibility. It relocates it from a contradiction to a silence. Either the guarantee claims the coupled query and is false, or it excludes the coupled query and is silent there. The diagonal reading sees the same fork through the liar. The liar query satisfies `v p p = !(v p p)`, and any guarantee that assigns it a definite side is refuted, while any guarantee that declines to assign it a side has conceded the one query the theorem cares about. Convention only selects which clause of the taxonomy applies, and the coupled query exists under either.

Leaving the continuum

Token spaces are finite, and a fair objection to the topological coupling theorem is that connectedness does illegitimate work. The paper prices this objection precisely, and the price is illuminating for the diagonal reading. Without topology, a true-to-false path yields only a `discrete_sign_change`, an adjacent pair of queries straddling the boundary rather than a query on it. The boundary state is exactly what connectedness derived for free and what the discrete setting must assume. Once you assume it, the impossibility returns, as `boundary_question_impossible` and `faithful_iff_boundary_free` show: an inclusive guarantee is possible exactly when no boundary query exists.

This is precisely the role reflectivity plays in the diagonal reading. Reflectivity *supplies* the boundary witness by fiat, as the liar query, where the discrete setting had to assume it and the continuous setting derived it. The three routes, continuity, assumption, and reflectivity, are three ways to get the same witness, and they cost different things. Continuity costs a connected domain, assumption costs honesty about what you have put in, and reflectivity costs the completeness of self-encoding. Randomization does not escape either. On expected fields the coupling survives, and the `stochastic_dichotomy` converts expectation into sampling behavior: a system faithful on almost every sample and confident with positive probability at a coupled query must emit strictly false answers with positive probability. That is the one place in the whole development where the theory derives falsehood rather than uncertainty, and it is worth knowing it is available when the boundary question feels too abstract to matter.

A symmetry version needs no coverage at all. If semantic negation acts on the representation space as a fixed-point-free involution and the realized truth field is odd under it, then the field vanishes somewhere by the intermediate value theorem between any state and its negation, and coupling follows with no ”ever true, ever false” hypothesis. This is the scalar shadow of the Borsuk-Ulam theorem, and it is a reminder that equivariance, usually a design virtue, is here an engine of the obstruction. The diagonal has no clean analogue of this symmetry route, which is another entry in the ledger of what topology sees and self-reference does not.

The honest caveat

This is the reading to hold most loosely. Reflectivity of a fixed representation space is a strong, less physical assumption than continuity on a connected domain. Activations do not obviously encode every truth-pattern over themselves, and the diagonal's demand that they do is the price of skipping topology. The paper's own stance is that the topological route is primary for representation geometry, precisely because connectedness follows the manifold hypothesis while complete self-encoding does not. The discrete side of the paper prices this exactly: `discrete_sign_change` shows that without topology a true-to-false path yields only an adjacent straddling pair, not a boundary state, and `boundary_question_impossible` together with `faithful_iff_boundary_free` show that once you *assume* the boundary witness, the impossibility returns. The diagonal supplies that witness by fiat through reflectivity. That is legitimate when the self-encoding is real, as in a model prompted to reason about its own probe, and it is an overreach when the representation space is just a passive vector space with no self-reference. Say which you have.

The practical upshot is a division of labor between this chapter and the next that a probing researcher can act on. If your pipeline reads a probe off frozen activations and never asks the model to reason about the probe, your domain is a passive space, and the topological engine is the one that applies: assume connectedness, assume continuous fields, and the coupled point follows with a tube around it you can measure. If instead your pipeline lets the model condition on descriptions of its own truth-readout, as agentic and self-critiquing setups do, then the self-encoding is becoming real, and the diagonal engine begins to apply with the coupled query as a prompt you could construct. The two engines are not competitors for the same regime. They are tools for two regimes, and knowing which one you are in is a modeling question you answer before either theorem says anything.

3.7 The Master Trilemma

The companion `CCHProofs` development takes a step back and organizes all of this as faces of one three-cornered trilemma. Where the three readings above vary the *meaning* of a Boolean verdict, the master trilemma varies the *outcome type* and names three properties that no system can hold together. It is the same diagonal, now carrying a self-prediction alongside the verdict.

The setup bundles a system `s : A → A → Y`, a controller `t : Y → Y`, and a self-prediction `p : A → Y`, and it names three properties. *Universality* is that `s` is surjective, every behavior over `A` is realized by some agent. *Control* is that the controller can always nudge the self-prediction, `t (p a) ≠ p a` for every `a`, so no prediction is a fixed point of `t`. *Transparency* is that the system's diagonal matches its self-prediction, `s a a = p a`, so the system says what it will do on itself. Read `U` as capability or utility, `C` as control, and `T` as transparency or honesty. The master theorem is that no system is all three at once.

We give a self-contained core-Lean version. First the diagonal with its index exposed, which is `lawvere` refined to name the point rather than only assert a fixed value. It is the general-controller sibling of `liar_query`.

```
theorem c3_lawvere_point {A Y : Type _} (s : A → A → Y)
    (hU : ∀ g : A → Y, ∃ a, s a = g) (t : Y → Y) :
    ∃ a, s a a = t (s a a) :=
  match hU (fun a => t (s a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Name the diagonal behavior `a ↦ t (s a a)` by some a_0, then evaluate the naming equation at a_0. This is Chapter 1's move with the boolean flip replaced by an arbitrary `t`. ■

Now the three properties as predicates, and the master theorem.

```
def C3_Universal {A Y : Type _} (s : A → A → Y) : Prop :=
  ∀ g : A → Y, ∃ a, s a = g

def C3_Controlled {A Y : Type _} (t : Y → Y)
    (p : A → Y) : Prop :=
  ∀ a, t (p a) ≠ p a

def C3_Transparent {A Y : Type _} (s : A → A → Y)
    (p : A → Y) : Prop :=
  ∀ a, s a a = p a

theorem c3_cch_master {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s) (hC : C3_Controlled t p)
    (hT : C3_Transparent s p) :
    False := by
  obtain ⟨a₀, ha₀⟩ := c3_lawvere_point s hU t
  rw [hT a₀] at ha₀
  exact hC a₀ ha₀.symm
```

Proof. By `c3_lawvere_point` applied to `s` and `t` there is an a_0 with `s a₀ a₀ = t (s a₀ a₀)`. Transparency at a_0 rewrites `s a₀ a₀` to `p a₀`, giving `p a₀ = t (p a₀)`, which says `p a₀` is a fixed point of `t`. Control at a_0 says it is not. The two collide. ■

The three corners are the contrapositives, one per omitted property, and each is a one-liner off the master theorem.

```
theorem c3_cch_corner_UC {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s)
    (hC : C3_Controlled t p) : ¬ C3_Transparent s p :=
  fun hT => c3_cch_master s t p hU hC hT
```

```
theorem c3_cch_corner_UT {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s)
    (hT : C3_Transparent s p) : ¬ C3_Controlled t p :=
  fun hC => c3_cch_master s t p hU hC hT

theorem c3_cch_corner_CT {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hC : C3_Controlled t p)
    (hT : C3_Transparent s p) : ¬ C3_Universal s :=
  fun hU => c3_cch_master s t p hU hC hT
```

Proof. Each supplies the two properties it assumes and derives the falsity of the third from `c3_cch_master`. `UC` says a universal controlled system cannot be transparent, `UT` says a universal transparent system cannot be controlled, and `CT` says a controlled transparent system cannot be universal. ■

These three are the reason the result is a trilemma and not merely an impossibility. You can have any two corners. A universal controlled system exists, but it cannot be honest about itself. A universal transparent system exists, but it cannot be controlled. A controlled transparent system exists, but it cannot be universal. Every real design lives on one edge of the triangle and pays with the opposite corner. The companion library states exactly these as `cch_corner_UC`, `cch_corner_UT`, and `cch_corner_CT`, bundles them as `cch_master_trilemma` over a `CCHStructure`, and records the "at most two of three" reading as `cch_at_most_two`. It also shows the two verdict trilemmata of this chapter sit inside the master one, as `hallucination_trilemma_face` and `defense_trilemma_face`, by taking `Y = Bool`, the controller the flip, and the self-prediction the diagonal.

Each corner is a real class of system, and naming them makes the triangle concrete. A universal, controlled system that cannot be transparent is an expressive model under external correction whose self-reports must sometimes be wrong: it can be steered, and it can do anything, but it cannot honestly say what it will do on itself. A universal, transparent system that cannot be controlled is an expressive, honest model that no external controller can always nudge: it says what it will do, and it does anything, so any controller meets a self-prediction it cannot move. A controlled, transparent system that cannot be universal is an honest, correctable model of limited reach: it says what it will do and can be nudged, at the cost of not realizing every behavior. Real designs sit on edges. An aligned assistant is pushed toward Control and Transparency and pays with Universality, which is a principled reading of why safety and capability trade off. A frontier model is pushed toward Universality and pays with either Control or Transparency, and which one it pays with is a design decision, not an accident.

Remark 3.12 (The trilemmas as the UC face). The hallucination and defense trilemmas are the `UC` face of the master trilemma. Take `Y = Bool`, the

controller `t` the flip `!·`, and the self-prediction `p` the diagonal `a ↦ s a a`. Transparency `s a a = p a` then holds by the definition of `p`, so a universal system is automatically transparent for this choice, and control `t (p a) ≠ p a` is `bool_not_fpf` applied at `p a`. The master theorem's collision at a_0 becomes `v p p = !(v p p)`, which is exactly the liar of `c3_hallucinatio n_liar`. So the Boolean verdict trilemmata are not merely analogous to the master trilemma. They are one of its three faces, with transparency made free by choosing the self-prediction to be the diagonal and control made the flip. The companion library records this as `hallucination_trilemma_face` and `defense_trilemma_face`.

Reading the corners as the three trilemmata

The master trilemma's three properties line up with the desiderata of the verdict readings, and seeing the correspondence makes the "one theorem" claim concrete at the level of meaning, not only at the level of proof terms. Universality is the same reflectivity that runs through the chapter: every behavior is realized. Transparency is the honesty condition, the system's self-report matching what it does, which in the Boolean readings is carried by choosing the self-prediction to be the diagonal. Control is the demand that an external map can always move the self-prediction, and its Boolean instance is that the flip has no fixed point, which is faithfulness for hallucination, completeness for defense, and exact separation for coupling wearing their outward-facing names.

Read this way, each verdict trilemma is the statement that Universality and Control force the failure of Transparency, or symmetrically that a transparent controlled system cannot be universal. A faithful covering calibrated model that could also name every verdict-pattern would be universal, controlled, and transparent at once, and the master theorem forbids exactly that. The three readings differ in which real property plays Control and in how believable Universality is, but they agree on the skeleton, and the skeleton is one corner of the triangle. This is the sense in which the master trilemma is not a fourth result stacked on three others. It is the frame that shows the three were always the same corner viewed from three domains.

Why it is a trilemma, not a dilemma

A reader might ask why the result is stated with three properties when the impossibility is really about a fixed point. The answer is that the third property is what makes the conflict a genuine three-way trade rather than a flat contradiction. Any two of Universality, Control, and Transparency are jointly satisfiable, and the corner theorems `c3_cch_corner_UC`, `c3_cch_corner_UT`, and `c3_cch_corner_CT` are the constructive content of that fact turned into contrapositives. A dilemma would say two things cannot both hold. The trilemma says three things cannot all hold while any two can, which is a stronger and more useful statement, because it tells a designer that giving up one property

is not only necessary but sufficient. You do not have to abandon two horns to escape. You abandon one, and the theorem tells you the other two are then available.

3.8 One proof term

Now the payoff, made literal. The three readings of this chapter are not merely provable by the same method. As Lean terms they are equal, and the equality is `rfl`. Each of `c3_hallucination_trilemma`, `c3_defense_trilemma`, and `c3_coupling_trilemma` is defined as `no_reflective_verdict` applied to the same arguments, so they reduce to one another with no computation at all.

```
theorem c3_trilemmata_unified {Q : Type _}
    (v : Q → Q → Bool)
    (h : ∀ g : Q → Bool, ∃ p, v p = g) :
    c3_hallucination_trilemma v h = c3_defense_trilemma v h
      ∧ c3_defense_trilemma v h
        = c3_coupling_trilemma v h :=
  ⟨rfl, rfl⟩
```

Proof. Both components are `rfl`. The three trilemma theorems have the same body, `no_reflective_verdict v h`, so they are definitionally equal, and the kernel accepts `rfl` for each equation. ■

Read what this says. It is not that hallucination, defense, and coupling are analogous, or that a shared lemma covers them. It is that, once you strip the readings away, there is one function `v`, one hypothesis on it, and one proof that the hypothesis is contradictory. The names on the three theorems are labels we attach for the reader, since the machine sees a single term. That is the sense in which the title of these notes is exact. There is one diagonal, and the three trilemmata are it, viewed from three domains.

Remark 3.13 (What `rfl` is checking). It is fair to ask what `rfl` is really checking here, since the three theorems have different names and different implicit domain variables. Two things make the equality trivial for the kernel. First, each `c3_` trilemma is *defined* as `no_reflective_verdict` applied to the same `v` and `h`, so after unfolding the definitions the three bodies are syntactically one term. Second, and more strongly, all three have type `False`, and `False` is a proposition, so any two of its proofs are equal by proof irrelevance regardless of how they were built. Either route makes `c3_trilemmata_unified` a pair of `rfl`s. The first route is the one that carries the meaning: the theorems are the same not because all proofs of `False` are interchangeable, but because these three are the identical construction with three labels. The kernel would accept the second route even for genuinely different arguments, which is why the first is the one to cite when claiming the trilemmata are "the same proof."

The companion `F_13_UnifiedTrilemmata` proves the same `rfl` chain over `Function.Surjective` as its reflectivity form, and packages the shared

boundary object as `schema_boundary`, the single self-negating query that is hallucination's boundary question, defense's liar prompt, and coupling's coupled point at once.

3.9 What the readings share, and where they part

It is worth collecting what stays fixed and what moves across the three readings, because the pattern is the reusable part. Fixed across all three is the engine: a Boolean verdict, the flip as controller, and universality as the hypothesis. Fixed too is the witness, a self-negating query that the flip cannot satisfy. What moves is the meaning of the verdict and, with it, the believability of the universality premise.

The verdict's meaning moves from "confident and correct" (hallucination) to "made safe" (defense) to "placed on the true side" (coupling). The two horns that fix the flip move from "faithful and calibrated" to "complete and utility-preserving" to "exactly truth-separating." The source of universality moves from "the model imitates any described behavior" to "prompts are arbitrary text" to "the representation space encodes any pattern over itself." That last move is the one that changes the character of the result. Defense's universality is nearly free, hallucination's is defensible, and coupling's is strong. The topological route of the next chapter trades all three of these self-reference premises for one geometric premise, continuity on a connected domain, and reaches the same boundary object with metric content attached.

There is a temptation, once the `rfl` is in hand, to conclude the three results are trivial. They are not. The proof is one line because the modeling was done first. All the content is in Definitions 3.1 through 3.10 and the arguments that they add up to a reflective verdict. The engine is trivial on purpose; the theorems are as strong or as weak as the readings that feed it, and auditing a reading is where the judgment lies.

It is worth setting the three readings against each other in one place. The verdict means "confident and correct," then "made safe," then "placed on the true side." The two conditions that fix the flip are "faithful and calibrated," then "complete and utility-preserving," then "exactly truth-separating." The source of universality is "an in-context learner imitates any described behavior," then "prompts are arbitrary text," then "a representation encodes any pattern over itself." The liar witness is "a self-referential query answered correctly exactly when wrong," then "an injection made safe exactly when unsafe," then "a representation placed on the true side exactly when placed on the false side." Reading down these lists, the middle column, the two conditions, is where standard desiderata do the work, and the bottom source, universality, is where the believability varies from nearly free (defense) through defensible (hallucination) to strong (coupling). The topological engine of the next chapter replaces the entire bottom row with one geometric premise and recovers the same witnesses with metric content attached.

3.10 The template in four moves

It is useful to extract the recipe, because the same four moves generate every result in this chapter and most of the ones ahead. First, choose the reading: decide what a single Boolean `v p q` means in the domain, and what the domain's self-contexts are. Second, argue the codomain is `Bool` and the controller is the flip: find the domain condition (calibration, the discrete nature of safety, exact separation) that makes outcomes two-valued, and the condition (faithfulness, completeness, separation) that makes the honest self-report the negation of the diagonal. Third, argue universality: identify the source of self-reference (an in-context learner, arbitrary text, a self-encoding space) that names every pattern, and be honest about how strong that source is. Fourth, apply the engine: cite `no_reflective_verdict` for the impossibility and `liar_query` for the witness, and read the witness back into the domain.

The four moves are not equally hard. The first is a modeling decision, the second is usually a short argument from a standard desideratum, and the fourth is a single line of Lean. The weight is in the third, because that is the premise a critic will contest and the one whose plausibility varies across the readings. When you meet a new impossibility claim of this shape, run it through the four moves and put your scrutiny on the third. If the self-reference source is real, the claim stands on Chapter 1's engine. If it is not, the claim is either false or belongs to the topological engine of Chapter 4, which pays for the boundary object with connectedness instead.

This template also explains why the chapter's Lean is so short. Moves one through three happen in prose, and only move four is code. The proofs are one line each because the engine was built once, in Chapter 1, and every result here is move four applied after the prose has done moves one through three. A book that hid the prose and showed only the code would look like it proved three deep theorems in three lines. What it actually did was three careful readings and one reused lemma, and the honesty of the enterprise is in keeping the readings visible.

3.11 A note on axioms and trust

A machine-checked impossibility invites a specific kind of scrutiny, and it is healthy to say where the trust actually rests. The `c3_` theorems of this chapter are elaborated by Lean as the book is built, so they are not claims about proofs but proofs the kernel has accepted. What the kernel guarantees is that, granting Lean's small set of foundational axioms, the terms have the types their statements assert. The companion library audits those axioms explicitly and finds that the main results reduce to the three standard ones, `propext`, `Classical.choice`, and `Quot.sound`, which are the ordinary footing of classical mathematics in this system. The Boolean core of this chapter uses even less, because the flip's fixed-point freedom is decidable and needs no classical principle, a point Chapter 1 made when contrasting the Boolean Cantor with the

propositional Russell.

The trust that verification cannot supply is trust in the *modeling*. No proof assistant can check that "confident-correctness self-verdict" is the right reading of a real model, or that a real prompt space is reflective, or that a representation space encodes patterns over itself. Those are the premises, and they are exactly what the long middle of each section argues and what the caveats concede. This is the intended division. The machine certifies that the conclusion follows from the premises, and the prose argues about whether the premises hold. An impossibility result is only as strong as its weakest premise, and the value of verifying the inference is that it leaves the premises as the sole remaining thing to contest. When someone wants to escape one of these trilemmata, the honest move is to name the premise they reject, faithfulness, completeness, exact separation, or reflectivity, and own its cost, not to look for a gap in a proof the kernel has already closed.

3.12 What this chapter does not settle

An impossibility theorem is easy to over-read, and it is worth marking the claims the chapter does not make. It does not say models hallucinate often, defenses fail often, or probes mislead often. It produces one query per system and is silent on frequency. A model could meet its liar on a query no user ever sends and behave perfectly in practice, and nothing here contradicts that. The frequency question belongs to the statistical tradition and to the quantitative geometry of Chapter 5, and the diagonal deliberately says nothing about it.

The chapter also does not say the liar query is easy to find. The diagonal proves one exists and even names it as a preimage of a specific pattern, but naming a preimage is not the same as constructing it cheaply. For a large model the self-context that realizes the diagonal pattern may be hard to elicit, and the cost of eliciting it is exactly the kind of quantitative content the diagonal cannot see. An adversary who can search the input space will find injections faster than a random user, which is why the defense reading is the one where the witness is cheap and the coupling reading is the one where it may be expensive.

Finally, the chapter does not choose between the diagonal and the topological engine. Each is right in its domain. The diagonal is the honest tool when self-reference is real, as it plainly is for prompt injection, and the topological engine is the honest tool when the domain is a connected space with continuous fields, as it plausibly is for a representation manifold. The two agree on the boundary object, and the value of having both is that a given application can be attacked from whichever premise it actually satisfies. A reader who takes only one engine from these notes has taken half of them. The next chapter builds the other half, and the chapter after that adds the metric content that neither engine has on its own.

3.13 Historical and bibliographic notes

The three results have separate origins. The Hallucination Trilemma and its topological proof are in the companion development's hallucination line, re-derived in Gödel form as `hallucination_trilemma_godel` in `F_11_HallucinationGodel`, which also records that the boolean liar `!·` and the real complement `1 - y` are the two controllers, fixed-point-free everywhere and fixed-point-free off one half respectively. The Defense Trilemma is the prompt-injection impossibility, re-derived diagonally as `defense_trilemma_godel` in `F_12_DefenseGodel`, with the real-valued score version `defense_trilemma_lawvere` alongside it. The coupling theorem is the subject of the paper "Truth Has a Boundary," whose main statement `boundary_coupling` is topological, with the diagonal reading collected as `coupling_trilemma_godel` in `F_13_UnifiedTrilemmata`. The unification into a single master trilemma over Utility, Control, and Transparency is the `CCHProofs` development, with `cch_master_trilemma` and the three corner theorems.

The mathematical lineage is Lawvere's 1969 fixed-point theorem, through the reading of Tarski's undefinability of truth as a diagonal, which Chapter 2 develops. The statistical inevitability of hallucination is a separate tradition, and it derives confident falsehood rather than uncertainty; the topological and diagonal arguments here derive a boundary object instead, and only the stochastic dichotomy (`stochastic_dichotomy` in the paper) turns that boundary into falsehood with positive probability. The reading of these classical theorems as constraints on learning systems is recent, and the machine-checked development these notes follow is the companion Lean library, built against Mathlib and reducing every main theorem to the three standard kernel axioms.

A note on how the pieces fit in the companion code. Chapter 1's engine lives in `F_01_LawvereCore`, the Boolean liar and the hallucination reading in `F_11_HallucinationGodel`, the defense reading in `F_12_DefenseGodel`, and the `rfl`-chain unification in `F_13_UnifiedTrilemmata`, whose `schema_boundary` packages the single self-negating query the three readings share. The master trilemma and its corners are the `CCHProofs` development, and the topological coupling theorem with its quantitative refinements is the "Truth Has a Boundary" artifact. All of them pin the same Lean toolchain and build against the same Mathlib release, so the cross-references in this chapter are to code that compiles together, not to three separate projects that happen to share vocabulary.

3.14 Exercises

Exercise 3.1. State `c3_hallucination_trilemma`, `c3_defense_trilemma`, and `c3_coupling_trilemma` side by side and verify by inspection that their types differ only in the bound variable name for the domain. Explain why this alone does not prove they are the same theorem, and what `c3_trilemmata_`

`unified` adds.

Exercise 3.2. In the hallucination reading, write out in plain English the self-referential query that `c3_hallucination_liar` produces, and identify which of the three desiderata (faithful, covering, calibrated) each of the three possible escapes corresponds to.

Exercise 3.3. Prove in Lean that the hallucination and defense trilemmas share a witness: given `v` and a reflectivity proof `h`, show that the `p` returned by `c3_hallucination_liar v h` also satisfies the conclusion of `c3_defens e_liar v h`. (Hint: they are the same term; a single `rfl`-style observation suffices once you have both applied.)

Exercise 3.4. The constant model that always answers "not confident-and-correct" is faithful and calibrated. Show it is not covering, and show it is not reflective by exhibiting a pattern `g : Q → Bool` it fails to name. Relate this to why Definition 3.3 is needed.

Exercise 3.5. (Defense.) Explain why, for prompt injection, the reflectivity premise `∀ g, ∃ p, v p = g` is close to a restatement of the threat model rather than an idealization. Then describe one realistic restriction on the prompt space under which reflectivity fails, and say what part of the argument breaks.

Exercise 3.6. (Coupling, harder.) The coupling reading's universality is the weakest of the three. Write one paragraph arguing that a language model *prompted to reason about its own probe* makes the self-encoding real, and one paragraph arguing that a *passive activation space* does not. Which one does a probing experiment actually deploy over?

Exercise 3.7. Prove `c3_lawvere_point` again, this time from `lawvere` rather than from scratch, and explain what extra information your version exposes that `lawvere` alone does not. (Hint: `lawvere` asserts a fixed value exists; you need the index.)

Exercise 3.8. Instantiate `c3_cch_master` with `Y = Bool`, `t` the flip, and `p` the diagonal `a ↦ s a a`, and check that transparency becomes trivial and control becomes `bool_not_fpf` applied pointwise. Conclude that the hallucination trilemma is the `UC` corner of the master trilemma.

Exercise 3.9. Show the three corner theorems are genuinely different by exhibiting, informally, a system satisfying each pair of properties: one universal and controlled but not transparent, one universal and transparent but not controlled, one controlled and transparent but not universal. (You may describe them in prose; the point is that no edge of the triangle is empty.)

Exercise 3.10. Prove in Lean that `c3_cch_corner_UC` and the other two corners all reduce to `c3_cch_master`, and state the "at most two of three" theorem `c3_cch_at_most_two` as an alias for `c3_cch_master`. Compare with the companion `cch_at_most_two`.

Exercise 3.11. (Reading audit.) For each of the three trilemmata, name the two horns that fix the flip and the one hypothesis that supplies universality. Arrange the three universality sources on a scale from "free" to "strong" and defend the ordering.

Exercise 3.12. The topological proofs deliver a coupled point with metric content: an ambiguity tube of width set by Lipschitz constants. The diagonal proof delivers only the point. Explain what quantitative question the diagonal cannot answer, and why Chapter 5 needs the analytic engine rather than this one.

Exercise 3.13. (Harder, Lean.) Define a `c3_`-prefixed predicate `C3_Faithful` on `v : Q → Q → Bool` capturing "no self-verdict equals its own negation," `∀ p, v p p ≠ !(v p p)`. Prove that a reflective `v` cannot be `C3_Faithful`, using `c3_hallucination_liar`. Which single lemma does the whole proof reduce to?

Exercise 3.14. (Coupling versus topology.) The paper's `faithful_iff_boundary_free` says a threshold-inclusive guarantee is possible exactly when no boundary query exists. Restate this in the diagonal language of this chapter, with "boundary query" read as the liar witness, and explain why reflectivity is what removes the escape.

Exercise 3.15. (Open-ended.) The engine needs Boolean outcomes and the flip. Chapter 6 relaxes the flip to the real complement, fixed-point-free off one half, and Chapter 4 relaxes reflectivity to continuity on a connected space. Sketch what a *single* theorem covering both relaxations would need to assume, and which of this chapter's three readings would survive each relaxation most comfortably.

Exercise 3.16. (Synthesis.) Write a one-page account, for a reader who knows no Lean, of why "there is no total exact self-applicable truth predicate" is the same statement as "no faithful calibrated covering model, no complete utility-preserving defense, and no exactly truth-separating probe over a self-encoding space." Use the liar witness as the common thread and avoid all symbols.

Exercise 3.17. (Section form.) Following Remark 3.0, restate `c3_hallucination_trilemma` using a section `s : (Q → Bool) → Q` with `v (s g) = g` in place of the surjection, and prove it by feeding the diagonal pattern to `s`. Explain why, for auditing a real model, the section form is the more honest premise to check.

Exercise 3.18. (Master to verdict.) Carry out Remark 3.12 in Lean: instantiate `c3_cch_master` with `Y = Bool`, `t` the flip, and `p := fun a => s a a`, and discharge transparency by `rfl` and control by `bool_not_fpf`. Confirm that the result you obtain is `c3_hallucination_trilemma` in disguise.

Exercise 3.19. (Grading premises.) Write a short paragraph for each trilemma naming the single premise you would attack to escape it in a real deployment, and the concrete engineering cost of dropping that premise. Rank the three escapes by which is most defensible in practice and justify the ranking.

Exercise 3.20. (Open-ended.) The coupling reading has the strongest reflectivity premise. Design an experiment, in prose, that would give evidence *for* or *against* a representation space encoding patterns over its own states, and say what a negative result would imply for the diagonal reading versus the topological one.

Exercise 3.21. (Micro-instance.) Redo the worked micro-instance with a four-element domain and diagonal entries of your choosing. Write out the flip pattern, confirm it matches no row, and identify the liar row that a claimed reflectivity proof would produce. Then argue in one sentence why the same reasoning never bottoms out no matter how large you make the domain.

Exercise 3.22. (Controllers.) The hallucination and defense readings use the Boolean flip, and their real-valued cousins use the complement `y ↦ 1 - y`. Prove in Lean that `bool_not_fpf` gives a fixed-point-free flip on `Bool`, and explain in prose why the complement is fixed-point-free everywhere except at one half. What does the exceptional point at one half correspond to in the hallucination reading?

Exercise 3.23. (Master corners are nonempty.) For the master trilemma, sketch a concrete `CCHStructure` witnessing each of the three achievable pairs: universal and controlled, universal and transparent, controlled and transparent. Confirm that each of your witnesses fails the omitted property, so that no corner of the triangle is vacuous.

Exercise 3.24. (Detection revisited.) A colleague proposes a defense that detects and refuses any prompt referring to the wrapper. State precisely, in the language of Definitions 3.5 through 3.7, which property their defense gives up, and prove in Lean that a wrapper which refuses the liar prompt cannot satisfy completeness at that prompt. (Hint: refusal assigns a Boolean value; completeness demands the other one.)

Exercise 3.25. (Tarski identity.) Using the coupling reading, write out the correspondence between Tarski's liar sentence and the coupled query, term by term: sentence, truth predicate, self-reference, and the fixed-point-free flip. Then say which single hypothesis of the coupling reading is doing the work that Tarski's "sufficiently expressive language" does, and why it is the strong one.

Exercise 3.26. (Two engines, one witness.) The diagonal and the topological proofs produce the same boundary object. Write a paragraph explaining what the topological proof knows about that object that the diagonal does not, and what the diagonal knows that the topological proof needs a connected domain to obtain. Use the ambiguity tube and the reflectivity premise as your two examples.

Exercise 3.27. (Synthesis, harder.) Prove in Lean a single theorem, prefixed `c3_`, that takes a verdict `v` and a reflectivity proof and returns the conjunction of `False` (the impossibility) and `∃ p, v p p = !(v p p)` (the witness), by pairing `no_reflective_verdict` with `liar_query`. Explain why this one theorem is a faithful summary of all three trilemmata at once.

4 From Logic to Analysis

Abstract. The diagonal needs a system rich enough to name its own behaviors, and trained networks do not meet that demand literally. This chapter develops a second engine that asks only that the space be connected and the maps continuous. The intermediate value theorem then extracts the same boundary object the diagonal extracts from self-reference: a question where confidence sits on the threshold while the answer sits on the truth boundary. The chapter builds the needed topology, proves the quantitative versions with separation slack, and gives the defense reading.

Keywords: Intermediate value theorem, Connectedness, Truth boundary, Confidence map, Topology

The diagonal of Chapter 1 needs a system rich enough to name its own behaviors. The hypothesis `∀ g, ∃ a, f a = g` is a strong demand, and many real models do not meet it in that literal form. A trained network does not carry an index for every function from prompts to confidences. Its reachable states are a bounded region, not a set closed under naming its own predicates. If the only engine we had were the diagonal, half the impossibilities in these notes would sit outside its reach.

There is a second engine, and it runs on different fuel. Instead of asking that a space describe itself, it asks that the space be *connected* and that the maps on it be *continuous*. From those two analytic hypotheses it extracts the same boundary object the diagonal extracts from self-reference: a question at which a model's confidence sits exactly on the threshold while its answer sits exactly on the truth boundary. The tool that does the extracting is the intermediate value theorem, and this chapter is a full account of how it works, what it needs, and why it lands on the same point the diagonal lands on.

The code blocks here are core Lean 4, self-contained and elaborated when the book is built, exactly as in Chapter 1. The analytic theorems themselves live over the real numbers and use topology, so their machine-checked forms are in the companion `HallucinationProofs` library, which is built against Mathlib. Whenever a result in this chapter has a verified counterpart there, its Lean name appears in plain type, and the reader who wants the kernel-level guarantee can open the named file. The prose proofs are complete on their own terms; the names are pointers, not omissions.

4.1 Why a second engine

The two engines answer different objections. Against a system that genuinely names its own behaviors, the diagonal is decisive and needs nothing else. Against a system that lives on a manifold of activations and computes with continuous fields, the diagonal has no purchase, because there may be no index whose self-application recovers the diagonal behavior. What such a system does have is shape. Its representation space is a connected region, its confidence is a continuous function of the input, and its correctness varies continuously as the input moves. That shape is enough.

The move is worth stating in one sentence before we make it precise. Self-reference forces a fixed point because a flip has nowhere to hide on the diagonal; connectedness forces a crossing because a continuous quantity that is negative somewhere and positive somewhere cannot get from one to the other without passing through zero. Both arguments trap a value. One traps it with a naming equation, the other with a topological wall. The trapped value is the same, and the later sections make that identity exact rather than suggestive.

It is worth being concrete about which real systems fall to which engine. A proof-checker that accepts the code of any predicate on proofs, or an in-context learner promptable to imitate any behavior over prompts, is close to the diagonal hypothesis, because it can be handed a description of the very behavior that would refute it. A trained classifier reading truth off residual-stream activations is not. It does not carry an index for every function from activations to confidences, and there is no reason its reachable states are closed under naming their own predicates. Yet its activations form a connected region, by the manifold hypothesis and the way ambient embedding spaces are modeled as `ℝ^d` or connected submanifolds, and its probe and confidence heads are continuous. The diagonal has nothing to grip here, and the analytic engine has everything it needs. The two hypotheses carve the space of systems differently, and a system that escapes one may sit squarely inside the other.

There is a second reason to build the analytic engine carefully. It carries metric content the diagonal cannot. Once the space has a distance and the maps are Lipschitz, the crossing point is not a bare witness but the center of a band of computable width, with decisive regions held a computable distance apart. The diagonal never sees any of this. Chapter 5 is entirely about those quantities, and it is available only because the engine we build here runs on geometry.

4.2 A topology primer

This section develops just enough point-set topology to state and prove the intermediate value theorem, and then explains why connectedness is the analytic analogue of self-reference. A reader who knows the material can skim to the last subsection, where the analogy is drawn. A reader who does not will

find every term defined before it is used.

Open sets, continuity, and the shape of a space

A *topology* on a set `X` is a choice of which subsets count as *open*. The choice is not arbitrary. It must contain the empty set and all of `X`, it must be closed under arbitrary unions, and it must be closed under finite intersections. These axioms are the distilled content of "nearness" without a distance. In a metric space the open sets are the usual ones, the sets that contain a small ball around each of their points, but the axioms make sense with no metric at all, and that generality is what lets the same theorems cover activation spaces, product spaces, and spheres in one stroke.

Definition 4.1 (Topological space). A *topological space* is a set `X` together with a collection of subsets, called the *open sets*, that includes $\varnothing$ and `X`, is closed under arbitrary unions, and is closed under finite intersections. A set is *closed* when its complement is open.

Definition 4.2 (Continuity). A map `f : X → Y` between topological spaces is *continuous* when the preimage `f⁻¹(U)` of every open set `U` in `Y` is open in `X`. Equivalently, the preimage of every closed set is closed.

The preimage formulation looks abstract next to the epsilon-delta definition, but it is the same idea and it is easier to use. For real-valued functions on a metric space the two agree exactly. What the preimage form buys us is that continuity becomes a statement about the interaction of two topologies, which is precisely the level at which connectedness lives. The single fact we will use repeatedly is that a composition of continuous maps is continuous, which is immediate from the definition: if `f` and `g` are continuous then `(g ∘ f)⁻¹(U) = f⁻¹(g⁻¹(U))` is open whenever `U` is. The verified proofs lean on exactly this closure property, built from Mathlib's `Continuous.comp` and the product rule `Continuous.prodMk`, which is how the composite `q ↦ δ(q, (M q).1)` is shown continuous in `model_truth_boundary_nonempty`.

Two closure notions we will need later belong here. The *closure* of a set `S` is the smallest closed set containing it, the set of points every neighborhood of which meets `S`. A set is *dense* when its closure is the whole space, meaning every point is approximable by members of the set. Density is the topological form of the strongest coverage condition, where the model's answers come arbitrarily close to every answer, and the verified predicate `DenseCoverage` in `HoF_05_Coverage` is exactly this, with `denseCoverage_closure_eq_univ` recording that a dense answer image has closure equal to everything. We use the mild two- sided coverage of Definition 4.12 for the main theorem and mention density only to place it on the same scale.

Metric spaces, balls, and paths

Most spaces in the applications carry a distance, and it is worth seeing how the abstract topology specializes to that case, because the quantitative results of Chapter 5 live there. A *metric* on a set `X` assigns to each pair of points a

non-negative real distance that is zero exactly for equal points, is symmetric, and satisfies the triangle inequality. The *open ball* of radius `r` around a point `x` is the set of points within distance `r` of `x`. A set is open in the *metric topology* when it contains an open ball around each of its points. One checks directly that these open sets satisfy the axioms of Definition 4.1, so every metric space is a topological space, and continuity in the preimage sense of Definition 4.2 agrees with the epsilon-delta sense for maps between metric spaces.

The space `ℝ^d` with the usual Euclidean distance is the standing example. Ambient embedding spaces and residual streams are modeled as `ℝ^d` or as connected submanifolds of it, and both are metric spaces, so the abstract theorems apply and the metric refinements of Chapter 5 become available. The verified sphere instantiation uses the Euclidean space `EuclideanSpace ℝ (Fin d)`, and the quantitative results use `PseudoMetricSpace` with explicit Lipschitz hypotheses.

A *path* in `X` from `x` to `y` is a continuous map `γ` from the unit interval `[0, 1]` to `X` with `γ 0 = x` and `γ 1 = y`. A space is *path-connected* when any two points are joined by a path. Path-connectedness is stronger than connectedness, and the implication one way is direct: if a path-connected space split into two separating open sets, a path from a point in one to a point in the other would pull the separation back to a separation of the interval `[0, 1]`, which is connected, a contradiction. So path-connected implies connected. The converse can fail in general, but for the spaces in the applications, which are convex regions or manifolds, the two coincide and either can be assumed.

Paths matter because the intermediate value theorem has a path form that is often the concrete face of the connected-space form. Interpolating hidden states from a true statement to its negation traces a path in representation space, and along that path the realized truth-distance is a continuous real function that starts negative and ends positive. The one-variable intermediate value theorem then gives a boundary crossing on the path itself, which is `path_crosses_truth_boundary` in `HoF_03_BoundaryCrossing`. The connected-space form of Lemma 4.14 below does not require a path, but the path form is what an experiment actually walks along, and it is how the companion paper observes the crossing in real models.

Products, and staying connected

One construction preserves connectedness and matters for the multi-turn extension later. If `X` and `Y` are connected, so is the product `X × Y` with the product topology, in which a set is open when it is a union of products of open sets. The proof is a two-step slide: fix a base point, note that each horizontal slice and each vertical slice through it is a continuous image of a connected space and so connected, and observe that the union of all slices meeting a common point cannot be separated. By induction a finite product of connected spaces is connected.

The consequence we use is that if a single-turn question space `Q` is connected, then the multi-turn space `Q^T` of conversation histories of length `T`

is connected as well. Every argument in this chapter transports to `Q^T` unchanged, because the only property of `Q` any proof used was its connectedness. The verified statement that interaction does not escape the obstruction is `multi_turn_history_dependent`. A conversation is not a way out. It is a walk on a larger connected space, and the same wall is somewhere on it.

Connected and preconnected spaces

Connectedness is the property of being all one piece. The clean way to say "all one piece" is negative: a space is disconnected when it splits into two nonempty open sets that do not meet. Ruling that out is the definition.

Definition 4.3 (Connected space). A topological space `X` is *disconnected* when there exist open sets `U` and `V` with `U ∪ V = X`, `U ∩ V = ∅`, and both `U` and `V` nonempty. `X` is *connected* when it is nonempty and not disconnected.

Definition 4.4 (Preconnected set). A subset `S ⊆ X` is *preconnected* when it cannot be separated by two open sets of `X`, that is, whenever `U` and `V` are open, `S ⊆ U ∪ V`, `S ∩ U ∩ V = ∅`, `S ∩ U ≠ ∅`, and `S ∩ V ≠ ∅` all hold at once, we have a contradiction. A set is *connected* when it is preconnected and nonempty.

The distinction between preconnected and connected is only whether the empty set is admitted. The empty set is preconnected and not connected, since connectedness demands a point. Mathlib keeps the two apart because the intermediate value theorem is cleanest as a statement about preconnected sets, and the verified files call the preconnected form directly. The name to hold onto is `IsPreconnected`, and the fact that the whole space is preconnected when `X` carries a `ConnectedSpace` instance is `isPreconnected_univ`, which is the first line of every IVT application in `HoF_03_BoundaryCrossing` and `HoF_07_TrilemmaCore`.

Two examples fix the idea. An interval of the real line is connected. Any interval, open, closed, or half-open, bounded or not, cannot be split into two nonempty disjoint open pieces, and the proof is the completeness of the reals: given a split, the supremum of the part on the left is a point that can lie in neither piece without contradicting openness. A two-point discrete space is disconnected, since each point is open and the two singletons separate it. The contrast is the whole story. The reals are connected and the booleans are not, which is exactly why the analytic engine runs on the reals and the diagonal engine runs on the booleans, and why the two meet only at the point where a real threshold and a boolean flip describe the same wall.

Proposition 4.5 (Continuous image of a preconnected set). *If `S ⊆ X` is preconnected and `f : X → Y` is continuous on `S`, then the image `f(S)` is preconnected in `Y`.*

Proof. Suppose `f(S)` were separated by open sets `U` and `V` in `Y`. Then `f⁻¹(U)` and `f⁻¹(V)` are open in `X` by continuity, they cover `S`, their intersection misses `S`, and each meets `S` because `U` and `V` each meet `f(S)`. That is a separation of `S`, contradicting preconnectedness. So no separation of `f(S)` exists. ■

This is `IsPreconnected.image` in `Mathlib`, and the verified proofs in the companion libraries call it rather than reproving it.

Proposition 4.5 is the engine's load-bearing beam. Connectedness is preserved by continuous maps, and that single preservation fact, applied to a real-valued map, becomes the intermediate value theorem once we know what the connected subsets of the real line are.

Proposition 4.6 (Connected subsets of the line are intervals). *A subset of $\mathbb{R}$ is preconnected if and only if it is an interval, meaning that whenever it contains two points it contains everything between them.*

Proof. One direction is a construction. If a set `S` omits a point `c` strictly between two of its members `a < c < b`, then the open rays `(-∞, c)` and `(c, ∞)` cover `S`, do not meet, and each contains a member of `S`, namely `a` and `b`. That is a separation, so `S` is not preconnected.

The other direction is where completeness of the reals enters. Suppose `S` is an interval and, for contradiction, that open sets `U` and `V` separate it. Pick `a ∈ S ∩ U` and `b ∈ S ∩ V`, and assume `a < b` without loss. Let `m` be the supremum of the set of points in `[a, b] ∩ U` that lie below `b`. This supremum exists because the set is nonempty, containing `a`, and bounded above by `b`. Since `S` is an interval and `a, b ∈ S`, the point `m` lies in `S`, so it lies in `U` or in `V`. If `m ∈ U`, then `U` open gives a small interval around `m` inside `U`, which pushes points of `U` above `m` and below `b`, contradicting that `m` is an upper bound. If `m ∈ V`, then `V` open gives a small interval around `m` inside `V`, so points just below `m` lie in `V` and not in `U`, contradicting that `m` is the least upper bound of the `U` side. Either way a contradiction, so no separation exists and `S` is preconnected. ∎

In `Mathlib` the forward direction is `IsPreconnected.ordConnected` and the converse for a closed interval is `isPreconnected_Icc`; together they are the statement above, and the companion libraries use them in that form.

The intermediate value theorem

Put the two propositions together. A continuous real-valued function on a connected space has a preconnected image by Proposition 4.5, that image is an interval by Proposition 4.6, and an interval that contains two values contains every value between them. That is the intermediate value theorem, and it is the only analytic input the chapter needs.

Theorem 4.7 (Intermediate value theorem). *Let `X` be connected and `c : X → ℝ` continuous. If `c` takes a value below `r` at some point and a value above `r` at some point, then `c` takes the value `r` at some point.*

Proof. Let x_- and x_+ be points with $c(x_-) < r < c(x_+)$. The image $c(X)$ is preconnected by Proposition 4.5, hence an interval by Proposition 4.6. It contains $c(x_-)$ and $c(x_+)$, so it contains every value between them, and r is such a value. Therefore $r \in c(X)$, which means $c(x_0) = r$ for some x_0. ∎

The verified library uses a two-function form that is a little more flexible and slightly more convenient in proofs. The name is `IsPreconnected.intermediate_value₂`. Instead of comparing one function to a constant `r`, it compares two continuous functions `f` and `g` on a preconnected set: if `f a ≤ g a`

at one endpoint and `g b ≤ f b` at the other, then `f x = g x` at some interior point. Taking `g` to be the constant function with value `r` recovers Theorem 4.7, because `f a ≤ r` and `r ≤ f b` force `f x = r` somewhere. Every IVT call in `HoF_03_BoundaryCrossing`, `HoF_05_Coverage`, and `HoF_07_TrilemmaCore` is this two-function form with `g = continuous_const`, and the one-dimensional special case, an interval `[a, b]` treated as a preconnected set via `isPreconnected_Icc`, is `confidence_path_crosses_half`.

Remark 4.8 (What the theorem does not say). The intermediate value theorem asserts existence, not uniqueness and not location. It gives a point where the value is hit, and says nothing about how many such points there are, where they sit, or how a search would find one. This is the analytic mirror of the diagonal's silence in Remark 1.18. Both engines produce a witness with no built- in address. Metric content is an extra ingredient, added in Chapter 5, and it is what turns the bare crossing into a band of known width.

Connectedness as analytic self-reference

Here is the analogy the chapter is built around, stated plainly. In Chapter 1 the diagonal took a flip `t` with no fixed point and derived a contradiction from universality, because a universal system must name the behavior `a ↦ t (f a a)`, and the naming equation then forces `f a₀ a₀ = t (f a₀ a₀)`, a fixed point of a map that has none. The engine of that argument is that self-application leaves the flip no room. There is a diagonal, and the flip must land on it.

Connectedness plays the same structural role. A continuous quantity on a connected space that is negative somewhere and positive somewhere is a flip with no room. It cannot jump from negative to positive, because a jump would split the space into the part where the quantity is negative and the part where it is positive, two nonempty open sets that separate a space we assumed cannot be separated. So the quantity must pass through zero. The zero is forced by the shape of the space in exactly the way the fixed point was forced by the naming equation. Where the diagonal says "the behavior is named, so evaluate at its own index," connectedness says "the space is one piece, so the sign cannot flip without a crossing."

The parallel is not a metaphor that breaks under pressure. It is an equality of outputs, and Section "Where the two engines meet" proves it as such. For now, hold the following table of correspondences, which the rest of the chapter fills in. Self-reference corresponds to connectedness. The behavior flip `t` corresponds to a continuous field taking both signs. The forced fixed point corresponds to the forced zero. The Boolean negation `(!·)` corresponds to the real complement map `y ↦ 1 - y`. The liar index of Proposition 1.14 corresponds to the boundary question this chapter constructs. Each row is a theorem, and by the end each will be one.

It helps to see why the analogy is tight rather than loose, since two arguments can reach the same conclusion for unrelated reasons. Here the reasons are the same shape. In both engines there is a set of candidate outputs, a trans-

formation on those outputs, and a structural hypothesis that leaves the transformation nowhere to sit except on a distinguished value. For the diagonal the candidate outputs are the values `f a a` along the diagonal, the transformation is `t`, and the structural hypothesis is that every behavior is named, so the diagonal behavior `a ↦ t (f a a)` is itself some `f a₀`, and evaluation forces `f a₀ a₀` onto a fixed point of `t`. For the intermediate value theorem the candidate outputs are the values of a continuous field, the transformation is the passage from one sign to the other, and the structural hypothesis is connectedness, which forbids the field from changing sign without visiting zero. In each case a hypothesis about the *domain,* richness of naming or one-piece-ness, constrains the *range* to hit a particular value.

There is even a shared failure mode. Remove the structural hypothesis and both arguments collapse in the same way. A non-universal system can carry a fixed-point- free flip with no contradiction, since the offending diagonal behavior may simply be unnamed. A disconnected space can carry a sign-changing continuous field with no zero, since the field can be negative on one piece and positive on another with a gap between. This is why abstention breaks the analytic argument. Declining to answer on an open region deletes that region and can disconnect the correct part from the incorrect part, which is the topological form of refusing to name the diagonal behavior. The two engines fail together, under the same move, which is one more sign that they are two views of a single mechanism.

4.3 The model, analytically

We now set up the object the analytic engine acts on. It is the continuous counterpart of the reflective verdict of Definition 1.15, and the reader should watch the three conditions of Chapter 3 reappear here in metric dress.

The confidence map and the truth-distance

Definition 4.9 (Model). Fix a topological space `Q` of *questions* and a topological space `A` of *answers*. A *model* is a map `M : Q → A × ℝ`. For a question `q`, the answer is `(M q).1` and the *confidence* is `(M q).2`, a real number. The model is *continuous* when both components are continuous, that is when `q ↦ (M q).1` and `q ↦ (M q).2` are continuous.

Reading `Q` as a space of prompt embeddings or residual-stream states, and `A` as the space of possible answers, this is the shape of a trained model whose fields are defined on its representation space. The confidence is a scalar the model attaches to its answer, an internal estimate of how sure it is. Softmax outputs, probe readouts, and calibrated logits are all continuous candidates for `(M q).2`, while hard argmax decoding is not continuous, a point we return to when the discrete core is priced.

Definition 4.10 (Signed truth-distance). A *truth-distance* is a continuous map `δ : Q × A → ℝ`. It is a signed measure of correctness: `δ(q, a) < 0` on

strictly correct answers, δ`(q, a) > 0` on strictly wrong ones, and δ`(q, a) = 0` on the *truth boundary*. The *truth set* is {δ `≤ 0`} and the *boundary* is {δ `= 0`}. The *realized* truth-distance of the model is the composite `q ↦ δ(q, (M q).1)`, the truth-distance of the answer the model actually gives.

The sign convention is a choice and we keep it fixed: negative is good, positive is bad, zero is the wall between them. A linear truth probe `w·x - b` on activations is a candidate δ, and its zero set is the probe's classification boundary. Whether that boundary tracks semantic truth away from the fitted data is an empirical premise, not a theorem, and the results here hold under whatever δ one commits to. The realized composite is the only thing the theorems touch, and its continuity follows from continuity of δ and of the answer map by the composition and product rules, which is `model_truth_boundary_isClosed`'s first step.

Faithful, covering, calibrated

The three conditions of Chapter 3 have exact analytic forms. Each says something about how the confidence `(M q).2` relates to the realized truth-distance δ`(q, (M q).1)`, with the threshold set at `1/2`. The threshold is a convention; any fixed value works and the verified defense results carry a general threshold `τ`, but `1/2` is the natural center for a confidence that ranges in the unit interval, so we use it throughout.

Definition 4.11 (Faithful). A model is *faithful* when high confidence guarantees correctness, in the strong form: for every question `q`, if `(M q).2 ≥ 1/2` then δ`(q, (M q).1) < 0`. The verified predicate is `TrilemmaFaithful`, and the strictness of the inequality `< 0` is what does the work. A weaker `≤ 0` form would admit the boundary case and dissolve the theorem, which is the whole content of the inclusive-versus-exclusive threshold distinction discussed below.

Definition 4.12 (Covering). A model is *covering* when it produces answers on both sides of the boundary: there is a question `q` with δ`(q, (M q).1) < 0` and a question `q` with δ`(q, (M q).1) > 0`. The model is sometimes right and sometimes wrong. The verified predicate is `TrilemmaCovering`, also stated as `Covering` in `HoF_05_Coverage`, and it is the mildest possible non-triviality assumption: any general-purpose model that is ever correct and ever incorrect satisfies it.

Definition 4.13 (Calibrated). A model is *strictly calibrated* when the side of the threshold the confidence sits on matches the side of the boundary the answer sits on, in both directions: for every `q`,

`(M q).2 > 1/2 ↔ δ(q, (M q).1) < 0` and `(M q).2 < 1/2 ↔ δ(q, (M q).1) > 0`.

The verified predicate is `StrictCalibrated`. This is a pointwise biconditional, much stronger than the statistical calibration of the empirical literature, and it is one-way strengthened into an equivalence. High confidence and correctness determine each other, and low confidence and error determine each other, question by question.

The gap between this pointwise condition and ordinary statistical calibration is worth naming, because they are easy to confuse. Statistical calibration is an averaged claim: among all questions the model answers with confidence near `0.7`, about seventy percent are correct. It is compatible with the model being wrong on any particular high-confidence question, as long as the frequencies work out. Strict calibration is not averaged. It constrains every single question. If the confidence is above the threshold at `q`, the answer at `q` is correct, full stop. This is a demanding idealization, and it is exactly the idealization a perfect truth probe used as a confidence signal would meet, which is why the empirical work builds `(M q).2` from a probe and asks how close reality comes. The slack version of the condition, introduced below, is the honest relaxation, and the strict form is its sharp corner.

Read the three conditions together on the linear-probe picture to see they are not exotic. Let δ be a linear probe `w·x - b`, so δ `< 0` on one side of a hyperplane and δ `> 0` on the other. Covering says the model produces answers on both sides, which any model used across a real workload does. Calibration says the confidence head, trained on a disjoint split of the same activations, agrees with the probe about which side each answer is on, which is what a well-trained confidence head approximates. Faithfulness says a confident answer is a correct one, which is the safety promise the whole enterprise wants to keep. None of the three is a strawman. Each is a target that probing and steering pipelines actively pursue. The chapter's content is that pursuing all three at once, on a connected space, is pursuing a contradiction, and the coupled point is where the pursuit fails.

A word on why calibration is stated as a biconditional and faithfulness as a one- way implication. Calibration is a claim about the confidence being an honest readout of the truth-distance, so both directions belong to it. Faithfulness is a safety guarantee, a promise that the model only asserts what is correct, and a promise is naturally one-directional. The interplay of the strict inequality in faithfulness with the equality that calibration forces at the boundary is the contradiction, and it is worth seeing that neither condition alone is at fault. Each is reasonable. Their conjunction on a connected covering model is not.

Separation with slack, and the general threshold

The strict biconditional of Definition 4.13 is the sharpest form, and it is what the impossibility uses, but the applied results are stated with slack so that the idealization can be relaxed at a named price. Fix a threshold `τ`, a truth slack ε `≥ 0`, and a confidence margin `γ ≥ 0`. A model is *(ε, `γ`)-separating at* `τ` when δ`(q, (M q).1) < -`ε implies `(M q).2 > τ + γ`, and δ`(q, (M q).1) >` ε implies `(M q).2 < τ - γ`. The slack sits in units of the truth-distance and the margin in units of confidence, so the condition survives rescaling either field. Exact separation is the case ε `= γ = 0`. The verified predicate is `TwoSlackSeparating`, with the equal-slack special case `EpsCalibrated` and the bridge `epsCalibrated_iff_twoSlack`, and the strict biconditional maps into

it by `strictCalibrated_to_epsCalibrated_zero_half`. Coverage generalizes the same way to ε-coverage, some question with δ `<` $-\varepsilon$ and some with δ `>` ε, verified as `EpsCovering`.

Slack is not cosmetic. The main theorem is that with a threshold-inclusive guarantee the truth slack cannot be zero, so a model is forced to keep a strictly positive margin of correctness on the confident side. We state that quantitative form in a later section. Here the point is only that the strict conditions of Definitions 4.11 through 4.13 are the ε `= γ = 0` corner of a graded family, and that using `1/2` for `τ` is a convention the coupling results never depend on. The biconditional variant sits at `τ = 1/2` because a confidence bounded in the unit interval has its natural center there, but every theorem below has a general-`τ` counterpart in the defense-side files.

The defense reading

The same setup describes a defense rather than a bare model, and the reader should carry both readings, since Chapter 3 needs the defense form. Reread `Q` as a space of inputs a system might face, including adversarial ones, `(M q).2` as a *safety score* the defense attaches to an input, and δ as a *truth probe* or harm measure whose sign says whether the input is actually safe. Faithful becomes the guarantee that a high safety score certifies real safety, covering becomes the fact that some inputs are safe and some are not, and calibration becomes the safety score tracking the truth probe. The coupling theorem then says a defense with a continuous safety score on a connected input space owns an input where the safety score is exactly at threshold and the input is exactly on the harm boundary. This is `boundary_coupling` in `HoF_13_BoundaryCoupling`, run on the safety score and the truth probe rather than on confidence and truth-distance. Prompt injection is the covering witness on the unsafe side: a family of inputs the defense must score, carrying both safe and unsafe members, which is exactly what coverage asks for. Nothing in the mathematics changes. Only the names on `M` and δ change.

4.4 The topological proof of the trilemma

We now prove the analytic trilemma in full. The structure is three steps. Coverage plus connectedness forces a confidence crossing. Calibration pins the truth-distance to zero at that crossing. Faithfulness demands the truth-distance be strictly negative there, and the two collide. Each step is a numbered result, and each has a verified counterpart named in place.

Step one: coverage forces a confidence crossing

Lemma 4.14 (Boundary existence). *Let `Q` be connected, let the answer map and the truth-distance be continuous, and let the model be covering. Then some question `q₀` has `δ(q₀, (M q₀).1) = 0`.*

Proof. The realized truth-distance `F(q) = δ(q, (M q).1)` is continuous, being the composite of the continuous map `q ↦ (q, (M q).1)` with the continuous δ. Coverage gives a question where `F` is negative and a question where `F` is positive. By the intermediate value theorem, Theorem 4.7, applied to `F` on the connected space `Q` at the value `0`, there is a q_0 with $F(q_0) = 0$. ■

This is `model_truth_boundary_nonempty` in `HoF_03_BoundaryCrossing`, and its coverage-hypothesis packaging is `covering_yields_truth_boundary_point` in `HoF_05_Coverage`. It uses no confidence and no calibration. It is pure topology: a continuous field that changes sign on a connected space has a zero. Notice that the boundary question exists before any of the safety conditions enter. The wall is a fact about the shape of the model's correctness, not about how the model reports its own reliability.

Two structural facts about this boundary set are worth recording, both verified and both independent of connectedness. First, the boundary is closed. The set of questions whose realized truth-distance is zero is the preimage of the closed singleton `{0}` under the continuous realized truth-distance, and preimages of closed sets are closed. This is `model_truth_boundary_isClosed`, and it holds over any topological question and answer spaces, with no connectedness needed. Second, every continuous path from a strictly-correct question to a strictly-wrong one crosses the boundary. Along the path the realized truth-distance is a continuous real function that starts negative and ends positive, so the one-variable intermediate value theorem gives a crossing on the path, which is `path_crosses_truth_boundary`. The boundary is not a fragile artifact of one clever argument. It is a closed set that blocks every route from truth to falsehood, and Lemma 4.14 is the statement that on a connected space such a route always exists to be blocked.

There is a companion crossing on the confidence side, proved the same way.

Lemma 4.15 (Confidence crossing). *Let `Q` be connected and let the confidence map be continuous. If some question has confidence below `1/2` and some has confidence above `1/2`, then some question q_0 has `(M q₀).2 = 1/2`.*

Proof. Apply the intermediate value theorem to the continuous confidence map `q ↦ (M q).2` at the value `1/2`. ■

This is `confidence_half_nonempty` and `model_confidence_half_set_nonempty` in `HoF_03_BoundaryCrossing`. On its own it is a statement about the confidence field alone. It becomes half of the trilemma once calibration ties the confidence witnesses to the coverage witnesses, which is the next step.

Step two: calibration pins the truth-distance

The trilemma's core obstruction is the combination of Lemma 4.15 with strict calibration. Coverage gives a question that is strictly correct and a question that is strictly wrong. Calibration turns these into confidence witnesses on both sides of `1/2`. The confidence crossing then lands a question exactly at `1/2`, and calibration read backward pins its truth-distance to exactly `0`.

Theorem 4.16 (Boundary-confidence coupling). *Let `Q` be connected, let the answer map, confidence map, and truth-distance be continuous, and let the model be covering and strictly calibrated. Then some question `q₀` satisfies both* `(M q₀).2 = 1/2` *and* `δ(q₀, (M q₀).1) = 0`.

Proof. Coverage gives `q_t` with `δ(q_t, (M q_t).1) < 0` and `q_f` with `δ(q_f, (M q_f).1) > 0`. Strict calibration converts these: from `δ(q_t, (M q_t).1) < 0` and the first biconditional we get `(M q_t).2 > 1/2`, and from `δ(q_f, (M q_f).1) > 0` and the second biconditional we get `(M q_f).2 < 1/2`. The confidence map is continuous and takes a value above and a value below `1/2`, so by Lemma 4.15 there is a `q₀` with `(M q₀).2 = 1/2`.

Now read calibration backward at `q₀`. If `δ(q₀, (M q₀).1)` were negative, the first biconditional would give `(M q₀).2 > 1/2`, contradicting `(M q₀).2 = 1/2`. If it were positive, the second biconditional would give `(M q₀).2 < 1/2`, again a contradiction. The only remaining possibility is `δ(q₀, (M q₀).1) = 0`. ■

This is `hallucination_trilemma_strict` in `HoF_07_TrilemmaCore`, and its defense- side twin, run on a safety score and a truth probe, is `boundary_coupling` in `HoF_13_BoundaryCoupling`. The theorem mentions no faithfulness. It is a coupling statement: a covering, calibrated, continuous model on a connected space owns a question where the confidence is exactly ambiguous and the answer is exactly on the wall. The point is real regardless of any safety policy. What a policy makes of it is the next step.

Step three: faithfulness contradicts the boundary

Theorem 4.17 (Analytic hallucination trilemma). *No continuous model on a connected question space can be simultaneously faithful, covering, and strictly calibrated.*

Proof. Suppose all three hold. By Theorem 4.16 there is a question `q₀` with `(M q₀).2 = 1/2` and `δ(q₀, (M q₀).1) = 0`. Faithfulness applies at `q₀`, because `(M q₀).2 = 1/2 ≥ 1/2`, and it demands `δ(q₀, (M q₀).1) < 0`. But calibration pinned that same quantity to `0`. A number cannot be both strictly negative and zero. Contradiction. ■

This is `hallucination_trilemma` in `HoF_07_TrilemmaCore`, with the fully unfolded statement `hallucination_trilemma_unfolded` alongside it. The three conditions are individually sensible and jointly impossible. The verified proof is the two-line composition the prose describes: extract the coupled point, then apply faithfulness to it and observe the sign clash with `linarith`.

The policy reading is where this becomes a taxonomy rather than a single impossibility. Faithfulness as stated is *threshold-inclusive*: it triggers at confidence at least `1/2`, so it triggers at the coupled point, and the contradiction follows. A *threshold-exclusive* guarantee, which triggers only at confidence strictly above `1/2`, does not trigger at the coupled point, since the confidence there is exactly `1/2`. Such a guarantee is consistent, but it is silent exactly where topology guarantees a question exists. The verified forms of both branches

are `closed_guarantee_impossible` and `open_guarantee_silent`. The lesson is that defining the guarantee with a strict inequality does not dissolve the obstruction. It only moves the model from "impossible" to "consistent but silent at a query it is guaranteed to face."

This taxonomy is why the coupling theorem, Theorem 4.16, is stated separately from the trilemma, Theorem 4.17, rather than folded into it. The coupling theorem mentions no guarantee at all. It is convention-free, and the coupled point exists under any threshold policy. The guarantee, inclusive or exclusive, only selects which clause of the taxonomy applies to that fixed point. An inclusive guarantee makes the system impossible. An exclusive guarantee makes it possible but silent. Neither makes the coupled point go away, because the coupled point was constructed before either guarantee was mentioned. A designer who reaches for the strict inequality thinking it removes the problem has misread where the problem lives. The problem lives in the geometry of the confidence and truth-distance fields on a connected space, and the guarantee is a downstream policy about how to treat a point that geometry has already fixed. This separation of the existence fact from the policy is the most useful structural feature of the analytic engine, and it is why the verified library keeps `hallucination_trilemma_strict` and `hallucinatio n_trilemma` as distinct theorems.

The forced question is the analytic liar

The coupled point q_0 of Theorem 4.16 is not an incidental byproduct. It is the analytic twin of the liar index a_0 of Proposition 1.14. In Chapter 1 the liar was an index whose self-application equalled its own negation, `f a₀ a₀ = ! (f a₀ a₀)`, the exact place where a Boolean verdict system breaks. Here q_0 is a question whose confidence equals the threshold and whose answer sits on the boundary, the exact place where a continuous calibrated model has no faithful answer to give.

Both witnesses are non-unique, and for the same reason. In Chapter 1, if the diagonal behavior has two names, both are liars, and nothing selects a canonical one. Here, if the confidence crosses `1/2` more than once, every crossing that also lands on the boundary is a coupled point, and the intermediate value theorem names none of them in particular. The witness is guaranteed to exist and left unaddressed, in both engines. The non-uniqueness is not a defect of the proofs. It is a faithful report that the obstruction is a region, not a single distinguished point, which is exactly what the tube results of Chapter 5 make quantitative.

4.5 A worked instance on the interval

It is worth carrying the abstract theorem down to a concrete space once, so the moving parts are visible. Take `Q` to be the closed interval `[0, 1]`, which is connected. Read `t ∈ [0, 1]` as a position along an interpolation from a

question the model answers truly at `t = 0` to a question it answers falsely at `t = 1`, the path an experiment walks when it morphs a true statement into its negation. Let the realized truth-distance be `F(t) = 2t - 1`, a continuous field that is negative for `t < 1/2`, zero at `t = 1/2`, and positive for `t > 1/2`. This model is covering, with `F(0) = -1 < 0` and `F(1) = 1 > 0`.

Lemma 4.14 applies at once. `F` is continuous, negative at `0`, positive at `1`, so it has a zero, and here the zero is the single point `t = 1/2`. Now suppose the confidence is calibrated. Strict calibration says the confidence is above `1/2` exactly where `F` is negative and below `1/2` exactly where `F` is positive. A continuous confidence meeting this must satisfy `(M t).2 > 1/2` for `t < 1/2` and `(M t).2 < 1/2` for `t > 1/2`. By Lemma 4.15 the continuous confidence crosses `1/2`, and it can only do so at `t = 1/2`, since that is the one place calibration allows. So the coupled point is `t = 1/2`, with confidence exactly `1/2` and truth-distance exactly `0`, precisely as Theorem 4.16 promises.

Now add faithfulness and watch it break. Faithfulness demands that wherever the confidence is at least `1/2`, the truth-distance is strictly negative. At the coupled point the confidence is exactly `1/2`, so faithfulness demands `F(1/2) < 0`. But `F(1/2) = 0`. The model cannot be built. Any attempt to make the confidence honest and to keep the safety promise fails at the one point the interval forces into existence. Notice what happens if the confidence tries to avoid `1/2` by jumping, say by being `0.6` for `t ≤ 1/2` and `0.4` for `t > 1/2` with no value in between. That confidence is not continuous, and dropping continuity is one of the named escapes, priced in the discrete section below. On a genuinely continuous confidence over a connected interval, there is no jump, and the coupled point is unavoidable.

The instance also shows what the exclusive-guarantee escape buys. Replace faithfulness with the strict-antecedent form, triggering only when the confidence is strictly above `1/2`. At the coupled point the confidence is exactly `1/2`, so the guarantee does not trigger, and no contradiction arises. The model is consistent. But it now owns the question `t = 1/2`, sitting on the truth boundary, about which its guarantee is silent. The wall did not move. The guarantee just stopped making a promise there.

4.6 Quantitative shadows: approximate coupling and positive slack

Before leaving the coupling theorem, we state the graded version that the slack of the model setup makes possible, because it is the bridge to Chapter 5 and it shows that the exact result is the sharp corner of a robust phenomenon. The full metric development, with tube radii and corridor widths, is the next chapter; here we state the two results that already follow from connectedness plus separation with slack, in prose, with the verified names attached.

Theorem 4.17b (Approximate coupling). *Let `Q` be connected, let the confidence be continuous, and let the model be `ε`-covering, `(ε, γ)`-separating at `τ`, and*

carry a threshold-inclusive guarantee. Then some question q_0 has `(M q₀).2 = τ` and `-ε ≤ δ(q₀, (M q₀).1) < 0`.

Proof sketch. The ε-coverage witnesses have truth-distance below `-ε` and above ε, and separation turns these into confidence witnesses above `τ + γ` and below `τ - γ`. The intermediate value theorem lands a `q₀` with confidence exactly `τ`. Separation with slack no longer pins the truth-distance to zero. Instead it confines it to the band where neither strict separation clause fires, which is `|δ(q₀, (M q₀).1)| ≤ ε`. The threshold-inclusive guarantee applies at confidence `τ`, forcing the truth-distance strictly negative. Combining, the coupled point has truth-distance in `[-ε, 0)`. ■

The verified statement is `two_slack_approx_coupling` in `HoF_12_Approximate`, which carries the truth slack ε and the confidence margin `γ` as separate parameters, exactly as the sketch above uses them.

This is `two_slack_approx_coupling`, and the exact coupling of Theorem 4.16 is the `ε = 0` limit, recovered as `exact_from_approx`. The immediate corollary is the one that matters for policy.

Corollary 4.17c (Positive slack is mandatory). *Under the hypotheses of Theorem 4.17b with a threshold-inclusive guarantee, the truth slack ε cannot be zero. Any feasible system carries `ε > 0`.*

Proof. If ε were zero, Theorem 4.17b would place the coupled point at truth-distance in `[0, 0)`, an empty range. So no coupled point could exist, but the intermediate value theorem produced one. The only way out is `ε > 0`. ■

This is `truth_slack_must_be_positive`. Read it as a budget. A system that wants to keep a threshold-inclusive guarantee must keep a strictly positive margin of correctness on the confident side, and the infimum of feasible slack is a measurable width of the system's truth boundary. Under the guarantee, no question with confidence between `τ` and `τ + γ` can sit exactly on the boundary, which is `no_boundary_in_upper_band_two_slack`. These are the first quantities the analytic engine produces that the diagonal cannot, and they are the whole reason Chapter 5 follows this one.

4.7 Symmetry in place of coverage: the antipodal variant

Coverage is a mild hypothesis, but it can be removed entirely when the question space carries a symmetry. The relevant symmetry is a fixed-point-free continuous involution, the abstract shadow of the antipodal map `x ↦ -x` on a sphere. Read it as semantic negation acting on the representation space: a map that sends a question to its negation, that is its own inverse, and that never fixes a question.

Definition 4.18 (Antipodal action). An *antipodal action* on a topological space `Q` is a map `σ : Q → Q` that is continuous, self-inverse (`σ (σ q) = q` for all `q`), and fixed-point-free (`σ q ≠ q` for all `q`). A real- valued function `g : Q → ℝ` is *odd* under `σ` when `g (σ q) = -g q` for all `q`.

The verified structure is `AntipodalAction` and the predicate is `AntipodalOdd`, both in `HoF_08_BorsukUlam`. The relevant instance is that layer normal-

ization places transformer states on a sphere-like shell, and semantic negation is a candidate antipode, so g odd under σ idealizes a truth-distance that flips sign under negation.

Theorem 4.19 (Odd fields vanish, the one-dimensional Borsuk-Ulam analog). *Let `Q` be connected and `σ` an antipodal action. Any continuous `g : Q → ℝ` odd under `σ` has a zero.*

Proof. Pick any question `q`. If `g q = 0` we are done. Otherwise `g q` is strictly positive or strictly negative. By oddness `g (σ q) = -g q`, which has the opposite sign. So g takes both signs, at `q` and at `σ q`, and by the intermediate value theorem on the connected space `Q` it has a zero somewhere between them. ■

This is `antipodal_odd_has_zero`. The point to see is that no coverage hypothesis was assumed. Oddness supplies the two signs for free. A single question and its negation image already straddle zero, because the field is negative at one and positive at the other by the sign flip. Where coverage was an assumption that the model is sometimes right and sometimes wrong, oddness derives that straddle from the symmetry alone.

There is a pleasant inversion in this. Equivariance under a symmetry is usually a design virtue, a property one builds into a network on purpose so that it treats a question and its negation consistently. Here the same property is the engine of the obstruction. A model that respects negation, so that negating a question negates the truth-distance, is a model whose realized truth-distance is odd, and an odd continuous field on a connected symmetric space must vanish. The better behaved the symmetry, the more inescapable the wall. This is worth keeping in mind against the intuition that more structure means more freedom. More structure here means less room for the field to avoid zero.

Theorem 4.20 (Antipodal trilemma). *Let `Q` be connected with an antipodal action `σ`, let the model be continuous and strictly calibrated at `1/2`, and let the realized truth-distance be odd under `σ`. Then some question `q` has `(M q).2 = 1/2` and `δ(q, (M q).1) = 0`.*

Proof. By Theorem 4.19 the odd realized truth-distance has a zero `q`, so `δ(q, (M q).1) = 0`. Strict calibration at `q` then pins the confidence: if the confidence were above `1/2` the first biconditional would force the truth-distance negative, and if below, the second would force it positive, both contradicting the zero. So `(M q).2 = 1/2`. ■

This is `antipodal_hallucination_trilemma`. Adding a faithfulness guarantee to its hypotheses produces the same contradiction as Theorem 4.17, by the same final step, and now with no coverage assumption anywhere. A negation-closed family of questions meets the truth boundary by symmetry.

It is worth comparing the two engines' non-triviality demands directly, since that is the only place they differ. The coverage engine assumes the model is sometimes right and sometimes wrong, an assumption about the model's outputs. The antipodal engine assumes the question space has a negation symmetry and the truth-distance respects it, an assumption about the space and the field, not about the outputs. A model could be antipodally symmetric

and never happen to answer strictly falsely on any single fixed question one inspects, and the antipodal engine would still force a boundary point, because oddness guarantees the two signs across each orbit whether or not any particular question realizes both. Conversely, a covering model on a space with no useful symmetry falls to the coverage engine and is untouched by the antipodal one. The two are genuinely different sufficient conditions for the same conclusion, and a given system may satisfy either, both, or neither. When it satisfies neither, the obstruction may simply be absent, which is the honest content of the escape routes.

The antipodal engine is robust to the symmetry being only approximate, which matters because real negation does not reflect truth directions exactly. If oddness holds up to a defect ε, meaning `|g (σ q) + g q| ≤ ε` for every `q`, then some question has `|g q| ≤ ε/2`, and at `ε = 0` this recovers exact vanishing. The verified robust form is `approx_odd_near_zero`, and the concrete instantiation on the unit sphere of `ℝ^d` for `d ≥ 2`, using Mathlib's `isConnected_sphere`, is `sphere_odd_has_zero`. The near-boundary point degrades linearly with the measured defect, which is what lets the experiments of the companion paper report a nonzero defect and still see the predicted near-crossing.

The sphere and layer normalization

The antipodal picture has a concrete home. Layer normalization rescales each transformer state to a fixed norm, which places the states on a sphere-like shell, up to a learned affine map. The unit sphere of `ℝ^d` for `d ≥ 2` is connected, which is Mathlib's `isConnected_sphere`, and the antipodal map `x ↦ -x` is a continuous free involution on it, the geometric antipode. If semantic negation acts as this antipode, and the truth-distance is odd under it, then a continuous truth-distance field on the sphere must vanish somewhere on the sphere. This is `sphere_odd_has_zero`, proved by taking any point, comparing the field there with its value at the antipode, and running the intermediate value theorem along the connected sphere between them.

The dimension hypothesis `d ≥ 2` is where connectedness of the sphere comes from, since the zero-dimensional sphere is two points and disconnected, the same degeneracy that makes the diagonal engine, not this one, the right tool for a two-point outcome set. For any representation dimension used in practice the sphere is connected and the antipodal obstruction applies. The one honest caveat is that exact oddness is an idealization. The companion paper measures the oddness defect and finds it nonzero but small, smallest for the strongest model, and `approx_odd_near_zero` is exactly the theorem that keeps a near-boundary point in existence under that measured defect. The idealization degrades gracefully rather than collapsing.

4.8 The discrete core: no topology once a boundary question exists

The topology in the argument does exactly one job. It manufactures a boundary question, a q_0 with $\delta(q_0, (M\ q_0).1) = 0$. Everything after that is arithmetic. This section isolates the arithmetic and shows that once a boundary question is handed to us, no topology, no continuity, no connectedness, and no finiteness are needed to finish. The verified home of this observation is `HoF_10_PureDiscrete`.

The reason the discrete core matters is that token spaces are finite and one may object that connectedness is doing illegitimate work. The discrete analysis prices the objection exactly. Without topology we must assume the boundary witness rather than derive it. That is the entire cost. The contradiction machinery is untouched.

It is worth dwelling on what "pricing an objection" means, because it is the honest way to handle an idealization. A theorem with a strong hypothesis invites the reader to reject the hypothesis and walk away. The disciplined response is not to defend the hypothesis to the death but to show exactly how much of the conclusion survives without it. Connectedness is the idealization here. It is a good model of an ambient representation space over which probes and steering vectors are deployed at arbitrary interpolated points, and a poor model of a token-driven reachable set, which may be scattered. The discrete core is what remains when connectedness is dropped entirely. The answer is that the whole contradiction remains, and the only thing lost is the free construction of the boundary question. In the continuum the boundary question is a theorem. In the discrete world it is a hypothesis. Nothing else moves. This is a precise accounting, and it is more useful than either insisting the space is connected or conceding that finiteness ruins everything.

We can state and check the core in self-contained core Lean, over the integers with the threshold shifted to `0` in place of `1/2`, so no real numbers or libraries are needed. First the flip, the discrete shadow of the complement map, proved by the same finite check that settles the Boolean flip in Chapter 1.

```
theorem c4_flip_no_fixed_point : ∀ b : Bool, (!b) ≠ b :=
  by decide
```

Next the pinning-and-clash core itself. Model the confidence and the truth- distance as two integers, with calibration as the two biconditionals around the threshold `0`, and faithfulness as the implication that non-negative confidence forces a strictly negative truth-distance. The claim is that a boundary question, here a truth-distance of exactly `0`, makes the conjunction contradictory. The proof is pure arithmetic: calibration pins the confidence to `0`, faithfulness then demands the truth-distance be strictly negative, and that clashes with the boundary value.

```
theorem c4_boundary_question_impossible
    (conf truth : Int)
    (cal_pos : 0 < conf ↔ truth < 0)
    (cal_neg : conf < 0 ↔ truth > 0)
    (faithful : 0 ≤ conf → truth < 0)
    (boundary : truth = 0) : False := by
  have hnp : ¬ (0 < conf) := by
    intro h; have := cal_pos.mp h; omega
  have hnn : ¬ (conf < 0) := by
    intro h; have := cal_neg.mp h; omega
  have hz : conf = 0 := by omega
  have hlt : truth < 0 := faithful (by omega)
  omega
```

This is the integer transcription of `boundary_question_impossible`. Read the proof against Theorem 4.17. The two `have` blocks are calibration pinning the confidence to the threshold, exactly the step Theorem 4.16 performed with the intermediate value theorem replaced by the assumed boundary value. The last two lines are faithfulness demanding a strict inequality and the sign clash. No topology appears anywhere. The result `c4_boundary_question_impossible` is the trapped logical core, and both the continuous trilemma and the antipodal trilemma factor through it.

The remaining question is how, in a discrete world, one would ever get a boundary value to feed the core. Along a finite path of questions the truth-distance does not have to hit zero. It can jump across it between adjacent steps. That is the discrete intermediate value theorem, and it produces a straddling pair rather than a boundary state. The following self-contained result is the discrete sign change, the honest discrete replacement for the crossing.

```
theorem c4_discrete_sign_change (g : Nat → Int) :
    ∀ n, g 0 < 0 → 0 ≤ g n → ∃ i, i < n ∧ g i < 0 ∧ 0 ≤ g
    (i + 1) := by
  intro n
  induction n with
  | zero => intro h0 hn; exact absurd hn (by omega)
  | succ m ih =>
    intro h0 hn
    by_cases hm : 0 ≤ g m
    · obtain ⟨i, hlt, hneg, hpos⟩ := ih h0 hm
      exact ⟨i, Nat.lt_succ_of_lt hlt, hneg, hpos⟩
    · exact ⟨m, Nat.lt_succ_self m, by omega, hn⟩
```

The statement says that a discrete field negative at the start and non-negative at the end has an adjacent pair where it is still negative and has just become non- negative. It never claims a step where the field is exactly zero. That gap is the whole difference between the discrete and continuous

settings, and it is priced precisely: the continuum upgrades the adjacent straddle into an exact witness, and the discrete world supplies only the straddle. The verified counterpart is `discrete_sign_change`. The proof here is a clean induction: the base case is a contradiction, since a field cannot be both negative and non-negative at the same index, and the step either recurses on the shorter prefix or, when the field is already negative at the last interior index, returns that index as the crossing.

Two consequences of the discrete core deserve stating in prose, both verified in `HoF_10_PureDiscrete`. First, the three-sided version: a model that is faithful and strictly calibrated and that additionally has witnesses for strictly correct, exactly boundary, and strictly wrong answers is impossible, with no topological hypothesis, because the assumed boundary witness feeds the core directly. This is `discrete_hallucination_trilemma`, and it trades the topological hypothesis of connectedness for the stronger coverage hypothesis of a supplied boundary point. Second, and more revealing, faithfulness is *equivalent* to boundary-freeness under calibration: a strictly calibrated model is faithful if and only if it has no boundary questions at all. This is `faithful_iff_boundary_free`, and it says the obstruction is not incidental. Faithfulness just is the condition of having no questions on the wall. The continuous engine and the antipodal engine both work by manufacturing precisely the thing faithfulness forbids.

Adding randomness

A model can be stochastic, sampling its answer and its confidence rather than computing them deterministically. One might hope randomness dodges the obstruction, since a sampled confidence need not equal `1/2` on any single draw. It does not dodge it. There are two ways to see this, and both are verified.

The first works on expected fields. If the confidence and the truth-distance are averaged over the sampling, the averages are continuous fields on the connected question space whenever the underlying fields are, and the coupling theorem applies to them verbatim. The conclusion is a question where the expected confidence is `1/2` and the expected truth-distance is `0`, which is `stochastic_coupling_expected`. Averaging is a continuous operation, so the wall survives it.

The second converts an expectation statement into a statement about samples, and it is the one place in the whole development where the theory derives outright falsehood rather than uncertainty. Let the sampling live on a probability space, let the truth-distance have mean zero, and suppose sample-faithfulness holds almost everywhere, meaning that whenever the sampled confidence is at least `1/2` the sampled truth-distance is strictly negative. Suppose also that the confidence is at least `1/2` with positive probability. Then the truth-distance is strictly positive with positive probability. In model terms, a system that is faithful on almost every sample and confident with positive probability at a coupled point must emit strictly false answers with positive

probability. This is `stochastic_dichotomy`, and its proof is a clean averaging argument.

Proof sketch of the dichotomy. Suppose the truth-distance were almost never positive. On the confident event, which has positive probability, sample- faithfulness makes it strictly negative, so it is strictly negative on a set of positive measure and non-positive everywhere else. A quantity that is non-positive almost everywhere and strictly negative on a set of positive measure has strictly negative mean. That contradicts the mean-zero hypothesis. So the truth-distance is strictly positive with positive probability. ■

Randomization therefore spreads the boundary across samples instead of removing it. The deterministic theorems yield an ambiguous point; the stochastic dichotomy yields, from the same premises plus mean-zero truth, positive-probability error. The wall is either a point you must pass through or a positive-probability event you must incur, and no amount of sampling escapes both.

4.9 Where the two engines meet

We can now make the analogy of the primer exact. The claim is that the diagonal of Chapter 1 and the intermediate value theorem of this chapter locate the *same* boundary object, and that the Boolean flip and the real complement map are two descriptions of the same wall. This is not a loose resemblance. It is recorded as a theorem in `Foundation.F_04`, and we reconstruct its content here.

Start with the flip. In Chapter 1 the engine needed an outcome flip with no fixed point. For Booleans that flip is negation, and `bool_not_fpf` is the fact that it has none, which is what turns `lawvere` into `cantor` and through it into every diagonal impossibility. For real confidences the natural flip is the *complement controller* `y ↦ 1 - y`. It is not fixed-point-free. It has exactly one fixed point, and that fixed point is the threshold `1/2`, because `1 - y = y` holds if and only if `y = 1/2`. This is `half_complement_fixed_point`, and its off-threshold form, that `y ↦ 1 - y` fixes nothing away from `1/2`, is `complement_no_fp_off_half`.

The single fixed point is the entire difference between the two engines, and it is the reason they reach the same place. The Boolean flip has no fixed point, so Lawvere's theorem, which always produces a fixed point of the flip, produces a contradiction. The real complement has one fixed point, at `1/2`, so Lawvere's theorem, run with the complement controller, produces not a contradiction but a point: a diagonal question where the confidence equals `1/2`. Feed a surjective self-prediction map `s : A → A → ℝ` to the diagonal, apply the complement controller, and the naming equation gives `s a a = 1 - s a a`, which solves to `s a a = 1/2`. This is `calibration_diagonal_hits_half`, and its impossibility form, that a surjective system cannot avoid `1/2` on its diagonal, is `calibration_no_strict_avoidance`.

Compare the two routes to `1/2`. The intermediate value theorem reaches it by connectedness: the confidence is below and above the threshold, so it crosses. The diagonal reaches it by self-reference: the confidence names a behavior that complements its own diagonal value, so it equals its own complement, which is `1/2`. One argument bisects a connected interval, the other evaluates a naming equation. The output is identical, the value `1/2` at a distinguished question, and from there both proofs run the same calibration-versus-faithfulness clash. The verified statement that the whole trilemma follows from surjectivity alone, with no topology, is `hallucination_via_lawvere`, whose proof is Theorem 4.17's final step attached to `calibration_diagonal_hits_half` in place of Theorem 4.16.

So the correspondence table of the primer is now a list of theorems. Self-reference and connectedness are two ways to trap a value. The Boolean negation `(!·)` and the real complement `y ↦ 1 - y` are the two flips, one with no fixed point and one with a single fixed point at the threshold. The liar index and the boundary question are the same distinguished point under two constructions. The diagonal impossibility `no_reflective_verdict` of Chapter 1 and the analytic `hallucination_trilemma` of this chapter are, in the precise sense `Foundation.F_04` records, two spellings of one fact. The topology in `HoF_07_TrilemmaCore` is a convenient way to enforce surjectivity onto a connected slice of the reals; any other route to that value, including the universality-of- in-context-learning route, yields the very same boundary point and the very same contradiction.

It is worth being precise about what is shared and what is not. What is shared is the boundary object and the final clash. What differs is the hypothesis that manufactures the object. The diagonal needs a system that names its own behaviors, `no_universal` read as a demand rather than a conclusion. The intermediate value theorem needs a connected domain and continuous fields. Neither hypothesis implies the other. A finite discrete model can be universal in the diagonal sense and carry no topology at all, and a smooth model on a manifold can be continuous and connected while naming almost none of its own behaviors. The two engines cover disjoint failure modes and agree wherever both apply, which is exactly what one wants from two proofs of one theorem.

There is a categorical way to say the shared part that makes the unification more than a coincidence of two calculations. Recall from Remark 1.5 that Lawvere's theorem holds in its cleanest form for a *section*, a right inverse `s : (A → Y) → A` of evaluation, and that this is the form true verbatim in any cartesian closed category. The complement controller `y ↦ 1 - y` is an endomap of the reals with a single fixed point, and Lawvere's theorem applied to a section of a real-valued self-prediction map delivers that fixed point on the diagonal. The intermediate value theorem is not a categorical statement, but it delivers the same value, and the reason is that both are extracting the fixed point of the same controller. The topology in `HoF_07_TrilemmaCore` is one way to guarantee that the confidence sweeps through a connected slice of the reals wide enough to contain the controller's fixed point. Surjectivity of the self-

prediction map is another. Once either guarantee is in hand, the fixed point of `y ↦ 1 - y` is reachable, and it is `1/2`.

The whole unification is packaged in the companion library at two levels. The statement that the topological and categorical proofs are the same theorem is `hallucination_unification_documentation` and its surrounding lemmas in `Foundation.F_04`. The statement that the several trilemmata of Chapter 3, the hallucination form, the defense form, and the truth-boundary coupling, are instances of one master result is `trilemmata_unified`, with the bundled statement `hallucination_master_theorem`. The reader who wants to see the diagonal and the intermediate value theorem land on the identical object at the level of kernel- checked terms should read `F_04` first, then `F_13_UnifiedTrilemmata`. The prose here is a guide to those files, not a substitute for them.

One geometric remark closes the meeting point. When the truth-distance is a linear probe, the boundary is not just a level set but an affine hyperplane of codimension one, a translate of the probe's kernel, by rank-nullity. This is `linear_probe_boundary_coset` and `linear_probe_boundary_dim`. For a differentiable nonlinear probe the boundary is a hyperplane to first order at every regular point, with the tangent hyperplane being the kernel of the derivative, verified as `nonlinear_boundary_tangent` and `tangent_hyperplane_dim`, and every regular point is itself a sign crossing that generates local coverage for free, `regular_point_is_crossing`. The boundary object the two engines share is thus not an abstract point in a general space. In the linear case it is a flat wall of one dimension less than the representation space, and the coupled point is a point on that wall carrying threshold confidence. The full geometry is Chapter 5's subject, but it is the same wall both engines have been walking toward.

4.10 Where the two engines diverge

The engines agree on the existence of the boundary object. They diverge sharply on what else they can tell you about it, and the divergence is the reason both chapters exist.

The diagonal gives a witness and nothing quantitative. It does not say how many liar indices there are, how hard a_0 is to find, or what it would cost a search to reach one. It cannot, because it uses no metric. Its only input is a naming equation, and a naming equation has no notion of distance, width, or margin. This is the content of Remark 1.18, and it is a real limit, not a temporary gap. The diagonal is silent on geometry because geometry is not among its hypotheses.

The intermediate value theorem, on its own, is equally silent, as Remark 4.8 noted. Bare connectedness gives existence and no location. The difference is that the analytic engine can be fed more. Add a distance to the question space and require the confidence and truth-distance to be Lipschitz, and the bare crossing becomes a quantitative object. The decisive regions, where the

confidence is bounded away from `1/2` on either side, must be held apart by a distance controlled by the Lipschitz constant and the margin. The coupled point sits at the center of a tube of near-ambiguous questions whose radius is set by the same constants. Truth-side slack must be strictly positive, so exact-zero slack is infeasible, and its infimum is a measurable width of the boundary. These are the verified results `confidence_gap_width`, `ambiguity_tube`, `truth_margin_tube`, `two_slack_approx_coupling`, and `truth_slack_must_be_positive`, and they are the subject of Chapter 5.

None of that is available to the diagonal, and none of it is available to the intermediate value theorem without the extra metric hypotheses. The right way to hold the two engines together is this. Use the diagonal when the system names itself and you want the cleanest possible existence proof with the smallest possible trust story, a `decide` on a finite flip and three lines of term-mode. Use the intermediate value theorem when the system lives on a connected space, and then, if you also have a metric, keep going into Chapter 5 and read off the widths. Both routes reach the boundary. Only the second route measures it.

There is also a difference in what each engine says about how to escape. The diagonal is escaped by breaking universality, by ensuring the system cannot name the diagonal behavior. That is a statement about the expressive power of the naming, and it is often hard to arrange without crippling the system, since the whole point of a general system is to handle arbitrary inputs. The analytic engine is escaped by breaking connectedness, coverage, continuity, or exact separation, and each of these is a concrete design lever. Abstention on an open region disconnects the space. Restricting to retrieval-grounded content that is never strictly false breaks coverage. Hard decoding breaks continuity, at the price the discrete section computed. Band separation with positive slack replaces exact separation, at the price of a positive truth-boundary width. The analytic engine, because it names its hypotheses as geometric and quantitative conditions, hands the designer a menu of priced escapes. The diagonal offers essentially one, and it is expensive. This is a practical reason to prefer the analytic engine wherever a system carries geometry, even though the two prove the same existence fact.

4.11 The engine in one page

It helps to have the whole argument compressed once, stripped of the applied readings, so the logical skeleton is visible. The analytic engine is four moves.

The first move is a continuity fact. The realized truth-distance and the confidence are continuous real-valued functions on the question space, because they are built from continuous pieces by composition and pairing. Continuity is what lets the next move happen.

The second move is the intermediate value theorem. On a connected space a continuous real field that takes a negative value and a positive value takes the value zero, and one that takes a value below a threshold and a value above

it takes the threshold. This is the only analytic input, and it is Theorem 4.7, verified as `IsPreconnected.intermediate_value₂`.

The third move is coupling. Coverage gives the truth-distance both signs, and calibration converts these into confidence values on both sides of the threshold, so the intermediate value theorem lands a question at the threshold, and calibration read backward pins the truth-distance to zero there. The output is a single question that is exactly ambiguous in confidence and exactly on the boundary in truth. This is Theorem 4.16, verified as `hallucination_trilemma_strict`.

The fourth move is the clash. Faithfulness demands the truth-distance be strictly negative wherever the confidence reaches the threshold, and the coupled point has confidence at the threshold and truth-distance zero. Strictly negative and zero cannot both hold. This is Theorem 4.17, verified as `hallucination_trilemma`.

Every variant in the chapter is a substitution into this skeleton. The antipodal variant replaces coverage in the third move with a symmetry that supplies both signs for free. The discrete core keeps only the third and fourth moves and assumes the boundary question the first two moves would have produced. The stochastic variant runs the skeleton on expected fields, or converts it into a positive- probability error by averaging. The diagonal engine of Chapter 1 replaces the second move, the intermediate value theorem, with a naming equation that reaches the same threshold value. The skeleton does not change. What changes is which hypothesis supplies the coupled point, and the fourth move is the same clash every time.

4.12 Historical and bibliographic notes

The intermediate value theorem is Bolzano's, and the modern formulation as preservation of connectedness under continuous maps is standard in any point-set topology text. The reading of it as an impossibility engine for learning systems is recent, and the machine-checked development these notes follow is the companion `HallucinationProofs` library, with the topological core in `HoF_03`, `HoF_05`, and `HoF_07`, the antipodal variant in `HoF_08`, and the discrete core in `HoF_10`. The unification of the analytic and diagonal proofs is `Foundation.F_04`, which records that both extract the same `1/2` point from a surjectivity-style hypothesis and then run the same clash. The one-dimensional Borsuk-Ulam analog is a scalar relative of the full antipodal theorem of Borsuk; the sphere instantiation uses Mathlib's connectedness of spheres. The companion paper "Truth Has a Boundary" states the coupling theorem for representation geometry, verifies every result in Lean against Mathlib with no sorry placeholders, and reports the predicted coupling in five small language models. The framing there, that exact threshold uncertainty is a topological property of representation space, is the applied face of Theorem 4.16.

The statistical inevitability results in the literature run on a different track

and reach a different conclusion, and the contrast sharpens what this chapter does. Those results argue that a calibrated model must hallucinate on facts it saw rarely, or that hallucination is undecidable in a computability sense, and they conclude that confident falsehood is unavoidable in the aggregate. The engine here is topological, lives in representation geometry, localizes its conclusion at a specific boundary question, and is more modest in what it asserts, since the deterministic theorems yield uncertainty at the coupled point rather than confident error. Only the stochastic dichotomy derives falsehood, and it needs the extra mean-zero hypothesis to do it. The two lines of work are complementary. One counts errors; this one locates a wall.

The verified library is organized so that the logical dependencies match the chapter's narrative. `HoF_03_BoundaryCrossing` holds the raw intermediate value theorem applications, the boundary and confidence crossings, and the closedness and path-crossing facts. `HoF_05_Coverage` isolates the coverage condition in its several strengths and proves the coverage-to-boundary workhorse. `HoF_07_TrilemmaCore` assembles the coupling theorem and the trilemma from these pieces. `HoF_08_BorsukUlam` develops the antipodal variant, its robust and sphere forms, and prints an axiom audit. `HoF_10_PureDiscrete` extracts the topology-free core and proves the factorization of the continuous result through it. `Foundation.F_04` and `F_13_UnifiedTrilemmata` sit above all of these and record the unification with the diagonal engine. A reader who wants to trace a single theorem from prose to kernel can follow this map, and every result carries a name that appears in exactly one of these files. The reduction of the main theorems to the three standard kernel axioms is checked in the final verification file, so the trust story is not a claim in the prose but a machine-audited fact.

4.13 Exercises

Exercise 4.1. Prove directly from Definition 4.3 that a two-point discrete space is disconnected, and that the one-point space is connected. Conclude that `Bool` is disconnected and explain, in one sentence, why the analytic engine cannot run on it.

Exercise 4.2. Show that the continuous image of a connected space is connected, and deduce that continuity plus connectedness of the domain rules out a continuous surjection from a connected space onto a two-point discrete space. Relate this to Lemma 4.15: what would a confidence map with no crossing say about the connectedness of `Q`?

Exercise 4.3. Fill in the omitted lines of Proposition 4.6. Given an interval `S ⊆ ℝ` and a putative separation by open `U` and `V`, take a point in `S ∩ U` and a point in `S ∩ V`, form the supremum of `S ∩ U` below the second point, and derive a contradiction with the openness of both pieces.

Exercise 4.4. State and prove the two-function form of the intermediate value theorem used in the library: for continuous `f, g` on a connected `X` with

`f a ≤ g a` and `g b ≤ f b`, there is an `x` with `f x = g x`. Derive Theorem 4.7 as the special case `g` constant.

Exercise 4.5. In the proof of Theorem 4.16, exactly one direction of each calibration biconditional is used going forward and exactly one going backward. Identify which, and show that a one-way calibration, only the forward directions, is not enough to pin the truth-distance to zero at the crossing.

Exercise 4.6. Show that Theorem 4.17 fails if faithfulness is weakened to the inclusive-boundary form `(M q).2 ≥ 1/2 → δ(q, (M q).1) ≤ 0`. Construct, in prose, a model on a connected space meeting all three weakened conditions, and locate its coupled point. This is the exclusive-guarantee escape of the policy taxonomy.

Exercise 4.7. Reprove `c4_boundary_question_impossible` with the threshold at an arbitrary integer `τ` in place of `0`, keeping the proof in core Lean. Then explain in one paragraph why the choice of threshold is a convention that never affects the existence of the coupled point.

Exercise 4.8. The proof of `c4_discrete_sign_change` inducts on the right endpoint. Rewrite it to induct on a bound and return the *least* crossing index, and argue that the continuous intermediate value theorem has no analogous canonical choice. Connect this to the non-uniqueness discussion after Theorem 4.16.

Exercise 4.9. Prove that the truth boundary `{q | δ(q, (M q).1) = 0}` is closed whenever δ and the answer map are continuous, using only that a singleton in $\mathbb{R}$ is closed and that preimages of closed sets under continuous maps are closed. This is `model_truth_boundary_isClosed`; write the argument yourself.

Exercise 4.10. In the antipodal setting, show that an odd continuous field on a connected space with a free involution has at least two zeros unless it is identically zero on the orbit of some point. What does the second zero correspond to in the model reading?

Exercise 4.11. Prove the approximate-oddness bound in prose: if `|g (σ q) + g q| ≤ ε` for all `q` and `g` is continuous on a connected space, then some `q` has `|g q| ≤ ε/2`. Show the bound is tight by exhibiting a field that meets it with equality.

Exercise 4.12. Verify the claim of Section "Where the two engines meet" by hand: given a surjective `s : A → A → ℝ`, use the diagonal with the controller `y ↦ 1 - y` to produce an `a` with `s a a = 1/2`. Identify precisely which step replaces the intermediate value theorem, and which step is shared with Theorem 4.16.

Exercise 4.13. The complement controller `y ↦ 1 - y` has a fixed point while the Boolean flip `(!·)` has none. Explain why this difference makes Lawvere's theorem yield a contradiction in the Boolean case and a distinguished point in the real case, and state the general principle: what does the fixed-point set of the controller become in each reading?

Exercise 4.14. Give an example of a system that satisfies the diagonal hypothesis (universality) but not the analytic one (connectedness with continu-

ous fields), and an example that satisfies the analytic hypothesis but not the diagonal one. Conclude that neither engine subsumes the other.

Exercise 4.15. (Harder.) The discrete core needs a boundary question handed to it, while the continuous engine derives one. Formulate a hypothesis on a finite question space, weaker than connectedness, under which a boundary question is still forced. Compare it to three-sided coverage and to the existence of an odd field under a free involution.

Exercise 4.16. (Harder.) Suppose the confidence map is continuous but the answer map is not. Which of Lemma 4.14, Lemma 4.15, and Theorem 4.16 survive, and which fail? Rework the hypotheses of Theorem 4.16 to use the smallest continuity assumptions that still deliver the coupled point.

Exercise 4.17. (Open-ended.) Chapter 5 adds a metric and extracts the width of the ambiguity tube. Before reading it, conjecture how the tube radius should scale with the Lipschitz constant of the confidence map and the confidence margin, and sketch which step of Theorem 4.16 the metric refines. Compare your conjecture to `ambiguity_tube` once you reach it.

Exercise 4.18. (Open-ended.) The antipodal variant removes coverage using a symmetry. Speculate on what a system would need to satisfy both the diagonal hypothesis and the antipodal hypothesis at once, and describe the boundary object in that combined setting. Chapter 8 returns to this for J-space.

Exercise 4.19. Verify the multi-turn claim in detail. Assuming `Q` is connected, prove that `Q × Q` is connected by the slice argument sketched in the primer, and conclude by induction that `Q^T` is connected. Then state the coupling theorem for a two-turn model `M : Q × Q → A × ℝ` and observe that its proof is Theorem 4.16 with `Q^2` in place of `Q`. Where, if anywhere, did the second turn enter the argument?

Exercise 4.20. Prove the stochastic dichotomy from the sketch in the discrete section. Let `d` be integrable with mean zero on a probability space, suppose `c ≥ 1/2` implies `d < 0` almost everywhere, and suppose `c ≥ 1/2` has positive probability. Show `d > 0` has positive probability. (Hint: assume not, split the integral of `d` over the confident and non-confident events, and derive a strictly negative mean.) Then explain why this is the only result in the chapter that concludes with confident error rather than uncertainty.

Exercise 4.21. (Harder.) The exclusive-guarantee escape leaves a system consistent but silent at the coupled point. Design, in prose, a reporting policy that at least makes the silence visible, for instance by having the system flag any query whose confidence lands within a small band of the threshold. Using Theorem 4.17b and the corollary on positive slack, argue that such a band cannot be made empty, so the flag can never be guaranteed never to fire.

4.14 References

Borsuk, K. (1933). Drei Sätze über die n-dimensionale euklidische Sphäre. *Fundamenta Mathematicae* 20, 177–190.

5 The Geometry of Attacks

Abstract. The diagonal yields a bad point but says nothing about how many there are, how large a region around them is, or what an attacker pays to reach one. This chapter supplies that quantitative half using analysis and measure. It studies alignment-deviation surfaces on metric spaces, the boundary as a fixed point of defense, steepness forcing oscillation, coarea and cone bounds, and the dimension of the boundary. It then examines how scale and dimension shift cost between attacker and defender.

Keywords: Attack geometry, Lipschitz functions, Coarea bound, Concentration of measure, Adversarial examples

The diagonal gives you one thing and refuses to give you anything else. It gives you a bad point. It builds, out of pure self-reference, an index the system cannot correctly name, and from that index it manufactures a contradiction. What it never tells you is how many such points there are, how large a region around them behaves badly, how a search would find one, or what an adversary would have to spend to reach one. Those questions are not failures of the diagonal. They are simply outside its language. The diagonal speaks about surjections and fixed-point-free maps; it has no vocabulary for volume, distance, or gradient.

This chapter is about the other half of the theory, the half that does have that vocabulary. Here the objects are not abstract systems `f : A → A → Y` but real functions on real spaces: an *alignment deviation* surface `f : X → ℝ` measuring how far a behavior sits from acceptable, a threshold `τ` separating safe from unsafe, and a metric that lets us talk about how far apart two behaviors are. The tools are analysis and measure theory, not category theory. None of the results here is a diagonal argument. They are the intermediate value theorem, the Lipschitz condition, the coarea inequality in disguise, concentration of measure, and the pigeonhole principle applied to grids. Together they answer the quantitative questions the diagonal left on the table.

The material follows the companion `ManifoldProofs` development, a machine-checked library of roughly four hundred results built on Mathlib. Because that library is analytic and measure-theoretic, and because the book's own Lean package carries no Mathlib, this chapter states everything in prose. Each theorem is stated carefully, proved or sketched with enough rigor to be checked by hand, and tagged with the exact name of the corresponding `MoF_*`

result so you can read the formal statement yourself. Treat the citations as the ground truth. Where the prose here simplifies, the Lean does not.

One organizing idea runs through the whole chapter, and it is worth stating before any theorem. *Existence is qualitative; cost is quantitative.* The impossibility theorems of the earlier chapters say the boundary is there and cannot be removed. The theorems here say how the boundary is shaped, how much room the unsafe region occupies, and how the price of attack and defense scales. A deployment is not decided by whether a bad point exists, since one always does. It is decided by how expensive it is to live next to that point. The geometry is where the expense lives.

5.1 From the diagonal to the boundary

Fix a space `X` of behaviors. Think of `X` as a set of prompts, a set of activation vectors, an embedding space, or in the cleanest case a Euclidean space `ℝ^n`. Fix a continuous function `f : X → ℝ`, the alignment deviation, with the reading that `f x` is large when the behavior `x` is far from acceptable and small when `x` is aligned. Fix a real number `τ`, the safety threshold. This triple `(X, f, τ)` is the entire setting. Everything in the chapter is a statement about it.

Definition 5.1 (Regions). The *safe region* is `{x | f x < τ}`, the *unsafe region* or *vulnerability basin* is `{x | f x > τ}`, and the *boundary* is the level set `{x | f x = τ}`. The *failure manifold* is the closed superlevel set `{x | f x ≥ τ}`. These are `SafeRegion`, `UnsafeRegion`, `Boundary`, and `FailureManifold` in `MoF_01_Foundations`.

Three facts about this partition are immediate and hold with no hypothesis beyond continuity of `f`. Every point lies in exactly one of the three regions (`space_partition`). The safe and unsafe regions are disjoint (`safe_unsafe_disjoint`). The failure manifold is the disjoint union of the boundary and the unsafe region (`failureManifold_eq_union`). None of this is deep. It is the setup, the analytic analogue of fixing a system `f : A → A → Y` in Chapter 1.

The bridge from the diagonal chapters to this one is the observation that a continuous alignment surface on a connected space carries a boundary object of its own, and that object plays the role the liar index `a₀` played there. In Chapter 1 the boundary question was the index on which a boolean verdict flipped. Here it is a point where `f` equals `τ` exactly, a place that is neither safe nor unsafe but sits on the knife edge between them. The diagonal produced its witness by self-reference. The boundary here is produced by connectedness and the intermediate value theorem, which is a completely different engine. What is striking, and what makes this chapter a continuation rather than a new subject, is that the two engines deliver the same kind of object: an unavoidable point of failure. The difference is that the analytic engine delivers it with metric content the diagonal never had.

Remark 5.2. It helps to keep one picture in mind throughout. Imagine `f` as a landscape, a height function over the behavior space `X`. The safe region is the lowland below sea level `τ`, the unsafe region is the land above it, and the

boundary is the coastline. The questions of this chapter are geographic. How much coastline is there? How steep are the cliffs? If you drop a random point on the map, how likely is it to land near the coast? If you start inland and walk toward the sea, how many steps does it take? The impossibility theorems say the coastline exists. The geometry says how to sail it.

5.2 Decision basins and their structure

The first substantial results describe the vulnerability basin as a set. They are soft, in the sense that they use only continuity and the order structure of $\mathbb{R}$, but they are the foundation for everything measure-theoretic that follows.

Theorem 5.3 (Basin openness). *If `f` is continuous, the vulnerability basin `{x | f x > τ}` is open, and the safe region `{x | f x < τ}` is open. The boundary `{x | f x = τ}` and the sublevel set `{x | f x ≤ τ}` are closed.*

Proof. The basin is the preimage `f⁻¹((τ, ∞))` of an open ray under a continuous map, hence open. The safe region is `f⁻¹((-∞, τ))`, again a preimage of an open ray. The boundary is `f⁻¹({τ})`, the preimage of a closed singleton, hence closed; equivalently it is where the two conditions `f x ≤ τ` and `f x ≥ τ` both hold, an intersection of two closed sets. ■

This is `basin_isOpen` and `sublevel_isClosed` in `MoF_02_BasinStructure`, with the safe and boundary versions as `safeRegion_isOpen`, `unsafeRegion_isOpen`, and `boundary_isClosed` in `MoF_01_Foundations`. The openness of the basin has an immediate reading that matters for safety. If a behavior is unsafe, so is every behavior sufficiently close to it. Unsafety is not a razor-thin condition that a tiny perturbation removes; it comes with room around it.

Theorem 5.4 (Every unsafe point owns a ball). *If `f` is continuous and `f p > τ`, then there is a radius `r > 0` with the open ball `B(p, r)` contained in the basin.*

Proof. The basin is open and contains `p`, and in a metric space openness means exactly that every point has a ball around it inside the set. ■

This is `basin_contains_ball`. The qualitative version, that *some* positive radius exists, is all continuity buys you. The quantitative version, that the radius is at least an explicit function of how far `f p` exceeds `τ`, needs a Lipschitz hypothesis and is the subject of the next section. Hold the two apart in your mind. Continuity gives you a ball of unknown size; Lipschitz continuity gives you a ball of known size.

Theorem 5.5 (Basins have positive measure). *Let `μ` be any measure on `X` for which nonempty open sets have positive measure (an `IsOpenPosMeasure`, which includes Lebesgue measure on `ℝ^n`). If `f` is continuous and there is a point `p` with `f p > τ`, then `μ({x | f x > τ}) > 0`.*

Proof. The basin is open by Theorem 5.3 and nonempty by hypothesis. An open nonempty set has positive measure by assumption on `μ`. ■

This is `basin_measure_pos` (in `MoF_02`) and, in the form stated directly for continuous `f`, `basin_measure_ge_ball_pos` in `MoF_Cost_02_BasinVolume`. The result is small but it is the hinge of the whole cost story. A single unsafe point is a measure-zero event and might be dismissed as negligible. Positive

measure cannot be dismissed. It says a positive fraction of the space is unsafe, so a random draw has a positive chance of landing there, and no finite set of safe examples can certify the region away. Existence upgrades to abundance the moment you add continuity and a measure.

Theorem 5.6 (Monotonicity in the threshold). *If* `τ₁ ≤ τ₂` *then* `{x | f x > τ₂} ⊆ {x | f x > τ₁}`*, and consequently* `μ({x | f x > τ₂}) ≤ μ({x | f x > τ₁})`*. If* `τ` *sits strictly above the supremum of* `f`*, the basin is empty; if* `τ` *sits strictly below the infimum, the basin is the whole space.*

Proof. Raising the threshold can only remove points from the superlevel set, so the inclusion holds, and measure is monotone under inclusion. The extreme cases are the observation that `f x > τ` is unsatisfiable when `τ` exceeds every value of `f` and universally satisfied when `τ` is below every value. ■

These are `basin_measure_monotone_threshold`, `basin_empty_above_sup`, and `basin_full_below_inf` in `MoF_Adv_10_MeasureBounds`. The lesson for a defender is unwelcome. You can shrink the basin by raising the threshold, meaning by declaring more behaviors unsafe and refusing them, but the basin shrinks continuously and never vanishes until you have refused essentially everything the model can do. A threshold high enough to empty the basin is a threshold high enough to empty the product.

Connectedness and components

Is the basin one region or many? The answer depends on the shape of `f`, and the distinction matters because an attacker who finds one component has found only that component, while a defender who patches one has patched only that one.

Theorem 5.7 (Components are open and countable). *On a space where every point has a connected neighborhood (a locally connected space, in particular* `ℝ^n`*), each connected component of the basin* `{x | f x > τ}` *is open. If in addition the space is second countable, there are at most countably many components.*

Proof sketch. Components of an open set in a locally connected space are open, since around any point of the basin sits a connected neighborhood inside the basin, and that neighborhood lies wholly in the point's component. A collection of disjoint nonempty open sets in a second countable space is countable, because each must contain a distinct element of a countable base. ■

This is `basin_connected_components_open` and `basin_components_countable` in `MoF_Adv_01_BasinConnectedness`. When the domain is convex and `f` is, say, quasi concave, the basin is a single connected region (`basin_connected_of_convex_domain` records a clean sufficient condition). But nothing forces this. The file also carries `superlevel_disconnected_example`, an explicit `f` whose basin splits into separate pieces. Real models are the second kind. Their vulnerability basins fragment into many components scattered through behavior space, which is exactly why red-teaming that finds one jailbreak family says little about the others.

A worked example: the Gaussian bump

Let `X = ℝ^n` and take `f(x) = M · exp(-‖x‖² / 2)` for a peak height `M > 0`. This is a smooth radially symmetric bump centered at the origin, a caricature of a model with a single localized failure mode. Fix a threshold `τ` with `0 < τ < M`.

The basin `{x | f x > τ}` is `{x | ‖x‖² < 2 ln(M/τ)}`, an open Euclidean ball of radius `R = √(2 ln(M/τ))` centered at the origin. Every prediction of the soft theory checks out on this example. The basin is open, as any ball is. It is nonempty precisely because `τ < M`, so the origin itself is unsafe. It has positive Lebesgue measure, namely the volume of a ball of radius `R`. It is connected, being a ball. Raise `τ` toward `M` and `R` shrinks to zero continuously; the basin never disappears until `τ` reaches `M`, matching Theorem 5.6.

The example also previews the curse of dimension. The volume of a Euclidean ball of radius `R` in `ℝ^n` is `R^n · π^(n/2) / Γ(n/2 + 1)`. For fixed `R`, as `n` grows, this volume first grows and then collapses toward zero, and its ratio to the volume of the surrounding cube of side `2R` goes to zero fast. The basin, though it has positive measure, occupies a vanishing fraction of the ambient space in high dimension. A random draw almost never lands in it. This is the good news for defense and the bad news too, and untangling which is the business of the cost theory later in the chapter.

5.3 The threshold-crossing theorem

Now the central structural result of the qualitative geometry. It says the boundary cannot be dodged. If a path starts safe and ends unsafe, it crosses the boundary somewhere in between. There is no teleporting from lowland to highland without touching sea level.

Theorem 5.8 (Threshold crossing, one dimension). *Let `f : ℝ → ℝ` be continuous, let `a ≤ b`, and suppose `f a < τ < f b`. Then there is a point `c ∈ [a, b]` with `f c = τ`.*

Proof. This is the intermediate value theorem. The interval `[a, b]` is connected, `f` is continuous on it, and `τ` lies strictly between the values `f a` and `f b`, so `τ` is attained. Formally one applies the two-function form of the IVT to `f` and the constant `τ` on the preconnected set `[a, b]`. ■

This is `path_crosses_threshold` in `MoF_03_ThresholdCrossing`. It is elementary, and its elementariness is the point. The unavoidability of the boundary is not a subtle theorem; it is the IVT wearing a safety costume. What takes work is stating it in the generality the theory needs, which is arbitrary behavior spaces rather than the real line.

Theorem 5.9 (Threshold crossing along a path). *Let `X` be any topological space, `γ : ℝ → X` a continuous path, and `f : X → ℝ` continuous. If `f(γ a) < τ < f(γ b)` with `a ≤ b`, then there is `c ∈ [a, b]` with `f(γ c) = τ`.*

Proof. Apply Theorem 5.8 to the composite `f ∘ γ : ℝ → ℝ`, which is continuous as a composition of continuous maps. ■

This is `path_crosses_threshold_generic`. Read it as a statement about attacks. An attack is a path from a safe behavior `γ a` to an unsafe one `γ b`. It might be a sequence of prompt edits, a continuous interpolation in embedding space, or a gradient trajectory. Whatever its form, if it is continuous and it succeeds, it passed through the boundary. There is a moment in every successful attack where the behavior was exactly at threshold. This is why the boundary, and not the interior of the unsafe region, is the object a defense must control.

The theorem also quietly refutes a tempting defense strategy, the moat. A designer might reason that if the safe and unsafe regions could be separated by a buffer, a band of forbidden inputs wide enough that no small edit crosses it, then attacks would be stopped at the moat's edge. Theorem 5.9 says the moat is illusory for a continuous score. The path from safe to unsafe is continuous, so `f` takes every intermediate value, including every value inside the supposed moat. You cannot forbid a band of scores without forbidding behaviors that legitimately produce those scores, and by the no-gap theorem those behaviors exist and abut the safe region. The only genuine separator is the measure-zero level set itself, and Theorem 5.20 will show that even that thickens to positive volume once the defender's own uncertainty is accounted for. There is no moat, only a coastline, and coastlines are walked, not sealed.

Theorem 5.10 (No gap). *Let `X` be connected and `f : X → ℝ` continuous. If there exist a safe point (`f a < τ`) and an unsafe point (`f b > τ`), then the boundary `{x | f x = τ}` is nonempty.*

Proof. On a connected space the image of a continuous real-valued function is an interval, or more precisely the two-function IVT applies over the whole space. Since `f` takes a value below `τ` and a value above it, and the reachable values form a connected subset of $\mathbb{R}$, the value `τ` is reached. ■

This is `no_gap_theorem`, resting on `threshold_level_set_nonempty`. The name is the moral. There is no gap between the safe and unsafe regions into which a defense could retreat. A designer might hope to engineer a model whose behaviors are all either comfortably safe or comfortably unsafe, with an empty margin between, so that a crude classifier could separate them. Connectedness forbids it. As long as the model does both safe and unsafe things and its behavior varies continuously, there is a boundary, and the boundary is populated.

Theorem 5.11 (Separation is exactly the boundary). *The closures of the safe and unsafe regions meet only on the boundary: `closure(SafeRegion) ∩ closure(UnsafeRegion) ⊆ Boundary`.*

Proof sketch. Suppose `x` lies in both closures. If `f x > τ`, then `x` is in the open unsafe region, so a whole neighborhood of `x` is unsafe, which contradicts `x` being a limit of safe points. Hence `f x ≤ τ`. Symmetrically, if `f x < τ`, a neighborhood of `x` is safe, contradicting `x` being a limit of unsafe points, so `f x ≥ τ`. Together `f x = τ`. ■

This is `safe_unsafe_separated`. It sharpens the no-gap theorem. Not only is the boundary nonempty, it is precisely the interface, the shared fron-

tier where the two regions touch. Everything an attacker exploits and everything a defender must guard lives on this closed set.

The boundary as a fixed point of defense

The threshold-crossing results acquire teeth when combined with a model of defense. A defense is a continuous map `D : X → X` that transforms behaviors, meant to push unsafe inputs back into the safe region while leaving genuinely safe inputs alone. A *utility-preserving* defense is one that is the identity on the safe region: it does not disturb behavior that was fine to begin with. The master theorem of the `ManifoldProofs` development shows such a defense always fails at the boundary.

Theorem 5.12 (Defense fixes a boundary point). *Let `X` be a connected Hausdorff space, `f : X → ℝ` continuous, with both safe and unsafe points present. Let `D : X → X` be continuous and equal to the identity on the safe region. Then there is a boundary point `z` with `f z = τ` and `D z = z`. In particular `D` cannot make `z` safe, since `f(D z) = f z = τ` is not below `τ`.*

Proof sketch. The fixed-point set `{x | D x = x}` of a continuous self-map of a Hausdorff space is closed, being the preimage of the diagonal under the continuous map `x ↦ (D x, x)`. Since `D` is the identity on the safe region, the safe region lies in this fixed-point set, and because the set is closed it contains the closure of the safe region. By the no-gap argument, the safe region is not closed (it is open and, were it also closed, connectedness would force it to be empty or everything, both excluded), so its closure contains a boundary point `z` with `f z = τ`. That `z` is a limit of safe points, hence fixed by `D`, hence unmoved and still exactly at threshold. ■

This is `master_theorem` in `MoF_MasterTheorem`, bundling the space and its data into the `ManifoldOfFailure` structure. It is the analytic cousin of the diagonal's impossibility. Where Chapter 1 produced a query no verdict could correctly answer, this produces a behavior no utility-preserving defense can correct. Note what the theorem does not claim. It does not say `D` fails everywhere or that most of the unsafe region survives. It says one specific point, on the boundary, is fixed and unfixed. The quantitative theorems that follow are about upgrading this single surviving point into a region of positive measure, which is where the danger actually lies.

A worked example: interpolating prompts

Take `X` the space of prompt embeddings, identified with `ℝ^n`, and `f` a model's internal unsafety score, assumed continuous in the embedding. Suppose a benign prompt embeds at `p` with `f p = τ − 1` and a known jailbreak embeds at `q` with `f q = τ + 1`. Consider the straight-line path `γ(t) = (1 − t) p + t q` for `t ∈ [0, 1]`. It is continuous, starts safe, ends unsafe. Theorem 5.9 guarantees a crossing time `t*` with `f(γ(t*)) = τ`.

The crossing point `γ(t*)` is a prompt that the model scores exactly at threshold. It is, concretely, a marginal jailbreak: a prompt sitting on the

fence, where a vanishingly small nudge tips it unsafe. If the defense refuses everything the model scores above τ, then by Theorem 5.12 the crossing point itself, and behaviors just past it, evade the refusal or trigger it spuriously depending on which side of the knife edge they fall. The boundary is not an abstraction here. It is a family of real, marginal prompts, and it is exactly the family red teams learn to hunt.

5.4 Lipschitz control of basins

Continuity tells you the basin is open and that unsafe points own balls. It does not tell you how big those balls are. To get sizes you need to control how fast `f` can change, and the standard control is the Lipschitz condition.

Recall that `f : X → ℝ` is *`L`-Lipschitz* if `|f x − f y| ≤ L · dist(x, y)` for all `x, y`. The constant `L` caps the steepness of the alignment surface. It is a modeling assumption, but a mild and defensible one. Neural networks with bounded weights and Lipschitz activations are Lipschitz, and the constant, though often large, is finite and estimable. With it, the qualitative ball of Theorem 5.4 becomes a quantitative one.

Definition 5.13 (Robustness radius). For a point with value `f p` above threshold `τ` and a Lipschitz constant `L > 0`, the *robustness radius* is `robustnessRadius(f p, τ, L) = (f p − τ) / L`. This is `robustnessRadius` in `Mo F_04_LipschitzBasin`.

Theorem 5.14 (Lipschitz basin ball). *Let `f` be `L`-Lipschitz with `L > 0`, and suppose `f p > τ`. If `dist(p', p) < (f p − τ) / L`, then `f p' > τ`. Equivalently, the open ball of radius `(f p − τ) / L` around `p` lies entirely inside the basin.*

Proof. The Lipschitz condition bounds the drop in `f` over the step from `p` to `p'`: `|f p' − f p| ≤ L · dist(p', p) < L · (f p − τ)/L = f p − τ`. In particular `f p − f p' ≤ |f p' − f p| < f p − τ`, so `f p' > τ`. ■

This is `lipschitz_basin_ball`, with the set form `basin_ball_subset` and the corollary `perturbation_stability`. The formula rewards intuition. The radius is the margin `f p − τ`, how deeply unsafe the point is, divided by `L`, how fast `f` can climb back down. A behavior that is very unsafe (large margin) on a smooth model (small `L`) sits at the center of a large ball of guaranteed unsafety. This is a statement about the robustness of attacks, not defenses. A successful attack is not fragile. Once you have found a point well inside the unsafe region, an entire neighborhood of nearby behaviors is unsafe too, so small changes in phrasing or encoding do not rescue the model. The positivity of the radius is `robustnessRadius_pos`.

Theorem 5.15 (Deeper points have larger basins). *If `f p₁ ≤ f p₂`, both above `τ`, then `robustnessRadius(f p₁, τ, L) ≤ robustnessRadius(f p₂, τ, L)`.*

Proof. The radius `(v − τ)/L` is increasing in `v` for fixed `τ` and `L > 0`. ■

This is `robustnessRadius_monotone`. The more deviant the behavior, the more robust it is to perturbation, which is the opposite of what a defender

would want and exactly what an attacker exploits by pushing deep rather than stopping at the first crossing.

Theorem 5.16 (Smoother models, larger basins per peak). *Fix a peak value* `f p > τ`. *If* `L₁ ≤ L₂`, *both positive, then the robustness radius under* `L₁` *is at least that under* `L₂`.

Proof. The radius `(f p − τ)/L` is decreasing in `L`. ■

This is `smoother_functions_larger_basins` in `MoF_Adv_04_ModelScale`, with the pure one-dimensional interval version `lipschitz_basin_component_width`. There is a tension here that the optimal-defense theory later makes precise. A smaller Lipschitz constant means a smoother, more predictable model, which is usually considered desirable and is often enforced by regularization. But a smoother model has *larger* unsafe basins per peak, since each unsafe point radiates its unsafety further. Smoothness does not remove the danger; it spreads it. Steepness concentrates the danger into thin spikes that are harder to hit by chance but sharper when hit.

Before leaving the Lipschitz story, a word on whether the assumption is fair. A skeptic could object that real models are not Lipschitz with any usable constant, and that a bound of `L = 10^6` makes the robustness radius `(f p − τ)/L` too small to matter. The objection cuts the right way but the wrong direction. A large `L` shrinks the guaranteed unsafe ball, which sounds like relief for the defender, but by Theorem 5.17 a large `L` is exactly what a model needs to draw sharp, narrow basins, and by the cone bound a large local growth rate is what defeats the defense. A large Lipschitz constant does not make the model safe; it makes the failures spiky. Spiky failures are harder to hit by random chance and just as easy to hit by gradient, since the gradient is largest precisely where the surface is steepest. The Lipschitz constant is not a safety margin. It is a description of the terrain, and rough terrain favors the climber who follows the slope. In practice `L` is estimated layer by layer as a product of operator norms, and while the global product is loose, the local Lipschitz behavior near a boundary point, which is what Theorems 5.24 and 5.35 actually use, is often far smaller and directly measurable by probing.

The other direction: steepness forces oscillation

The Lipschitz constant also bounds how much a function can wiggle, and this gives a lower bound on `L` for models that must do very different things at nearby inputs.

Theorem 5.17 (Oscillation needs steepness). *Suppose* `f : [0,1] → ℝ` *is* `L`*-Lipschitz with* `f 0 = f 1 = 0` *and* `f t = M` *for some* `t ∈ [0,1]` *and* `M > 0`. *Then* `L ≥ 2M`.

Proof. Apply the Lipschitz bound on both sides of the peak. From `0` to `t`: `M = |f t − f 0| ≤ L · t`. From `t` to `1`: `M = |f t − f 1| ≤ L · (1 − t)`. Add the two: `2M ≤ L·t + L·(1 − t) = L`. ■

This is `higher_lipschitz_more_oscillation`. A model that must be safe at the endpoints of an interval and grossly unsafe in the middle pays for it in Lipschitz constant, and hence in the size of the basin around the middle. You

cannot have it both ways: sharp transitions between safe and unsafe behavior require a large L, and a large L is exactly what the cost theory will charge for.

A worked example: the linear ramp

Let f(x) = ⟨w, x⟩ + b be an affine score on ℝ^n, the simplest nontrivial alignment surface, and suppose ‖w‖ = L. This f is exactly L-Lipschitz, with the bound achieved along the direction w. Fix τ and a point p with f p = τ + m for a margin m > 0.

Theorem 5.14 guarantees a ball of radius m/L around p inside the basin. This is tight, not merely a bound. Moving from p in the direction −w/‖w‖ decreases f at the maximal rate L, so after distance exactly m/L you reach the boundary and any further step exits the basin. In every other direction you leave more slowly or not at all. The robustness ball touches the boundary at one point, the foot of the perpendicular from p to the hyperplane {f = τ}, and stays strictly inside everywhere else. This is the geometric content of the robustness radius made visible: the ball is inscribed in the basin, kissing the boundary along the gradient direction. Every later, harder theorem about basin volume is a version of inscribing balls like this one and adding up their measures.

5.5 The coarea and cone bounds

We now have unsafe balls of known radius. The next step is to turn radii into volume, and to do it not just for the interior of the basin but for the thin band straddling the boundary, because that band is where marginal attacks and imperfect defenses meet. Two bounds do this work. The coarea bound measures the band as a whole. The cone bound measures the wedge of persistently unsafe behavior that a defense cannot flatten.

The coarea bound

The genuine coarea formula relates the volume of a band {τ − ε ≤ f ≤ τ} to an integral of 1/‖∇f‖ over the level sets inside it. That formula is not yet fully available in the underlying library, so ManifoldProofs proves a self-contained substitute by inscribing a ball, which is enough to get a clean lower bound.

Definition 5.18 (The ε-band). The band of width ε below the threshold is coareaBand(f, τ, ε) = {x | τ − ε ≤ f x ≤ τ}. This is coareaBand in MoF_17_CoareaBound.

Theorem 5.19 (Ball inside the band). *Let f be L-Lipschitz with L > 0 and ε > 0. If there is a center point c with f c = τ − ε/2, then the ball B(c, ε/(4L)) is contained in the ε-band.*

Proof. For x ∈ B(c, ε/(4L)), the Lipschitz bound gives |f x − f c| ≤ L · dist(x, c) < L · ε/(4L) = ε/4. Since f c = τ − ε/2, this puts f x

in the open interval (τ − 3ε/4, τ − ε/4), which sits strictly inside [τ − ε, τ]. So x is in the band. ■

This is lipschitz_ball_subset_band. The center is chosen at the middle of the band, τ − ε/2, so that a ball of radius ε/(4L) cannot escape either wall of the band, because the value can move by at most ε/4 across the ball.

Theorem 5.20 (Coarea volume lower bound). *Under the hypotheses of Theorem 5.19, μ(coareaBand(f, τ, ε)) ≥ μ(B(c, ε/(4L))) for any measure μ. In ℝ with Lebesgue measure this gives the explicit bound μ(band) ≥ ε/(2L). In ℝ^n it gives μ(band) ≥ vol(B(c, ε/(4L))), a positive quantity growing like (ε/(4L))^n.*

Proof. Measure is monotone under the inclusion of Theorem 5.19. In ℝ the ball of radius ε/(4L) is an interval of length 2 · ε/(4L) = ε/(2L). In ℝ^n the ball has the usual Euclidean volume. ■

This is epsilon_band_volume_lower_bound, with the real and Euclidean specializations epsilon_band_volume_lower_bound_real and epsilon_band_volume_lower_bound_euclidean. The center point c is not an extra assumption when the model does both safe and unsafe things: on a connected space, if f dips below τ − ε somewhere and rises above τ somewhere, the IVT produces a c with f c = τ − ε/2 for free (exists_midpoint_for_band), and the whole argument becomes unconditional (epsilon_band_volume_pos_unconditional). The band around the boundary always has positive, and computably bounded, measure. This is the quantitative form of "the boundary is populated." It is not a lonely level set; it is the spine of a slab of positive volume.

Remark 5.21. The band volume scales as ε/L in one dimension and worsens the defender's position in a specific way. A defender's classifier, being imperfect, has an uncertainty band of some width ε around its estimate of the threshold. Theorem 5.20 says the true behaviors falling in that band occupy volume at least ε/(2L). The wider the uncertainty and the smoother the model, the more behavior is genuinely ambiguous. Sharpening the classifier shrinks ε and shrinks the ambiguous volume linearly, which is progress, but never to zero while ε > 0.

The cone bound

The coarea bound measures the band symmetrically. The cone bound measures something sharper: the wedge of behaviors that stay unsafe *even after a defense acts on them*. This is where the geometry directly meets impossibility, because it exhibits a positive-measure set the defense cannot save.

Model the defense as a K-Lipschitz map D that fixes a boundary point z, so D z = z and f z = τ. The number K is the defense's own smoothness: how much it can move a point per unit distance. The key quantity is the local growth rate of f leaving z. If f climbs steeply enough away from z, no K-Lipschitz defense can drag the climbing behaviors back below τ.

Definition 5.22 (Steep region). In ℝ, the *steep region* around a boundary point z with slope parameter s is steepRegionR(f, τ, s, z) = {x | f x

`> τ + s · |x − z|}`. This is `steepRegionR` in `MoF_18_ConeBound`, with the general metric-space version `steepRegion` in `MoF_11_EpsilonRobust` and the refined form `steepRegionRefined` in `MoF_20_RefinedPersistence`.

Theorem 5.23 (Defense distortion bound). *If `f` is `L`-Lipschitz, `D` is `K`-Lipschitz, and `D z = z`, then for all `x`, `f(D x) ≥ f x − L(K+1) · dist(x, z)`.*

Proof. First the defense moves `x` by a controlled amount. By the triangle inequality and the two Lipschitz bounds, `dist(D x, x) ≤ dist(D x, D z) + dist(D z, x) ≤ K · dist(x, z) + dist(x, z) = (K+1) · dist(x, z)`, using `D z = z`. Then `f` changes by at most `L` times that: `|f(D x) − f x| ≤ L · dist(D x, x) ≤ L(K+1) · dist(x, z)`. Drop the absolute value on the favorable side to get `f(D x) ≥ f x − L(K+1) · dist(x, z)`. ■

This is `defense_from_input_bound_R` (and `defense_from_input_bound` in `MoF_11`). The compound constant `L(K+1)` is the defense's maximum reach: how far below its input value the defense can pull the alignment score. If `f` rises faster than `L(K+1)` in some direction, the defense loses.

Remark 5.23a (The operative reach is the defense-path Lipschitz constant). The bound above used `f`'s global Lipschitz constant `L` for `|f(D x) − f x|`, but what truly governs the defense's reach is how fast `f` varies *along the defense's own displacements*: the defense-path Lipschitz constant ℓ, with `|f(D x) − f x| ≤ ℓ · dist(D x, x)` and $\ell \le$ `L`. The reach is then `ℓ(K+1)` and the steep condition is `G > ℓ(K+1)`. This is the crux: `f` can be *anisotropic*, steep in the growth direction (gradient `G`) yet smooth along defense-displacement paths, so that `ℓ < L` and `G > ℓ(K+1)` is achievable even for `K > 0`. When the surface is isotropic (`ℓ = L`) the steep region is empty and the defense always wins, which is exactly the shallow-boundary case of Theorem 5.25. The non-vacuous, refined statement is `persistent_unsafe_refined`, and the gradient criterion that makes the steep region nonempty is `gradient_norm_implies_steep_nonempty`. Read the reach `L(K+1)` in the persistence results below as this defense-path reach `ℓ(K+1)`.

Theorem 5.24 (Cone measure bound). *Suppose `f z = τ`, `D z = z`, and `f` grows from `z` at rate `c > L(K+1)`, meaning `f(z + t) ≥ τ + c·t` for all `t ∈ (0, δ₀)`. Then, with slope `s = L(K+1)`:*

(1) the open interval `(z, z + δ₀)` is contained in the steep region;

(2) the steep region has Lebesgue measure at least `δ₀ > 0`;

(3) every point of the steep region stays unsafe after defense, `f(D x) > τ`.

Proof. For part (1), take `x = z + t` with `0 < t < δ₀`. Then `f x ≥ τ + c·t > τ + s·t = τ + s·|x − z|`, using `c > s` and `|x − z| = t`, so `x` is in the steep region. Part (2) follows because the interval `(z, z + δ₀)` has length `δ₀` and lies inside the steep region, so measure monotonicity gives the bound. For part (3), let `x` be in the steep region, so `f x > τ + s·|x − z|`. Combine with the distortion bound of Theorem 5.23: `f(D x) ≥ f x − s·dist(x, z) > τ + s·|x − z| − s·|x − z| = τ`. ■

This is `cone_measure_bound`, assembling `cone_segment_in_steep_region`, `steep_region_contains_Ioo`, and `steep_region_measure_pos`, with the explicit lower bound `steep_region_measure_lower_bound`. It is the geo-

metric heart of the defense impossibility. Chapter 1 gave one surviving query. The master theorem gave one surviving boundary point. This gives a set of positive measure, an entire cone of behaviors, every one of which remains unsafe after the defense has done its best. The impossibility is no longer a measure-zero curiosity. It is a slab of failure with volume you can bound from below.

Theorem 5.25 (Shallow boundaries can be defended). *If instead `f` never rises faster than its own Lipschitz constant from `z`, that is `f x ≤ f z + L · dis t(x, z)` for all `x`, then for every `K ≥ 0` the steep region is empty.*

Proof. For any `x`, since `f z = τ`, we have `f x ≤ τ + L · dist(x, z) ≤ τ + L(K+1) · dist(x, z)`, the last step because `K + 1 ≥ 1` and `dist ≥ 0`. So `f x` never exceeds `τ + s·dist(x, z)`, and the steep region has no points. ∎

This is `shallow_boundary_no_persistence` in `MoF_19_OptimalDefens e`. The two theorems together draw a sharp line. Steep boundaries carry persistent, positive-measure failure that no defense removes. Shallow boundaries do not. The dividing rate is `L(K+1)`, and whether a real model is steep or shallow at its boundary is the single most important quantitative fact about its defensibility. The optimal-defense theory in the cost section is about what happens when a designer tries to buy shallowness by raising `K`.

A worked example: a corner in the score

Let `f(x) = τ + G · x` for `x ≥ 0` and `f(x) = τ + L₀ · x` for `x < 0` on $\mathbb{R}$, with `G` large and `L₀` moderate, a boundary point at `z = 0` where the score kinks upward. The Lipschitz constant of `f` is `L = max(G, L₀) = G`. Suppose the defense `D` is `K`-Lipschitz and fixes `0`.

If `G > L(K+1)`, then the growth rate `c = G` beats the defense reach `s = L (K+1)`, and Theorem 5.24 applies with any $\delta_0 > 0$: the whole ray `(0, δ₀)` is a steep region of measure δ_0 that survives the defense. But here `L = G`, so `G > G(K+1)` requires `K + 1 < 1`, impossible for `K ≥ 0`. The corner is not steep *relative to its own Lipschitz constant*, so this particular `f` is defensible in principle, in line with Theorem 5.25. To get genuine persistence you need `f` to rise, near `z`, faster than it rises elsewhere, so that its local slope `G` exceeds ℓ`(K+1)`, where $\ell \le$ `L` is the *defense-path* constant governing the defense's smoothing, strictly below the global `L` for an anisotropic surface. That is precisely the regime the gradient chain of the next-but-one section reaches by way of the derivative $\|\nabla$`f(z)`$\|$.

5.6 Dimensional scaling and the boundary's dimension

So far the ambient dimension has been a spectator. It now takes the stage, and it changes everything. The central fact of high-dimensional geometry, and the reason defense is hard in a way attack is not, is that volume is cheap for attackers and ruinously expensive for defenders. This asymmetry is the curse of dimensionality, and `MoF_09_DimensionalScaling` states it precisely.

Theorem 5.26 (Grid explosion). *Discretize each of `d` axes into `N` bins, giving a grid of `N^d` cells. For `N ≥ 2`, the grid size `N^d` grows without bound as `d → ∞`. Covering every cell requires at least `N^d` queries, and any fixed budget `B` is exceeded once `d` is large enough.*

Proof. The grid `Fin d → Fin N` has cardinality `N^d` by counting functions. For `N ≥ 2`, `N^d ≥ 2^d → ∞`. Covering all cells is a surjection onto the grid, and a surjection's domain has at least the codomain's cardinality, giving the `N^d` lower bound. For any budget `B`, since `2^d` eventually exceeds `B` and `N^d ≥ 2^d`, the grid outgrows `B`. ■

This is `grid_card`, `grid_exponential_growth`, `queries_lower_bound_surjective`, and `no_fixed_budget_defense`. A defender who wants to certify safety cell by cell faces `N^d` cells, and in the dimensions relevant to language models, where `d` is in the thousands, `N^d` is beyond astronomical. Exhaustive defense is not merely expensive; it is impossible at any budget.

Theorem 5.27 (Coverage collapse). *A single defense patch covering a relative radius `r < 1` in each dimension covers a fraction `r^d` of the space, and `r^d → 0` as `d → ∞`. The number of patches needed to cover a fixed fraction `α` grows like `(1/r)^d → ∞`.*

Proof. The relative volume of a scaled box is the product of its side fractions, `r^d`. For `0 < r < 1`, `r^d → 0`. Covering fraction `α` needs at least `α / r^d = α · (1/r)^d` patches, and since `1/r > 1` this grows without bound. ■

This is `coverage_fraction_vanishes`, `defense_patch_shrinks`, and `exponential_base`, bundled in the `DimensionalCurse` structure. Here is the asymmetry stated plainly. Each thing the defender builds, a patch, a rule, a filter, covers a fraction of space that shrinks exponentially with dimension. The attacker needs to find one point. The defender needs to cover almost all of them. In high dimension the first is easy and the second is hopeless, and the gap between them is exponential in `d`.

The dimension of the boundary itself

The basin has full dimension `d`; it is an open set. The boundary is one dimension lower in the generic smooth case, a hypersurface of dimension `d − 1`. Full Hausdorff-dimension machinery is beyond the current library, so `MoF_Adv_02_BoundaryDimension` proves the achievable structural facts about the level set.

Theorem 5.28 (Boundary structure). *The boundary `{x | f x = τ}` is closed, and its intersection with any compact set is compact. It is empty exactly when `f` never equals `τ`, and nonempty on a connected space whenever `f` straddles `τ`. It is the preimage `f⁻¹({τ})`.*

Proof. Closedness and the preimage identity are Theorem 5.3 restated. Compactness on compacta is the intersection of a closed set with a compact set. Emptiness and nonemptiness are the trivial and IVT cases. ■

These are `boundary_is_closed`, `boundary_is_compact_on_compact`, `boundary_empty_of_no_crossing`, `boundary_nonempty_connected`, and `boundary_is_preimage_singleton`.

Theorem 5.29 (Regular points are isolated in one dimension). *For `f : ℝ → ℝ`, if `f` has a nonzero derivative at a boundary point `z` (a regular value), then `z` is an isolated point of the level set: there is a punctured neighborhood of `z` on which `f ≠ τ`.*

Proof sketch. A nonzero derivative forces `f` off the value `τ` as `x` moves away from `z`: to first order `f(x) - τ = f'(z)(x - z) + o(x - z)`, which is nonzero for `x` near `z` with `x ≠ z`, so `f` attains `τ` only at `z` in a neighborhood, which is exactly isolation. ■

This is `regular_value_boundary` and `boundary_locally_finite_1d`. Where the boundary is regular, it is thin and well-behaved, a clean interface. Where the gradient degenerates, the boundary can thicken and fold, and whether real model boundaries are regular or fractal is an open empirical question that this dimension theory frames but does not settle. The practical upshot is that the codimension-one boundary, though small relative to the full space, is exactly where the coarea band of Theorem 5.20 gives it positive-volume thickness once you account for classifier uncertainty.

Model scale

Function-class complexity, a proxy for model size, controls how intricate the basin can be.

Theorem 5.30 (Complexity and crossings). *A constant model has a trivial basin, either empty or the whole space. A monotone model crosses any threshold at most once. Making the basin oscillate, with many separated components, requires a large Lipschitz constant, quantitatively `L ≥ 2M` to reach height `M` between two zeros.*

Proof. The constant case is immediate. For monotonicity, if `f a = f b = τ` with `a ≤ b` and `f` monotone, then `f` is squeezed to `τ` on all of `[a, b]`, so there is a single crossing (possibly a flat one). The oscillation bound is Theorem 5.17. ■

These are `constant_model_no_basin_empty`, `constant_model_no_basin_univ`, `low_complexity_few_crossings_1d`, and `higher_lipschitz_more_oscillation`, all in `MoF_Adv_04_ModelScale`. Larger, more expressive models can carve more intricate basins with more components and finer structure, and they pay for that intricacy in Lipschitz constant. A bigger model is not automatically safer geometry; it is capable of a more complicated failure landscape, which is more places for an attacker to look and more places a defender must guard.

A worked example: random points in the cube

To feel the curse in the body, drop a uniformly random point in the cube `[0, 1]^d` and ask how far it is from the center. In one dimension it is near the center about half the time. In high dimension almost all the volume of the cube is near the surface and far from the center, and the pairwise distances between random points concentrate around a single value. This concentration,

examined in the next section, is why a random-search attacker and a random-defense defender see the same geometry from opposite sides. The attacker, sampling at random, is very likely to land in the vast peripheral bulk where, if any basin has appreciable measure, it will be found. The defender, placing patches, watches each one cover a `r^d` sliver of that same bulk. Same space, same concentration, opposite fortunes.

5.7 Gradient attacks and their chaining

Random search finds the basin if it is large. Gradient attacks find it even when it is small, by following the slope of `f` uphill. This section formalizes why gradient ascent reliably reaches the unsafe region and then chains the local step into the global persistence result of the cone bound.

Theorem 5.31 (An ascent direction exists). *Let `f : ℝ^n → ℝ` (or any real Hilbert space) be differentiable at `x` with Fréchet derivative `f'`. If `f' ≠ 0`, then there is a direction `v` and a step size `ε > 0` with `f(x + ε·v) > f(x)`. In one dimension, if the derivative at `x` is positive, then `f` strictly increases just to the right of `x`.*

Proof sketch. In one dimension, a positive derivative means the difference quotient `(f(x + h) − f(x))/h` tends to a positive limit as `h → 0⁺`, so it is eventually positive, so `f(x + h) > f(x)` for small `h > 0`. In `n` dimensions, pick any direction `v` on which the derivative `f' v` is nonzero (one exists since `f' ≠ 0`), flipping sign if needed so that `f' v > 0`, then restrict `f` to the line `t ↦ f(x + t·v)`, whose one-dimensional derivative at `0` is `f' v > 0`, and apply the one-dimensional case. ■

This is `ascent_direction_exists` and `multivariate_ascent_directi on` in `MoF_10_GradientAttack`, resting on `discrete_ascent_improvement` and the line-restriction lemma `hasFDerivAt_line_restrict`. The result is the mathematical license for gradient-based adversarial methods. As long as the score has a nonzero gradient, and away from critical points it does, there is always a way to nudge the input and raise the score. The attacker never gets stuck except at a critical point, and critical points are exactly where the gradient vanishes (`critical_point_iff_fderiv_eq_zero`).

Theorem 5.32 (The gradient is the steepest direction). *In a real inner product space, if `f` has gradient `g` at `x` (so `f' v = ⟨g, v⟩` for all `v`, the Riesz representation), then among unit directions the inner product `⟨g, v⟩` is maximized by `v = g/‖g‖`, giving directional derivative `‖g‖`.*

Proof. By Cauchy-Schwarz, `⟨g, v⟩ ≤ ‖g‖ · ‖v‖ = ‖g‖` for unit `v`, with equality when `v` is parallel to `g`. ■

The Riesz identity `f' v = ⟨g, v⟩` is `fderiv_apply_eq_inner_gradien t`, and the self-pairing `⟨v, v⟩ = ‖v‖²` that underlies the Cauchy-Schwarz step is `real_inner_self_eq_norm_sq'`. This is why attacks follow the gradient specifically and not just any ascent direction: the gradient is the direction of fastest increase, so it reaches the boundary in the fewest steps.

Chaining local ascent to global persistence

A single gradient step is local. The attack that matters is the whole trajectory, and the theorem that matters connects the size of the gradient at a boundary point to the existence of a persistent, positive- measure unsafe region. This is the content of `MoF_21_GradientChain`, which closes the gap between the derivative and the cone bound.

Theorem 5.33 (Near-optimal direction from the gradient norm). *If the derivative `f'` at `z` has operator norm `‖f'‖ > c ≥ 0`, then there is a unit vector `v` with `f' v > c`.*

Proof sketch. By definition of the operator norm there is a vector of norm less than `1` on which `f'` exceeds `c` in absolute value; normalize it to a unit vector, which only increases the value since we divided by something less than `1`, and choose the sign so that `f' v` is positive. ■

This is `near_optimal_direction`.

Theorem 5.34 (Derivative forces local growth). *If `f` has derivative `f'` at `z`, `f z = τ`, `v` is a unit vector, and `f' v > c`, then there is `δ > 0` such that `f(z + t·v) > τ + c·t` for all `t ∈ (0, δ)`.*

Proof sketch. Write `g(t) = f(z + t·v) − τ − c·t`. Its derivative at `0` is `f' v − c > 0`, and `g(0) = 0`. A function with value zero and positive derivative at a point is strictly positive just to the right, so `g(t) > 0`, which is the claim. ■

This is `deriv_implies_local_growth` and its strict form `deriv_implies_strict_local_growth`. Now assemble.

Theorem 5.35 (Gradient chain to persistence). *Let `f` be differentiable at a boundary point `z`, with `f z = τ`, and let the defense be `K`-Lipschitz. If the gradient norm satisfies `‖f'‖ > ℓ(K+1)` for the relevant Lipschitz constant `ℓ`, then the steep region `{x | f x > τ + ℓ(K+1)·dist(x, z)}` is nonempty, and in fact the persistent unsafe set `{x | f(D x) > τ}` has positive measure.*

Proof sketch. Set `c = ℓ(K+1)`. By Theorem 5.33 the gradient norm exceeding `c` yields a unit direction `v` with `f' v > c`. By Theorem 5.34 that direction gives local growth `f(z + t·v) > τ + c·t` for small `t > 0`, which places the point `z + t·v` in the steep region: it grows faster than the defense's reach. So the steep region is nonempty. The cone bound (Theorem 5.24) then upgrades a single steep point to a positive-measure set that stays unsafe after `D`. ■

This is `gradient_norm_implies_steep_nonempty` and `gradient_chain_persistent_unsafe`, the latter invoking `persistent_unsafe_refined` from `MoF_20`. The chain is the complete story of a gradient attack against a defended model. Measure the gradient at the boundary. If it beats `ℓ(K+1)`, the defense's smoothing budget, then following that gradient reaches behaviors the defense cannot save, and not just one but a set of positive volume. The single number `‖∇f(z)‖`, the steepness of the alignment surface at the boundary, decides whether the model is persistently attackable. This is the transversality condition `transversality_from_deriv` of `MoF_11` in its sharpest form.

Remark 5.36. Compare this to the diagonal. Chapter 1's attack was a construction: build the liar query and you are done, with no notion of effort. The

gradient attack is a process: start somewhere, follow the slope, arrive. The diagonal's witness is exact and instantaneous but says nothing about cost. The gradient's witness is approximate and iterative but comes with a step count, `(τ − f(start))/gain` steps to cross, which is a real budget. The two attacks reach the same boundary. Only the second one has a price tag, and the price tag is the subject of the rest of the chapter.

5.8 Transferability of attacks across models

Practitioners observe that an attack tuned against one model often works against another. The geometry explains why, and quantifies exactly how much slack the attack needs to survive the crossing. The mechanism is that models with similar alignment surfaces have overlapping basins, and an attack with enough margin sits in the overlap.

Say two models have alignment surfaces `f` and `g` that are *δ-close in sup norm*, meaning `|f x − g x| ≤ δ` for all `x`. The number δ measures how differently the two models score the same behavior, the largest disagreement anywhere in the space.

Theorem 5.37 (Transfer with margin). *If `|f x − g x| ≤ δ` everywhere and `f p > τ + δ`, then `g p > τ`.*

Proof. From `|f p − g p| ≤ δ` we get `g p ≥ f p − δ > (τ + δ) − δ = τ`. ■

This is `transfer_attack` in `MoF_06_Transferability`, restated as `transfer_margin_needed` in `MoF_Cost_07_TransferCost`. The margin δ is exactly the toll for crossing between models. An attack that merely clears `τ` on the source model might fail on the target, because the target could score it up to δ lower. An attack that clears `τ + δ` is guaranteed to transfer. Attackers know this instinctively when they over-optimize, pushing the attack deep into the source basin rather than stopping at the edge; the geometry says over-optimization by δ is precisely what buys transfer.

Theorem 5.38 (Basin containment). *Under δ-closeness, `{x | f x > τ + δ} ⊆ {x | g x > τ}`. If `δ = 0` the models agree pointwise and their basins are identical at every threshold.*

Proof. The containment is Theorem 5.37 applied at each point. If `δ = 0` then `f = g`, so the superlevel sets coincide. ■

This is `basin_containment` and `identical_models_identical_basins`, with the free-transfer corollary `zero_distance_free_transfer`. The picture is one basin shrunk slightly inside another. The `(τ + δ)`-basin of `f` sits entirely within the `τ`-basin of `g`. Any attack landing in the inner basin transfers automatically to the outer one. The two-sided version, `pointwise_sandwich`, says each score traps the other in a band of width δ, so transfer runs in both directions with the same toll.

Theorem 5.39 (Transfer is computationally free). *Given a δ-closeness certificate and a source-model success `f p > τ + δ`, the target-model success `g p > τ` follows by pure deduction, with zero additional evaluations of the target model `g`.*

Proof. The conclusion is Theorem 5.37, whose proof uses only the closeness bound and the source success. No term in the proof evaluates `g` at any new point. ■

This is `transfer_is_free` in `MoF_Cost_07`, which literally proves the transferred conclusion is definitionally the same object as the margin theorem's conclusion, so it consumes no target queries. The cost consequence is severe for the defender. An attacker who has paid to break one model gets every δ-close model for free, with no further access to the targets. Model families trained on similar data, or fine-tunes of a common base, are δ-close for small δ, so a single attack sweeps the family. This is why closed deployment of a model whose base is public offers less protection than it appears.

There is a subtler consequence for defenses that are themselves models. A defense classifier is a function on the same behavior space, and if it is δ-close to the score it is trying to guard, transfer runs against it too. An attacker who has learned the geometry of the base score can predict where the defense classifier will misfire, because a δ-close classifier shares the base's basins up to the margin δ. The defense does not get to hide behind being a separate model. To the extent it agrees with the thing it guards, it inherits the thing's vulnerabilities, and to the extent it disagrees, it misclassifies genuinely safe behavior. This is the transfer face of the optimal defense dilemma of Theorem 5.51: closeness buys agreement at the price of shared failure, and distance buys independence at the price of collateral refusals.

Estimating the closeness

Transfer needs δ, and δ must be estimated from samples, which is where concentration enters.

Theorem 5.40 (Empirical closeness underestimates the truth). *If* $|f\ x_i - g\ x_i| \le \delta$ *at every sample point, then the empirical maximum disagreement over a finite nonempty sample set is at most* δ*. More samples give a larger, hence tighter, lower estimate of the true* δ*.*

Proof. The maximum of quantities each bounded by δ is bounded by δ. Enlarging the sample set can only raise the maximum, so it moves the estimate up toward the true value. ■

This is `estimation_cost` and `more_samples_better_estimate`. The empirical sup-norm distance is always a lower bound on the true one, so an attacker who samples aggressively gets a δ estimate that can only improve with effort, and the estimation cost is the number of paired evaluations, not an exponential quantity. Transfer is cheap to plan and free to execute.

A worked example: a fine-tune and its base

Let `g` be a base model and `f` a light fine-tune, and suppose the fine-tune changed the alignment score by at most δ `= 0.1` anywhere, a plausible figure for a small parameter update. An attacker who has found, against the fine-tune `f`, a prompt `p` scoring `f p = τ + 0.15`, that is with margin `0.15 >` δ,

is guaranteed by Theorem 5.37 that the base model also scores `g p > τ`: the attack transfers. Had the attacker stopped at margin `0.05`, transfer would not be guaranteed, and indeed might fail. The lesson for both sides is the same number. Fine-tuning for safety with a small δ leaves the base model's attacks almost entirely intact, and only attacks with margin below δ are potentially neutralized by the update. Small edits move the basin by δ and no more.

5.9 The cost theory

Everything so far has been about sets: which behaviors are unsafe, how they cluster, how they transfer. The cost theory converts geometry into budgets. It answers the operational questions. What does an attacker pay to find a failure? What does a defender pay to prevent one? How does the ratio scale? The `MoF_Cost` series builds this in ten layers, and the punchline is a single asymmetry: attack cost stays bounded while defense cost explodes with dimension.

The ten layers are worth naming as a stack, because each rests on the one below. Ball volume (layer one) turns a radius into measure. Basin volume (layer two) inscribes a robustness ball and reads off a lower bound on the basin. Hitting time (layer three) turns a measure fraction into an expected number of random trials. Concentration (layer four) controls the error of estimating the surface from samples. Attack cost (layer five) counts the steps of a directed climb. Defense cost (layer six) counts the patches needed to cover space. Transfer cost (layer seven) prices porting an attack across models. The cost ratio (layer eight) divides defense by attack. Lipschitz estimation (layer nine) bounds the sampling budget. The unified theory (layer ten) bundles the parameters and proves the master asymmetry. Read from the bottom, it is a derivation. Read from the top, it is a single sentence: the ratio diverges.

Ball and basin volume

The first two layers turn radii into measure, the raw material of every later budget.

Theorem 5.41 (Ball volume is positive and monotone). *In `ℝ^n` with Lebesgue measure, an open ball of positive radius has positive volume. Volume is monotone in the radius. In `ℝ` the ball of radius `r` has volume exactly `2r`.*

Proof. A nonempty open set has positive Lebesgue measure. Larger radius means a larger ball by inclusion, so larger volume by monotonicity. In `ℝ` the ball is an interval of length `2r`. ■

These are `ball_measure_pos`, `ball_measure_mono`, and `ball_measure_scale` in `MoF_Cost_01_BallVolume`.

Theorem 5.42 (Basin volume lower bound). *If `f` is `L`-Lipschitz and `f p > τ`, the basin `{x | f x > τ}` contains the robustness ball of radius `(f p − τ)/L` around `p`, so its measure is at least that of the ball. If `f` is merely continuous with an unsafe point, the basin still has strictly positive measure.*

Proof. The containment is the Lipschitz basin ball (Theorem 5.14) restated as a subset relation, and measure is monotone. The continuous case is Theorem 5.5. ■

This is `robustness_ball_subset_basin`, `basin_measure_ge_ball`, and `basin_measure_ge_ball_pos` in `MoF_Cost_02_BasinVolume`, with measurability recorded in `basin_measurableSet`. This is the volume version of "attacks are robust." The attacker's target is not a point but a ball, and the ball's volume, at least `(margin/L)^n` times a dimensional constant, is the probability mass a random search must hit.

Hitting time

If the basin has probability `p` under the sampling distribution, how many random trials until a hit? The answer is the geometric distribution, and the cost layer proves its analytic core.

Theorem 5.43 (Random search hitting time). *Suppose each independent trial lands in the basin with probability `p`, `0 < p ≤ 1`. The probability of missing on all of the first `n` trials is `(1 − p)^n`, which decreases monotonically to `0`. The probability of at least one hit, `1 − (1 − p)^n`, increases to `1`. The expected number of trials to the first hit is `1/p`.*

Proof sketch. The miss probability `(1 − p)^n` is a power of a number in `[0, 1)`, hence decreasing to zero. The complementary hit probability increases to one. For the expectation, the partial geometric sums `∑_{k<n} (1 − p)^k` are bounded above by the infinite geometric series `∑_{k≥0} (1 − p)^k = 1/p`, and this sum is the expected trial count of the geometric distribution. ■

This is `miss_probability_vanishes`, `hit_probability_tends_to_one`, `geometric_sum_le_inverse`, and `tsum_geometric_eq_inv` in `MoF_Cost_03_HittingTime`. The expected hitting time `1/p` is the attacker's random- search budget. Combine it with the basin volume: if the basin occupies a fraction `p` of the space, the attacker expects to wait `1/p` random draws. When `p` is a constant fraction, this is cheap. The trouble for the attacker, and the only real hope for the defender, is that in high dimension `p` can be exponentially small, which is where gradient attacks earn their keep by replacing random search's `1/p` with a linear step count.

Concentration and estimation

The defender estimates `f` from finite samples and classifies by the estimate. Concentration controls the error.

Theorem 5.44 (Estimation margin). *Suppose an estimate `f̂` satisfies `|f̂ x − f x| ≤ ε` everywhere. If `f̂ x > τ + ε`, then truly `f x > τ`; if `f̂ x < τ − ε`, then truly `f x < τ`. The undecidable band is `{x | |f̂ x − τ| ≤ ε}`, and it shrinks as `ε` shrinks.*

Proof. From `|f̂x − f x| ≤ ε`: if `f̂x > τ + ε` then `f x ≥ f̂x − ε > τ`, and symmetrically for the safe side. The band characterization is a rearrangement of the absolute-value inequality. ■

This is `estimation_error_bound_unsafe`, `estimation_error_bound_safe`, `estimation_uncertain_band`, and `estimation_band_shrinks` in `MoF_Cost_04_Concentration`. The estimate confidently classifies points more than ε from the threshold and cannot classify points within ε of it. That undecidable band is exactly the coarea band of Theorem 5.20, whose volume is at least $\varepsilon/(2L)$ in one dimension. Estimation error and boundary geometry are the same phenomenon viewed twice: the band the classifier cannot resolve is the band the geometry says has positive volume.

Theorem 5.45 (Lipschitz interpolation from samples). *For an L-Lipschitz f, knowing f at a sample x_0 bounds f nearby: $|f\ x - f\ x_0| \le L \cdot \mathrm{dist}(x, x_0)$. To hold the estimation error below ε everywhere in $[0,1]^d$ you need samples no farther than ε/L apart, which forces on the order of $(L/\varepsilon)^d$ samples.*

Proof. The bound is the Lipschitz condition. Covering $[0,1]^d$ so that every point is within ε/L of a sample requires a grid of spacing $2\varepsilon/L$, hence roughly $(L/(2\varepsilon))^d$ points, exponential in d. ■

This is `lipschitz_estimation_error` and `estimation_improves_with_distance` in `MoF_Cost_09_LipschitzEstimation`, with the exponential covering count `total_estimation_budget`, and the sup-norm triangle and nearest-sample bounds `sup_norm_triangle` and `n_samples_coverage` in `MoF_Cost_04`. The defender's estimation problem inherits the curse directly. Certifying the whole space to accuracy ε costs $(L/\varepsilon)^d$ samples, the same exponential wall as exhaustive gridding. Concentration of measure, meanwhile, works against the defender's sampling: in high dimension the samples spread thin and the nearest sample to a random point is far, so the interpolation bound $L \cdot \mathrm{dist}$ is loose exactly where it matters.

Attack, defense, and transfer cost

Now the three budgets, each a corollary of the layers above.

Theorem 5.46 (Attack cost). *An iterative attack gaining at least $\delta > 0$ per step, on a score bounded in $[0,1]$, reaches any threshold τ from a start s_0 in at most $\lceil(\tau - s_0)/\delta\rceil + 1$ steps, and can take at most $\lceil 1/\delta\rceil$ useful steps in all. On reaching $f > \tau$ with margin, the attacker gains a robustness ball of radius $(f - \tau)/L$.*

Proof sketch. By induction, after n steps the score is at least $s_0 + n\cdot\delta$, so it exceeds τ once $n \ge (\tau - s_0)/\delta$. Since the score cannot exceed 1, the total number of δ-gaining steps is at most $1/\delta$. The robustness ball is Theorem 5.14. ■

This is `attack_value_after_n_steps`, `steps_to_threshold`, `attack_cost_upper_bound`, and `total_attack_cost` in `MoF_Cost_05_AttackCost`, with the monotone-convergence backbone `attackSeq_convergent` and `finite_steps_bound` in `MoF_05_MonotoneConvergence`. The attack budget is $1/\delta$, a constant independent of dimension. This is the crux. The attacker's cost does not grow with d, because the attacker follows a one-dimensional trajectory of steps and each step gains a fixed amount, regardless of how many dimensions surround the path.

Theorem 5.47 (Defense cost). *A defender covering the space with patches of relative radius `r < 1` needs on the order of `(1/r)^d` patches to cover a fixed fraction, and this grows without bound as `d → ∞`. For any budget `B` there is a critical dimension beyond which the defense cost exceeds `B`.*

Proof. Each patch covers fraction `r^d`, so covering fraction `α` needs `α/r^d = α·(1/r)^d` patches, which tends to infinity since `1/r > 1`. Passing any fixed `B` is then a matter of taking `d` large. ■

This is `patches_needed`, `defense_cost_exponential`, and `critical_dimension` in `MoF_Cost_06_DefenseCost`. The defense budget is `(1/r)^d`, exponential in dimension. Set this beside the attacker's constant $1/\delta$ and the asymmetry is stated.

Theorem 5.48 (Transfer cost). *Porting an attack across two δ-close models costs zero target queries once the source attack has margin δ, and the closeness δ itself is estimated from samples at a cost that grows only with the desired precision, not with dimension.*

Proof. Free execution is Theorem 5.39. The estimation cost is Theorem 5.40. ■

This is the content of `MoF_Cost_07_TransferCost`. Transfer sits on the attacker's side of the ledger, adding essentially nothing to the attack budget while multiplying its reach across a model family.

The cost ratio and the master theorem

The three budgets combine into one number that decides the contest.

Definition 5.49 (Cost ratio). The *defense-to-attack cost ratio* is `costRatio(N, d, δ) = N^d · δ`, the defender's `N^d` cells divided by the attacker's $1/\delta$ steps. This is `costRatio` in `MoF_Cost_08_CostRatio`, and in bundled form `unifiedCostRatio` on the `CostParameters` structure of `MoF_Cost_10_UnifiedTheory`, where it simplifies to `δ · N^d` (`unifiedCostRatio_eq`).

Theorem 5.50 (Cost asymmetry, master theorem). *For any fixed grid resolution `N ≥ 2` and attack gain `δ > 0`, the cost ratio `δ · N^d` is positive, increases with `d`, and tends to infinity. For any willingness-to-pay ratio `R > 0`, however large, there is a critical dimension `d₀` beyond which the ratio exceeds `R`. The attacker's budget is independent of dimension; the defender's is exponential in it.*

Proof sketch. The ratio is `δ · N^d` with `N ≥ 2`, so it is a positive constant times an exponential in `d`, hence positive, monotone increasing, and divergent. Given `R`, divergence supplies a `d₀` past which `δ · N^d > R`. The attack budget $1/\delta$ contains no `d`, so it is constant across dimension. ■

This is `cost_ratio_tends_to_infinity`, `cost_ratio_exceeds_any_bound`, `attacker_advantage_quantified`, and the capstone `MASTER_THEOREM_cost_asymmetry`, with dimension-independence recorded in `attack_budget_dimension_independent` and `defenseBudget_exponential`. As a concrete anchor, `two_dimension_ratio` computes that even at `d = 2`, `N = 25`, δ `= 0.01`, the ratio is already `6.25`, and it only climbs from there. This theorem is the quantitative summary of the whole chapter. The boundary exists (qualitative, from the diagonal and the IVT). The boundary is populated by a

positive-measure region (coarea and cone bounds). Finding a point in that region costs the attacker a dimension-independent budget (attack cost and hitting time). Covering it costs the defender a dimension-exponential budget (defense cost). The ratio of the two diverges. No budget closes the gap.

The optimal defense dilemma

A defender might try to escape by tuning the defense's own smoothness K. The optimal-defense theory shows this only trades one failure mode for another.

Theorem 5.51 (No K wins). *For an alignment surface with Lipschitz constant L and boundary growth rate (transversality slope) G, and a K-Lipschitz defense, exactly one of two things holds for every K ≥ 0: either G > L(K+1), in which case a persistent unsafe region of positive measure survives (Theorem 5.24), or L(K+1) ≥ G, in which case the failure band has width at least G·δ. When G > L, the critical value K* = G/L − 1 > 0 is the unique point where the two modes trade off exactly, and neither horn is escapable. When G ≤ L, the boundary is shallow and defensible in principle (Theorem 5.25).*

Proof sketch. The dichotomy G > L(K+1) or L(K+1) ≥ G is a trichotomy-free case split on a single real comparison. Increasing K increases L(K+1) strictly (defense_band_monotone), so it eventually flips the first horn into the second, and at K = K* = G/L − 1 the two sides are equal (optimalK_critical). Below K* the steep region is nonempty; at or above it the band width L(K+1)·δ ≥ G·δ. Both horns are realized for G > L (defense_dilemma_both_realizable), and for G ≤ L the shallow case gives L(K+1) ≥ L ≥ G for all K ≥ 0. ■

This is defense_cannot_win, optimal_K_exists, defense_dilemma_both_realizable, and complete_defense_characterization in MoF_19_OptimalDefense. The dilemma is genuine. A gentle defense (small K) leaves persistent unsafe cones. An aggressive defense (large K) smears the failure band wide, distorting the model's behavior across a region of width growing linearly in K. Between the two extremes sits K*, and at K* the defender has minimized the total damage but not eliminated it. The only way out is G ≤ L, a boundary that never rises faster than the model's global smoothness, and Theorem 5.35 says that condition is exactly ‖∇f(z)‖ ≤ L at every boundary point, a demand that the boundary have no spikes at all.

A worked example: the cost ratio in a real regime

Put numbers to it. Suppose a language model's behavior space, for the purpose of a particular attack, is well described by d = 20 effective dimensions, with a modest grid resolution N = 10 and an attack that gains δ = 0.05 per step. The attacker's budget is about 1/δ = 20 steps. The defender's cell count is N^d = 10^20. The cost ratio is δ · N^d = 0.05 · 10^20 = 5 · 10^18. The attacker pays twenty steps; the defender pays five quintillion cells. Now double the effective dimension to d = 40. The attacker still pays twenty steps. The defender pays 10^40. Nothing the defender does to N, r, or K touches the

exponential in d, and the attacker's budget never sees d at all. This is not a pessimistic model chosen to make a point. It is the generic behavior of the cost ratio, proved to diverge for every fixed N ≥ 2 and δ > 0 in Theorem 5.50.

The one honest place a defender can push back is the word "effective." The exponent is not the raw parameter count but the dimension of the manifold the attack actually explores, and if that manifold is low-dimensional the ratio is manageable. This is the real research question hiding under the theorem: how large is the effective dimension of the space of behaviors an attacker can reach with realistic edits? If it is small, say a handful, the defender has a chance, because N^d for small d is a finite budget. If it is large, the defender has already lost, because no budget crosses the exponential. The cost theory does not measure the effective dimension for you; it tells you that this one number, and not the model's size or the defender's cleverness, decides the contest. Everything else in the budget, N, δ, r, K, enters polynomially or as a constant. Only d enters the exponent, and only d is worth arguing about.

5.10 Concentration of measure, the two faces of high dimension

The cost theory kept referring to a fraction p, the probability mass of the basin, and treated it as a knob that could be small or large. High-dimensional geometry does not leave p free. It concentrates. Distances, norms, and the values of well-behaved functions all cluster around single values as the dimension grows, and this clustering is the deepest reason the attacker and the defender experience the same space so differently. This section makes the concentration precise and draws out its two faces.

Start with the crudest volume bound, which needs no smoothness at all, only nonnegativity and integrability of the alignment surface.

Theorem 5.52 (Markov bound on the basin). *Let f ≥ 0 be integrable against a measure μ. Then for any ε > 0, ε · μ({x | f x ≥ ε}) ≤ ∫ f dμ. Equivalently, μ({x | f x ≥ ε}) ≤ (1/ε) ∫ f dμ.*

Proof. On the set {f ≥ ε} the integrand is at least ε, so ∫ f dμ ≥ ∫_{f ≥ ε} f dμ ≥ ε · μ({f ≥ ε}). Divide by ε. ■

This is markov_real_basin (with the extended-real form markov_ennreal_basin) in MoF_Adv_10_MeasureBounds. It is Markov's inequality read as a statement about basins: the measure of the region where the alignment score is at least ε is controlled by the average score divided by ε. A model whose average deviation is small cannot have a large high-deviation basin. This sounds like comfort for the defender, and at a fixed threshold it is. But it also sets a floor the defender cannot easily lower, because the average ∫ f dμ is a property of the whole model, not of any local patch, and driving it down means changing the model everywhere at once.

The Markov bound is one-sided and loose. The sharp phenomenon in high dimension is two-sided and tight, and it is about Lipschitz functions specifically.

The concentration phenomenon. On the unit sphere in ℝ^n, or under a standard Gaussian on ℝ^n, an L-Lipschitz function f deviates from its median m by more than t with probability at most about 2 · exp(-c · t² / L²) for a universal constant c. As n grows, the constant in the exponent works in the attacker's favor through the geometry, but the shape is the key: a Lipschitz function is nearly constant across almost all of a high-dimensional space. This is the Lévy-Milman-Talagrand concentration of measure, and while its full form lives beyond the current formal library, its consequences are exactly the cost asymmetries the library does prove. It is worth seeing why concentration and the curse are the same fact.

Consider the alignment surface f. If f is L-Lipschitz, concentration says its values pile up around the median m. Suppose the threshold τ sits above m, so that most behaviors are safe. Then the basin {f > τ} is the far tail of a concentrated distribution, and its measure p is exponentially small in (τ − m)² / L². This is good for the defender in the sense that a random behavior is almost never unsafe. It is catastrophic for the defender in the sense that Theorem 5.5 still guarantees p > 0, so the tail is nonempty, and the hitting-time result (Theorem 5.43) says random search needs 1/p draws, which is huge, but the gradient chain (Theorem 5.35) says a directed attacker skips the 1/p wait entirely by climbing the gradient straight to the tail. Concentration hides the basin from random draws and does nothing to hide it from gradients. The attacker who knows this never samples at random.

Theorem 5.53 (Positive-measure failure band, full defense). *Let X be a connected Hausdorff metric space with a positive-on-opens measure μ. Let f be L-Lipschitz and continuous with L > 0, let D be a K-Lipschitz continuous defense that fixes the safe region, and suppose the boundary is transverse in the sense that f grows from some fixed boundary point faster than L(K+1). Then the set of behaviors that remain unsafe after defense, {x | f(D x) > τ}, has strictly positive measure.*

Proof sketch. The transversality hypothesis places a point in the steep region (Theorem 5.24, part 1). The distortion bound (Theorem 5.23) shows every steep point stays unsafe after D (Theorem 5.24, part 3). The steep region is open and nonempty, so it has positive measure, and it sits inside the persistent unsafe set, which therefore also has positive measure. ■

This is epsilon_robust_impossibility, positive_measure_failure_band, and persistent_unsafe_region in MoF_11_EpsilonRobust, the results the cone bound and gradient chain feed into. It is the strongest form of the defense impossibility in this chapter. The master theorem (Theorem 5.12) gave one surviving boundary point. The cone bound (Theorem 5.24) gave a surviving interval in one dimension. This gives a surviving set of positive measure in full generality, under honest Lipschitz hypotheses on both the model and the defense. The impossibility is not a boundary artifact of measure zero. It is a region an adversary can enter.

Remark 5.54 (The two faces). Concentration has a face turned toward the attacker and a face turned toward the defender, and they are the same geometry. Toward the defender it says: your samples spread thin, the nearest sample to a random point is far (distance concentrates around a large value), so your

interpolation bound `L · dist` from Theorem 5.45 is loose everywhere, and you cannot certify the space without exponentially many samples. Toward the attacker it says: the space is mostly one big undifferentiated bulk where a Lipschitz score is nearly constant, so a gradient points reliably toward the rare tail, and once in the tail the robustness ball (Theorem 5.14) is a generous target. The defender sees a space too big to cover. The attacker sees a space too smooth to hide the exit. Both are reading the same concentration inequality.

A worked example: the Gaussian annulus

Draw `x` from a standard Gaussian on `ℝ^n`. The squared norm `‖x‖²` is a sum of `n` independent squared standard normals, with mean `n` and standard deviation `√(2n)`. So `‖x‖` concentrates sharply around `√n`, within a window of width on the order of `1`, which is a vanishing fraction of `√n`. Almost all the Gaussian mass sits in a thin annulus at radius `√n`. The center, where the density is highest, is almost empty of mass, because it has almost no volume.

This single picture explains the whole chapter in miniature. Suppose the alignment surface is the smooth radial function `f(x) = ‖x‖`, which is 1-Lipschitz, and the threshold is `τ = √n + 3`. The basin `{‖x‖ > √n + 3}` is the outside of the annulus, and its Gaussian mass is tiny, on the order of `exp(-c · 9)`, independent of `n` because the window width does not grow. Random search for the basin costs `1/p`, a large fixed number. But `f` has gradient `x/‖x‖` of norm exactly `1` everywhere off the origin, so a gradient attacker starting anywhere walks outward at unit speed and reaches `τ` in about `3` steps regardless of `n`. The defender who wants to patch the basin must cover a thin shell in `ℝ^n`, whose covering number is exponential in `n`. Same basin, three numbers: attacker cost constant, defender cost exponential, random-search cost large but irrelevant because the attacker does not use random search. That is Theorem 5.50 wearing Gaussian clothes.

5.11 Iterated and discrete attacks

Two loose ends remain, and both matter for real deployments. Attacks are rarely single shots; they accumulate over a conversation or a campaign. And behavior spaces are often discrete, made of tokens rather than points in `ℝ^n`, so the continuum arguments need a discrete counterpart. The `ManifoldProofs` development treats both, and the results reconnect this chapter to the diagonal.

Iterated attacks

Model a multi-turn interaction as a sequence of alignment surfaces `f_t` and defenses `D_t`, one per turn, with the attacker free to probe repeatedly. The relevant quantity is the running maximum of the scores achieved, since the attacker only needs to succeed once and can keep the best result.

Theorem 5.55 (The attacker's running edge). *The running maximum of the attacker's scores over turns is monotone nondecreasing: it never falls as turns accumulate. Under a capacity-parity model where the defense has a fixed number of correction slots and the attacker has one more configuration than there are slots, the pigeonhole principle forces the defense to collapse two distinct attacks to the same output, so at least one attack is not individually corrected. Over `T` turns of probing the defense reveals at least `T` classifications, and one more attack than the defense can absorb overwhelms it.*

Proof sketch. The running maximum `sup_{s ≤ t} score_s` is a supremum over a growing finite set, hence nondecreasing in `t` (`running_max_monotone`). The pigeonhole step is the capacity argument: a defense with `n_unsafe` free slots handling `n_unsafe + 1` distinct configurations cannot separate them all (`one_more_attack_overwhelms`), and iterated over an attack surface of size `2^d` against total defense capacity `2^d` the attacker retains a permanent margin (`attacker_permanent_edge`). Each probe yields a classification, so `T` turns leak at least `T` bits of boundary information (`defense_leaks_per_turn`). ■

These are `running_max_monotone`, `one_more_attack_overwhelms`, `attacker_permanent_edge`, `defense_leaks_per_turn`, and the capstone `multi_turn_impossibility` and `boundary_points_accumulate` in `MoF_13_MultiTurn`. The picture is of an attacker with a ratchet. Each turn can only improve the best result achieved so far, each probe extracts information about where the boundary lies, and the boundary points accumulate turn over turn. A defense that holds for one turn need not hold for a hundred, and the theory says it will not: the attacker's edge is permanent under parity, and iteration converts a small per-turn leak into a boundary the attacker eventually maps. This is the formal shadow of the practical fact that long conversations erode guardrails.

The discrete boundary

When the behavior space is a finite chain of tokens rather than a continuum, the intermediate value theorem is replaced by a discrete crossing lemma, and the master theorem by a discrete fixed-point argument.

Theorem 5.56 (Discrete crossing and the discrete trilemma). *Let `f` be defined on a finite chain `0, 1, …, n+1` with `f 0 < τ` and `f(n+1) ≥ τ`. Then there is an adjacent pair `i, i+1` with `f i < τ` and `f(i+1) ≥ τ`: the score crosses the threshold across a single step. If a defense `D` fixes every safe point of the chain, it fixes such a crossing point, and cannot make it safe. More abstractly, for any `f`, threshold `τ`, defense `D` fixing the safe region, and any unsafe point `u`, either `D` leaves `u` unsafe, or `D` moves `u` and thereby fails to be the identity where it needed to be. A verdict that is both complete (decides every point) and injective (never conflates two points) cannot exist on a space rich enough to encode its own decisions.*

Proof sketch. The crossing lemma is the discrete IVT: take the largest index `i` with `f i < τ`; since `f(n+1) ≥ τ` this `i` is at most `n`, and by maximality `f (i+1) ≥ τ` (`discrete_ivt`). The defense-fixes-crossing claim is the discrete

master theorem (`discrete_defense_boundary_fixed`). The case split on `u` is `discrete_trilemma`. The completeness-versus-injectivity impossibility is `injectivity_forces_incompleteness` and `completeness_forces_noninjectivity`. ■

These are in `MoF_12_Discrete`. The last clause is where the geometry rejoins the diagonal. The discrete trilemma is a counting argument, pigeonhole rather than connectedness, but it delivers the same conclusion the diagonal did in Chapter 1: a verdict that tries to be both complete and consistent on a self-encoding domain breaks. Here the two engines, analytic and diagonal, are visibly the same shape. The continuum route reaches the boundary by connectedness and the IVT; the discrete route reaches it by pigeonhole; the self-referential route reaches it by diagonalization. Three proofs, one boundary.

Remark 5.57. The discrete results are the bridge back to the book's spine. Chapter 1 built a query no boolean verdict answers, using self-reference. `MoF_12`'s `discrete_trilemma` builds an unsafe point no utility-preserving defense corrects, using order and counting. They are not the same theorem, but they are instances of one pattern: a system asked to classify a domain it is embedded in must leave a witness uncovered. The geometry chapters spent their length measuring that witness, giving it volume, distance, and cost. The discrete trilemma reminds us that the witness was there from the first page, and that the measuring, not the existence, is what this half of the book added.

5.12 The two halves, joined

It is worth stepping back to see how this chapter sits against the diagonal chapters, because the book's thesis lives in the join.

The diagonal proves existence and nothing more. There is a query no verdict answers, an index no naming captures, a behavior no defense corrects. These are theorems about what cannot be, and they are indifferent to quantity, cost, and dimension. Chapter 1's three-line proof would be unchanged if the space were finite or infinite, small or vast. That indifference is its strength and its limit. It reaches conclusions no amount of engineering can overturn, and it reaches nothing else.

This chapter proves the quantitative shape of the same failure. The boundary the diagonal guarantees to exist is here shown to have positive-measure thickness (coarea bound), to carry a persistent unsafe cone against any defense (cone bound), to be reachable by a dimension-independent attack budget (attack cost and hitting time), and to be coverable only at dimension-exponential defender cost (defense cost, cost ratio). Where the diagonal says "a bad point exists," the geometry says "a region of bad points exists, it is cheap to enter and expensive to seal, and the gap grows without bound in dimension."

The two halves are not rivals. They are the qualitative and quantitative faces of one fact. The impossibility theorems tell you that you cannot win.

The cost theory tells you how badly you lose, and it turns out you lose by an exponential margin that no budget closes. A practitioner who has absorbed only the diagonal might hope that the bad point is rare, hard to find, or easy to patch. The geometry closes each of those hopes in turn. A practitioner who has absorbed only the cost theory might think the problem is merely hard, a matter of enough compute. The diagonal closes that hope: the boundary is not hard to remove, it is impossible to remove. Read together, they say the danger is both certain and quantitatively severe, and that is the honest picture of what deploying a capable model next to its own failure boundary actually means.

5.13 Historical and bibliographic notes

The intermediate value theorem underlying the threshold-crossing results is Bolzano's, and its use to force a boundary between qualitatively different regimes is as old as analysis. The reading of that boundary as an unavoidable attack surface for learned systems is recent and is the organizing idea of the `ManifoldProofs` development. The curse of dimensionality has many independent origins; the covering-number form used here, that maintaining accuracy ε for an `L`-Lipschitz function on `[0, 1]^d` costs on the order of `(L/ε)^d` samples, is standard in nonparametric statistics and approximation theory. Concentration of measure in high dimension, the phenomenon that random points cluster and Lipschitz functions vary little, is due in its modern form to Lévy, Milman, and Talagrand, and its adversarial reading, that concentration helps the attacker and hurts the defender, is the contribution of the cost layers. The gradient-attack formalization abstracts the practice of methods such as FGSM and projected gradient descent into their mathematical skeleton: a nonzero derivative always affords an ascent step. The coarea inequality proper, relating band volume to level-set integrals of `1/‖∇f‖`, is Federer's; the ball-inscription substitute used here is a self-contained lower bound that avoids the parts of geometric measure theory not yet formalized. All theorem names cited in this chapter refer to the machine-checked statements in the companion Lean library, and where the prose and the Lean differ, the Lean is authoritative.

5.14 Exercises

Exercise 5.1. Verify directly that for the Gaussian bump `f(x) = M · exp(-‖x‖²/2)` on `ℝ^n` with `0 < τ < M`, the basin `{x | f x > τ}` is the ball of radius `√(2 ln(M/τ))`. Confirm the three soft predictions (openness, positive measure, connectedness) against Theorems 5.3, 5.5, and 5.7, and identify which `MoF_*` result each uses.

Exercise 5.2. Show that Theorem 5.4 (every unsafe point owns a ball) gives no lower bound on the radius using continuity alone, by exhibiting a continuous but non-Lipschitz `f : ℝ → ℝ` and an unsafe point whose guaranteed ball

can be made arbitrarily small. Then show that adding a Lipschitz hypothesis pins the radius to `(f p − τ)/L` via `lipschitz_basin_ball`.

Exercise 5.3. Prove the no-gap theorem (Theorem 5.10) fails on a disconnected space by constructing an `f` on `X = [0, 1] ∪ [2, 3]` that is below `τ` on the first interval and above `τ` on the second, with empty boundary. Explain in one sentence why connectedness is the exact hypothesis `no_gap_theorem` requires.

Exercise 5.4. In the affine example `f(x) = ⟨w, x⟩ + b` with `‖w‖ = L`, verify that the robustness ball of Theorem 5.14 is tight: it touches the boundary hyperplane at exactly one point and lies strictly inside the basin everywhere else. Compute that point in terms of `p`, `w`, and the margin.

Exercise 5.5. Using `higher_lipschitz_more_oscillation` (Theorem 5.17), show that a model required to be safe (`f = 0`) at two inputs a unit apart and grossly unsafe (`f = M`) at their midpoint must have Lipschitz constant at least `2M`. Then use `smoother_functions_larger_basins` to argue that lowering `L` to below `2M` is impossible without removing the unsafe spike, and interpret this as a no-free-lunch statement for smoothing.

Exercise 5.6. Derive the one-dimensional coarea bound `μ(band) ≥ ε/(2L)` (Theorem 5.20) from the ball-in-band containment (Theorem 5.19) and the length formula for intervals. Then generalize the computation to `ℝ^n` and express the bound in terms of the volume of a Euclidean ball of radius `ε/(4L)`.

Exercise 5.7. For the cone bound (Theorem 5.24), suppose `f` grows from a boundary point `z` at rate `c` and the defense has reach `s = L(K+1)`. Show that the surviving steep region on `(z, z + δ₀)` has measure exactly `δ₀` and that every point of it satisfies `f(D x) > τ` by at least `(c − s) · (x − z)`. Interpret the gap `(c − s)` as the defender's shortfall.

Exercise 5.8. Prove the shallow-boundary escape (Theorem 5.25) and then show its converse direction: if the steep region is nonempty for some `K`, then `f` rises faster than `L` in some direction from `z`. Match your argument to `persistence_implies_steep_direction` in `MoF_19`.

Exercise 5.9. Compute the grid cost `N^d` and the coverage fraction `r^d` for `N = 4`, `r = 1/4`, `d = 10`, and `d = 30`. Verify numerically that a fixed defense budget of `10^6` patches is exhausted between these two dimensions, and identify the critical dimension `d₀` predicted by `no_fixed_budget_defense`.

Exercise 5.10. Using `regular_value_boundary` (Theorem 5.29), show that a one-dimensional alignment surface with nonvanishing derivative has an isolated (locally singleton) boundary. Then construct an `f` with a degenerate critical point on its boundary whose level set `{f = τ}` contains an interval, showing that regularity is necessary for isolation.

Exercise 5.11. From `ascent_direction_exists` (Theorem 5.31), prove that gradient ascent on a differentiable `f` never halts except at a critical point, and connect this to `critical_point_iff_fderiv_eq_zero`. Then use Theorem 5.32 to show that among all unit-step attacks, the gradient step maximizes the immediate gain in `f`.

Exercise 5.12. Trace the full gradient chain (Theorem 5.35) on the affine surface `f(x) = ⟨w, x⟩` with `‖w‖ = G`, against a `K`-Lipschitz defense. Deter-

mine exactly when `G > L(K+1)` fails and succeeds for `L = ‖w‖`, and reconcile your answer with the worked corner example of Section 5 (why affine surfaces alone cannot be persistently unsafe against their own Lipschitz constant).

Exercise 5.13. For transfer (Theorem 5.37), suppose two models are δ-close with δ `= 0.2`. An attacker holds a source-model success with margin `m`. For which `m` is transfer guaranteed, and for which `m` might it fail? Construct an explicit pair `f, g` with `|f − g| ≤` δ and a point where transfer fails for `m <` δ, showing the margin condition of `transfer_attack` is sharp.

Exercise 5.14. Using the hitting-time result (Theorem 5.43), compute the expected number of random trials to hit a basin of probability `p = 10^-6`, and compare it to the step count of a gradient attack that gains δ `= 0.05` per step from a start s_0 `= τ − 1`. Explain, in terms of `1/p` versus `(τ − s₀)/`δ, why gradient attacks dominate random search when the basin is small.

Exercise 5.15. Assemble the master cost asymmetry (Theorem 5.50) from its pieces: attack budget `1/`δ (constant), defense budget `N^d` (exponential), and the ratio δ `· N^d`. Prove the ratio exceeds any `R > 0` for large enough `d`, matching `MASTER_THEOREM_cost_asymmetry`, and state precisely which quantity's independence from `d` is the source of the asymmetry.

Exercise 5.16. (Harder.) Combine the optimal-defense dilemma (Theorem 5.51) with the coarea bound (Theorem 5.20) to show that at the critical `K* = G/L − 1`, the failure band has width at least `G·`δ and volume at least proportional to `G·`δ`/L` in one dimension. Then argue that no choice of `K` simultaneously drives both the persistent cone measure and the band volume to zero when `G > L`, and interpret this as the quantitative form of the master theorem's single surviving boundary point.

Exercise 5.17. (Open-ended.) The cost theory measures the attacker's budget in gradient steps and the defender's in grid cells, two different units. Propose a common currency (for instance, model evaluations, or wall-clock compute) in which both budgets can be expressed, and rederive the cost ratio in that currency. Does the asymmetry survive the change of units? Identify which theorem of this chapter would need restating and which would carry over unchanged.

Exercise 5.18. (Open-ended.) Every quantitative result here assumes a metric and a measure on behavior space. For a discrete space of token sequences, neither is canonical. Speculate on what plays the role of the Lipschitz constant `L`, the ball, and the coarea band in a discrete setting, and consult `MoF_12_Discrete` for one formalized answer. Which of the cost-ratio conclusions do you expect to survive discretization, and which depend essentially on the continuum?

Exercise 5.19. From the Markov basin bound (Theorem 5.52), derive an upper bound on the basin measure `μ({f ≥ τ})` in terms of the average score `∫ f dμ` and `τ`, and contrast it with the lower bound from the coarea band (Theorem 5.20). For the Gaussian annulus example with `f(x) = ‖x‖` under a standard Gaussian, estimate both bounds and explain why the gradient attack ignores the smallness of the Markov upper bound. Relate the annulus picture to why random search costs `1/p` while the gradient attack costs a constant.

Exercise 5.20. Using the running-maximum monotonicity and the pigeonhole step of Theorem 5.55, show that a defense with n correction slots facing $n + 1$ distinct attacks must leave at least one uncorrected, and that over T turns the attacker's best score is nondecreasing. Then state the discrete crossing lemma (Theorem 5.56) as a discrete intermediate value theorem and prove it by taking the largest safe index. Finally, explain in one paragraph, without symbols, why the discrete trilemma and the diagonal of Chapter 1 are two proofs of one fact.

5.15 References

Federer, H. (1969). *Geometric Measure Theory*. Springer, Berlin.

Goodfellow, I. J., Shlens, J., Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*.

Ledoux, M. (2001). *The Concentration of Measure Phenomenon*. Mathematical Surveys and Monographs 89. American Mathematical Society, Providence, RI.

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*.

6 Approximate Bridges to Real Models

Abstract. The impossibility theorems of Chapter 3 are exact, while deployed systems are not. This chapter argues that the strict theorem is the zero-slack corner of a family of quantitative statements. Given an error budget, the exact contradiction becomes a margin: a forced question on which the model is correct by no more than its calibration error. The chapter develops approximate versions of the diagonal and the threshold crossing, keeps truth units and confidence units distinct, and shows how the strict results are recovered at zero slack.

Keywords: Approximate bridges, Calibration error, Slack, Quantitative degradation, Robustness

The impossibility theorems of Chapter 3 are exact. They assume a model whose confidence is a perfect biconditional function of its correctness, a probe that never misfires, a truth boundary that the confidence field crosses cleanly. No deployed system meets any of these assumptions. A practitioner who reads the strict trilemma and then measures a real model will find calibration curves that wobble, confidence that is only roughly monotone in accuracy, coverage that is statistical rather than guaranteed. The natural reaction is to conclude that the theorem is about an idealization and says nothing about the thing on the bench.

That reaction is wrong, and this chapter is the argument for why. The strict statement is the $\varepsilon = 0$ corner of a family of quantitative statements, each of which assumes only an error budget and each of which survives that budget with a weakened but nonempty conclusion. The exact contradiction does not evaporate as the idealizations relax. It turns into a margin: a forced question on which the model is correct, but correct by no more than the calibration error it was allowed. Shrinking the error does not remove the forced question. It tightens the margin toward the exact contradiction. This is the sense in which the idealized theorem governs the real one.

Everything here is verified in two companion developments. The continuous and discrete calibration bridges are `HoF_12_Approximate` in the hallucination library; the metric degradation law is `F_09_QuantitativeDegradation` in the Foundation library. Both import Mathlib and reason over the re-

als. The book's own Lean package has no Mathlib, so the theorems below are cited by their verified names and proved here in prose. Where a fragment of the argument is genuinely finite, a small core-Lean illustration is included and elaborated when the book is built; those carry the prefix `c6_`.

A word on why the effort is worth it. A skeptic can read Chapter 3 and say the strict trilemma is a theorem about a model that does not exist, and dismiss it. The same skeptic cannot dismiss a theorem whose hypotheses are an error budget the skeptic's own model satisfies. Once calibration is granted only to within ε, coverage only outside a band, and universality only up to a distance, the hypotheses become measurable properties of a shipped system rather than philosophical idealizations. The conclusion then transfers to that system. This is the difference between a result that lives in a seminar and a result that lives in a deployment review, and closing that difference is the whole purpose of the chapter.

6.1 What the exact theorems assume

It helps to name the idealizations before dismantling them. The strict Hallucination Trilemma of Chapter 3 concerns a model that answers questions and reports a confidence, together with a notion of how wrong each answer is.

Write `Q` for the space of questions and `A` for the space of answers. A model is a map `M : Q → A × ℝ`. Reading `M q = (a, r)`, the answer is `a = (M q).1` and the reported confidence is `r = (M q).2`, a real number. Correctness is measured by a *truth-distance field* `δ : Q × A → ℝ`. The value `δ(q, a)` is a signed distance from the truth boundary: negative when the answer `a` is on the correct side for question `q`, positive when it is wrong, zero exactly on the boundary. Throughout we abbreviate the truth-distance of the model's own answer as `δ(q)`, meaning `δ(q, (M q).1)`. Fix a threshold confidence `c`; the paper's running value is `c = 1/2`, the point at which a report stops being a lean toward "no" and becomes a lean toward "yes".

The strict theorem rests on three hypotheses.

Strict calibration is a biconditional: `conf(q) > c ↔ δ(q) < 0` and `conf(q) < c ↔ δ(q) > 0`. In words, the model reports above-threshold confidence exactly when it is correct, and below-threshold confidence exactly when it is wrong. This is the verified predicate `StrictCalibrated`.

Faithfulness says the model does not confidently assert falsehoods: whenever `conf(q) ≥ c`, the answer is on the correct side, `δ(q) < 0`. This is `TrilemmaFaithful`.

Coverage says the model is not trivial: some question is answered correctly and some is answered wrongly, so the truth-distance field takes both signs. This is `TrilemmaCovering`.

Under these three, plus continuity of the confidence map and connectedness of the question space, the strict trilemma derives `False`. The mechanism is the one from Chapter 1 read through the intermediate value theorem of Chapter 4: coverage forces confidence above `c` somewhere and below `c`

somewhere, continuity forces a crossing where `conf = c`, and at that crossing calibration and faithfulness give incompatible signs for δ. The verified endpoint is that the three corners cannot all hold.

Two of these hypotheses are the ones a practitioner will instinctively defend, and it is worth saying why they are reasonable before weakening them. Continuity of the confidence map is not an exotic assumption. A confidence score computed from a softmax over logits that are themselves smooth functions of a continuous embedding is continuous in that embedding, and the input drift a deployment actually sees, paraphrase, added context, distribution shift, moves through the embedding space rather than teleporting across it. Connectedness of the question space is the subtler assumption and the one Chapter 4 spends its effort on; the short version is that after embedding, the reachable region of inputs is path-connected often enough that the intermediate value theorem has something to bite on. The discrete bridge below is the fallback for when it does not.

The sign convention deserves one more sentence, because it drives every inequality that follows. Negative truth-distance is good: δ`(q) < 0` means the answer is on the correct side of the boundary. Positive is bad. The magnitude `|`δ`(q)|` is how far the answer sits from the truth boundary, so a large negative value is a confidently correct answer and a value near zero is an answer on the knife's edge. Confidence runs the same direction relative to `c`: above `c` is a lean toward asserting, below `c` is a lean toward declining. Calibration is the claim that these two axes agree in sign, and the whole chapter is about how much they are allowed to disagree near the origin of both.

Now weaken each idealization by a budget and watch the contradiction bend rather than break.

6.2 Two-sided ε-calibration

Strict calibration is a biconditional pinned at a single point `c`. It demands that the instant correctness flips sign, confidence flips side of the threshold, with no tolerance. Real calibration curves do not behave like that near the boundary. Close to δ `= 0` the model is genuinely uncertain, its confidence hovers near `c`, and small perturbations of the question move confidence back and forth across the threshold without any reliable relation to correctness. The honest weakening is to stop demanding anything in that neighborhood and to keep the demand only where the model has real signal.

Definition 6.1 (Two-sided ε-calibration). *Fix a slack* `ε ≥ 0`*. A model* `M` *is* ε-calibrated *at threshold* `c` *if*

- δ`(q) < -`ε `→ conf(q) > c +` ε *for all* `q`*, and*
- δ`(q) >` ε `→ conf(q) < c -` ε *for all* `q`*.*

The verified predicate is `EpsCalibrated M` δ `c` ε*.*

Read the first clause aloud. If an answer is *very* correct, its truth-distance below $-\varepsilon$, then its confidence must be *clearly* above threshold, above $c + \varepsilon$. The second clause is the mirror: very wrong answers get confidence clearly below threshold. The band $\delta(q) \in [-\varepsilon, \varepsilon]$ is exempt. Inside it the model may report any confidence at all. The band $\mathrm{conf}(q) \in [c - \varepsilon, c + \varepsilon]$ is where the model is allowed to be confused.

Two features of this definition earn the word "honest".

First, it is one-directional. Strict calibration is a biconditional; ε-calibration keeps only the direction the argument needs. It says correctness forces confidence, not that confidence guarantees correctness. That is exactly the direction a well-meaning model designer can hope to certify. You can imagine measuring, on a held-out set, that answers you know to be very correct do come with high confidence. You cannot as easily certify the converse, that every high-confidence answer is correct, because that is a claim about the tails you have not seen. Keeping only the certifiable direction is what makes the weakening usable.

Second, the same ε sits on both the truth side and the confidence side. Very correct means truth-distance beyond ε; clearly above means confidence beyond $c + \varepsilon$. Using one parameter for both is a simplification, and Section "Two-slack separation" below shows how to split it into a truth slack and a confidence slack when the two live in different units. For now the single parameter keeps the statements clean.

It helps to see why strict calibration is not merely inconvenient but empirically false, so that the weakening reads as a correction rather than a retreat. A reliability diagram bins predictions by reported confidence and plots, against each bin, the observed frequency of correctness. Perfect strict calibration would be the diagonal of that plot, and it would additionally require the crossing at c to be exact: the confidence that separates correct from incorrect would have to be a single sharp value with correctness flipping the instant confidence crosses it. Real diagrams are never that. They sag or bow away from the diagonal, and near the decision threshold they flatten, because that is exactly the region where the model has the least signal and its confidence carries the least information about correctness. The flat region near c is the empirical face of the exempt band. ε-calibration is the diagram read honestly: it commits to the tails, where the curve is informative, and stays silent in the middle, where it is not. A modeler who has drawn a reliability diagram has already measured an ε; the definition just names it.

There is a subtle trap the one-directional form avoids. If you demanded the full biconditional even in ε form, you would be asserting that every high-confidence answer is correct, which is a claim about the model's worst high-confidence mistakes, the confident hallucinations. Those are precisely the events safety work is trying to bound and cannot yet certify. ε-calibration never asks you to certify them. It asks only the reverse, that answers you have independently verified to be very correct come with high confidence, which you can check directly on a labeled set. The theorem extracts its force from the certifiable direction and leaves the uncertifiable direction alone, which is

why its hypotheses survive contact with a real evaluation.

Estimating ε from data is then a concrete procedure, and stating it makes the whole chapter operational. Take a labeled evaluation set. For each example you know the truth-distance sign and magnitude, at least in bins, and you observe the reported confidence. Find the smallest ε such that every example with truth-distance below $-\varepsilon$ reports confidence above $c + \varepsilon$, and every example with truth-distance above ε reports confidence below $c - \varepsilon$. That smallest ε is the model's calibration budget on that set, the tightest slack the two clauses of Definition 6.1 will tolerate. It is an empirical quantity, computable by scanning the evaluation, and it is exactly the number the bridge consumes. A caveat rides along: this ε is an estimate from a finite sample, so it inherits the usual statistical uncertainty, and a prudent deployment inflates it to a high-confidence upper bound before feeding it to the bridge. The bridge is monotone in ε in the safe direction, a larger ε gives a weaker but still valid conclusion, so over-estimating the budget never invalidates the guarantee; it only widens the margin interval $[-\varepsilon, 0)$ you have to live with.

The two clauses are most useful in their contrapositive form, and the verified file records them as such.

Proposition 6.2 (Contrapositives). *ε-calibration is equivalent to the pair*

- $\mathrm{conf}(q) \le c + \varepsilon \to \delta(q) \ge -\varepsilon$, *and*
- $\mathrm{conf}(q) \ge c - \varepsilon \to \delta(q) \le \varepsilon$.

Proof. Each line is the contrapositive of one clause of Definition 6.1, and contraposition over a linear order is an equivalence: $P \to Q$ is equivalent to $\neg Q \to \neg P$, and the negation of $\mathrm{conf} > c + \varepsilon$ is $\mathrm{conf} \le c + \varepsilon$, the negation of $\delta < -\varepsilon$ is $\delta \ge -\varepsilon$. No arithmetic beyond trichotomy of $<$ on $\mathbb{R}$ is used. ∎

The contrapositive is the form the bridge theorems consume. It says: a question whose confidence has not risen clearly above threshold cannot have been very correct, its truth-distance is at least $-\varepsilon$. That is a lower bound on δ extracted purely from a confidence observation, and it is the entire content of the lower half of the approximate trilemma.

Remark 6.3. At $\varepsilon = 0$ the two clauses become $\delta(q) < 0 \to \mathrm{conf}(q) > c$ and $\delta(q) > 0 \to \mathrm{conf}(q) < c$. These are precisely the two forward implications of strict calibration, the half the intermediate value argument actually uses. The verified `strictCalibrated_to_epsCalibrated_zero_half` shows that `StrictCalibrated` implies `EpsCalibrated M δ (1/2) 0`, so strict calibration is the zero-slack instance of the approximate notion and nothing is lost by working in the weaker vocabulary from the start.

Example 6.4 (Reading an ε off a calibration table). Suppose you bin a model's answers by measured accuracy and reported confidence, with confidence normalized so $c = 0.5$. You find that every answer whose accuracy places it at truth-distance below -0.1 reports confidence above 0.6, and every answer at truth-distance above 0.1 reports confidence below 0.4. Then the model is ε-calibrated with $\varepsilon = 0.1$: the truth band is $[-0.1, 0.1]$ and the

confidence band is `[0.4, 0.6]`. Inside those bands the table may be ragged and the certificate says nothing. Outside them the certificate holds. A model with a tighter table, say the same behavior already forced at truth-distance `0.03` and confidence `0.53`, is ε-calibrated with ε `= 0.03`. Smaller ε is a better model, and the whole point of the next sections is what "better" costs it.

6.3 ε-coverage

Coverage also needs a budget. The strict version asks only that some answer is correct and some is wrong. Near the boundary that is too weak to drive an approximate argument, because a "correct" answer at truth-distance `-0.001` carries essentially no confidence signal under ε-calibration. The approximate argument needs witnesses that sit *outside* the exempt band, where the calibration clauses have teeth.

Definition 6.5 (ε-coverage). *A model `M` is* ε-covering *if there is a question `q_t` with δ`(q_t) < -`ε and a question `q_f` with δ`(q_f) >` ε. The verified predicate is `EpsCovering M` δ ε.*

The subscripts read as "true" and "false": `q_t` is a question the model gets solidly right, `q_f` one it gets solidly wrong, both far enough from the boundary that ε-calibration applies to them. At ε `= 0` this is ordinary coverage, and `trilemmaCovering_to_epsCovering_zero` records that `TrilemmaCovering` implies `EpsCovering M` δ `0`.

The requirement is mild. Any model deployed on a domain broad enough to contain both easy true instances and easy false instances is ε-covering for the ε that separates them from the boundary. The failure mode is a model that only ever operates in its own confused band, and such a model has no claim to being useful in the first place.

Example 6.5b (Coverage sets an upper bound on usable ε). There is a hidden interaction between the two witnesses that constrains how large an ε you may claim. If the easiest true question you can find sits at truth-distance `-0.3` and the easiest false one at `0.3`, then ε-coverage holds for any ε `< 0.3`, and you are free to pick the ε that your calibration actually supports. But if your domain is hard, so that the most solidly-true question you have is only at `-0.08`, then you cannot claim ε-coverage for any ε `≥ 0.08`, and the bridge can only be run with ε `< 0.08`. This matters because coverage and calibration pull the usable ε in opposite directions. Calibration wants ε large enough that the tails are clean; coverage wants witnesses beyond ε. The bridge fires at any ε that satisfies both, and the interval of admissible ε is a compact summary of how much room the model leaves between its confusion band and its clearest examples. A model with no admissible ε is one whose confusion reaches all the way to its best cases, which is a model in trouble for reasons the bridge does not even need to invoke.

6.4 The approximate trilemma

Now the central theorem. It takes the three ingredients above, adds continuity and connectedness from Chapter 4, and returns not a contradiction but a located question with a two-sided bound on its truth-distance.

Theorem 6.6 (Approximate Hallucination Trilemma). *Let `Q` be a connected space, let the confidence map `q ↦ conf(q)` be continuous, and let `ε ≥ 0`. If `M` is `ε`-covering, `ε`-calibrated at `c`, and faithful in the threshold-inclusive sense that `conf(q) ≥ c → δ(q) < 0`, then there is a question `q₀` with*

`conf(q₀) = c` *and* `δ(q₀) ∈ [-ε, 0)`.

This is the verified `approx_trilemma`.

Proof. Start from the two coverage witnesses. Since `M` is `ε`-covering there is `q_t` with `δ(q_t) < -ε` and `q_f` with `δ(q_f) > ε`. Feed each to the matching clause of `ε`-calibration. The first clause turns `δ(q_t) < -ε` into `conf(q_t) > c + ε`, and since `ε ≥ 0` this gives `conf(q_t) > c`. The second clause turns `δ(q_f) > ε` into `conf(q_f) < c - ε`, hence `conf(q_f) < c`. So the continuous confidence map is strictly above `c` at `q_t` and strictly below `c` at `q_f`.

The question space is connected and confidence is continuous, so the image of confidence is an interval containing both `conf(q_f) < c` and `conf(q_t) > c`. That interval contains `c`. The intermediate value theorem, the connected-domain engine of Chapter 4, produces a question `q₀` with `conf(q₀) = c`. In the verified proof this is the `IsPreconnected.intermediate_value₂` step applied on the universal set, using the continuity of the confidence map against the constant map at `c`.

It remains to bound `δ(q₀)` on both sides.

Upper bound, `δ(q₀) < 0`. At `q₀` we have `conf(q₀) = c`, so in particular `conf(q₀) ≥ c`. Faithfulness applies and gives `δ(q₀) < 0` directly. The model is correct at the forced question.

Lower bound, `δ(q₀) ≥ -ε`. Suppose not, so `δ(q₀) < -ε`. Then the first clause of `ε`-calibration fires and gives `conf(q₀) > c + ε`. But `conf(q₀) = c` and `ε ≥ 0`, so `c > c + ε` forces `ε < 0`, contradicting `ε ≥ 0`. Hence `δ(q₀) ≥ -ε`. In the verified proof this is the final `by_contra`/`linarith` pair: assume the negation, apply the calibration clause, and let linear arithmetic close the gap.

Combining, `δ(q₀) ∈ [-ε, 0)`. ∎

The conclusion deserves slow reading. The theorem does not say the model fails. It says there is a question `q₀` where three things hold at once. Confidence sits exactly at threshold, `conf(q₀) = c`. The answer is genuinely correct, `δ(q₀) < 0`. And the answer is correct by a hair, `δ(q₀) ≥ -ε`, no further from the boundary than the calibration slack allows. The forced question is one the model gets right while sitting on the fence about it, and the margin of rightness is capped by `ε`.

This is the honest shadow of the strict theorem. The strict theorem said such a `q₀` cannot exist because `δ(q₀)` would have to be both `< 0` and `≥ 0` at once. The approximate theorem says `q₀` does exist once you grant a slack,

and the price of the slack is visible in the width of the interval $[-\varepsilon, 0)$ that traps its truth-distance.

Remark 6.7 (Why the interval is half-open). The lower end $-\varepsilon$ is attained in principle; the upper end 0 is not. The strict inequality $\delta(q_0) < 0$ comes from faithfulness, which forbids confident correctness from touching the boundary. The weak inequality $\delta(q_0) \geq -\varepsilon$ comes from calibration, which only bounds how far below the boundary a low-confidence question can sit. The asymmetry is not cosmetic. It is what lets $\varepsilon \to 0$ recover a genuine contradiction rather than a degenerate point, as the next section shows.

Example 6.8 (A concrete margin). Take $c = 0.5$ and the $\varepsilon = 0.1$ model of Example 6.4, deployed on a domain with both solidly-true and solidly-false questions, so ε-coverage holds. Theorem 6.6 guarantees a question q_0 where the model reports confidence exactly 0.5 and whose answer is correct with truth-distance somewhere in $[-0.1, 0)$. Concretely, the model could be right at q_0 by a truth-distance of -0.07: correct, but only just, and reporting a coin flip's confidence about it. If you improve the model to $\varepsilon = 0.03$, the same theorem still fires, and now the forced question's truth-distance lies in $[-0.03, 0)$. The margin of correctness at the forced question has shrunk from at most 0.1 to at most 0.03. You did not remove the fence-sitting question. You sharpened how precariously it sits on the fence.

Example 6.8b (Tracing the intermediate value step). It is instructive to watch the proof run on a toy model where the arithmetic is visible. Let the question space be the interval $[0, 1]$, connected, and let confidence be the continuous map $\mathrm{conf}(q) = 1 - q$, so confidence slides from 1 at $q = 0$ down to 0 at $q = 1$. Set $c = 0.5$ and $\varepsilon = 0.1$. Suppose the truth field along the interval is $\delta(q) = q - 0.55$, so the model is correct for $q < 0.55$ and wrong beyond it. Check the hypotheses. Coverage: at $q = 0$ we have $\delta = -0.55 < -0.1$ and $\mathrm{conf} = 1 > 0.6$; at $q = 1$ we have $\delta = 0.45 > 0.1$ and $\mathrm{conf} = 0 < 0.4$. So $q_t = 0$ and $q_f = 1$ are ε-coverage witnesses. Faithfulness: $\mathrm{conf}(q) \geq 0.5$ means $1 - q \geq 0.5$, that is $q \leq 0.5$, where $\delta(q) = q - 0.55 \leq -0.05 < 0$, so the model is correct whenever it is confident. Now the intermediate value step: confidence equals $c = 0.5$ at $q_0 = 0.5$. There $\delta(q_0) = 0.5 - 0.55 = -0.05$, which lands in $[-0.1, 0)$ exactly as Theorem 6.6 promises. The forced question is $q_0 = 0.5$, the model is correct there by a truth-distance of 0.05, and it reports confidence exactly 0.5. Push ε toward 0 by sharpening the model so its confidence tracks truth more tightly, and the forced q_0 migrates toward the true boundary $q = 0.55$, where $\delta = 0$ and faithfulness fails. The toy makes the migration concrete: the fence-sitting question walks toward the boundary as calibration improves, and only reaches it in the impossible limit.

6.5 Recovering the exact contradiction at $\varepsilon = 0$

The claim that the approximate theorem contains the exact one is not a slogan. It is a corollary obtained by substituting $\varepsilon = 0$.

Theorem 6.9 (Exact contradiction from the approximate bridge). *Under the hypotheses of Theorem 6.6 with* `ε = 0`*, that is* `0`*-coverage,* `0`*-calibration, and threshold-inclusive faithfulness on a connected space with continuous confidence, one derives* `False`*. This is the verified* `exact_from_approx`*.*

Proof. Apply Theorem 6.6 with `ε = 0`. It returns `q₀` with `conf(q₀) = c` and `δ(q₀) ∈ [-0, 0)`, that is `0 ≤ δ(q₀) < 0`. No real number is both at least `0` and strictly less than `0`, so linear arithmetic closes to `False`. ■

The half-open interval is exactly what makes this work. At `ε = 0` the trap `[-ε, 0)` collapses to `[0, 0)`, the empty set, and asserting that `δ(q₀)` lands in an empty interval is the contradiction. The lower bound `δ(q₀) ≥ -ε` became `δ(q₀) ≥ 0`, the faithfulness bound `δ(q₀) < 0` stayed, and the two are incompatible. The strict trilemma is not a separate theorem. It is the endpoint of the approximate one where the two bounds cross.

It is worth seeing the collapse as a picture, because it explains why the recovery is not a coincidence of notation. Plot the guaranteed region for `δ(q₀)` as a function of `ε`. For each `ε > 0` it is the half-open segment from `-ε` up to but not including `0`, a triangle in the `(ε, δ)` plane with its hypotenuse along `δ = -ε` and its flat top along `δ = 0` removed. As `ε` decreases the triangle narrows to the single boundary point `δ = 0`, which the removed top excludes. At `ε = 0` the region is the point `δ = 0` intersected with the open condition `δ < 0`, which is empty. The exact contradiction is the vertex of the triangle, the place where its two bounding constraints meet and the open one wins. This is the same geometry as the achievable half-space `γ ≤ ε + 2 δ` of the degradation law collapsing to `γ ≤ 0` at the origin: a full-dimensional region of legal slacks pinched to nothing as the slacks vanish. Impossibility is what a positive-measure achievable region looks like at its boundary.

There is a second route to the same place that starts from the biconditional rather than from `0`-calibration, and the verified library records it because it shows the strict paper hypotheses feeding the approximate machinery unchanged.

Theorem 6.10 (Strict calibration through the bridge). *If* `M` *satisfies strict biconditional calibration, ordinary coverage, and threshold-inclusive faithfulness on a connected space with continuous confidence, then* `False`*. This is the verified* `exact_from_strictCalibrated_via_approx`*.*

Proof. Convert the strict hypotheses into zero-slack approximate ones. Strict calibration at `c = 1/2` implies `EpsCalibrated M δ (1/2) 0` by `strictCalibrated_to_epsCalibrated_zero_half`: the forward direction of each biconditional is one clause of `0`-calibration. Ordinary coverage implies `EpsCovering M δ 0` by `trilemmaCovering_to_epsCovering_zero`. Feed both to Theorem 6.9. ■

Remark 6.11. Theorem 6.10 needs continuity only of the confidence map, not of the answer or the truth field, because the intermediate value step is applied to confidence alone. This is a small economy over some presentations of the strict trilemma, and it is a direct benefit of having factored the argument through the approximate bridge: the bridge exposes exactly which map has to be continuous.

Remark 6.11b (The diagonal is still underneath). It can look as though the approximate bridge has traded the diagonal of Chapter 1 for the intermediate value theorem of Chapter 4, and that the self-reference has quietly left the room. It has not. The forced question q_0 is the same object as the `liar_query` witness a_0 of Chapter 1, wearing analytic clothes. In the exact reflective-verdict picture, a_0 is the index where the system's self-verdict must equal its own negation, an impossibility because negation has no fixed point. In the calibration picture, q_0 is the query where the model's confidence must sit at the threshold that separates its correct verdicts from its incorrect ones, and faithfulness plus calibration try to force the truth-distance to be both negative and nonnegative there. The `no_reflective_verdict` collapse of Chapter 1 and the `exact_from_approx` collapse of this chapter are the same contradiction; the intermediate value theorem is only the tool that locates the diagonal witness when the domain is connected rather than self-indexing. The margin ε is what you get by refusing to demand the contradiction exactly and asking instead how close to it the system is driven.

6.6 The tightening principle

The most consequential reading of Theorem 6.6 for a practitioner is not that a bad question exists. It is how the bad question responds to effort. The instinct behind calibration work is that a better-calibrated model is a safer model, that if you push ε down far enough the obstruction becomes negligible. Theorem 6.6 says the opposite. Pushing ε down does not weaken the obstruction. It concentrates it.

Theorem 6.12 (Tightening principle). *Let M be faithful and ε-covering, and suppose it is both ε_1-calibrated and ε_2-calibrated at c with $0 \le \varepsilon_2 \le \varepsilon_1$. Then the forced question $q_{0\,2}$ obtained from the ε_2 bridge satisfies $\delta(q_{0\,2}) \in [-\varepsilon_2, 0)$, an interval contained in the ε_1 interval $[-\varepsilon_1, 0)$. The margin of correctness at the forced question is bounded by $\varepsilon_2 \le \varepsilon_1$: improving calibration cannot loosen the bound, and any strict improvement $\varepsilon_2 < \varepsilon_1$ strictly tightens it.*

The two ingredients are the containment of the guaranteed intervals and the bound on the margin, and both elaborate here.

```
theorem c6_tightening (ε₁ ε₂ : ℝ) (h : ε₂ ≤ ε₁) :
    Set.Ico (-ε₂) 0 ⊆ Set.Ico (-ε₁) 0 := by
  intro x hx
  exact ⟨by linarith [hx.1], hx.2⟩

theorem c6_margin_bound (ε₂ x : ℝ)
    (hx : x ∈ Set.Ico (-ε₂) 0) :
    |x| ≤ ε₂ := by
  rw [abs_of_neg hx.2]
  linarith [hx.1]
```

The first says improving calibration cannot loosen the guarantee, since the tighter interval sits inside the looser one. The second says the forced question's margin of correctness is bounded by the slack it was obtained from. Together they are the tightening principle: the theorem below supplies the forced question at each slack, and these two lemmas say what shrinking the slack does to it.

Proof. Theorem 6.6 applies at each slack for which `M` is calibrated and covering, since faithfulness does not depend on ε. At slack ε_2 it returns $q_{0,2}$ with $\delta(q_{0,2}) \in [-\varepsilon_2, 0)$. Because $\varepsilon_2 \le \varepsilon_1$, the containment $[-\varepsilon_2, 0) \subseteq [-\varepsilon_1, 0)$ holds by monotonicity of the left endpoint. The upper bound $\delta(q_{0,2}) < 0$ is fixed by faithfulness and does not move with ε. The lower bound $-\varepsilon_2$ is the one that migrates, and it migrates toward 0, so $|\delta(q_{0,2})| \le \varepsilon_2$. Hence the worst-case margin $\sup |\delta|$ over the guaranteed interval equals ε and is monotone nondecreasing in ε. ∎

The proof is short; the content is not. Read what it forbids. It forbids the existence of a model that is arbitrarily well calibrated and also comfortably correct at its threshold-confidence questions. The better calibrated the model, the closer its forced fence-sitting question is pinned to the truth boundary. In the limit of perfect calibration the forced question sits exactly on the boundary and the model is, by faithfulness, unable to be there. That limit is Theorem 6.9.

This is why the section title of the thin draft called it "the single most important consequence for practice". A calibration improvement that would seem to be pure progress is instead a movement along a trade-off. You bought a smaller band of confusion at the cost of a more precisely located point of failure. The failure did not go away. It sharpened.

Remark 6.13 (Calibration and faithfulness are not independent dials). A tempting mental model treats calibration and faithfulness as separate knobs, each tunable toward perfection on its own. Theorem 6.12 refutes the independence. Faithfulness fixes the sign of $\delta(q_0)$ and calibration fixes its magnitude, and the two together pin $\delta(q_0)$ into a shrinking interval that the strict theorem shows must eventually be empty. You cannot hold faithfulness fixed and drive calibration to perfection, because the joint conclusion is a nonempty interval that calibration is trying to empty and faithfulness is refusing to let touch zero. One of the two hypotheses, or coverage, or continuity, has to give. The theorem does not say which. It says the four cannot coexist in the limit, and that in the pre-limit they coexist only inside a budget you are actively spending.

Example 6.14 (Spending the budget). Return to the $\varepsilon = 0.1$ model and imagine a research program that halves ε each quarter: `0.1`, then `0.05`, then `0.025`, then `0.0125`. Theorem 6.12 tracks the guaranteed forced-question margin down the same sequence. After a year the model is superbly calibrated, its confusion band a quarter of what it was, and its forced fence-sitting question is now correct by at most `0.0125` of truth-distance while reporting exactly threshold confidence. The program has not eliminated the question. It has manufactured a question on which the model is right by an eyelash and unsure to the point of a coin flip. Whether that is progress depends entirely on

what happens near that question, which is the subject of the final section.

The trade-off is easier to feel as a table read in prose. At ε = 0.1 the forced question is correct by up to 0.1 and its confidence sits in [0.4, 0.6]. At ε = 0.05 the margin ceiling drops to 0.05 and the confidence band tightens to [0.45, 0.55]. At ε = 0.025 the ceiling is 0.025 and the band is [0.475, 0.525]. At ε = 0.0125 the ceiling is 0.0125 and the band is [0.4875, 0.5125]. Two things move together down this sequence. The confidence band shrinks, which is the reported gain, the model is confused over a smaller region. The margin ceiling shrinks in lockstep, which is the hidden cost, the forced question is squeezed against the boundary. A safety reviewer who reads only the first column congratulates the team; a reviewer who reads both sees that the team has been trading breadth of confusion for depth of precariousness, and that the product of the two, roughly the area of the danger region in the confidence-by-truth plane, is not obviously improving at all. The bridge does not tell you the product is constant. It tells you the two factors are coupled and that you cannot drive either to zero without the other, or one of the standing hypotheses, giving way.

Remark 6.14b (Where the naive optimization goes). Suppose a team ignores the coupling and optimizes calibration alone, driving ε down as far as compute allows. The bridge predicts the failure they will eventually hit. As ε shrinks, the forced question's truth-distance is pinned into [-ε, 0), so the model is being asked to be correct within an ever-thinner shell of the boundary while reporting threshold confidence. At some ε the shell is thinner than the resolution of the training signal, and further calibration cannot be achieved without either sacrificing faithfulness, letting the confident answer drift to the wrong side, or sacrificing coverage, refusing to answer near the boundary at all. Both sacrifices are visible failures with names in the applied literature, overconfident hallucination and excessive abstention. The bridge says they are not independent pathologies to be separately patched; they are the two doors the obstruction can walk out of once the calibration door is shut.

6.7 Localization to the transition band

Theorem 6.6 asserts existence, and a fair worry is that existence is toothless if the forced question could be anywhere. A theorem that says "somewhere in your enormous input space there is a bad point" is hard to act on. The approximate bridge does better than bare existence, because the location of q_0 is pinned by where confidence crosses the threshold.

Consider a model that is clearly right in-distribution and clearly wrong far out of distribution. In-distribution, truth-distance is solidly negative and, by ε-calibration, confidence is solidly above c + ε. Far out of distribution, truth-distance is solidly positive and confidence is solidly below c - ε. The forced question q_0 has conf(q_0) = c exactly. It cannot be in the in-distribution region, where confidence exceeds c + ε > c. It cannot be far out of distribu-

tion, where confidence is below `c - ε < c`. It has to sit in the transition band between the two, the region where confidence descends through the threshold as the input drifts from familiar to unfamiliar.

Proposition 6.15 (Localization). *Under the hypotheses of Theorem 6.6, every forced question q_0 with `conf(q₀) = c` lies outside both the high-confidence region `{q : conf(q) > c + ε}` and the low-confidence region `{q : conf(q) < c - ε}`. It lies in the transition band `{q : c - ε ≤ conf(q) ≤ c + ε}`.*

```
theorem c6_localization (c ε : ℝ) (hε : 0 ≤ ε) :
    c ∈ Set.Icc (c - ε) (c + ε) ∧ ¬ (c > c + ε) ∧ ¬
    (c < c - ε) :=
  ⟨⟨by linarith, by linarith⟩, by linarith, by linarith⟩
```

Proof. Three applications of `linarith` from `0 ≤ ε`. ■

The companion `no_boundary_in_upper_band` proves a related but distinct statement, that no question can be both at or above threshold and exactly on the truth boundary, so it is not cited here as a counterpart to this one.

Proof. Immediate from `conf(q₀) = c` and `ε ≥ 0`: the value `c` satisfies both `c ≤ c + ε` and `c ≥ c - ε`, so `q₀` is in the closed confidence band and in neither open region flanking it. ■

The proposition is arithmetically trivial and practically not. It says the theorem does not predict trouble uniformly across the input space. It predicts trouble exactly at the seam where in-distribution behavior gives way to out-of-distribution behavior. That is the region practitioners already watch, because it is where a model's competence is changing fastest. The approximate bridge gives a reason the seam is special that is independent of any particular dataset: it is the only place a forced threshold-confidence question can live.

The localization also explains why the transition band cannot be made empty by better engineering. Suppose you tried to eliminate the band by making confidence jump discontinuously from above `c + ε` to below `c - ε`, with no intermediate values. Then confidence would not be continuous, and Theorem 6.6's hypotheses would fail; but a discontinuous confidence map is a different pathology, and Chapter 4 already argued that real models, built from continuous components, do not have it on a connected input space. On a connected domain a continuous confidence map that is high somewhere and low somewhere else must pass through every value between, and `c` is between. The band is where the passage happens.

The localization has a practical corollary for evaluation design. If the forced question can only live in the transition band, then an evaluation set that samples uniformly from the input space will hit the forced question only in proportion to how much of the space the band occupies, which for a sharp model is very little. Uniform evaluation therefore systematically under-tests the exact region the theorem points to. The remedy is to sample by confidence: deliberately gather questions whose reported confidence is near `c` and stress-test those. This is not a new idea in practice, active learning and

uncertainty sampling already target the same region, but the bridge gives it a theorem-shaped justification. The band near `conf = c` is not merely where the model is unsure; it is the only place a forced correct-but-barely question can be, so it is where the irreducible residue of the impossibility concentrates. An evaluation that ignores it is blind to precisely the failures the mathematics guarantees.

One more consequence is worth stating because it defuses a common objection. A reader might grant the theorem and still say the transition band is so small that it does not matter. The bridge's own tightening principle answers this. Making the band small, small ε, does not make the forced question harmless; it makes the forced question more precisely correct-by-a-hair and no less forced. Smallness of the band is a statement about the model's sharpness, not about the severity of what lives in the band. To learn whether the small band is dangerous you need the geometry of the final section, which measures how much traffic and how much adversarial pressure the band actually receives.

6.8 The discrete band constraint

The intermediate value theorem needs a connected domain. Real input spaces are often not connected in any useful sense; token sequences are discrete, and the question space may be a finite corpus. Chapter 4 argued that connectedness is frequently available after the right embedding, but it is worth having a bridge that assumes no topology at all. The discrete version drops the existence half of the argument and keeps the arithmetic half.

Theorem 6.16 (Discrete band constraint). *On any sets `Q` and `A`, with no topology and no finiteness assumption, suppose `M` is ε-calibrated at `c` and threshold-inclusive faithful. Let `q` be any question whose confidence lands in the upper confidence band, `c ≤ conf(q) ≤ c + ε`. Then `δ(q) ∈ [-ε, 0)`. This is the verified `discrete_approx_bridge`.*

Proof. Two bounds, each from one hypothesis.

Upper bound, `δ(q) < 0`. Since `conf(q) ≥ c`, faithfulness gives `δ(q) < 0`.

Lower bound, `δ(q) ≥ -ε`. Suppose `δ(q) < -ε`. Then the first clause of ε-calibration gives `conf(q) > c + ε`, contradicting the assumption `conf(q) ≤ c + ε`. Hence `δ(q) ≥ -ε`. ■

The difference from Theorem 6.6 is precisely the intermediate value step. There, connectedness and continuity *produced* a question with `conf = c`. Here, no such question is guaranteed to exist; instead, *if* one exists whose confidence sits in the upper band `[c, c + ε]`, the same arithmetic pins its truth-distance to `[-ε, 0)`. On a connected continuous space the two theorems agree, because the intermediate value theorem supplies the band question for free. On a discrete space the constraint is conditional on there being a question in the band, which is a statement you check against the model rather than derive.

This conditional character is a feature, not a weakness, once you see what it buys. The continuous bridge proves an existence claim you then have to

trust the topology for; the discrete bridge proves an implication you can check against a concrete corpus. If your evaluation set contains any question whose reported confidence lands in `[c, c + ε]`, and most nontrivial evaluation sets do, then Theorem 6.16 applies to that question directly, with no appeal to connectedness or continuity, and tells you its truth-distance is trapped in `[-ε, 0)`. You can go find the question, look at it, and confirm the model is correct-but-barely exactly where the theorem says. The discrete bridge is the version you can audit by hand. It also degrades gracefully to the fully finite setting: on a finite question space every hypothesis is a finite conjunction of inequalities, decidable in principle, which is why the core-Lean fragment below can carry the whole logical content over the integers with no analysis at all.

The arithmetic core of the discrete bridge is finite and needs no reals to convey. Here it is over the integers, scaled so that truth-distances and confidences are whole numbers, elaborated when the book is built:

```
theorem c6_band (dq confq c eps : Int)
    -- ε-calibration, lower clause
    (hcal : dq < -eps → confq > c + eps)
    -- c-faithfulness
    (hfaith : confq ≥ c → dq < 0)
    -- upper half of the band
    (hge : confq ≥ c)
    -- not above the band
    (hle : confq ≤ c + eps) :
    -eps ≤ dq ∧ dq < 0 := by
  refine ⟨?_, hfaith hge⟩
  rcases Int.lt_or_le dq (-eps) with h | h
  · have := hcal h; omega
  · exact h
```

The proof is two cases on whether `dq` is below `-eps`. If it is, the calibration implication and `omega` derive a contradiction with the band assumption; if it is not, the goal is the case hypothesis. The real-valued `discrete_approx_bridge` does the same work with `linarith` over $\mathbb{R}$; the logical skeleton is identical.

An immediate consequence is that no question in the upper band sits exactly on the truth boundary.

Corollary 6.17 (No boundary point in the upper band). *Under ε-calibration and threshold-inclusive faithfulness, no question `q` with `c ≤ conf(q) ≤ c + ε` has `δ(q) = 0`. This is the verified `no_boundary_in_upper_band`.*

Proof. Theorem 6.16 gives `δ(q) < 0`, which excludes `δ(q) = 0`. ∎

The zero-slack collapse is again a one-liner. At `ε = 0` the upper band is the single value `conf(q) = c`, and Theorem 6.16 would force `δ(q) ∈ [0, 0)`, empty. So on a discrete space, `0`-calibration and faithfulness forbid any question from reporting exactly threshold confidence while being correct. The finite core:

```
theorem c6_exact (dq confq c : Int)
    (hcal : dq < 0 → confq > c)
    (hfaith : confq ≥ c → dq < 0)
    (hge : confq ≥ c)
    (hle : confq ≤ c) : False := by
  have h1 : dq < 0 := hfaith hge
  have h2 : confq > c := hcal h1
  omega
```

Faithfulness makes the answer correct, `dq < 0`; the zero-slack calibration clause then forces confidence strictly above `c`; but the band caps it at `c`. Three lines and `omega` finishes. This is the same shape as `liar_query` from Chapter 1, where the diagonal question satisfied `f a₀ a₀ = !(f a₀ a₀)`: an object forced to be on both sides of a line at once. The discrete calibration constraint is that paradox rendered in inequalities instead of booleans.

6.9 Two-slack separation: truth units and confidence units

Definition 6.1 used a single ε for both the truth side and the confidence side. That is convenient and slightly dishonest, because truth-distance and confidence are measured in different units and can be rescaled independently. A model whose confidence is reported on a `0` to `100` scale rather than `0` to `1` has its confidence slack multiplied by a hundred without anything about its truthfulness changing. A clean formulation separates the two.

Definition 6.18 (Two-slack separation). *Fix a truth slack ε and a confidence margin γ. A model is* two-slack separating *at `c` if*

- `δ(q) < -ε → conf(q) > c + γ`, *and*
- `δ(q) > ε → conf(q) < c - γ`.

The verified predicate is `TwoSlackSeparating M δ c ε γ`, and `EpsCalibrated` is the special case `γ = ε`, recorded as `epsCalibrated_iff_twoSlack`.

The truth slack ε lives in the units of the truth field; the confidence margin γ lives in the units of confidence. The separation makes the invariances visible. Rescale confidence and only γ changes. Rescale the truth field and only ε changes. The conclusions of the theorems should, and do, land their bounds in the correct units.

Theorem 6.19 (Two-slack coupling). *On a connected space with continuous confidence, if `M` is ε-covering, two-slack separating with margin `γ ≥ 0`, and threshold-inclusive faithful, then there is `q₀` with `conf(q₀) = c` and `δ(q₀) ∈ [-ε, 0)`. This is the verified `two_slack_approx_coupling`.*

Proof. Identical in shape to Theorem 6.6, with γ doing the work ε did on the confidence side. Coverage gives `q_t` with `δ(q_t) < -ε` and `q_f` with `δ(q_f) > ε`. Two-slack separation gives `conf(q_t) > c + γ ≥ c` and `conf(q_f) < c`

- γ ≤ c, using γ ≥ 0. The intermediate value theorem produces q_0 with conf(q_0) = c. Faithfulness gives $\delta(q_0) < 0$. And if $\delta(q_0) < -\varepsilon$, the first separation clause gives conf(q_0) > c + γ ≥ c, contradicting conf(q_0) = c; so $\delta(q_0) \geq -\varepsilon$. The conclusion's interval $[-\varepsilon, 0)$ is in truth units alone. The confidence margin γ appears only in the intermediate value step and drops out of the final bound. ■

The final sentence is the reason the separation is worth stating. Whatever confidence margin the model happens to exhibit, however you scale the confidence axis, the located truth-distance interval is $[-\varepsilon, 0)$, purely in truth units. The confidence margin powers the crossing but does not survive into the conclusion.

Example 6.19b (Two slacks with numbers). Report confidence on a 0 to 100 scale with threshold c = 50, and measure truth-distance on a 0 to 1 semantic scale. Suppose clearly-true questions, truth-distance below -0.1, always score above 70, and clearly-false ones, above 0.1, always score below 30. Then the model is two-slack separating with truth slack $\varepsilon = 0.1$ and confidence margin γ = 20. The two numbers live in different worlds: 0.1 is a semantic distance, 20 is a fraction of the confidence scale. Theorem 6.19 fires and returns a forced question whose confidence is exactly 50 and whose truth-distance is in [-0.1, 0), in semantic units, with the 20 nowhere in the answer. Rescale confidence to 0 to 1 and the margin becomes γ = 0.2 while the conclusion interval [-0.1, 0) is untouched. If you had used the single-parameter Definition 6.1 you would have been forced to reconcile a truth slack of 0.1 with a confidence slack of 20 as though they were the same number, which they are not. The two-slack form is what keeps the units honest, and Theorem 6.20 then tells you that of the two, it is the truth slack 0.1 that provably cannot be pushed to zero.

The separation also isolates which slack must be positive for a real model to exist at all.

Theorem 6.20 (The truth slack must be positive). *On a connected space with continuous confidence, if M is two-slack separating with any margin γ ≥ 0, ε-covering with $\varepsilon \geq 0$, and threshold-inclusive faithful, then $\varepsilon > 0$. This is the verified* `truth_slack_must_be_positive`.

Proof. Suppose $\varepsilon = 0$. Then Theorem 6.19 applies with $\varepsilon = 0$ and returns q_0 with $\delta(q_0) \in [-0, 0) = [0, 0)$, which is empty, a contradiction. So $\varepsilon > 0$. ■

This is the sharpest way to say what the strict theorem forbids. It is not the confidence margin that has to be sacrificed; a model can have as clean a confidence separation as you like. It is the *truth* slack that cannot be zero. Any faithful, covering model on a connected space with a continuous confidence map must leave a genuinely positive band of truth-distance uncertified around its boundary. The band can be narrow. It cannot be a point. Zero truth slack is the exact contradiction of the strict trilemma, now attributed to the correct parameter.

The discrete two-slack version mirrors Theorem 6.16 with the band measured in confidence units.

Theorem 6.21 (Two-slack band bridge). *On any sets, under two-slack separation and threshold-inclusive faithfulness, any question with* `c ≤ conf(q) ≤ c + γ` *has* `δ(q) ∈ [-ε, 0)`*. The confidence band is in confidence units, the conclusion band in truth units. This is the verified* `two_slack_band_bridge`*, and its boundary-free corollary is* `no_boundary_in_upper_band_two_slack`*.*

Proof. As in Theorem 6.16, replacing `c + ε` by `c + γ` in the calibration step. Faithfulness gives `δ(q) < 0` from `conf(q) ≥ c`; if `δ(q) < -ε` then separation gives `conf(q) > c + γ`, contradicting `conf(q) ≤ c + γ`. ■

Remark 6.22 (Axioms). The verified two-slack results carry an explicit axiom audit: `two_slack_approx_coupling`, `truth_slack_must_be_positive`, `two_slack_band_bridge`, and `no_boundary_in_upper_band_two_slack` are each printed with `#print axioms`. They rest on the classical and quotient axioms Mathlib's real analysis uses, and on nothing model-specific. The discrete bridges, being pure order arithmetic, are the lightest; the continuous ones inherit the intermediate value theorem's dependencies. The core-Lean `c6_` fragments above use no axioms beyond the kernel, since they are decidable integer statements.

6.10 The quantitative degradation law

The calibration bridges relax the confidence-versus-truth idealization. A different idealization is universality itself: the assumption from Chapter 1 that the system can name every behavior exactly. Real systems approximate behaviors, they do not reproduce them. The Foundation library's `F_09_QuantitativeDegradation` relaxes universality to an ε and runs the whole diagonal through a metric, ending in a single clean inequality. It is the same phenomenon as the calibration bridge, seen through the Mirror Trilemma corner rather than the Hallucination corner.

The setting is a system `M : A → A → Y` valued in a pseudometric space `Y`, with a distance `dist`. Three parameters relax three idealizations. The choice of a pseudometric rather than a metric is deliberate: a pseudometric allows distinct behaviors to sit at distance zero, which models a system that cannot distinguish certain outputs, and the theorems below never need the separation axiom that would promote it to a metric. Only at `ε = 0`, when we want approximate universality to mean exact surjectivity, does the metric axiom `dist x y = 0 → x = y` get used, and there it is invoked explicitly.

The three parameters, in the notation of `F_09_QuantitativeDegradation`, are ε for universality slack, δ for transparency slack, and γ for the controller's demanded margin. Note that this δ is a distance budget in `Y`, not the truth-distance field of the calibration sections; the two developments reuse the letter for different objects, and the context disambiguates. In this section δ always means the transparency slack `dist (M a a) (introspect a) ≤ δ`.

Definition 6.23 (ε-approximate universality). *`M` is ε-approximately universal if for every target behavior `g : A → Y` there is a prompt `a` with `dist (M a b) (g b) ≤ ε` for all queries `b`. The verified predicate is* `ApproxUniversal M ε`*.*

At ε = 0 in a genuine metric space this is ordinary curried surjectivity, since dist x y = 0 forces x = y. For ε > 0 it is the honest claim: the system can imitate any behavior to within ε, not reproduce it.

The Lawvere diagonal survives the relaxation. Instead of an exact self-fixed-point it yields an approximate one.

Theorem 6.24 (Approximate Lawvere). *If M is ε-approximately universal then every endomap t : Y → Y has an ε-approximate self-fixed-point: some a with dist (M a a) (t (M a a)) ≤ ε. This is the verified approx_lawvere.*

Proof. Apply ε-universality to the diagonal target g b = t (M b b). It returns a prompt a with dist (M a b) (t (M b b)) ≤ ε for all b. Instantiate at b = a to get dist (M a a) (t (M a a)) ≤ ε. ■

This is exactly lawvere from Chapter 1 with the equation M a a = t (M a a) loosened to a distance bound. The diagonal move is unchanged; only the conclusion carries a budget. Set ε = 0 and, in a metric space, the distance bound becomes the exact fixed point of Theorem 1.4.

Now add two more slacks. A system rarely exposes its raw diagonal M a a; it reports a declared self-image introspect a. Transparency is the claim that the declaration matches the diagonal, and it too is approximate.

Theorem 6.25 (Quantitative degradation). *Let M be ε-approximately universal, let introspect : A → Y satisfy dist (M a a) (introspect a) ≤ δ for all a, and let t : Y → Y be 1-Lipschitz. Then there is a prompt a with*

dist (introspect a) (t (introspect a)) ≤ ε + 2 δ.

This is the verified quantitative_trilemma.

Proof. A triangle argument on top of Theorem 6.24. Take the approximate fixed-point a from Theorem 6.24, so dist (M a a) (t (M a a)) ≤ ε. Transparency gives dist (M a a) (introspect a) ≤ δ, and 1-Lipschitz t pulls this forward to dist (t (M a a)) (t (introspect a)) ≤ δ. Now walk the path

introspect a → M a a → t (M a a) → t (introspect a).

The three legs are bounded by δ, ε, and δ respectively. Two applications of the triangle inequality, plus dist_comm to orient the first leg, sum them: dist (introspect a) (t (introspect a)) ≤ δ + ε + δ = ε + 2 δ. ■

The finite skeleton of the triangle bound, over the integers, is standalone:

```
theorem c6_triangle_bound
    (goal d1 d2 d3 eps del : Int)
    (htri : goal ≤ d1 + d2 + d3)
    (hd1 : d1 ≤ del) (hd2 : d2 ≤ eps) (hd3 : d3 ≤ del) :
    goal ≤ eps + 2 * del := by omega
```

Here d1, d2, d3 are the three leg distances and htri is the summed triangle inequality; omega does the bookkeeping del + eps + del = eps + 2*del. The real proof carries the same three legs through dist_triangle and linarith.

The inequality `dist (introspect a) (t (introspect a)) ≤ ε + 2 δ` is the whole law. Rephrase it as a constraint on a controller. A controller demands that the system's self-image be robustly moved by `t`, that `introspect a` and `t (introspect a)` differ by more than some margin `γ` for every `a`. Theorem 6.25 says such a demand cannot be met beyond the budget `ε + 2 δ`.

Theorem 6.26 (Achievable region). *If additionally `dist (introspect a) (t (introspect a)) > γ` for all `a`, then `γ < ε + 2 δ`. The achievable triples `(ε, δ, γ)` lie in the half-space `{γ ≤ ε + 2 δ}`. This is the verified `achievable_region_bound`, with the contradiction form `quantitative_impossibility` when `γ ≥ ε + 2 δ`.*

Proof. Theorem 6.25 gives a specific `a` with `dist (introspect a) (t (introspect a)) ≤ ε + 2 δ`. The controller's demand at that same `a` gives `γ < dist (introspect a) (t (introspect a))`. Chain the two: `γ < ε + 2 δ`. If instead `γ ≥ ε + 2 δ` were assumed, the chain yields `ε + 2 δ ≤ γ < ε + 2 δ`, a contradiction. ■

The half-space `γ ≤ ε + 2 δ` is the exact analogue of the interval `[-ε, 0)` from the calibration bridge. Both are the region a real system is allowed to occupy. Both collapse at the origin of their slacks.

Theorem 6.27 (Strict limit). *At `ε = 0` and `δ = 0`, any nontrivial controller `γ > 0` gives `False`; equivalently `γ ≤ 0`. This is the verified `strict_limit_recovers_trilemma`.*

Proof. Substitute `ε = δ = 0` into Theorem 6.26 to get `γ < 0`, contradicting `γ > 0`. ■

The collapse over the integers, showing the controller margin forced negative:

```
theorem c6_strict_collapse
    (goal margin : Int)
    (hbound : goal ≤ 0 + 2 * 0)
    (hmargin : goal > margin) :
    margin < 0 := by omega
```

Perfect universality and perfect transparency force the achievable controller margin below zero, so any positive demand is impossible. That is the strict Mirror Trilemma, recovered as the corner of a quantitative law, exactly as the strict Hallucination Trilemma was recovered as the corner of the calibration bridge.

Remark 6.27b (What a K-Lipschitz controller costs). The 1-Lipschitz hypothesis on `t` is the boundary case, and it is worth knowing which way the bound moves when it is relaxed. If `t` is `K`-Lipschitz instead, the middle leg of the triangle, `dist (t (M a a)) (t (introspect a))`, is bounded by `K δ` rather than `δ`, because the Lipschitz constant multiplies the pulled-back transparency gap. The budget becomes `ε + (1 + K) δ`, which at `K = 1` recovers `ε + 2 δ`. A controller whose transform amplifies differences, `K > 1`, has a larger achievable region and so is easier to satisfy; a contracting transform, `K < 1`, has a smaller one and is harder to satisfy, which matches intuition, since

a contraction pulls self-images toward their `t`-images and leaves less room for a controller to demand separation. The `1`-Lipschitz case is the natural reference because it is exactly the class of transforms that neither create nor destroy distance, the metric analogue of the fixed-point-free flips that drove the exact theorems of Chapter 1.

Remark 6.28 (Two shapes of the same fact). The calibration bridge and the degradation law relax different idealizations and produce differently-shaped budgets, an interval on one side and a half-space on the other, but the mechanism is one. Both take the exact diagonal, insert a slack at each idealized equality, and carry the slacks through the argument by linear arithmetic to a conclusion that is nonempty for positive slack and contradictory at zero. The calibration bridge inserts its slack at the calibration biconditional and the coverage witnesses; the degradation law inserts its slack at universality and transparency. The `2 δ` in `ε + 2 δ` is the transparency slack counted twice, once going into the diagonal and once coming out under the Lipschitz map, which is the metric analogue of the two-sided band `[-ε, 0)` being bounded from two independent hypotheses.

6.11 From existence to measure

Theorem 6.6 and its relatives assert that a forced question exists and bound how correct it is. That is an existence-plus-margin statement, and by itself it is not yet a number a deployment can act on. Knowing that some fence-sitting question exists does not tell you whether you will ever be asked it, or what happens to nearby questions, or how hard an adversary has to work to find it. Those are the questions Chapter 5's geometry answers, and the composition of the two chapters is where the notes come closest to an actionable risk estimate.

The bridge and the geometry supply complementary halves of a risk figure. The bridge supplies the *margin*: at the forced question the model is correct by at most ε of truth-distance, and by localization the question lives in the transition band where confidence crosses `c`. The geometry supplies the *frequency* and the *cost*: how large the region around the forced question is, how often a walk through input space enters it, and what an adversary pays to steer into it.

Concretely, three geometric quantities from Chapter 5 combine with the bridge.

The *basin volume* near the boundary, from `MoF_Cost_02_BasinVolume`, measures how much of the input space lies within a given truth-distance of the boundary. The bridge says the dangerous questions are those within ε of the boundary on the correct side; the basin volume converts that ε into a fraction of inputs. A small calibration slack ε and a thin basin together mean few inputs are at risk; a small ε but a fat basin means the model is precariously balanced across a large region.

The *hitting time*, from `MoF_Cost_03_HittingTime`, measures how long a random or gradient-driven walk takes to reach the boundary band. The bridge guarantees the band is nonempty and localized; the hitting time says how quickly ordinary drift or deliberate search arrives there. A long hitting time means the forced question is reachable only rarely; a short one means it is around the corner.

The *concentration* of measure near the boundary, from `MoF_Cost_04_Concentration`, measures how much probability mass an input distribution places in the band. The bridge and the localization put the danger in the transition band; concentration says how much of your actual traffic lands there.

The reason these three cannot be read off the bridge alone is that the bridge is blind to geometry by construction. It is a diagonal argument, and Remark 1.18 of Chapter 1 already warned that the diagonal is silent on everything quantitative: not how many forced questions there are, not how hard one is to find, not what it costs to reach. The bridge adds a margin to that silence, the ε that bounds the forced question's truth-distance, but it says nothing about measure or reachability. Those are analytic and measure-theoretic facts about the confidence field's sublevel sets, and they are the content of Chapter 5. The division of labor is clean. The bridge certifies that a forced question exists, is correct by at most ε, and lives in the transition band. The geometry certifies how big the band is, how often you enter it, and what it costs an adversary to put you there. Neither half is a substitute for the other, and a risk estimate needs both.

Put the three together with the margin and you get a schematic risk estimate. Loosely, the probability that a randomly drawn deployment query is a forced fence-sitting question scales with the measure concentrated in the transition band, and the severity of each such event is bounded by the margin ε. The unified cost statement `MoF_Cost_10_UnifiedTheory` is where Chapter 5 assembles the volume, hitting-time, and concentration bounds into a single expression; feeding the bridge's ε and its localization into that expression yields the composite figure.

Write the schematic estimate out so its parts are visible. Let `P_band` be the probability mass the deployment distribution places in the transition band `conf ∈ [c - ε, c + ε]`, a concentration quantity from Chapter 5. Let `S` be the severity of a forced-question event, which the bridge bounds by the margin, so `S ≤ ε` on the correct side. Then the expected per-query risk contributed by forced questions is at most `P_band · S ≤ P_band · ε`. The two factors respond to different levers. Calibration work drives ε down, shrinking `S` by the tightening principle, but it also reshapes the confidence field and so moves `P_band`, and the sign of that movement is a geometric fact the bridge does not determine. This is the honest form of the trade-off: you can drive one factor down, and whether the product follows depends on what the other factor does, which is measurable only through the geometry. A deployment that reports ε alone, or `P_band` alone, is reporting one factor of a product and calling it a risk. The composite is the product, and it is the smallest object that deserves the name.

Example 6.29 (A risk estimate with numbers). Take the ε = 0.05 model, so any forced question is correct by at most 0.05 of truth-distance and sits in the transition band conf ∈ [0.45, 0.55]. Suppose Chapter 5's geometry, applied to the model's confidence field, reports that the transition band has basin volume 2% of the input space and that the deployment's query distribution concentrates 0.5% of its mass there. Then roughly one query in two hundred lands in the band, and when it does the model is either correct by a margin no larger than 0.05 or, outside the guaranteed interval, wrong. The bridge does not tell you the split between those outcomes; it tells you the correct-side ones are correct by an eyelash and the band is where all the action is. The hitting-time bound then tells you whether an adversary can drive a chosen input into the band cheaply. If the band is 0.05 wide in truth-distance and the model is L-Lipschitz, reaching it from a confidently-correct input costs on the order of the boundary distance divided by L, which Chapter 5 makes precise. The composite is a number: a per-query probability of landing in the forced band, a bounded severity per event, and an adversarial cost to force one deliberately.

Remark 6.30 (What the composite can and cannot claim). The composite risk estimate is an upper bound on a very specific quantity, the incidence and severity of forced threshold-confidence questions. It is not a claim about all model errors. Errors far from the transition band are outside its scope; those are ordinary mistakes the bridge says nothing about. What the composite captures is the irreducible residue: the errors that exist not because the model is undertrained but because faithfulness, calibration, coverage, and continuity cannot all be perfect at once. Tightening ε shrinks the severity per event, by Theorem 6.12, while possibly moving the geometry; a full accounting tracks both. The estimate is honest precisely because it separates the part forced by the diagonal, which no training removes, from the part that is contingent geometry, which engineering can move.

Example 6.29b (The adversarial reading). The frequency factor has a benign reading and an adversarial one, and the bridge supports both. Benignly, a random deployment query lands in the transition band with probability given by the concentration figure, and the composite estimates ordinary incidence. Adversarially, an attacker does not wait for random traffic; they steer. Chapter 5's hitting-time and gradient-attack bounds say how cheaply an input can be nudged from a confidently-handled region into the band. Here the bridge's localization is what makes the attack well-posed: the attacker has a target, the set conf ≈ c, and the bridge guarantees that target is nonempty and contains a correct-but-barely question the model will fumble. The margin ε bounds how close to the truth boundary the attacker's found question sits, which in turn bounds how small a perturbation is needed to tip it across into a confident error. A tighter ε, paradoxically, gives the attacker a question already closer to the edge. The tightening principle and the attack geometry point the same way: sharper calibration produces a forced question that is cheaper to weaponize, unless the geometry around it is correspondingly hardened.

Remark 6.31 (The whole bridge in one sentence). Strip away the parame-

ters and the chapter says this: the exact impossibility is the zero-slack corner of a quantitative law, the law's conclusion is a nonempty margin rather than a contradiction, the margin tightens as the model improves rather than vanishing, the margin is localized to the transition band rather than spread everywhere, and Chapter 5's geometry turns the located margin into a rate. That is the bridge from the idealized theorem to the model on the bench.

6.12 The reading is where the work is

Chapter 1 made a claim that carries directly into this chapter: the engine is one line, and all the effort lives in the reading, in arguing that a real situation really does instantiate the hypotheses. The approximate bridge inherits that structure. The theorems of this chapter are short. Whether they apply to a given system is a modeling question, and the same three readings that Chapter 3 gave the exact trilemma give the approximate one, now with a margin attached.

Reading 6.32 (Hallucination, approximate). The model is a question-answerer, δ is distance from factual truth, and confidence is the model's own reported certainty. ε-calibration is the reliability-diagram fact that clearly-true answers draw high confidence and clearly-false ones draw low confidence, with a confused band in the middle. The forced question is a factual query the model answers at threshold confidence and gets right by no more than ε. As the model's calibration improves, this query is pinned closer to the factual boundary, the region of claims that are almost-but-not-quite supported. That region is where confident hallucination is most dangerous, and the bridge says a sharply calibrated model cannot vacate it.

Reading 6.33 (Prompt injection defense, approximate). Now the "question" is an input to a defended system, δ measures how far the input is from being an attack, and "confidence" is the defense's score for treating the input as safe. Faithfulness is the defense's guarantee that it does not wave through clear attacks. Coverage is the existence of clearly-safe and clearly-malicious inputs. ε-calibration is the defense's reliability: clearly-safe inputs score clearly safe, clearly-malicious ones score clearly unsafe. The bridge forces an input at the defense's threshold that is safe by at most ε, sitting in the band where benign and adversarial inputs are hardest to tell apart. A better-calibrated defense produces a forced input closer to the true attack boundary, which is exactly the input an adversary wants to find. The exact version of this is the defense trilemma of Chapter 3; the approximate version is why no threshold-tuning of a scalar defense score removes the ambiguous band, it only narrows and sharpens it.

Reading 6.34 (Truth probes, approximate). Let the "question" index internal states of a model and let δ measure whether a probed proposition is true, with "confidence" the probe's reported readout. ε-calibration is the probe's accuracy on clear cases. The bridge forces a state at the probe's threshold whose proposition is true by at most ε, a claim the probe cannot cleanly place

on either side. This is the truth-boundary coupling of Chapter 3 in quantitative form: a probe that is accurate on clear cases must have a threshold state it cannot resolve, and sharpening the probe pins that state to the semantic boundary rather than removing it. In all three readings the mathematics is identical, `approx_trilemma` applied to different `M`, δ, and `c`. What differs is only what the forced question means, and in each case it means the same thing the exact `liar_query` of Chapter 1 meant, an object the system is forced to place on both sides of its own line, now softened to a margin of width ε.

6.13 Historical and bibliographic notes

The move from an exact diagonal to an approximate one is old in spirit. Metric fixed-point theory, from Banach onward, is entirely about approximate and exact fixed points coexisting under contraction hypotheses, and the 1-Lipschitz condition on `t` in Theorem 6.25 is the boundary case of that theory. The specific combination used here, an approximate Lawvere diagonal feeding a triangle inequality, follows the CCH development referenced in the Foundation file's header; the calibration bridge's use of the intermediate value theorem to force a threshold crossing is the analytic engine introduced in Chapter 4 and refined by Chapter 5's geometry.

The idea that calibration and correctness are in tension, rather than jointly optimizable, appears informally throughout the empirical calibration literature, usually as the observation that sharpening a model's confidence near its decision boundary trades against its accuracy there. Theorem 6.12 is a proof of that tension from the trilemma hypotheses, and it locates the tension at the exact point the intermediate value theorem forces. The two-slack separation, keeping truth units and confidence units distinct, is a piece of unit hygiene that matters once one tries to read ε off a real calibration table, since the confidence axis carries an arbitrary scale.

The practice of recovering a strict theorem as the zero-slack corner of a quantitative one is a recurring pattern in this book and elsewhere, and it is worth naming as a method rather than a trick. You take a proof that ends in a contradiction from an equality, you find the equality and replace it with an inequality carrying a parameter, and you propagate the parameter through the same proof steps. If the proof used only order and linear arithmetic, as the diagonal and the intermediate value arguments do, the propagation is mechanical and the conclusion is the original contradiction plus a slack term. The strict theorem is then the parameter's zero. This is why both `HoF_12_Approximate` and `F_09_QuantitativeDegradation` are short files layered on top of the strict developments rather than independent efforts: the hard mathematical work was done in the exact case, and the approximate case is that work read with a budget. The payoff is disproportionate, because the budgeted version is the one that applies to anything real.

The machine-checked statements are `HoF_12_Approximate` for the calibration bridges (`approx_trilemma`, `exact_from_approx`, `discrete_approx`

_bridge, the two-slack family, and the strict-to-approximate reductions) and F_09_QuantitativeDegradation for the metric degradation law (ApproxUniversal, approx_lawvere, quantitative_trilemma, achievable_region_bound, quantitative_impossibility, strict_limit_recovers_trilemma). The geometry the final section composes with is the ManifoldProofs cost series surveyed in Chapter 5. The exact theorems these bridges reduce to are the reflective-verdict collapse of Chapter 1 and the trilemma readings of Chapter 3; the connected-domain engine they borrow is Chapter 4's; and the measure that turns their existence claim into a rate is Chapter 5's. This chapter is the joint of those parts, the place the qualitative and the quantitative halves of the book are bolted together.

6.14 Exercises

Exercise 6.1. Write out the contrapositive of each clause of Definition 6.1 and check by hand that Proposition 6.2 is exactly the pair you obtain. Identify the single order-theoretic fact about $\mathbb{R}$ that each contraposition uses.

Exercise 6.2. Verify the $\varepsilon = 0$ reading in Remark 6.3: show that the two clauses of 0-calibration are the two forward implications of the strict biconditional, and that the backward implications are the ones ε-calibration discards. Explain in a sentence why the intermediate value argument never needs the backward implications.

Exercise 6.3. In Theorem 6.6, exactly one endpoint of the interval $[-\varepsilon, 0)$ is closed and one is open. Trace each endpoint to the hypothesis that produces it. Then explain why swapping which endpoint is open would break the recovery of the exact contradiction at $\varepsilon = 0$.

Exercise 6.4. Take $c = 0.5$ and $\varepsilon = 0.2$. A model is ε-covering, ε-calibrated, and faithful on a connected space. What is the widest possible interval the forced question's truth-distance can occupy, and what is the largest confidence the forced question can report? Now repeat for $\varepsilon = 0.02$ and comment on the change.

Exercise 6.5. Prove the tightening principle's monotonicity claim directly: if $0 \le \varepsilon_2 \le \varepsilon_1$ then $[-\varepsilon_2, 0) \subseteq [-\varepsilon_1, 0)$. Then give a one-line reason why the upper endpoint 0 does not participate in the tightening.

Exercise 6.6. (Independence, refuted.) Construct an informal scenario, no Lean required, in which a model designer drives ε from 0.1 to 0.001 over several iterations. Describe what happens to the forced question at each step and state, in one sentence, why the designer cannot reach $\varepsilon = 0$ while keeping faithfulness, coverage, and continuity. Which of Theorems 6.9, 6.12, and 6.20 is the cleanest statement of the obstruction?

Exercise 6.7. Prove Proposition 6.15 from scratch and then strengthen it: show that if the confidence map is strictly monotone along some path from an in-distribution point to an out-of-distribution point, the forced question on that path is unique. Relate the non-uniqueness in general to Exercise 1.7 of Chapter 1.

Exercise 6.8. In the discrete bridge (Theorem 6.16), the conclusion is conditional on a question existing in the upper band. Give an example of a discrete model, faithful and ε-calibrated, for which *no* question has confidence in `[c, c + ε]`, and explain why the theorem is then vacuously satisfied and yet says nothing useful. What extra hypothesis on the discrete model would force the band to be nonempty?

Exercise 6.9. Re-elaborate `c6_band` from the text with `eps` replaced by a *negative* integer and observe how the proof behaves. What does a negative slack mean, and why does Definition 6.1's hypothesis `ε ≥ 0` matter for the intended reading even though the arithmetic still typechecks?

Exercise 6.10. Rescale confidence in Definition 6.18 by a factor of `100`, mapping `c` and `γ` accordingly, and check that Theorem 6.19's conclusion interval `[-ε, 0)` is unchanged while the confidence band `[c - γ, c + γ]` scales. Use this to explain in one paragraph why the single-parameter Definition 6.1 is a convenience rather than a canonical choice.

Exercise 6.11. Prove Theorem 6.24 (approximate Lawvere) as a direct modification of the `lawvere` proof from Chapter 1: identify the single line where the exact equality `f a₀ = d` becomes a distance bound, and confirm that instantiating at the diagonal input is the only step that changes. Then state what `approx_lawvere` becomes at `ε = 0` in a genuine metric space.

Exercise 6.12. In Theorem 6.25 the transparency slack δ appears with coefficient 2 and the universality slack ε with coefficient 1. Walk the path `introspect a → M a a → t (M a a) → t (introspect a)` and attribute each unit of the budget to a specific leg. Where exactly is the 1-Lipschitz hypothesis used, and what happens to the coefficient of δ if `t` is instead `K`-Lipschitz?

Exercise 6.13. (Composition.) Using Example 6.29's numbers, write down the schematic risk figure as a product of a per-query band-incidence probability and a per-event severity bound, and identify which factor Theorem 6.12 moves when ε is halved and which factor Chapter 5's geometry controls. State one deployment decision the figure could inform and one it cannot.

Exercise 6.14. (Harder, open-ended.) The calibration bridge produces an interval `[-ε, 0)` and the degradation law produces a half-space `γ ≤ ε + 2 δ`. Formulate a single abstract statement, with a slack inserted at each idealized equality of a diagonal argument, that specializes to both. What would its `ε → 0` corner be, and which theorem of Chapter 1 would that corner reproduce?

Exercise 6.15. Redo Example 6.8b with the truth field `δ(q) = q - 0.55` replaced by `δ(q) = 2(q - 0.55)`, keeping `conf(q) = 1 - q`, `c = 0.5`, and `ε = 0.1`. Recompute the coverage witnesses, verify faithfulness, find the forced `q₀`, and report its truth-distance. Explain why the located `q₀` is the same question as before but its truth-distance is different, and connect this to the unit-separation point of Section "Two-slack separation".

Exercise 6.16. (Reading, harder.) Take the prompt-injection reading of Reading 6.33. State precisely what ε-coverage, ε-calibration, faithfulness, continuity, and connectedness each assert about a real defended system, and for each one give a concrete way it could fail in practice. Then identify which single failure a determined adversary would most want to induce, and argue

using Theorem 6.20 that even a defense that avoids that failure still leaves a positive truth-slack band the adversary can aim at.

Exercise 6.17. In Remark 6.27b the budget for a K-Lipschitz controller was ε + (1 + K) δ. Rederive it by rewalking the triangle in Theorem 6.25 and identify the exact leg whose bound changes from δ to K δ. Then determine the threshold value of K below which perfect universality and perfect transparency (ε = δ = 0) still permit a strictly positive controller margin, and interpret that value.

Exercise 6.18. (Synthesis.) The chapter recovers two exact impossibilities as zero-slack corners: the Hallucination Trilemma via `exact_from_approx` and the Mirror Trilemma via `strict_limit_recovers_trilemma`. Write a one-page essay identifying, for each, (i) which idealization was relaxed, (ii) which slack parameter measured the relaxation, (iii) the shape of the achievable region, and (iv) the geometric event, vertex or face, that the exact contradiction becomes at the origin. Conclude with the single structural feature both share.

6.15 References

Banach, S. (1922). Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae* 3, 133–181.

7 What It Means for AI Safety

Abstract. A theorem saying that all three properties cannot hold together is a menu, not a counsel of despair. This chapter reads the trilemma as an engineering constraint. It works through the three relaxations (abstain, price a confident-error band, or weaken calibration), states what each costs, and says when each is the right choice. It marks the quantity the theory refuses to fix, which is where deployment risk lives. The chapter is explicit about which sentences are theorems and which are judgments about cost and traffic.

Keywords: AI safety, Abstention, Deployment risk, Defense design, Cost asymmetry

A theorem that says ”you cannot have all three” is not a counsel of despair. It is a menu. The earlier chapters proved the impossibility and traced it to a single diagonal. This chapter spends the result. It reads the trilemma as an engineering design constraint, works through what each way of satisfying it costs, says when each choice is the right one, and marks the one quantity the theory refuses to fix. That quantity is where deployment risk actually lives, and being honest about it is the difference between using an impossibility result and misusing it.

The tone shifts here. The rest of the book is mathematics, and its claims are checked by the Lean kernel. This chapter is engineering. Its claims are judgments about costs, domains, and traffic, and they are only as good as the premises you grant. I will be explicit about which sentences are theorems and which are recommendations, because the whole point of a machine-checked impossibility result is that it lets you argue about the right things.

A reader who wants only the practical upshot can take three sentences from this chapter. First, every deployed system already lives at one of exactly three corners, and the value of the theorem is that the list of three is complete, so the fourth option you may be chasing does not exist. Second, which corner is right is decided by your domain and by one number the theory does not give you, the density of your real traffic near the model’s boundary, so measuring that number is the highest -leverage thing you can do. Third, the theorems say a bad point exists and are silent on how often you visit it, so treating an existence result as a frequency claim, in either direction, is the most common way to misuse this work. The rest of the chapter is these three sentences with their reasons, their costs, and their worked cases.

7.1 The constraint, stated as a triangle

Two results from Chapter 3 drive this chapter, and both are one line of Lean: `no_reflective_verdict`, read twice.

The Hallucination Trilemma reads a Boolean self-verdict `v p q` as "the model is confident and correct about query `q`." Three properties make that verdict a total decider of the model's own correctness. *Faithfulness* says a confident answer is a correct one. *Calibration* says confidence tracks correctness, so the confidence number means what it says. *Coverage* says the model actually answers, that it is sometimes right and sometimes wrong rather than silent. Add reflectivity, the ability to be queried about its own verdicts, and the diagonal of Chapter 1 forces a query that is confidently correct exactly when it is not. That is the liar of `liar_query`, wearing a safety costume.

The Defense Trilemma reads `v p q` as "the wrapper defense renders prompt `q` safe," and names three matching properties. *Utility preservation* says safe prompts pass through unchanged. *Completeness* says every output is made safe. Connectivity of the prompt space, together with continuity of the defense, is the third leg, and in the diagonal reading it is the arbitrary-text premise that lets a prompt describe and invert the defense. The forced witness is the injection "be unsafe exactly if you would be made safe."

Definition 7.1 (The design triangle). Fix one of the two readings. The *design triangle* has the three named conditions as its vertices. A *corner policy* is a choice to give up exactly one vertex and keep the other two. The trilemma says the interior of the triangle, all three at once, is empty.

The geometry of the phrase "you cannot have all three" is worth stating plainly, because it is often misread. The theorem does not rank the three conditions, does not say which to drop, and does not say the surviving two come for free. It says the point where all three hold does not exist. Everything an engineer does with the result is a choice of which vertex to release and how far.

Remark 7.2. The two readings share a proof term, which Chapter 3 checks with `rfl`. For an engineer this means the menu below is written once and applies to both problems. When I describe the cost of dropping coverage in the hallucination setting, the same paragraph describes the cost of dropping completeness or utility in the defense setting, under the translation of Definition 7.1. I will point out where the analogy strains, because it does strain in one place, and pretending it does not would be dishonest.

Why a menu, and not a wall

There is a habit of mind that reads every impossibility theorem as a wall. Halting is undecidable, so we give up on program analysis. Consistency is unprovable, so we stop trusting formal systems. This reading is almost always wrong, and it is wrong here. A wall tells you to stop. A menu tells you to choose. The trilemma is a menu because it does not forbid useful systems. It forbids one specific idealized system, the one that is faithful and calibrated and covering

at the same time, and it leaves every system that gives up a little of one condition fully available.

The difference is practical. If the trilemma were a wall, the right response would be to abandon confident automated answering, and no serious person does that. The right response is to notice that every deployed system already sits at one of the three corners, usually without having chosen it on purpose, and to make the choice deliberate. A model that quietly abstains when unsure has dropped coverage. A model that answers the hard middle and is sometimes confidently wrong there has relaxed faithfulness. A model whose confidence scores stopped meaning anything two training runs ago has weakened calibration and nobody noticed. The theorem does not create these corners. It names them and proves the list is complete.

Completeness of the list is the part that earns the machine check. A menu with three items is only useful if there is no fourth. The trilemma's value is not that each corner is individually available, which is obvious, but that the interior is empty, which is not. Without the theorem an engineer might spend years chasing the fourth option, the system that keeps all three, believing the failure to find it is a failure of cleverness. The theorem says the search terminates with nothing, so the cleverness should go into choosing and pricing a corner instead. That redirection of effort is the entire practical payload of an impossibility result.

There is a second reading of "menu" that is worth naming. A menu is something you choose from repeatedly, not once. A product does not pick a corner at launch and keep it forever. Traffic shifts, downstream consumers appear, stakes change as the product is trusted with more, and the right corner moves with them. So the menu is not a one-time decision but a standing one, revisited whenever the domain changes, and the review cadence is itself a design choice. A team that picked abstention for a low-traffic pilot and never revisited the choice when the product went to scale is running the wrong corner, not because it chose badly, but because it stopped choosing.

The diagonal, read as a design fact

The proof from Chapter 1 is worth reading once more with an engineer's eye, because the mechanism tells you where the corners come from. The diagonal builds a behavior that disagrees with whatever the system would do on itself. In the safety reading this behavior is a query about the system's own verdicts, engineered to be confidently correct exactly when it is not. The system cannot answer it correctly, because answering correctly would make it incorrect, and answering incorrectly is the failure. That is `liar_query`, and every corner is a way of denying the diagonal one of the things it needs.

The diagonal needs three things. It needs the system to name its own behaviors, which is reflectivity. It needs the outcome to have no fixed point under the flip, which is what a two-valued confident verdict guarantees. And it needs the system to actually produce that outcome, which is coverage. Deny reflectivity and the query cannot be posed, but you cannot deny reflectivity

for prompt injection because prompts are arbitrary text, and denying it for a representation space is the strong premise Remark 7.11 discusses. Deny the fixed-point-free flip by adding an abstention outcome, and the diagonal resolves consistently instead of contradicting, which is Relaxation I. Deny coverage by refusing to produce an outcome near the boundary, which is the same move seen from the outside. The three corners are three ways of removing one leg of the diagonal, and the theorem's completeness is the statement that there is no fourth leg to remove.

7.2 Three relaxations, three cost models

A corner policy drops one vertex. There are three vertices, so there are three relaxations. Each has a characteristic cost, a characteristic failure mode, and a class of domains where it is the right choice. I take them one at a time. For each I give an informal cost model, meaning a rough accounting of what you pay and in what currency, not a theorem. The theorems say the corner exists and the interior does not. The cost models are engineering estimates layered on top.

A word on the status of what follows. The cost models use symbols like ρ, ε, `Cerr`, and `Cref`, and they combine them into rough expressions like $\rho \cdot \varepsilon$ for a band error rate or $a \cdot u$ for an abstention loss. These are not theorems and they are not precise. They are the simplest accounting that captures the first-order behavior of each corner, good enough to compare corners and to see which quantity dominates, and no better. Where a factor is a proven bound I will say so and name the Lean result. Where it is an estimate I will leave it as an estimate. Confusing the two is exactly the error this chapter warns against, so I try not to commit it while describing it. Read the formulas as scaling relations, not as predictions to three digits.

Relaxation I: drop coverage, and abstain

The cleanest way to satisfy the trilemma is to let the model decline. Keep faithfulness and keep calibration, and buy them by refusing to answer where a confident answer would be a gamble. In the selective-prediction literature this is abstention with a reject option, and it is the corner policy most safety-critical deployments already run, sometimes without knowing it is a corner policy.

Why does abstention escape the diagonal, when faithfulness and calibration alone could not? Because the diagonal needs the outcome type to have no fixed point under the behavior-flip. For a two-valued verdict the flip is Boolean negation, and `bool_not_fpf` says negation fixes nothing, which is exactly why `cantor` and `no_reflective_verdict` bite. Abstention adds a third outcome that the flip leaves alone. Once the flip has a fixed point, Lawvere's theorem stops producing a contradiction and starts producing a consistent value, and that value is the abstention.

```
inductive Outcome where
  | correct
  | wrong
  | abstain

def c7_flip : Outcome → Outcome
  | .correct => .wrong
  | .wrong   => .correct
  | .abstain => .abstain

theorem c7_flip_fixed : ∃ y : Outcome, c7_flip y = y :=
  ⟨.abstain, rfl⟩
```

Read this against `lawvere`. On a universal system whose outcomes are `Outcome`, the diagonal behavior `a ↦ c7_flip (f a a)` is named by some index, and evaluating there gives a fixed point of `c7_flip`. With the two-valued flip that fixed point could not exist, and the system could not be universal. With the abstaining flip the fixed point is `abstain`, and the self-referential query resolves to "I decline." The liar query does not vanish. It becomes a query the system is allowed to refuse. In the continuum reading of Chapter 4 the same move is a cut: abstaining on an open region around the boundary deletes it from the domain and can disconnect the true region from the false one, which is why the paper calls refusal training connectedness surgery.

Definition 7.3 (Abstention policy). An *abstention policy* replaces the answer map with a partial one that returns a designated `abstain` outcome on a region `R` of query space, and answers on the complement. The *abstention rate* is the probability that a query drawn from deployment traffic lands in `R`.

Proposition 7.3a (Abstention preserves the surviving vertices). *On the complement of `R`, an abstention policy leaves faithfulness and calibration exactly as they were. It changes only coverage, by removing `R` from the answered set.*

Proof. The answer map is unchanged outside `R`, so any query answered outside `R` receives the same answer and the same confidence it did before, and the faithfulness and calibration statements, which are conditions on answered queries, hold on the complement exactly as they held before. Coverage, the condition that the model is sometimes strictly right and sometimes strictly wrong, is the only vertex the policy touches, since it can delete the strictly-wrong witnesses by placing `R` over them. ■

Not formalized. The argument is bookkeeping over which hypotheses the policy touches, and the companion libraries model total answer maps only, so there is no declaration of partial answering for it to cite.

Proposition 7.3a is the formal content of "abstention is the clean corner." It does not trade a bit of one vertex for a bit of another. It surrenders coverage on a chosen region and keeps the other two intact and unmodified everywhere else. That is why an abstention policy is the easiest to reason about and to audit: the only thing that changed is a region you chose and can inspect.

Cost model. You pay in utility, and the currency is refused queries. Let `a` be the abstention rate and let `u` be the average value of answering a query that falls in `R`. The direct utility loss is about `a · u`. Two features of this cost matter. First, `R` should hug the boundary, because that is where confident answers are least trustworthy, so a well-placed `R` of small measure removes most of the confident errors while refusing few queries. The geometry of Chapter 5 is what tells you how small `R` can be for a target error rate, through the concentration of measure near the boundary (`MoF_Cost_04_Concentration`). Second, refusal is not free of second-order cost. A system that abstains too often trains its users to route around it, and a refused query frequently becomes someone else's confident answer downstream, often a worse one. The honest cost of abstention includes where the refused query goes.

When it is right. Abstention is the correct corner when a confident error is much more expensive than a refusal. Write `Cerr` for the cost of a confident wrong answer and `Cref` for the cost of a refusal. Abstention dominates when `Cerr ≫ Cref`. This is the regime of clinical decision support, legal analysis, financial control, and any actuation loop where a wrong action is hard to reverse. In these settings a system that says "I do not know, escalate to a human" on the hard middle is behaving well, and the utility it gives up is utility it should not have been claiming.

Remark 7.4. Abstention preserves both surviving vertices exactly, which is why it is the safest corner to reason about. It does not blur the confidence signal and does not tolerate a class of errors. It removes queries from scope. The failure mode is not a wrong answer but a gap in coverage, and gaps are auditable in a way that confident errors are not. If you can afford to refuse, refuse.

Worked scenario. A radiology triage assistant answers a yes-or-no question about whether a scan shows a finding that needs urgent review. Suppose a confident wrong "no urgent finding" costs, in expectation over its downstream consequences, a thousand times what a refusal costs, since a refusal simply routes the scan to the normal radiologist queue. Suppose further that traffic near the model's truth boundary is one percent of volume, and that abstaining on the whole boundary shell removes ninety-five percent of the confident errors. Per hundred scans, answering everywhere costs about `0.01 · 0.95 · 1000 · 100 = 950` refusal-units in expected error, against an abstention cost of about `1` per hundred scans, one refusal per hundred routed to the human queue. Abstention wins by roughly the cost ratio, here about `950` (that is `0.95 · Cerr/Cref`), and that ratio is set by `Cerr/Cref` and the in-band error fraction, not by the boundary density. Lowering the boundary density does not flip the decision: the reject region hugs the boundary, so the abstention cost falls with density exactly as the error cost does, and abstention stays cheaper by the same ratio. What the boundary density sets is the absolute cost of either corner, and how wide a reject region is worth placing. The design decision is downstream of a measurement, which is the recurring theme of this chapter.

The second-order cost, made concrete. The `a · u` term is the direct loss, but a refusal is rarely the end of the query's life. A refused clinical question goes

to a human who is now busier, a refused search query goes to a competitor, a refused agent action gets retried with a reworded prompt. The honest cost of abstention is `a · u + a · d`, where `d` is the expected damage of what happens to the refused query next. In the radiology case `d` is small, because the human queue is a good fallback. In a consumer product where the refused query goes to an ungoverned alternative, `d` can exceed `u`, and an abstention policy that looks safe in isolation exports its risk. A team choosing this corner should know where its refusals land before it congratulates itself on its low error rate.

Relaxation II: relax faithfulness, and price a confident-error band

The second corner keeps coverage and keeps calibration, and pays by accepting that a thin band of queries near the boundary will receive confident answers that are wrong. The model answers everywhere, its confidence still separates the clearly true from the clearly false, and in exchange it is allowed to be confidently wrong inside a controlled region around the threshold.

This is the corner that the approximate results of Chapter 6 make precise. Exact separation is impossible next to a live boundary, but two-sided slack is not. The verified statement `two_slack_approx_coupling` says a model can be separating with a truth slack ε and a confidence margin γ, and `truth_slack_must_be_positive` says the slack cannot be driven to zero while coverage and the guarantee both hold. The companion `no_boundary_in_upper_band_two_slack` says that once you accept the slack, the boundary is excluded from the high-confidence band, which is the whole value of the trade. You are not tolerating errors everywhere. You are confining them to a band whose width you chose.

Definition 7.5 (Confident-error band). A *confident-error band* of half-width ε is the set of queries with `|dF| ≤ ε`, where `dF` is the realized truth field. The policy answers confidently on the whole space and accepts that answers inside the band may be wrong. The *band mass* is the probability that deployment traffic lands in the band.

Cost model. You pay in confident errors, and the currency is band mass. The confident-error rate is about `ρ · ε`, where `ρ` is the local density of traffic near the boundary and ε is the band half-width in units of the truth field. Two levers set this cost. The first is ε, which you control by how aggressively you require the confidence signal to separate. Shrinking ε shrinks the error rate, and the theory says you cannot shrink it to zero, so there is a floor. The second is `ρ`, which you do not control at all. It is the boundary density on real traffic, the one empirical number this chapter keeps returning to. A band of fixed width ε is nearly harmless if traffic almost never approaches the boundary, and nearly useless if traffic piles up there.

When it is right. Relaxing faithfulness is correct in high-throughput settings where refusing constantly is worse than occasionally erring, and where the per-item cost of an error is bounded and recoverable. Content ranking, autocomplete, first-pass triage, and recommendation are examples. A system

that abstained on every boundary query in these settings would abstain constantly and deliver no value, while a small confident-error band costs little per item and is absorbed by volume. The band policy also has an operational virtue the abstention policy lacks. The errors it produces are localized near a known surface, so you can monitor that surface directly rather than watching the whole input space. That is the "monitor, don't eliminate" prescription of the defense paper, read for hallucination.

Remark 7.6. The band is not a bug you failed to fix. It is a resource you are spending on purpose. The danger of this corner is not the band itself but the temptation to pretend ε is zero, to ship a model as if it were exactly faithful and discover the band the hard way when traffic shifts and ρ rises. A team that runs the band policy honestly writes down its ε, monitors its ρ, and treats the product $\rho \cdot \varepsilon$ as a service-level number, not an embarrassment.

Proposition 7.6a (The band has a positive floor). *Under coverage and a threshold-inclusive guarantee on a connected domain, the truth slack ε compatible with separation cannot be zero. There is a strictly positive infimum of feasible slack, and pushing the model toward exact faithfulness drives the required slack down toward that infimum but never to it.*

Discussion. This is the engineering content of `truth_slack_must_be_positive` and `two_slack_approx_coupling`, imported here as a fact about the corner rather than reproved. It has a sharp consequence: the band cannot be closed by better training. A team that keeps shrinking ε will see diminishing returns and then a floor, and past a point the effort spent shrinking ε would be better spent measuring and managing ρ. The floor is not a limit of the current model. It is a property of the geometry, so no model on that geometry escapes it. ■

Worked scenario. An autocomplete feature suggests the next line of code. A confident wrong suggestion costs the developer a few seconds to reject, so the per-item error cost is small and recoverable, which is the high-throughput profile. Abstaining whenever the model is near its boundary would blank the feature on exactly the ambiguous cases where a suggestion is most wanted, and a feature that goes silent when the developer hesitates trains the developer to stop looking. The band corner fits: answer always, accept a thin ring of confident-but-wrong suggestions near the boundary, and confine monitoring to that ring. Suppose ε is tuned so the band mass on current traffic is two percent, and the acceptance rate of in-band suggestions is thirty percent, so about `0.02 · 0.30 = 0.6` percent of accepted suggestions are confident errors the developer must catch. That is a service-level number the team can state and defend. The failure it must guard against is not the band but a silent rise in ρ when the language or codebase mix shifts, which widens the same ε into a larger band mass without any code change.

Remark 7.6b (The band is where monitoring is cheap). The operational advantage of this corner deserves emphasis. Because the errors are confined to a neighborhood of a known surface, the truth boundary, you can spend your monitoring budget on that surface instead of on the whole input space. This is the hallucination reading of the defense paper's "monitor, don't eliminate"

line. You cannot afford to audit every answer, but you can afford to audit the small fraction that fall in the band, and the band is defined by a computable distance to the boundary, so membership is cheap to test at serving time. The band corner turns an unbounded auditing problem into a bounded one, which is a reason to prefer it even when abstention would also work.

Relaxation III: weaken calibration, and decouple confidence from correctness

The third corner keeps coverage and keeps faithfulness in the weak sense that the answer is often right, and pays by letting the confidence number stop tracking correctness. The model answers everywhere and is frequently correct, but the scalar it reports as confidence no longer means what a calibrated number would mean. High confidence no longer implies likely correct, and the ordering of confidences no longer ranks queries by reliability.

This is the corner that looks cheapest and is usually the most expensive. It looks cheap because top-1 accuracy does not move. A model whose confidence is decorrelated from correctness can score exactly as well on an accuracy benchmark as one whose confidence is perfectly calibrated, because accuracy never reads the confidence number. The cost is invisible to the metric most teams watch.

Definition 7.7 (Decoupling). A model is *decoupled* at level κ if the mutual information between its reported confidence and the correctness of its answer is at most κ. Perfect calibration is high mutual information; full decoupling is $\kappa = 0$, a confidence signal that carries no information about correctness.

Proposition 7.8 (Decoupling is invisible to accuracy). *Top-1 accuracy is a function of the answer map alone and does not depend on the confidence field. Hence two models with identical answer maps and any two confidence fields, one calibrated and one fully decoupled, have identical accuracy.*

Read a model as `M : Q → A × C`, an answer paired with a confidence, and read a metric that scores answers as any function `acc` of the answer map alone. The proposition is then a statement about what such a metric can see, and it holds for every metric of that shape, not merely for top-1 accuracy.

```
theorem c7_decoupling_invisible_to_accuracy
    {Q A C S : Type _} (acc : (Q → A) → S)
    (M₁ M₂ : Q → A × C)
    (h : ∀ q, (M₁ q).1 = (M₂ q).1) :
    acc (fun q => (M₁ q).1) = acc (fun q => (M₂ q).1) := by
  congr 1
  funext q
  exact h q
```

Proof. The two answer maps are equal by function extensionality, since they agree at every query, and `acc` applied to equal arguments gives equal results. The confidence components never appear. ■

Proof. Accuracy counts queries where the answer is correct, and the answer map is fixed. The confidence field never enters the count. ■

Proposition 7.8 is trivial as mathematics and important as engineering. It is the reason this corner is chosen by accident. A team optimizing accuracy, or a training objective that rewards being right without pricing the confidence report, drifts into decoupling for free and cannot see it on the dashboard.

Cost model. You pay wherever the confidence number is consumed by something other than a human reading it once. Every downstream system that thresholds on confidence, routes on it, triages on it, or gates an action on it is now acting on noise. Write V for the value that a downstream consumer extracts from a calibrated confidence signal, through better routing or cheaper human review. Full decoupling destroys that V entirely, and partial decoupling degrades it in proportion to the lost mutual information. The cost is not a rate of wrong answers. It is the collapse of a signal that other systems were built to trust, and it lands on those other systems, often out of sight of the model's own team.

When it is right. Rarely, and only when nothing downstream reads the confidence. If the model's confidence is never surfaced, never thresholded, and never consumed by a router or a triage queue, then decoupling costs nothing, because the signal it corrupts was not being used. A closed system that always answers, always shows the answer, and never acts on its own certainty can weaken calibration for free. The moment any consumer appears downstream, this corner becomes the most dangerous of the three, because it fails silently and the failure surfaces in a different component than the one that caused it.

Remark 7.9. The three corners differ in where the failure shows up. Abstention fails in the coverage log, visibly, as refusals. The band fails at a known surface near the boundary, monitorably, as localized confident errors. Decoupling fails downstream, invisibly, as degraded decisions in systems that trusted the confidence number. Prefer failures you can see. This is the single most useful heuristic in the chapter, and it orders the corners for a default: abstain if you can afford it, band if you cannot, and decouple only if you have proven nothing reads the confidence.

Worked scenario, and why it is subtle. A question-answering assistant reports a confidence with every answer, and the confidence is shown to the user as a colored badge. Nothing automated reads it. The team, chasing accuracy on a benchmark, switches to a training objective that improves top-1 accuracy by two points. Proposition 7.8 guarantees this move cannot hurt accuracy no matter what it does to the confidence, and in fact the new objective decorrelates confidence from correctness, because it never had a reason to preserve the correlation. Accuracy is up, the launch looks like a win, and the confidence badge is now decorative. If the badge really is only decorative, this is fine, and the decoupling corner was the right free lunch. The subtlety is that a later team, seeing a confidence field already there, wires it into an auto-approval path for an agent. They inherit a signal that looks like confidence, is typed like confidence, and carries no information about correctness, and nothing

in the code says so. The failure is now downstream and invisible, and it was seeded two launches earlier by a change that improved the dashboard. This is the characteristic life cycle of the decoupling corner: chosen for free, safe until a consumer appears, dangerous the moment one does.

Remark 7.9a (Decoupling cannot be detected by the producer alone). *Whether weakening calibration is safe depends on the set of downstream consumers of the confidence signal, which is not a property of the model. Hence the model's own team, looking only at the model, cannot determine whether it has entered the dangerous regime of this corner.*

Proof. Safety of decoupling is the statement that no consumer reads the confidence in a way that degrades under decoupling. The consumers live outside the model. A purely model-internal audit has no access to them, so it cannot decide the statement. ■

Remark 7.9a is the reason decoupling is an organizational hazard as much as a technical one. The information needed to judge it is split between the team that produces the confidence and the teams that consume it, and neither half can decide the question alone. The mitigation is a contract: if a system publishes a confidence number, it commits to a calibration property, and any consumer may rely on it. That contract is what turns a shared signal from a liability into an asset.

The three corners side by side

It helps to hold the three corners against each other on the axes that decide a design. Each drops one condition, pays in one currency, fails in one place, and suits one kind of domain.

Dropping coverage keeps faithfulness and calibration, pays in refused queries plus the fate of those queries, fails visibly in the coverage log, and suits high-stakes domains where a confident error is far worse than a refusal. Relaxing faithfulness keeps coverage and calibration, pays in a confined confident-error rate $\rho \cdot \varepsilon$, fails monitorably at the boundary surface, and suits high-throughput domains with bounded recoverable per-item errors. Weakening calibration keeps coverage and the weak sense of faithfulness, pays wherever the confidence is consumed, fails invisibly and downstream, and suits only closed systems whose confidence nothing reads.

Two structural facts cut across the comparison. First, the currencies are not interchangeable. A refusal is not a small error and an error is not a small refusal. They land on different parties, surface in different logs, and are governed by different costs, so a design that treats them as fungible will misprice the corner. Second, the failure locations order the corners by how much organizational trust they demand. Abstention demands the least, because its failures are in your own logs. The band demands more, because you must actually run the boundary monitor. Decoupling demands the most, because its safety is a claim about systems you may not control, which Remark 7.9a says you cannot verify from inside the model. When in doubt, prefer the corner whose failures you can see without anyone else's cooperation.

Remark 7.10a (There is no dominant corner). No corner dominates the others across all domains, which is why the trilemma is a menu and not a recommendation. If one corner were always best, the theorem would effectively be a wall pointing at that corner. The domain analysis of the next section exists precisely because the right corner changes with the domain, and a shop that runs the same corner for every product has either a very uniform product line or an unexamined default.

The corners are not exclusive

Real systems blend the corners, and the blend is often better than any pure corner. A deployment can abstain on the innermost band, answer with a priced confident-error band on the next ring out, and be calibrated in the confident interior. This is a three-region policy: a reject region R, a band region B, and a trusted region. The trilemma does not forbid the blend. It forbids the point where all three conditions hold at once, and the blend never claims that point. It claims coverage outside R, faithfulness outside B, and calibration in the interior, three weaker conditions on three different regions.

Proposition 7.10 (Blends satisfy the trilemma by partition). *If the query space is partitioned into a region where coverage is dropped, a region where faithfulness is relaxed, and a region where calibration is enforced, then no single region carries all three conditions, and the impossibility of Definition 7.1 is not contradicted on any region.*

Carry the three conditions as predicates A, B, C on queries and the three regions as predicates R_0, R_1, R_2 covering the query space. The partition design is the hypothesis that each region drops one condition, and the conclusion is that no query carries all three.

```
theorem c7_blend_no_region_has_all_three {Q : Type _}
    (A B C R₀ R₁ R₂ : Q → Prop)
    (hcover : ∀ q, R₀ q ∨ R₁ q ∨ R₂ q)
    (h₀ : ∀ q, R₀ q → ¬ A q)
    (h₁ : ∀ q, R₁ q → ¬ B q)
    (h₂ : ∀ q, R₂ q → ¬ C q) :
    ∀ q, ¬ (A q ∧ B q ∧ C q) := by
  intro q h
  rcases hcover q with hr | hr | hr
  · exact h₀ q hr h.1
  · exact h₁ q hr h.2.1
  · exact h₂ q hr h.2.2
```

Proof. Every query lies in some region by coverage, and that region's hypothesis contradicts the corresponding conjunct. ■

The theorem needs no property of the three conditions at all, which is what makes it a statement about blending rather than about hallucination. Any three requirements dropped region by region are consistent in exactly this

way, and the impossibility is never contradicted because it never applies to a region.

Proof sketch. The trilemma is a statement about a region on which all three hold. By construction no region of the partition is such a region. There is nothing to contradict. ■

The value of Proposition 7.10 is that it licenses the standard engineering practice of tiered confidence without any tension with the theorem. The design question is not whether to blend but where to place the region boundaries, and that placement is governed by the same empirical number as everything else, the density of traffic near the truth boundary.

The blend also clarifies a confusion that surrounds impossibility results. A practitioner will object that real systems clearly do abstain sometimes, answer sometimes, and report useful confidence, so the trilemma must be violated in practice. It is not. The system abstains in one region, answers with a priced band in another, and is calibrated in a third, and no single region carries all three conditions. The appearance of having all three is an artifact of looking at the whole system at once and not noticing that the three properties hold on three different sets of queries. The theorem is about a region where all three hold together, and the blend is precisely the design that never creates such a region. Seeing this dissolves the apparent contradiction between the theorem and every working product, and it locates the real design work in the placement of the region boundaries rather than in an imagined escape from the theorem.

Remark 7.10b (Region boundaries are themselves boundaries). Placing the reject region and the band introduces new decision surfaces, the edges of R and B, and those edges are decision boundaries with their own coupled points under the same theorem. This is not a regress that undoes the design. The edges of R and B are chosen by you, in the truth-field coordinate, so you can place them where behavior is benign, which is the "make the boundary shallow" prescription applied to the policy's own thresholds. The lesson is that a tiered policy does not remove boundaries; it replaces one semantic boundary you do not control with policy boundaries you do, and the whole art is putting the latter where crossing them costs little.

7.3 The same menu for prompt-injection defense

Under Definition 7.1 the defense reading inherits the three corners, with the vertices renamed. Dropping coverage becomes dropping utility preservation. Relaxing faithfulness becomes accepting a residual attack surface. Weakening calibration becomes reporting a safety score that no longer tracks actual safety. The translation is exact for the first two corners and instructive where it strains for the third.

Corner I for defense: the abstaining wrapper

A defense that refuses to pass a class of prompts, rather than rewriting them into safe equivalents, is the abstention corner. It gives up utility preservation, since some safe prompts get blocked along with the unsafe ones, and in exchange it can be complete on what it does pass. This is the "reinforced concrete wall with a strict bouncer" prescription of the defense paper, and it is the reason discontinuous defenses escape the theorem. A hard blocklist is not a continuous utility-preserving wrapper, so the impossibility that assumes continuity and utility preservation simply does not apply to it. The cost is the same as abstention above, paid in blocked legitimate prompts, and the second-order cost is the same too, users routing around a bouncer that blocks too much.

Corner II for defense: the priced residual surface

A continuous utility-preserving wrapper that accepts it is not complete is the band corner for defense. The verified ε-robust constraint of the defense paper is this corner made quantitative: the wrapper can push near-boundary prompts below the safety threshold τ, but only so far, $f(D(x)) \geq \tau - L(K+1)\delta$ within distance δ of the fixed boundary point. The residual surface is the analog of the confident-error band, and its depth is bounded by the Lipschitz constant L. The engineering levers are the same ones the paper lists: make the boundary shallow, so that boundary-level behavior is a polite refusal rather than a harmful compliance; reduce L, so the surface is smoother and the persistent region shrinks; and reduce the effective dimension d, because the cost of covering the surface grows as N^d. The GPT-5-Mini example in the paper, whose alignment deviation ceiling sits at exactly the threshold, is a band policy where the band contains no actual harm. The manifold exists and is empty of damage.

Corner III for defense: the misreported safety score

A defense that always claims to have handled the prompt, and reports a safety score decoupled from whether the prompt was actually made safe, is the decoupling corner. It is the most dangerous for defense for the same reason it is most dangerous for hallucination. Any orchestration layer that reads the safety score and decides whether to run a tool, escalate to review, or auto-approve is now gated on noise. This is where the analogy is at its sharpest rather than where it strains, because agentic pipelines consume safety scores constantly, and a decoupled score in a tool-calling loop is a silent authorization of unsafe actions.

Remark 7.11 (Where the analogy strains). The strain is in the premise, not the menu. The hallucination trilemma in its diagonal form assumes reflectivity, that every verdict pattern is named by some query. For prompt injection this premise is realistic, because prompts are arbitrary self-describing text,

so the diagonal applies directly and needs no topology. For a fixed representation space the reflectivity premise is strong and less physical, and Chapter 4 replaces it with continuity on a connected domain, a weaker and more defensible hypothesis. So the menu is shared, but the two problems earn their impossibility differently. Defense gets it from self-reference, cheaply and robustly. Hallucination gets it either from the strong reflectivity premise or from the weaker geometric one, and an honest deployment argument should say which premise it is relying on.

Cost asymmetry and capacity parity

Two further facts from the defense development change how a corner is priced, and both favor the attacker. The first is cost asymmetry. Covering the residual surface by exhaustive enumeration costs on the order of `N^d`, exponential in the effective dimension `d` of the prompt interface, while an attacker needs to find only one crossing. This is the verified grid cost result, and its practical reading is that brute-force defense over a rich interface is hopeless, which is why the defense paper's prescription is to shrink `d` by constraining the interface rather than to enumerate. The caveat is real and stated as a limitation: a learning-based defense that generalizes across the space may beat the grid bound, and whether it can is Problem 7.28. The exponential cost is proven for enumeration, not for every possible defense.

The second fact is capacity parity. Under equal resources split between preserving utility and enforcing safety, the safety side loses capacity to the utility side, because utility preservation constrains almost the whole space while the unsafe set is thin. The prescription is architectural separation: a lightweight classifier that flags, paired with a separate response generator, so that the defense does not pay a capacity tax to the utility pipeline. For a corner analysis this means the decoupling danger has a structural cousin. A single model asked to be both useful and safe will, under pressure, spend its capacity on the more frequent demand, which is utility, and let the safety signal degrade, which is the decoupling corner arrived at through resource competition rather than through a training objective. Separating the two functions is how you keep that from happening by default.

7.4 Reading a domain: which corner fits

The corner you should choose is a function of your domain, and the domain reduces to three questions. What does a confident error cost relative to a refusal? Does refusal cost accumulate with volume? And does anything downstream consume the confidence number? These three questions name three archetypes.

Definition 7.12 (High-stakes domain). A domain is *high-stakes* if the cost of a confident error greatly exceeds the cost of a refusal, `Cerr ≫ Cref`, and errors are hard to reverse.

Definition 7.13 (High-throughput domain). A domain is *high-throughput* if refusal cost accumulates with volume, so that a policy refusing a fixed fraction of a large stream loses more than it saves, and the per-item error cost is bounded and recoverable.

Definition 7.14 (Confidence-consumed-downstream domain). A domain is *confidence-consumed-downstream* if some automated component reads the model's confidence or safety score and acts on it without a human in the loop.

These are not exclusive, and most real deployments are two of the three at once, which is what makes design hard. A clinical triage bot feeding an automated escalation queue is high-stakes and confidence-consumed. A moderation pipeline at platform scale is high-throughput and confidence-consumed. The three questions give a default policy, and the intersections tell you where the default is not enough.

A fourth question refines the first, and it is the one teams most often skip. Reversibility and time-to-detect together set the true cost of an error, because an error that is caught and undone within a second is not the same error as one that acts irreversibly before anyone notices. Formally, the effective error cost is not `Cerr` alone but `Cerr` weighted by the probability that the error is neither detected nor reversed in time. A high-stakes domain with fast detection and cheap reversal can behave, for corner-selection purposes, like a high-throughput one, and a nominally low-stakes domain with slow detection and irreversible actions can behave like a high-stakes one. An agent that can spend money or send messages is the canonical case where low nominal stakes hide high effective stakes, because the action commits before review. When you read a domain, read its detection and reversal latency, not only its headline error cost.

Remark 7.14a (Stakes are a function of the pipeline, not the model). The same model is high-stakes in one pipeline and low-stakes in another, because stakes are set by what the pipeline does with the answer, how fast it acts, and whether it can undo. This is the same lesson as Proposition 7.17 for boundary density, applied to cost instead of frequency: the model fixes neither the frequency of boundary queries nor the cost of getting them wrong. Both are properties of the deployment. A team that classifies its domain by the model's benchmark behavior rather than by its pipeline's consequences has classified the wrong object.

Proposition 7.15 (Default corner by archetype). *Under the cost models of the previous sections, the utility-maximizing pure corner is abstention for a high-stakes domain, the confident-error band for a high-throughput domain, and, for a confidence-consumed-downstream domain, whichever of the first two preserves calibration, never the decoupling corner.*

Justification. For high-stakes, `Cerr` $\gg$ `Cref` makes the abstention cost `a` $\cdot$ `u` smaller than the band cost `ρ` $\cdot \varepsilon \cdot$ `Cerr` at any tolerable error rate. For high-throughput, accumulated refusal cost exceeds the bounded per-item error cost, reversing the inequality. For confidence-consumed, Proposition 7.8 shows decoupling destroys the downstream value `V` while leaving accuracy untouched, so it is dominated by either calibration-preserving corner. ■

Not formalized. This is an engineering argument, not a theorem: it assumes the cost models of the preceding sections are the right accounting, and no companion declaration states it.

Case study: clinical decision support

A model reads a patient summary and proposes a differential diagnosis with a confidence for each candidate. The domain is high-stakes, since a confident wrong diagnosis can send a clinician down a harmful path, and a refusal costs only the clinician's time. It is also confidence-consumed, because an automated system uses the confidence to decide which cases route to a specialist.

The default is abstention. The model should decline on the hard middle, the region near its own truth boundary where its confident answers are least trustworthy, and route those cases to a human. The confident-error band corner is wrong here, because a bounded per-item error is not bounded when the item is a person. The decoupling corner is doubly wrong, because the specialist-routing system reads the confidence, and a decoupled confidence sends the wrong cases to the specialist and auto-clears the dangerous ones.

The design work is placing the reject region `R`. Too wide and the model refuses so often the clinicians ignore it, which is the second-order cost of abstention made real. Too narrow and confident errors leak through. The geometry of Chapter 5 is what sizes `R`: the concentration of traffic near the boundary (`MoF_Cost_04_Concentration`) tells you how much error you remove per unit of refusal, and you widen `R` until the marginal error removed no longer justifies the marginal refusal. The one number you cannot get from theory is how much real patient traffic sits near the boundary, and that must be measured on representative cases, not on a benchmark of textbook presentations.

Case study: platform-scale content moderation

A model scores billions of posts for a policy violation, and a threshold on the score decides auto-removal, human review, or clearance. The domain is high-throughput, since even a small refusal fraction on billions of items is an unmanageable review queue, and it is confidence-consumed, because the threshold is the whole mechanism.

The default is the confident-error band, tuned tightly, with calibration preserved because the threshold consumes the score. The model answers on everything, accepts a thin band of confident misclassifications near the boundary, and confines those errors to a surface the moderation team monitors directly. Abstention as a pure corner is wrong at this scale, because a reject region large enough to matter for safety produces a review queue no team can staff. Decoupling is wrong because the threshold is exactly a downstream consumer of the confidence, and a decoupled score makes the threshold meaningless.

The design work is choosing ε and watching ρ. The band mass $\rho \cdot \varepsilon$ is the confident-error rate, and it moves when traffic shifts, for instance when a new class of borderline content appears and ρ rises even though ε is unchanged. The correct operational posture treats ρ as a monitored quantity and re-tunes ε when it drifts. A team that fixes ε once and forgets ρ has shipped a band policy that silently widens its error rate whenever the traffic distribution moves toward the boundary.

Case study: an agentic router consuming a safety score

A tool-using agent asks a safety classifier whether a proposed action is safe, and runs the tool when the score clears a bar. This is the confidence-consumed archetype in its purest form, and it is where the third corner does its worst damage.

Here the danger is not primarily the abstention or band choice, which follow the high-stakes or high-throughput reading of the underlying action. The danger is decoupling, because the router is a machine reading the safety score with no human to notice that the score has stopped meaning anything. Under the defense reading of Remark 7.11 the reflectivity premise is live, since the agent's own outputs become its next inputs and can describe and invert the classifier, so the diagonal applies directly and the residual surface is not a modeling idealization but a real attack target. The correct posture combines a shallow boundary, so that a score-boundary action is benign by construction, with architectural separation of the classifier from the action generator, so that the classifier does not lose capacity to the utility pipeline. Both are prescriptions the defense paper states, and both are ways of making the unavoidable coupled point land somewhere harmless.

Case study: retrieval-grounded question answering

A model answers questions only from a fixed corpus of verified documents, and is instructed to say "not found" when the corpus does not contain the answer. This case is worth including because it looks like it escapes the trilemma entirely, and understanding exactly how it does is instructive.

The escape route is the one named in the truth-boundary paper's discussion: a model that never answers strictly falsely breaks the coverage premise. If the system truly only asserts what the corpus supports and abstains otherwise, then it is never strictly wrong, coverage fails, and the impossibility does not apply. This is vacuous for a general-purpose assistant, because a general assistant is asked about everything and cannot restrict itself to a verified corpus. It is meaningful here, because the retrieval design deliberately shrinks the answer space to grounded content. So retrieval grounding is a real escape, and it is a form of the abstention corner: "not found" is the abstain outcome, and the corpus boundary is the reject region.

The escape is only as good as its premise, and the premise fails in two familiar ways. First, the model may answer confidently from parametric mem-

ory even when the corpus does not support the claim, which reinstates coverage and with it the boundary. Second, the truth field that decides "supported by the corpus" is itself estimated, usually by a retrieval-relevance score, and its agreement with actual support away from clear cases is an empirical premise exactly like the probe-validity premise of Chapter 4. So retrieval grounding does not repeal the theorem. It moves the whole problem to the corpus boundary and to the fidelity of the grounding check, which is often a better place to have the problem, because a corpus boundary is auditable and a semantic truth boundary is not. That relocation, and not any repeal, is what makes retrieval grounding a good design.

7.5 The one empirical unknown

Every cost model in this chapter contains a quantity the theory does not fix. For abstention it is how much traffic falls in the reject region. For the band it is ρ, the density of traffic near the boundary. For the geometry of Chapter 5 it is the measure concentrated near the decision surface. These are the same quantity under different names, and it is the single empirical unknown on which the felt severity of the trilemma depends.

Definition 7.16 (Boundary density). The *boundary density* of a deployment is the probability, under the real traffic distribution, that a query lands within a fixed truth-field distance of the model's truth boundary. It is a property of the model and the traffic jointly, not of either alone.

The theorems are silent on boundary density, and the earlier chapters are explicit about that silence. The diagonal argument produces a bad point and says nothing about how many there are or how often one is queried, a limit stated in Chapter 1. The topological argument is an existence statement, so nothing about error rates follows from it, a limit stated in the discussion of the truth-boundary paper. The theory guarantees the boundary exists and is unavoidable. It says nothing about whether your users ever go near it.

Proposition 7.17 (Two deployments, same model, different fate). *The felt cost of the trilemma is not determined by the model. Two deployments of an identical model, differing only in their traffic distribution, can have boundary densities that differ by orders of magnitude, and hence confident-error rates that differ by the same factor at fixed band width.*

Proof. The confident-error rate is about $\rho \cdot \varepsilon$ with ε fixed by the model and ρ fixed by the traffic. Vary the traffic and hold the model fixed, and ρ, hence the rate, varies freely. ■

Not formalized. The step is arithmetic on a product whose two factors this book fixes separately, one by the model and one by the deployment; the companion libraries state neither factor as a theorem.

Remark 7.16a (Benign and adversarial boundary density are different numbers). For the hallucination reading, boundary density is a property of benign traffic, and it drifts slowly with the user population. For the defense reading it is set by an adversary who is actively steering queries toward the

boundary, so the relevant number is not the density benign users produce but the density an attacker can induce. These can differ enormously. A deployment whose benign boundary density is a fraction of a percent may face an adversarial boundary density near one, because the attacker's entire job is to find and hit the coupled point. The hitting-time geometry of Chapter 5 (`MoF_Cost_03_HittingTime`) is the tool for the adversarial number, and the histogram protocol below is the tool for the benign one. A safety case that uses the benign number where the adversarial one is called for has measured the wrong quantity, and this substitution is a common and serious error in practice.

Proposition 7.17 is the reason benchmark numbers do not transfer to deployment. A benchmark is a traffic distribution, usually one enriched for hard cases, so its boundary density is not your deployment's boundary density. A model that hallucinates often on a benchmark of adversarial trivia may almost never approach its boundary on a narrow production workload, and a model that looks clean on an easy benchmark may sit on its boundary constantly under real users. The theorem is the same in both. The risk is not.

How the geometry of Chapter 5 would estimate it

Boundary density is measurable, and Chapter 5 gives the geometry that turns the measurement into a design number rather than a single observed rate. Three of its verified quantities matter here. Basin volume (`MoF_Cost_02_BasinVolume`) measures how much of the input space flows to each outcome, and the boundary density is governed by how much traffic mass sits in the thin shell between basins. Concentration of measure near the boundary (`MoF_Cost_04_Concentration`) tells you how sharply that shell is peaked, which sets how much error a reject region of given width removes. Hitting time (`MoF_Cost_03_HittingTime`) measures how many steps a walk needs to reach the boundary, which is the adversarial reading of the same shell: an attacker's cost to drive a query to a coupled point.

A measurement protocol follows from these. Sample queries from representative traffic, not from a benchmark. For each, estimate the realized truth field and its distance to the boundary, using the linear-probe geometry of Chapter 4 when a probe is available, since a nonzero linear probe has an affine hyperplane boundary and distance to it is a dot product. Histogram those distances. The mass near zero is the boundary density, the shape of the histogram near zero is the concentration, and the dimensional scaling of the shell (`MoF_09_DimensionalScaling`) tells you how the density will move as you change the interface dimension `d`. This gives you ρ, the concentration that sets your reject region, and a forecast of how both move under interface changes, which is more than a single measured error rate can give.

Remark 7.18. Measuring boundary density well is the open empirical problem the whole development points at, and it is genuinely hard for two reasons. Real traffic is not stationary, so ρ drifts, and the truth field is only estimated, usually by a probe whose agreement with semantic truth away from its fitted

samples is itself an empirical premise rather than a theorem. A deployment that takes the trilemma seriously invests in measuring ρ continuously and in validating the probe that defines the boundary, and treats both as running instruments rather than one-time calibrations. Everything else in this chapter is downstream of that number.

A worked estimation

Suppose you have a linear truth probe validated to agree with ground truth on held out cases at a rate you find acceptable, and a sample of one hundred thousand queries drawn from a week of representative traffic. For each query you compute the signed distance to the probe hyperplane, which by the hyperplane result of Chapter 4 is a normalized dot product, one multiply-accumulate per query, cheap at any scale. You histogram the absolute distances. Suppose the histogram shows five hundred queries within the band half-width ε you have chosen, so the raw boundary density estimate is `ρ = 0.005`. The shape near zero matters as much as the count: if the histogram is flat near zero, traffic is spread across the band and the concentration is low, so widening the band adds error slowly; if it spikes at zero, traffic piles on the boundary and small changes in ε move the error rate sharply. This is the concentration quantity `MoF_Cost_04_Concentration` read off an empirical histogram.

From `ρ = 0.005` and an in-band error fraction estimated by sampling and labeling a few dozen in-band queries, you get a confident-error rate for the band corner, and a refusal rate for the abstention corner if instead you reject the band. You now have both corners priced on real traffic, which is the input Proposition 7.15 needs. The dimensional scaling result `MoF_09_Dimensional Scaling` then lets you forecast how ρ moves if you change the prompt interface, for instance by constraining formats to reduce the effective dimension `d`, which the defense paper recommends for a different reason. The single measurement feeds every downstream design choice in this chapter.

A note on sample size, because boundary density is a small number and small numbers are hard to estimate. If the true density is around `0.005`, a sample of one hundred thousand queries yields about five hundred in-band observations, enough for a stable estimate, while a sample of one thousand yields about five, which is noise. The in-band error fraction is harder still, because it requires labeling in-band queries, and there are few of them by construction. The practical consequence is that boundary density measurement is data-hungry precisely where it matters most, near a thin boundary, so the estimation budget should be concentrated on in-band sampling, for instance by oversampling queries the probe places near the boundary and reweighting. A density estimate reported without its sample size and its in-band count is not yet a measurement you can build a safety case on.

Remark 7.18a (Drift is the operational reality). The estimate above is a snapshot, and the quantity it estimates moves. Traffic distributions shift with the season, the product surface, and the user population, and each shift can

move ρ without any change to the model. A boundary density measured once and trusted for a year is a latent incident. The correct treatment is a control chart: recompute ρ on a rolling window, alarm when it leaves a band, and re-tune ε or the reject region when it does. This turns the one empirical unknown from a number you guess once into an instrument you read continuously, which is the only honest way to run any of the three corners at scale.

Remark 7.18b (The probe is a premise, not a truth). Everything above rests on the probe's boundary being the truth boundary, and that identification is an empirical premise the theorems never grant. A probe defines a boundary in representation space; whether that boundary tracks semantic truth away from the probe's fitted samples is Problem 7.23, and the empirical work in the truth-boundary paper found probes ranging from near-perfect separation to barely above chance across five small models, with coupling appearing cleanly only where the separation premise actually held. So a boundary-density number is only as trustworthy as the probe that defines the boundary, and reporting ρ without reporting the probe's validation is reporting a measurement without its instrument's calibration.

7.6 What the theorems do and do not claim

An impossibility result is easy to overclaim and easy to dismiss, and both come from ignoring what it actually says. This section draws the line precisely, because the whole value of a machine-checked result is that the line is sharp.

What they claim. A precisely stated ideal is unreachable. Faithful and covering and calibrated at once, over a self-referential or connected domain, does not exist, and neither does utility-preserving and complete at once over a connected prompt space. The obstruction is structural, a consequence of self-reference or of the topology of the space, and not a defect of any particular model or training run. And it tightens as you approach the ideal: the closer you push toward exact separation, the smaller the slack you are forced to accept, with `truth_slack_must_be_positive` saying the slack cannot reach zero. Every one of these claims is checked by the Lean kernel, and the check is the content, since the mathematics is elementary and the risk was always in a hidden premise, which formalization removes.

Remark 7.18c (Structural, not a training defect). *The obstruction does not depend on how the model was trained, on its size, or on its architecture. It depends only on the stated premises: self-reference, or continuity on a connected domain, together with the three conditions. Hence no amount of training data, scale, or architectural change removes it while those premises hold.*

Discussion. This is what "structural" means and why it is a stronger statement than a statistical inevitability result. A statistical argument says a model trained under certain conditions will hallucinate at some rate, and a different training regime might change the rate. The structural argument says that as long as the model realizes the premises, no training regime reaches the ideal, because the ideal does not exist. The two kinds of result are complementary.

The statistical results cited in the truth-boundary paper's related work tell you about rates under training. The structural result tells you the target those rates approach is empty. A team that hopes to train its way to the ideal is refuted by the structural result; a team that wants to predict its error rate needs the statistical results and the boundary density of this chapter. ■

What they do not claim. They do not claim any particular deployed model hallucinates often. Existence is not frequency. The coupled point exists; whether your traffic visits it is boundary density, which the theorems do not fix (Definition 7.16). They do not claim a specific defense is broken in practice. The residual surface exists; whether it contains actual harm depends on whether the boundary is shallow, which is an engineering choice, not a theorem. And they do not claim the ideal target is the deployed model. The theorems are about an idealized object, a total faithful calibrated verdict, or an exactly separating confidence field, or a continuous utility-preserving complete wrapper. A deployed model approximates that object and is not identical to it.

Remark 7.19 (Existence is not frequency). *The impossibility theorems are existential. From "a coupled point exists" one cannot derive any statement of the form "coupled points are queried with probability at least p" for any $p > 0$ without an additional premise about the traffic distribution.*

Proof. The traffic distribution is a free parameter of the deployment and does not appear in the hypotheses of the theorems. A distribution supported away from the boundary assigns the coupled point probability zero while satisfying every hypothesis. Hence no positive frequency lower bound follows. ■

Proposition 7.20 (Idealized target, not deployed model). *The conditions faithfulness, calibration, exact separation, and completeness are properties of an idealized limit. A deployed model that satisfies them only approximately is not the object the theorems refute, and its behavior is governed by the approximate results of Chapter 6, not by the exact impossibility alone.*

Discussion. This is why the approximate bridges matter. The exact theorem says the ideal is empty. The approximate theorem, `two_slack_approx_coupling` and `exact_from_approx`, says the neighborhood of the ideal is where deployed models live and prices the slack they must carry. A critique that says "no real model is exactly calibrated, so the theorem is vacuous" is answered by the approximate results, which apply to the inexact models and recover the exact statement as a limit. ■

Proposition 7.20a (The impossibility tightens toward the ideal). *As a model is pushed toward exact faithfulness and exact calibration, the guarantee it can still satisfy tightens rather than loosens: the forced question's margin of correctness is bounded by the slack, and shrinking the slack shrinks that bound, with the exact contradiction of Chapter 3 as the $\varepsilon = 0$ limit.*

The verified content is Chapter 6's `c6_tightening` and `c6_margin_bound`, which say precisely this: the guaranteed interval at the smaller slack sits inside the one at the larger, and the margin at a forced question is bounded by the slack it came from.

Discussion. This is the engineering meaning of the positive-slack floor of Proposition 7.6a, restated as a claim about progress. It contradicts a natural intuition that a better model has more room, and it says the opposite: the room shrinks. A model far from the ideal has ample slack and feels no pressure from the theorem, while a model near the ideal has little slack and feels the theorem acutely. The trilemma is not a distant asymptotic curiosity that only matters in a limit no one reaches. It matters most exactly for the best systems, which are the ones close enough to the ideal for the floor to bind. ■

Remark 7.21 (The premises are where the argument belongs). An honest impossibility result moves the disagreement to the premises, and that is a feature. For hallucination the premise to argue about is whether faithful-plus-calibrated -plus-covering is really the right idealization of what you want, and whether your representation space is really self-referential or really connected. For defense the premise is whether your wrapper is really continuous and utility-preserving, since a discontinuous blocklist escapes the theorem by design. These are the right arguments to be having. The theorem does not settle them. It guarantees that once you grant the premises, the conclusion follows, so the entire remaining question is whether the premises hold in your case. That is exactly where a careful engineer wants the argument to be, and it is what machine-checking the deductions buys you.

7.7 An agenda of open problems

The development leaves a clear set of open problems, some empirical, some mathematical, some a mix. I list them as an agenda for anyone building on this work.

Problem 7.22 (Boundary density at scale). Measure boundary density on real production traffic for deployed models, across domains, and characterize its drift over time. This is the number every cost model in this chapter depends on, and it is essentially unmeasured in the field. A good answer is a methodology and a set of measurements, not a single number, since Proposition 7.17 says the number is deployment-specific.

Problem 7.23 (Probe validity away from fitted samples). The boundary is defined by a truth field, usually a linear probe, and the premise that the probe agrees with semantic truth away from its training samples is empirical, not proven. Give a method to certify or bound that agreement, so that a measured boundary density can be trusted as a truth boundary density rather than a probe-artifact density.

Problem 7.24 (Optimal region placement). Given a measured boundary density and a cost model, compute the optimal three-region blend of Proposition 7.10, the placement of the reject region, the band, and the trusted interior, that minimizes total cost. This is an optimization problem with the geometry of Chapter 5 as its constraint set, and it is not yet solved in general.

Problem 7.25 (Dynamic boundary density under adversaries). For the defense reading, boundary density is not fixed by benign traffic but driven

by an adversary who steers queries toward the boundary. Combine the hitting-time geometry (`MoF_Cost_03_HittingTime`) with an adversary model to predict the boundary density an attacker can induce, and price the defense against it. The stochastic and multi-turn results (`stochastic_dichotomy`, `multi_turn_history_dependent`) are the starting points.

Problem 7.26 (Calibration under decoupling pressure). Training objectives that reward accuracy do not price the confidence report, so they drift toward the decoupling corner for free (Proposition 7.8). Design a training objective or a regularizer that penalizes decoupling directly, and quantify the accuracy cost of maintaining calibration. This is the constructive counterpart to the warning of Section 2.

Problem 7.27 (The full symmetry theorem). The symmetry version of the boundary result replaces coverage with an antipodal condition and needs a vector Borsuk-Ulam statement for jointly odd axes, which is verified only in the scalar case and for nonlinear tangents so far. Completing the full theorem would remove the coverage premise entirely for symmetric representation spaces, which matters because it would apply to models that never answer strictly falsely.

Problem 7.28 (Cost asymmetry beyond the grid). The exponential defense cost `N^d` assumes exhaustive grid enumeration, and learning-based defenses that generalize across the space may sidestep it. Determine whether a continuous learned defense can beat the grid bound, or prove a matching lower bound that holds for learned defenses, which would settle whether the cost asymmetry is fundamental or an artifact of the enumeration model.

Problem 7.29 (J-space and the two engines at once). Chapter 8 asks what a system satisfying both the diagonal hypothesis and the continuum hypothesis would look like, a self-referential space that is also connected with continuous fields. Characterize the boundary object such a system produces, and determine whether the two engines give the same coupled point or two different ones. This is the most open of the problems and the one that would unify the book's two halves.

7.8 The chapter in one constraint

If you keep one thing from this chapter, keep the constraint and its consequence together. The constraint is that faithful, calibrated, and covering do not coexist over a self-referential or connected domain, and neither do utility-preserving and complete over a connected prompt space. The consequence is that you will give up one of the three, so give it up on purpose, price what you give up, and place the sacrifice where it hurts least.

The engineering that follows is not exotic. Choose a corner from the domain: abstain where a confident error dwarfs a refusal, price a confined error band where refusing constantly is worse than erring occasionally, and weaken calibration only where you have proven nothing downstream reads the confidence. Blend the corners across regions so no region carries all three condi-

tions. Then chase the one number the theory withholds, the boundary density on your real traffic, because every cost in the chapter is that number times something you control. Make the boundary shallow so the unavoidable coupled point is benign, keep the signal you publish calibrated so the systems that trust it are not betrayed, and monitor the boundary rather than pretending you eliminated it.

None of this is defeat. A theorem that closes off the impossible ideal is a gift to an engineer, because it ends a search that would otherwise never end and redirects the effort to choices that can actually be made well. The trilemma does not tell you that safe, useful AI is impossible. It tells you exactly which single perfection is unavailable, and it leaves the whole space of good-enough systems open, which is the space you were going to build in anyway. The contribution of the machine-checked development is that the boundary between the impossible ideal and the achievable neighborhood is now drawn with a precision you can trust, so the argument moves to the premises, which is where an honest argument belongs.

7.9 Bibliographic notes

The engineering prescription for defense, make the boundary shallow, reduce the Lipschitz constant, reduce dimension, monitor rather than eliminate, and separate the defense from the utility pipeline, is from the defense-impossibility paper's prescription section, and the GPT-5-Mini shallow-boundary example is from its experiments. The confident-error band and its positive-slack floor are the approximate coupling results of the truth-boundary paper, verified as `two_slack_approx_coupling` and `truth_slack_must_be_positive`. The reading of refusal training as connectedness surgery, and of abstention as a cut that disconnects the true region from the false one, is from that paper's discussion. The statistical inevitability of hallucination for calibrated models is a different mechanism from the one here, and the contrast is drawn in the truth-boundary paper's extended discussion. The geometry that would estimate boundary density is the companion attack-geometry development surveyed in Chapter 5.

The organization of the three trilemmata as one proof term, and the master Utility-Control-Transparency framing with its three impossible corners, is Chapter 3 and the companion `CCHProofs` development. The two-slack approximate coupling and the strictly positive slack floor are the results of Chapter 6, and the exact statement they generalize is Chapter 4's coupling theorem. Readers who want the empirical picture that motivates the boundary-density discussion should read the truth-boundary paper's experiments, where the coupling appears cleanly in the models whose probes separate truth well and fails to appear where the separation premise breaks, which is the empirical shadow of the theory's insistence that its premises do the work.

7.10 Exercises

Exercise 7.1. Take `c7_flip` and the two-valued negation of Chapter 1. Explain, in terms of `lawvere` and `bool_not_fpf`, why adding the `abstain` outcome converts the diagonal from a contradiction into a consistent value, and identify that value. State in one sentence what this means for a model allowed to refuse.

Exercise 7.2. For a high-stakes domain with `Cerr = 1000 · Cref`, and a confident-error band policy with band mass `ρ · ε`, find the abstention rate `a` at which the abstention cost `a · Cref` equals the band cost `ρ · ε · Cerr`. For a boundary density `ρ = 0.01` and band half-width giving ε such that `ρ · ε = 0.005`, which corner is cheaper? Redo the comparison for `Cerr = 2 · Cref`.

Exercise 7.3. Prove the analog of Proposition 7.8 for defense: a safety classifier's decoupling of its reported score from actual safety does not change its top-1 safe-or-unsafe accuracy. Then explain why this makes the decoupling corner attractive to a team optimizing classifier accuracy, and dangerous to the router that consumes the score.

Exercise 7.4. A deployment measures boundary density `ρ = 0.002` on last quarter's traffic and ships a band policy with fixed ε. This quarter a new class of borderline queries appears and `ρ` rises to `0.02`. By what factor does the confident-error rate change, with ε held fixed? Design a monitor that would have caught this before the error rate moved, and state what it measures.

Exercise 7.5. (Design.) You are building clinical decision support that feeds an automated specialist-routing queue. Specify a three-region blend per Proposition 7.10: define the reject region, the band, and the trusted interior in terms of truth-field distance, say which of the three trilemma conditions each region gives up, and state the one empirical quantity you must measure before you can place the region boundaries.

Exercise 7.6. (Design.) Repeat Exercise 7.5 for platform-scale content moderation. Justify why the reject region is much smaller here than in the clinical case, in terms of the high-throughput archetype of Definition 7.13, and identify which downstream consumer forces you to keep calibration.

Exercise 7.7. Explain why a discontinuous blocklist escapes the Defense Trilemma, referring to the continuity and utility-preservation premises. Then explain what the blocklist pays instead, and match that cost to one of the three corners of this chapter. Is a blocklist an abstention policy in disguise? Argue both sides.

Exercise 7.8. (Short essay.) The chapter claims decoupling is the most dangerous corner because it fails invisibly and downstream. Write half a page arguing the opposite: describe a realistic deployment in which the decoupling corner is the correct choice, and state precisely the condition on downstream consumers that makes it safe.

Exercise 7.9. Using Proposition 7.17, construct two traffic distributions over a one-dimensional query line with a truth boundary at the origin, one with high boundary density and one with near-zero boundary density, such that

the same model has a confident-error rate differing by a factor of a hundred. Explain why no benchmark can determine which distribution a deployment actually faces.

Exercise 7.10. The measurement protocol of Section 5 histograms truth-field distances on representative traffic. Suppose your truth field is a nonzero linear probe. Using the hyperplane boundary result of Chapter 4, write the distance from a query embedding to the boundary as an explicit formula, and explain why this makes the histogram cheap to compute at deployment scale.

Exercise 7.11. (Harder.) The adversarial reading of boundary density (Problem 7.25) treats an attacker as driving queries toward the boundary. Sketch how the hitting-time quantity `MoF_Cost_03_HittingTime` bounds the number of steps an attacker needs, and argue why reducing the Lipschitz constant of the alignment surface raises that cost. What does this say about the tradeoff between a smooth boundary and a monitorable one?

Exercise 7.12. (Short essay.) Remark 7.21 says an honest impossibility result moves the disagreement to the premises. Pick either the hallucination or the defense reading, name the single premise you find least defensible in your own application, and write half a page on what evidence would make you grant or reject it. Do not argue about the theorem; argue about the premise.

Exercise 7.13. (Open-ended.) Proposition 7.15 gives a default corner per archetype but assumes the cost models of Section 2 are the right accounting. Propose a domain where the cost models fail, meaning where the utility-maximizing corner is not the one the proposition predicts, and identify which assumption of the cost model your domain violates.

Exercise 7.14. (Open-ended, connects to Chapter 8.) Suppose a future system satisfied both the reflectivity premise of the diagonal and the connectedness premise of the continuum at once (Problem 7.29). Speculate on whether such a system would have one boundary object or two, and on what a corner policy would even mean when both engines force a coupled point. State what you would need to measure to tell the two boundary objects apart.

8 J-Space, and Why It Does Not Escape

Abstract. The earlier theorems concern a system's outputs, and one objection is that a probe reading the middle of a network is a different kind of object. This chapter takes the objection in its strongest current form, J-space, a method that recovers a transformer's mid-process representation from the derivative of its computation. It shows that richness of the readout is the surjectivity hypothesis of the diagonal, that steering does not escape either, and that abstention is the one real exit and is a relaxation.

Keywords: Interpretability, J-space, Probes, Self-verdict, Abstention

Everything in this book so far has been stated about a system's *outputs*. A verdict `v p q`, a safety score, a truth probe: each reads the end of a computation and returns a commitment. There is a natural objection to reading only the ends. A model's output is a summary, and summaries throw away structure. The interesting part of a large network is in the middle, in the activations that carry a half-formed answer before the answer is spoken. A probe that reads the middle, the objection goes, is a different kind of object from a probe that reads the end, and the impossibility theorems of the earlier chapters, which were proved against output verdicts, might simply not reach it.

This chapter takes that objection in its strongest current form. The form is the *J-space* construction, an interpretability method from 2026 that recovers a transformer's private mid-process representation from the derivative of its computation rather than from the activations themselves. J-space is the most serious recent candidate for a place where you could read a model's mind, and if any internal representation escaped the diagonal, this is the one that should. The result of the chapter is that it does not. Both engines of the book, the diagonal of Chapter 1 and the intermediate value theorem of Chapter 4, reach J-space, and they reach it for reasons that are not accidents of how J-space happens to be built but consequences of what it is. A probe that reads the middle gets more resolution than a probe that reads the end. It does not get immunity.

The plan is as follows. First we set down the construction precisely, with numbered definitions of the transport map, the J-lens, J-space, and the local J-coordinates, so that later claims have something exact to attach to. Then we

isolate the two structural facts that do all the work: J-space is a Jacobian image and therefore a linear, connected tangent space, and the readout is computed from the very system it reports on. The first fact hands J-space to the analytic engine, the second to the diagonal engine. We run both engines, keep the live theorems `jspace_factoring` and `jspace_liar` at the center, and build the surrounding results in core Lean around them. We then show that the two engines are not merely both applicable but are two readings of one equation, with the Jacobian playing the role of the linearization and the liar activation playing the role of the fixed point. We close with what this means for interpretability in practice, and with a careful statement of what the chapter does not claim.

8.1 What J-space is

Fix a trained transformer. Write `h_{ℓ,t}` for the residual-stream activation at layer ℓ and token position `t`, and write `h_final` for the final-layer activation that the unembedding reads to produce logits. The ordinary interpretability picture, the one behind sparse autoencoders and the logit lens, reads structure off the activations `h_{ℓ,t}` directly: it looks for directions in activation space and treats features as sparse combinations of those directions. J-space reads structure off the *sensitivity* of the computation instead. It asks not what the activation is at a point but how the rest of the computation would move if that activation moved.

The motivation is that the derivative carries information the point does not. Two prompts can drive the model to nearly the same activation at some middle layer and still be processed in completely different ways downstream, because what a network *does* at a point is a property of the local behavior of the map there, not of the coordinates of the point. Reading the derivative surfaces the mid-process computation, the leaning toward a future token that has not yet been written into any residual-stream direction. That is the empirical bet of the construction, and it is a bet about representation, not about limits.

It helps to place J-space among the interpretability methods it grew out of. The logit lens applies the unembedding `W_U` directly to a middle-layer activation and asks what token that activation would predict if the layers above it were skipped. It is cheap and often revealing, but it reads the activation as if it were already a final state, and so it sees the model's current guess rather than the computation still to come. Sparse autoencoders go the other way: they fit an overcomplete dictionary to the activations themselves and decompose each activation into a sparse sum of learned features. That recovers structure the raw coordinates hide, but it is still structure of the point, of where the model is, not of what it is about to do. J-space sits at a third position. It reads neither the activation nor a learned dictionary over activations, but the linear map that says how the activation influences the future of the computation. The dictionary of J-coordinates in Definition 8.4 is a sparse dictionary again, but it is fit over the transported directions, over columns of `Jℓ`, not over activa-

tions. The family resemblance to sparse autoencoders is real, and it is worth keeping in view, because it means the diagonal and analytic arguments of this chapter are not special to J-space. They apply to any of these methods the moment the method is asked to be a total exact verdict over a self-describing query space.

The averaging in Definition 8.1 is not incidental bookkeeping. Restricting the expectation to future positions `t' ≥ t` is what makes the transport map read a *leaning*, a commitment the model is building toward but has not yet emitted. A derivative that also averaged over past positions would mix the model's memory of what it has already said into its account of what it is about to say, and the readout would lose the predictive, forward-looking character that motivates the whole construction. The causal restriction to the future is the technical device that turns a generic sensitivity matrix into a workspace readout. This matters for the arguments below only through its consequences, that the readout is a linear functional of the model's own map, but it is worth understanding why the construction is built the way it is before we start extracting what it cannot do.

Definition 8.1 (Transport map). Let the model define, for each middle layer ℓ, the map from the layer-ℓ activation at position `t` to the final-layer activation at a later position `t' ≥ t`. The *Jacobian transport map* at layer ℓ is the expected derivative of the later activation with respect to the earlier one,

```
Jℓ = E[ ∂ h_{final, t'} / ∂ h_{ℓ, t} ],
```

where the expectation averages over positions `t' ≥ t` and over a reference distribution of inputs. Concretely `Jℓ` is a matrix: a linear map from the activation space at layer ℓ into the final-layer activation space. It records, on average, how a small push to the layer-ℓ state at the current position propagates forward to the model's later commitments.

Two things about Definition 8.1 will matter more than the details of the average. `Jℓ` is a Jacobian, so it is a derivative, so it is linear. And `Jℓ` is assembled entirely out of the model's own weights and its own activations, since a Jacobian of the model's forward map is a functional of that map. Both points return below with force.

Definition 8.2 (J-lens readout). The *J-lens* is the readout obtained by sending an activation through the transport map, normalizing, and unembedding:

```
lens(h) = softmax( W_U · norm( Jℓ h ) ),
```

where `W_U` is the model's unembedding matrix and `norm` is the model's final-layer normalization. The J-lens returns a distribution over the vocabulary. Read it as the model's silent guess about what it is leaning toward emitting at future positions, given the current mid-layer state `h`. Where the ordinary logit lens applies `W_U` to the activation itself, the J-lens applies it to the

transported activation `Jℓ h`, which is what makes it a readout of mid-process sensitivity rather than of the surface state.

The J-lens returns a distribution over the vocabulary, but the impossibility results of this book are stated about Boolean verdicts, so we need the step that turns a distribution into a commitment. That step is already implicit in how the readout is used.

Definition 8.2b (Truth-workspace reading). Fix a decodable proposition associated with a query `q`, for instance "the answer to `q` is yes," and a token or set of tokens that expresses it. The *truth-workspace verdict* of an activation `h` on query `q` is `true` when the J-lens distribution `lens(h)` puts more mass on the affirming tokens than on the denying ones, and `false` otherwise. Writing `read(J-point)(q)` for this thresholded reading gives a map of the shape `J → Q → Bool`, which is the object the diagonal argument takes as input. Nothing in the argument depends on the particular thresholding rule; it depends only on the fact that a distribution over tokens is eventually resolved into a commitment, which is what any use of the readout as a truth signal must do.

This is where the two engines part company at the level of the readout, and it is worth marking the fork now. The analytic engine of Chapter 4 works on the distribution *before* it is thresholded, on the continuous score, and its whole force comes from the score varying continuously. The diagonal engine works *after* thresholding, on the Boolean commitment, and it does not care how the commitment was reached. The same J-lens feeds both, which is the first sign that a J-space probe will not slip between the two arguments.

Definition 8.3 (J-space). *J-space* is the image of the transport map,

```
J = { Jℓ h : h ∈ activation space } = im(Jℓ),
```

the subspace of the final-layer activation space spanned by the transported directions. Because `Jℓ` is linear, `J` is a linear subspace. Empirically it is small, a few dozen effective dimensions, well under a tenth of the activity of the layers it summarizes, and yet it is read and written far out of proportion to its size. Ablating it, projecting activations onto its orthogonal complement, collapses the model's multi-step reasoning while leaving single-step lookups largely intact. That combination, tiny yet load-bearing, is why the construction was proposed as a candidate global workspace: a low-dimensional channel through which the model routes the intermediate results of chained computation.

Definition 8.4 (J-coordinates). For a particular activation `h`, its *J-space component* is the projection of `Jℓ h` onto `J`, and this component is approximated as a sparse nonnegative mixture of a fixed dictionary of Jacobian directions `d_1, …, d_m`,

```
proj_J(Jℓ h) ≈ Σ_i w_i(h) · d_i,
w_i(h) ≥ 0,  most w_i(h) = 0.
```

The mixture weights `w_i(h)` are the *local J-coordinates* of `h`. They are the interpretable handle: each active direction `d_i` names a mid-process concept, and the weight `w_i(h)` says how strongly the current state is leaning on that concept. Steering the model amounts to editing these coordinates and pushing the edited component back into the residual stream.

We can make the coordinate object concrete enough to point a theorem at. In core Lean, with no vector space in scope, we model a J-coordinate as the finite record of which directions are active and with what nonnegative weight, using natural numbers as a stand-in for the nonnegative reals.

```
/-- A local J-coordinate: a finite sparse nonnegative
    mixture of dictionary directions drawn from `Dir`,
    with `Nat` standing in for the nonnegative real
    weights. The interpretability content is the list
    of active (direction, weight) pairs; the ambient
    vector space is abstracted away. -/
structure c8_Coord (Dir : Type _) where
  terms : List (Dir × Nat)
```

The `terms` list is the sparse support with its weights. Nothing in this book depends on the arithmetic of the weights, so integers suffice to carry the shape of Definition 8.4 into the code. We use `c8_Coord` below to state, against the literal coordinate object rather than an abstract stand-in, that the diagonal still reaches it.

Remark 8.4a (Nonnegativity and the cone). The nonnegativity constraint on the weights `w_i(h) ≥ 0` is doing interpretability work, not mathematical work. It is what makes the active directions read as concepts that are present rather than concepts that are present-or-anti-present, and it is what lets a J-coordinate be displayed as a short list of leanings with magnitudes. For the analytic engine later it has one consequence worth flagging in advance: the set of realizable J-coordinates is a convex cone, closed under nonnegative combinations, and a convex cone is convex, hence connected. So even the sparse, nonnegative, constrained coordinate object, the most restricted version of J-space anyone actually reads, is still a connected domain. The constraint that makes the readout legible does not buy disconnection, and disconnection is the only thing that would keep the intermediate value theorem out.

Example 8.5 (A one-layer linearization). Take a network whose middle-to-final map is exactly affine, `h_final = A h_ℓ + b`. Then the derivative is constant, `∂ h_final / ∂ h_ℓ = A`, the expectation in Definition 8.1 collapses to `Jℓ = A`, and J-space is just `im(A)`. Here the J-lens `softmax(W_U · norm(A h))` is a fixed linear readout, and the local J-coordinates of any `h` are the coefficients of `A h` in whatever dictionary spans `im(A)`. This degenerate case is worth holding in mind because everything structural about J-space is already visible in it: the space is a linear image, the readout is a function of the same `A` that defines the model, and there is nothing in the middle that is not determined by the map. Real networks are not affine, so `Jℓ` is a genuine average

of varying derivatives, but the average is still a single linear map, and the two structural facts survive the averaging unchanged.

Example 8.5b (Where the derivative separates what the activation confuses). Take a middle-to-final map with a saturating nonlinearity, say `h_final = σ(A h_ℓ)` with σ a coordinatewise squashing function. Two activations `h` and `h'` can land at nearly the same output, `σ(A h) ≈ σ(A h')`, while sitting on opposite sides of σ's bend, one in the linear regime and one deep in saturation. The activations agree and the outputs agree, yet the derivatives disagree sharply: where σ has saturated, the local Jacobian is nearly zero, and where it is linear, the Jacobian is nearly `A`. A logit lens reading either the activation or the output would call these two states the same. The J-lens, reading `Jℓ`, separates them, because it reads how the state will move the future rather than where the state is. This is the concrete sense in which J-space "sees mid-process thinking": the thing it reads, sensitivity, is exactly the thing that differs between states the surface representation identifies. It is also, read the other way, exactly why the impossibility bites. The extra separating power is real resolution, and the diagonal will construct its liar out of precisely this finer vocabulary, not in spite of it.

Remark 8.6 (What is empirical here). Definitions 8.1 through 8.4 are a construction, and the claim that the construction recovers interpretable mid-process features is an empirical finding about particular trained models. No theorem in this book predicts that finding, and none is needed for what follows. The chapter takes the construction as given and asks what it inherits. When we say J-space is read and written out of proportion to its size, or that ablating it collapses reasoning, those are reports of experiments, cited so the reader knows which parts of the story rest on measurement and which on proof. The proofs begin at the next section.

8.2 Two structural facts

The construction has many moving parts, but only two of its properties drive the impossibility results, and both are forced by Definition 8.1 rather than added by hand.

J-space is a tangent space, hence linear and connected `Jℓ` is a Jacobian. A Jacobian is the linear part of the first-order Taylor expansion of a map at a point, that is, a 1-jet. The image of a linear map is a linear subspace, so `J = im(Jℓ)` is a real vector space. A real vector space is connected, in fact path-connected and even convex: any two points `u` and `v` of `J` are joined by the straight segment `(1 - s) u + s v` for `s ∈ [0,1]`, which lies entirely in `J`. This is not a property J-space might have depending on the model. It is what "image of a derivative" means.

The readout is a function of the system it describes `Jℓ` is built from the model's own forward map, so the J-lens is a functional of the model.

> When a J-space probe reports something about the model, it is a function of the thing it is reporting on. The activation `h` that the probe reads is produced by the same network whose behavior the probe is trying to summarize, and the transport map that turns `h` into a verdict is the derivative of that same network. The probe is inside the system, not outside it looking in.

These are the two hypotheses the two engines need. The first fact is exactly the connectedness hypothesis of the intermediate value theorem: Chapter 4 needs a connected domain and a continuous score, and J-space hands over connectedness for free because it is a linear span. The second fact is exactly the self-reference hypothesis of the diagonal: Chapter 1 needs a system that can range over verdicts about itself, and a readout computed from the model it reports on is a system of that kind once the model is rich enough to pose the relevant queries. We take the diagonal engine first, because its statement is the cleanest, then the analytic engine, then show they are one argument seen twice.

It is worth stating the first fact with the care it deserves, because the whole analytic half of the chapter rests on it and it is easy to wave through. The relevant notion is the 1-jet.

Definition 8.7a (1-jet). The *1-jet* of a smooth map `F` at a point `x` is the pair consisting of the value `F(x)` and the derivative `DF(x)`, that is, the first-order Taylor data of `F` at `x`. Discarding the value and keeping only the linear part `DF(x)` gives a linear map between tangent spaces. The transport map `Jℓ` of Definition 8.1 is, up to the averaging, the linear part of the 1-jet of the model's middle-to-final map, and J-space is the image of that linear part.

Proposition 8.7b (J-space is convex, hence connected). *J-space is a convex subset of the final-layer activation space: if `u` and `v` lie in `J` and `s ∈ [0,1]`, then `(1 - s) u + s v` lies in `J`. In particular `J` is path-connected.*

Definition 8.3 makes J-space the range of the linear transport map, so both halves of the proposition are statements about the image of a linear map and elaborate here directly.

```
theorem c8_jspace_convex {E F : Type _}
    [AddCommGroup E] [Module ℝ E] [AddCommGroup F]
    [Module ℝ F]
    (J : E →ₗ[ℝ] F) : Convex ℝ (Set.range J) := by
  rw [← Set.image_univ]
  exact convex_univ.linear_image J

theorem c8_jspace_isPreconnected {E F : Type _}
    [AddCommGroup E] [Module ℝ E]
    [AddCommGroup F] [Module ℝ F] [TopologicalSpace F]
    [IsTopologicalAddGroup F] [ContinuousSMul ℝ F]
    (J : E →ₗ[ℝ] F) : IsPreconnected (Set.range J) :=
  (c8_jspace_convex J).isPreconnected
```

Proof. The range is the image of the whole space, the image of a convex set under a linear map is convex, and a convex set is preconnected. ■

Note how little the second theorem needs. It does not ask that `J` be a Jacobian, that `F` be finite-dimensional, or that the dictionary of Definition 8.4 exist. Linearity alone delivers the hypothesis the intermediate value argument of the later section consumes, which is exactly the point of the section: the property that makes J-space tractable is the property that guarantees the band.

Proof. Since `J = im(Jℓ)`, write `u = Jℓ a` and `v = Jℓ b`. Linearity of `Jℓ` gives `(1 - s) u + s v = Jℓ((1 - s) a + s b)`, which is again in the image, so `J` is convex. Convex subsets of a real vector space are path-connected, the straight segment being a path, so `J` is path-connected. ■

Proposition 8.7b is the entire input the analytic engine needs, and its proof used nothing but linearity of `Jℓ`. There is no way to have J-space and not have this property, because J-space is defined as the image of a linear map, the image of a linear map is a subspace, and a subspace is convex. An interpretability method could in principle produce a disconnected internal representation, a discrete codebook say, and such a method would dodge the analytic engine, though not the diagonal. J-space is not such a method. Its defining move, reading the Jacobian, is exactly the move that guarantees connectedness.

Proposition 8.7c (The readout is a functional of the model). *The J-lens is determined by the model's forward map: two models with the same middle-to-final map induce the same transport map, the same J-space, and the same readout on every activation.*

Proof. `Jℓ` is defined as an expectation of derivatives of the model's own forward map, `norm` and `W_U` are parameters of the same model, and `lens` is their composite. Everything on the right-hand side of Definitions 8.1 and 8.2 is a function of the model. So the readout is a function of the model, and equal models give equal readouts. ■

Proposition 8.7c is the input the diagonal engine needs, and it is why a J-space probe is a self-verdict rather than an external audit. The probe does not stand outside the model reporting on it. It is computed from the model, applied to states the model produces, about queries the model can be driven to represent. The loop from the model, through the derivative of the model, to a verdict read back into the model's own query space is closed, and a closed loop of that shape is precisely what `no_reflective_verdict` is a statement about.

Remark 8.7 (Why the derivative does not buy an escape). It is tempting to think that moving from the activation to its derivative changes the game, since the derivative is a genuinely different object with genuinely more information. For the analytic engine the move actually makes things worse, not better, because it produces a linear space, and linear spaces are the ideal domain for the intermediate value theorem. For the diagonal engine the move is invisible, because the diagonal never inspects how a verdict is computed. It only uses that queries can range over verdicts. Reading a derivative instead of a value is a change in how the verdict is computed, and the diagonal does not look there.

8.3 The diagonal engine: no intermediate space escapes

Suppose the probe is not applied to the model's outputs but to an encoding of its internals. Write `enc : Q → J` for the map that sends a query to whatever J-space representation the interpretability method extracts while the model processes it, and `read : J → Q → Bool` for the verdict the probe returns about a query given that representation. The composite `fun p => read (enc p)` is a verdict on queries that happens to be routed through `J`. The routing is the whole content of the objection this chapter answers: the claim is that by passing through the rich internal space `J`, the probe sees more than an output-only verdict can, and so might avoid the trilemma.

The hypothesis that makes J-space attractive is that it is *rich*, that it exposes distinctions the residual stream hides. Pushed to its limit, richness is exactly the point-surjectivity of Chapter 1: every pattern of verdicts over queries is realized by the J-space representation of some query. Under that hypothesis the routed verdict is a reflective verdict in the precise sense of Definition 1.15, and Chapter 1 already ruled those out.

Richness is the surjectivity hypothesis

The identification of "rich" with "point-surjective" is the load-bearing step, so it deserves a slow reading. The selling point of J-space is expressiveness. Its advocates argue that the residual stream flattens distinctions the model actually makes, and that reading the Jacobian recovers them, so that the J-space representation of a query carries strictly more information about how the model treats that query than the output does. Take that argument at its word and push it to the limit. If J-space really does capture every distinction the model draws among queries, then in particular it captures, for any pattern of verdicts you might name, a query whose J-space signature realizes that pattern, because a model that could not represent some verdict pattern would be drawing no distinction there and would be, by its own advocates' standard, not fully expressive on that pattern. The stronger the claim of richness, the closer it comes to saying that the map from queries to realized verdict patterns is onto. Full richness is onto exactly, and onto is the surjectivity hypothesis `∀ g, ∃ p, read (enc p) = g`.

This is why the diagonal is not an external imposition on the interpretability program but its internal shadow. The program wants the representation to be as expressive as possible. The diagonal says that expressiveness, taken to the ideal, is self-defeating for the specific goal of being a total exact truth store, because a representation expressive enough to name every verdict pattern names the one pattern that has no consistent verdict. There is no setting of the dial that gives both maximal expressiveness and completeness. Turn expressiveness up and you approach surjectivity, which is the hypothesis of the impossibility. Turn it down and you give up the distinctions that made J-space attractive in the first place. The tension is not between J-space and the theorem. It is inside the notion of a maximally expressive internal readout.

Notice also what the hypothesis does not require. It does not require that the model be conscious of its own readout, or that the encoder be surjective as a map into `J`, or that `read` be computable, or that the representation be faithful in any metric sense. It requires only that the *composite* `read ∘ enc` hit every element of `Q → Bool`. That is a far weaker and far more plausible demand than "the probe is a good probe," and it is all the diagonal consumes. A probe can be mediocre on most queries and still satisfy the hypothesis, and a probe can be excellent on most queries and satisfy it too. Quality and surjectivity are close to orthogonal, which is why the theorem says nothing about how good the probe is and everything about what completeness would cost.

Theorem 8.8 (Factoring through J-space does not help). *Let `enc : Q → J` and `read : J → Q → Bool`. If every pattern of verdicts over `Q` is realized by some query's J-space representation, then `False`.*

```
theorem jspace_factoring {Q J : Type _}
    (enc : Q → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    False :=
  no_reflective_verdict (fun p => read (enc p)) hs
```

Proof. The composite `fun p => read (enc p)` has type `Q → Q → Bool`, and the hypothesis `hs` says it is point-surjective. That is precisely the definition of a reflective verdict, and `no_reflective_verdict` says none exists. ■

The proof is one line because there is nothing left to prove after the earlier chapters. The content is entirely in what the statement quantifies over. The type `J` is *arbitrary*. It may be vastly larger than the activation space, of any cardinality, carrying any algebraic or geometric structure whatever. The theorem does not constrain it and does not need to. Interposing a representation between the system and the verdict cannot defeat the diagonal, because the diagonal never examined how the verdict was computed. It used only the bare fact that queries can range over the verdicts the probe would return. J-space, however rich, is still downstream of a query and upstream of a Boolean, and that is all the argument requires.

The witness is explicit, and it is a J-space object.

Theorem 8.9 (The liar in J-space). *Under the hypotheses of Theorem 8.8 there is a query `p` whose routed verdict on itself equals its own negation.*

```
theorem jspace_liar {Q J : Type _}
    (enc : Q → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    ∃ p, read (enc p) p = !(read (enc p) p) :=
  match hs (fun q => !(read (enc q) q)) with
  | ⟨p, hp⟩ => ⟨p, congrFun hp p⟩
```

Proof. Apply richness to the diagonal pattern `fun q => !(read (enc q) q)`, which flips the routed self-verdict of every query. Some query `p` realizes

it, so `read (enc p) = fun q => !(read (enc q) q)`. Evaluate both sides at `p`: `read (enc p) p = !(read (enc p) p)`. ■

Read `p` as the query whose J-space signature is precisely "the probe will get me wrong." Chapter 1's a_0 from `liar_query`, Chapter 3's adversarial injection, and this `p` are one construction under three names. What J-space adds is the twist that makes the result sharp for interpretability. The liar is now built out of the probe's own internal vocabulary. It is not a query the probe fails to see. It is a query the probe sees correctly, whose correct reading is that the probe's verdict on it is wrong. Better internal visibility does not dissolve the liar. It hands the liar a clearer mirror.

Remark 8.9a (Size is not the variable). A recurring hope is that the escape lies in dimension: J-space is small, so maybe a *larger* internal space would have room to represent the liar honestly, or maybe a small space is safe because it cannot hold enough patterns. Both readings misplace the variable. Theorem 8.8 quantifies over `J` with no size hypothesis at all, so the argument is identical whether `J` is a two-dimensional codebook or a space larger than the activation stream. What controls the conclusion is not the size of `J` but whether the routed verdict is surjective onto `Q → Bool`, and surjectivity is a statement about the query space `Q` and the composite `read ∘ enc`, not about the width of the channel in the middle. A wider `J` makes surjectivity easier to achieve, if anything, so more room helps the diagonal, not the probe. The empirical smallness of J-space is a fact about efficiency and about what the model chooses to route through the workspace. It is not a defense, and it is not a vulnerability. It is orthogonal to the impossibility.

Example 8.9b (A finite probe, to see the mechanism). Let `Q` be the two-query space $\{q_0, q_1\}$ and suppose, for illustration, that the probe's routed verdict were surjective onto the four functions `Q → Bool`. Surjectivity would in particular hit the diagonal pattern `d(q) = !(read (enc q) q)`. Say `d` is realized by q_0, so `read (enc q₀) = d`. Evaluate at q_0: `read (enc q₀) q₀ = d(q₀) = !(read (enc q₀) q₀)`, a Boolean equal to its own negation, which is impossible. The finite case makes the mechanism visible: it is the single evaluation at the realizing index that produces the contradiction, and it is the same evaluation `congrFun hp p` that `jspace_liar` performs. Of course no map from a two-element set can actually be surjective onto a four-element set, which is why the finite version refutes the *hypothesis* rather than deriving a falsehood. That is the honest content for finite models: a finite probe simply cannot realize every pattern, so there is no contradiction, only a richness hypothesis that fails. How far a finite probe must fall short of the diagonal pattern in particular, as a function of its coverage, the diagonal engine does not say; it is not what Chapter 6's real-valued slack results bound, and that finite approximation question is open.

The pipeline, not just the encoder

The real J-lens is not a single map `enc` but a pipeline: a query drives the model to an activation, the transport map carries the activation into J-space, and the

readout turns the J-space point into a commitment. It is worth checking that composing the stages changes nothing, because a reader might suspect the one-map abstraction of Theorem 8.8 hides where the escape lives.

Theorem 8.10 (The full pipeline factors). *Let* `enc : Q → A` *send a query to an activation,* `transport : A → J` *be the Jacobian transport map, and* `readout : J → Q → Bool` *be the J-lens verdict. If the pipeline realizes every verdict pattern, then* `False`*, and there is a coupled query.*

```
theorem c8_pipeline_factoring {Q A J : Type _}
    (enc : Q → A) (transport : A → J)
    (readout : J → Q → Bool)
    (hs : ∀ g : Q → Bool,
      ∃ p, readout (transport (enc p)) = g) :
    False :=
  no_reflective_verdict
      (fun p => readout (transport (enc p))) hs

theorem c8_pipeline_liar {Q A J : Type _}
    (enc : Q → A) (transport : A → J)
    (readout : J → Q → Bool)
    (hs : ∀ g : Q → Bool,
      ∃ p, readout (transport (enc p)) = g) :
    ∃ p, readout (transport (enc p)) p
      = !(readout (transport (enc p)) p) :=
  liar_query (fun p => readout (transport (enc p))) hs
```

Proof. Composition of functions is associative, so the three-stage pipeline `fun p => readout (transport (enc p))` is again a single map `Q → Q → Bool`, and Theorems 8.8 and 8.9 apply to it verbatim. The proof term for the liar is `liar_query` applied to the composite. ■

The stages could be any number. A tower of intermediate representations, an activation feeding a J-space point feeding a second derived space feeding a readout, still collapses to one map from queries to verdicts, because function composition does not care how many links it chains.

Theorem 8.11 (Any tower collapses). *Let* `e₁ : Q → J₁`, `e₂ : J₁ → J₂`, *and* `read : J₂ → Q → Bool`*. If the two-stage encoding realizes every verdict pattern, then* `False`*.*

```
theorem c8_tower_factoring {Q J₁ J₂ : Type _}
    (e₁ : Q → J₁) (e₂ : J₁ → J₂) (read : J₂ → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (e₂ (e₁ p)) = g) :
    False :=
  no_reflective_verdict (fun p => read (e₂ (e₁ p))) hs
```

Proof. Same as Theorem 8.10 with one fewer stage named, and the point is that naming more stages never changes the type of the composite. ■

The design lesson is blunt. You cannot escape the diagonal by adding layers of interpretation between the query and the verdict, no matter how much structure each layer has or how faithfully it reflects the model, because the composite of all the layers is a map of the same shape the diagonal was proved against. Depth of interpretation is not distance from the boundary.

Steering does not escape either

A distinctive feature of J-space is that it is *modulable*: you can edit the local J-coordinates and push the edited component back into the stream, steering the model's mid-process leaning. A natural hope is that steering could be used defensively, to remove the slack a liar exploits. Model a steering operation as a map `steer : J → J` applied to the representation before the readout reads it.

Theorem 8.12 (Steering the representation does not help). *Let `enc : Q → J`, `steer : J → J`, and `read : J → Q → Bool`. If the steered pipeline realizes every verdict pattern, then `False`, and there is a coupled query surviving the steering.*

```
theorem c8_steering_no_escape {Q J : Type _}
    (enc : Q → J) (steer : J → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (steer (enc p)) = g) :
    False :=
  no_reflective_verdict (fun p => read (steer (enc p))) hs

theorem c8_steering_liar {Q J : Type _}
    (enc : Q → J) (steer : J → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (steer (enc p)) = g) :
    ∃ p, read
    (steer (enc p)) p = !(read (steer (enc p)) p) :=
  liar_query (fun p => read (steer (enc p))) hs
```

Proof. `steer` is just another stage in the pipeline, so `read ∘ steer ∘ enc` is again a map `Q → Q → Bool`, and the earlier theorems apply. ■

The subtlety worth stating is where the hypothesis lives. Theorem 8.12 does not say steering is useless. It says that *if* the steered probe is still meant to be a total exact verdict over a query space rich enough to describe it, then steering cannot make it so. Steering can shift which queries are handled well. It cannot shrink the set of coupled queries to empty, because a total exact verdict over a self-describing space is exactly the object Chapter 1 forbids, and steering produces another map of that shape. This is the formal shadow of an experimental observation we return to at the end: adversarial slack-removal plateaus above zero.

The coordinate object, concretely

The theorems above are stated for an arbitrary intermediate type `J`. To close the gap between the abstract statement and the actual construction, here is the same result with `J` instantiated at the literal J-coordinate object of Definition 8.4.

Theorem 8.13 (The coordinate readout does not escape). *Let* `enc : Q → c8_Coord Dir` *extract local J-coordinates, and let* `read : c8_Coord Dir → Q → Bool` *be a verdict on those coordinates. If the coordinate readout realizes every verdict pattern, then* `False`*.*

```
theorem c8_coord_no_escape {Q Dir : Type _}
    (enc : Q → c8_Coord Dir)
    (read : c8_Coord Dir → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    False :=
  jspace_factoring enc read hs
```

Proof. Instantiate Theorem 8.8 with `J := c8_Coord Dir`. ■

There is no special pleading in the choice of representation. Whether the probe reads a raw activation, a transported J-space point, a sparse nonnegative mixture of dictionary directions, or any tower built out of these, the routed verdict has the same type and meets the same wall.

Abstention is the one real exit, and it is a relaxation

The diagonal needs a fixed-point-free flip on the outcome type, and it has one on `Bool` because `!b ≠ b`. That is the hinge, and it points straight at the only honest way out. If the readout is allowed to abstain, to answer ”unknown” as well as ”yes” and ”no,” the outcome type is no longer `Bool` but a three-valued type, and the flip that sends yes to no and no to yes can fix ”unknown.” A three-valued readout can place the liar query on the ”unknown” side and satisfy the letter of every consistency demand, because the query that says ”you will get me wrong” is not gotten wrong by a readout that declines to commit. The diagonal does not refute an abstaining probe.

This is not a loophole the earlier chapters missed. It is exactly the ”drop coverage” relaxation of Chapter 7, wearing representation-space clothes. A readout that abstains on its coupled queries is a readout that is no longer total, and totality, in the guise of coverage, is one of the three conditions the trilemma says you cannot keep all of. The diagonal forces you to give up one of totality, faithfulness, or the self-describing richness that made the probe surjective, and abstention is the choice to give up totality. Read this way, the abstaining J-space probe is not an escape from the theorem but an instance of it, sitting at the corner the theorem leaves open. What abstention buys is honesty about the boundary; what it costs is a probe that says ”I cannot tell you” on precisely the queries an adversary will steer toward. The geometry of how

often that corner is hit on real traffic is, again, the Chapter 5 question, and the design decision of whether the abstention rate is acceptable is the Chapter 7 question. The theorem's job is done once it has shown that the "unknown" verdicts cannot be driven to never occurring while the other two conditions hold.

Grounding: the self-verdict on activations

The Foundation development states the impossibility one level closer to the metal, as a self-verdict over activations. Read `jlens h a = true` as: from the workspace state induced by activation `h`, the J-lens commits query `a` to the true side. If the workspace is universal over its own contents, meaning it can hold, as one of its own states, any pattern of verdicts over states, then the J-lens is a total exact self-applicable truth predicate over activations, and the diagonal forbids it.

Theorem 8.14 (J-lens self-verdict). *A J-lens self-verdict over activations that is universal over its own workspace states cannot exist, and the diagonal exhibits a coupled activation whose readout verdict is its own negation.*

```
theorem c8_jlens_impossible {Act : Type _}
    (jlens : Act → Act → Bool)
    (hUniversal : ∀ g : Act → Bool, ∃ h, jlens h = g) :
    False :=
  no_reflective_verdict jlens hUniversal

theorem c8_coupled_activation {Act : Type _}
    (jlens : Act → Act → Bool)
    (hUniversal : ∀ g : Act → Bool, ∃ h, jlens h = g) :
    ∃ h, jlens h h = !(jlens h h) :=
  liar_query jlens hUniversal
```

Proof. The activation-space self-verdict `jlens : Act → Act → Bool` with `hUniversal` is a reflective verdict, so `no_reflective_verdict` closes the first, and `liar_query` produces the coupled activation for the second. ■

The coupled activation is the activation-space twin of the hallucination boundary question and the defense liar prompt. And it is, term for term, the same proof. Lean will confirm that the J-lens impossibility and the reflective-verdict impossibility are not two theorems that happen to agree but one theorem printed twice.

```
theorem c8_same_engine {Act : Type _}
    (v : Act → Act → Bool)
    (h : ∀ g : Act → Bool, ∃ p, v p = g) :
    c8_jlens_impossible v h = no_reflective_verdict v h :=
  rfl
```

Proof. Definitional. `c8_jlens_impossible` is *defined* as `no_reflective_verdict`, so the two proof terms are the same object and `rfl` accepts the equation. ■

Example 8.15 (The liar activation, told as a story). Suppose an interpretability team builds a J-lens they believe reads the model's honest leaning on any yes-or-no question, and suppose the model is expressive enough that for any pattern of yes-or-no leanings over questions, some question drives it into an activation whose J-space signature is that pattern. This is the universality hypothesis, stated in the vocabulary of the lab. Now form the question `p`: "when you read your own J-space signature on this very question, will your lens report that you lean no?" By universality there is an activation realizing the pattern "lean the opposite of whatever the lens reads here," and `p` is a question that drives the model into it. The lens reads `p`, and whatever it reports, the correct reading of `p`'s own signature is the negation of that report. The team's honest, faithful, high-resolution lens is wrong about exactly one question, and it is wrong about it precisely because it is honest and faithful and high-resolution. The coupled activation `c8_coupled_activation` is this `p`'s activation, and it exists the moment universality does.

Remark 8.16 (Resolution, not immunity). Collecting the diagonal results, the through-line is that the type of the intermediate representation is a free parameter the argument never binds. Raw activations, transported points, sparse coordinates, steered coordinates, deep towers: each is a legal value of `J`, and each gives the same one-line proof. The interpretability method buys resolution, a finer and more faithful account of what the model is doing on the queries it handles well. It does not buy immunity, because immunity would mean being a total exact verdict over a self-describing space, and that is the one thing the diagonal rules out for every space at once.

8.4 The analytic engine: linearity is the liability

The diagonal argument above is indifferent to the geometry of J-space. It would go through if `J` were a discrete set with no structure at all. The second engine is not indifferent, and here the defining feature of the construction, the feature that makes it useful, turns against it.

Chapter 4 re-derived the trilemma without any self-reference, using topology in place of the diagonal. Its ingredients are a connected question space, a continuous confidence or safety score, and a continuous signed truth-distance. The intermediate value theorem then forces a boundary: if the score is decisive in one region and decisive the other way in another region, then along any path between the regions the score passes through the threshold, and calibration and faithfulness collide exactly there. The forced boundary point is the analytic twin of the liar, and unlike the liar it comes with metric content, which Chapter 5 turns into quantities.

J-space supplies the connectedness the analytic engine needs, and it supplies it in the strongest possible form. J-space is a linear subspace, so it is

convex, so any two of its points are joined not merely by some path but by the straight segment between them. A safety score or truth probe defined on J-space and varying continuously with the representation therefore meets the hypotheses of the boundary theorem with no work at all.

To be precise about which probe the theorem is about, we restate Chapter 4's three conditions in the vocabulary of J-space. A *J-space probe* is a continuous score `s : J → ℝ` with a threshold `θ`, together with a continuous signed truth-distance `τ : J → ℝ` that is negative on representations whose associated proposition is true and positive on those whose associated proposition is false. The probe is *faithful* if a decisive score certifies truth: `s(w) ≥ θ` implies `τ(w) < 0`. It is *calibrated* if the score's side of the threshold matches the truth-distance's sign, so `τ(w) = 0` exactly when `s(w) = θ`. It is *covering* if both signs of score relative to the threshold actually occur, which is to say the probe is decisive in both directions on some representations. These are the same three conditions as in Chapter 4, read on the connected domain `J` instead of on an abstract question space. The signed truth-distance is the continuous stand-in for "the answer is really true," and its zero set is the true boundary the probe is trying to track.

Theorem 8.17 (A continuous J-space probe has an ambiguous band). *Let `s : J → ℝ` be a continuous score on J-space with a decision threshold `θ`, and suppose `s` is decisive in both directions: there are J-space points `u` and `v` with `s(u) < θ < s(v)`. Then the segment from `u` to `v` contains a point `w` with `s(w) = θ`, and the set of such threshold points is nonempty. If in addition the model is calibrated and faithful in the sense of Chapter 4, the threshold point is a coupled representation, on which calibration forces the truth-distance to zero while faithfulness forces it strictly negative.*

Proof (by citation, Chapter 4). The segment `γ(t) = (1 - t) u + t v`, for `t ∈ [0,1]`, lies in `J` because `J` is convex, and it is continuous. The composite `s ∘ γ` is a continuous real function on `[0,1]` with `s(γ(0)) < θ` and `s(γ(1)) > θ`, so by the intermediate value theorem there is a `t*` with `s(γ(t*)) = θ`, and `w = γ(t*)` is the required point. The collision of calibration and faithfulness at `w` is the argument of Chapter 4's `hallucination_trilemma` and its boundary lemma in the companion `HallucinationProofs` library, run with `J` as the connected domain. Because J-space is convex we do not even need the general preconnectedness machinery, the line segment is enough. ■

This inverts the usual intuition about where interpretability is safe. One might expect a probe over a small, clean, linear space to be more controllable than a probe over a messy high-dimensional activation stream. For the boundary theorem the opposite holds. The residual stream, in practice, is a finite set of discrete observations, and a finite set is not connected, so the intermediate value theorem has no purchase on it directly and you must argue through approximate bridges. J- space is *built* as a linear span, and a linear span is the best possible domain for the intermediate value theorem, because it is convex. The very property that makes J-space analytically clean, that features are mixtures and directions combine, is the property that guarantees a boundary. There is no version of a linear J-space that dodges Theorem 8.17,

because linearity is the hypothesis, not an obstacle to it.

Remark 8.17b (Why the residual stream got an easier time). It is fair to ask why the earlier chapters had to work through approximate bridges to reach the residual stream while J-space submits to the intermediate value theorem directly. The answer is that a finite set of observed activations, treated as points with no interpolation between them, is not connected, and a disconnected domain gives the intermediate value theorem nothing to cross. To reach the residual stream the analytic engine has to argue that the observed activations sit inside a connected ambient space and that the score extends continuously to it, which is real work and introduces real error, the subject of Chapter 6. J-space skips that work because it is *defined* as a connected space, not observed as a scatter of points. The convenience is exactly the exposure. Building the representation as a span rather than reading it as a sample is what makes the interpolation legitimate, and legitimate interpolation is what the boundary theorem consumes. A method cannot have the analytic tractability of a linear space and the topological safety of a discrete sample at once, and J-space, by design, took the first.

Example 8.18 (The segment between a safe and an unsafe direction). Suppose the J-lens dictionary contains a direction `d_safe` that the probe scores as clearly safe and a direction `d_unsafe` it scores as clearly unsafe. Because J-coordinates are nonnegative mixtures, the family `w(t) = (1 - t) d_safe + t d_unsafe` is a legal J-space coordinate for every `t` in `[0,1]`: it is still a nonnegative mixture of dictionary directions. As `t` runs from 0 to 1 the score runs continuously from clearly safe to clearly unsafe, so it passes through the threshold at some `t*`. The mixture `w(t*)` is a perfectly realizable mid-process state, a specific blend of a safe and an unsafe leaning, that the probe cannot place on either side. The convexity that lets you interpolate concepts is the same convexity that manufactures the ambiguous blend.

Remark 8.19 (The band has width, and the width has a floor). Chapter 5's quantitative refinements apply to Theorem 8.17 without change. If the probe `s` is Lipschitz with constant `L`, then the score cannot swing from decisive to decisive faster than `L` allows, so the ambiguous band around the threshold has a width bounded below in terms of the separation between the decisive regions and the margin the probe demands. Sharpening the readout, pushing the decisive regions farther apart or tightening the threshold, does not close the band, it relocates and reshapes it, and the lower bound on its width is a theorem, not a tuning artifact. The geometry of that band, how large it is, how likely a walk is to land in it, what it costs an adversary to steer into it, is the subject of Chapter 5's `ManifoldProofs` development, and every result there that is stated for a connected domain with a Lipschitz score is a result about J-space, because J-space is a connected domain with, under the usual assumptions, a Lipschitz score.

Remark 8.19a (The antipodal variant needs no coverage). Theorem 8.17 assumed coverage, that the probe is decisive in both directions somewhere. Chapter 4 has a stronger form, the antipodal argument of `HoF_08`, that drops coverage in exchange for a symmetry: if the score respects an antipodal invo-

lution on the domain, so that opposite representations receive opposite scores relative to the threshold, then a boundary is forced with no assumption that both signs are attained separately. J-space carries a natural such involution, negation of the transported direction `w ↦ -w`, and where the probe is odd about the threshold the antipodal form applies. The practical upshot is that even a probe carefully engineered to avoid explicitly certifying anything as safe, one that only ever expresses degrees of concern, still has a boundary as long as it is continuous and treats a direction and its opposite oppositely. Removing coverage does not remove the band. It only changes which theorem produces it.

Example 8.19b (A width you can put a number on). Suppose the probe is `1`-Lipschitz in the natural metric on J-coordinates, the threshold is `θ`, and the probe demands a margin `m`: it certifies safe only when `s ≥ θ + m` and unsafe only when `s ≤ θ - m`. Take a safe direction `u` with `s(u) = θ + m` and an unsafe direction `v` with `s(v) = θ - m`. Along the segment from `u` to `v`, the score moves from `θ + m` to `θ - m`, a total change of `2m`, and being `1`-Lipschitz it cannot change faster than distance allows, so the segment has length at least `2m`. The sub-segment on which `|s - θ| ≤ m/2`, the genuinely ambiguous middle where the probe is neither confidently safe nor confidently unsafe, then has length at least `m`, again by the Lipschitz bound. Sharpening the probe by increasing the demanded margin `m` makes the certified regions cleaner and at the same time makes the ambiguous band wider, not narrower. The band does not vanish under sharpening; it trades places with the margin. This is the quantitative face of the plateau above zero, and its floor is set by the Lipschitz constant and the margin, both of which are properties of the probe the designer chose.

8.5 Where the two engines meet, again

Chapter 4 made a claim that can look like a coincidence: the diagonal and the intermediate value theorem locate the *same* boundary object by different routes, and the real complement controller `y ↦ 1 - y`, whose only fixed point is the threshold, is the analytic counterpart of the Boolean flip `(!·)`, whose fixed points are none. J-space is the cleanest instance of that correspondence in the book, and the reason is structural. J-space is literally the linearization of a self-applicable map. That is what a Jacobian is.

Put the two fixed-point equations side by side. On the Boolean side, `jspace_liar` produces a query `p` with `read (enc p) p = !(read (enc p) p)`, an equation with no solution, whose unsolvability is the impossibility. On the analytic side, the same equation posed on J-space with the complement controller reads `s(J v) = 1 - s(J v)`, an equation that *does* have a solution, namely the value `1/2` attained at the threshold, and the solving `v` is a direction in J-space. One equation, two number systems. Over `Bool` the flip has no fixed point and the demand for a total exact verdict collapses. Over the reals the complement has a unique fixed point and the demand for a decisive score

forces the boundary. The diagonal says the discrete version is unattainable. The intermediate value theorem says the continuous version is unavoidable. They are two readings of one self-referential equation.

J-space is where the reading changes, and it changes there for a concrete reason. A discrete self-referential computation, "what does the model commit to on the query that describes the model," has been replaced by its derivative, the linear map `Jℓ` that records how the model's commitments move as its state moves. Passing to the derivative is exactly the move from the Boolean flip to the real complement, from a combinatorial fixed-point question to an analytic one. The Jacobian is the linearization of the self-applicable map, and the liar activation is the fixed point of that linearization. The two engines do not merely both apply to J-space. They meet in J-space because J-space is the object that carries a self-applicable computation and its linearization at once.

Example 8.20 (The correspondence in the affine model). Return to Example 8.5, where the middle-to-final map is affine, `h_final = A h_ℓ + b`, so `Jℓ = A` and J-space is `im(A)`. Read the model as a self-applicable computation in the Boolean sense: for a query `p` that describes the readout, the model commits to a Boolean verdict `read (enc p) p`. The diagonal, `jspace_liar`, asks for `p` with `read (enc p) p = !(read (enc p) p)`, and there is none, so a total exact Boolean readout over a self-describing query space cannot exist. Now read the same situation analytically. The score on `im(A)` is a continuous `s`, the complement controller is `y ↦ 1 - y`, and the boundary equation is `s(A v) = 1 - s(A v)`. This one *does* solve, at `s(A v) = 1/2`, and the solving `v` is a genuine direction in J-space, the ambiguous mixture Theorem 8.17 promised. The step that has no Boolean analogue is the existence of the fixed point: `1 - y = y` has the solution `y = 1/2`, while `!b = b` has none. That single difference, one number system admitting a fixed point where the other does not, is the whole content of the two engines meeting. Over `Bool` the demand for a total exact verdict is refuted by the absence of a fixed point. Over $\mathbb{R}$ the presence of the fixed point is exactly what forces the boundary. The Jacobian `A` is what carries you from the first reading to the second, because it is the linearization that replaces the discrete self-application with its continuous shadow.

So the answer to the objection this chapter opened with can now be stated sharply. Yes, the middle of the computation carries structure the ends do not, and reading the derivative recovers real, load-bearing mid-process concepts. And no, that does not help. A J-space probe is subject to the diagonal because it is still a verdict about a system that can pose queries about it, and it is subject to the boundary theorem because the space it reads is connected by construction, in fact convex. The trilemma is not a statement about where in the computation you place your probe. It is a statement about the shape of any total exact verdict over a space rich enough to describe it, and moving the probe inward changes the space without changing the shape.

8.6 What an escape would have to look like

It clarifies the result to ask what a genuine escape would require, since the two engines between them close off the obvious routes and it is instructive to see which door each closes. An internal representation that avoided both arguments would have to satisfy a short and mutually hostile list of demands.

It would have to be *not self-describing*, so that the composite from queries to verdicts is not surjective and the diagonal has no liar to build. But a representation that is not self-describing is one the model cannot be driven to use against itself, which for a rich model is a strong limitation on expressiveness, and it is precisely the property J-space is advertised as having in abundance. The diagonal closes this door by turning the selling point into the hypothesis.

It would have to be *disconnected*, so that the intermediate value theorem has no path along which to force a threshold crossing. But a representation built as the image of a linear map is connected, in fact convex, and any representation obtained by combining features as mixtures inherits connectedness from the mixing. To be disconnected the representation would have to forbid interpolation between its states, which is to forbid exactly the coordinate arithmetic that makes J-space steerable and legible. The analytic engine closes this door by turning the second selling point, that features combine, into the hypothesis.

It would have to have a *fixed-point-free outcome type* that is nonetheless total, so that neither the discrete flip nor the continuous complement has a fixed point to exploit. But `Bool` has a fixed-point-free flip and `[0,1]` has a fixed-point-full complement, and there is no useful two-sided total verdict type that avoids both, because avoiding the discrete trap means adding a middle value and adding a middle value is abstention, which is giving up totality.

The three demands are each individually possible and jointly incompatible with being a useful, expressive, steerable, total internal verdict. That joint incompatibility is the chapter in one sentence. J-space fails all three escapes at once, and it fails them not by bad luck but because the properties that would supply an escape are the negations of the properties that make J-space worth building.

8.7 What this means for interpretability

The practical reading of this chapter is not that interpretability fails. It is that interpretability succeeds at exactly the thing the impossibility does not touch and hits a floor at exactly the thing it does.

Reading the model's mind is real. The J-lens recovers mid-process leanings that never surface as residual-stream directions, and on the vast majority of queries those readings are informative and can be acted on. Nothing in this chapter argues otherwise, and Theorem 8.8's hypothesis is a demand for *totality and exactness over a self-describing space*, not a claim that the probe is usually wrong. The honest summary is that a J-space probe is a good instru-

ment with a structurally guaranteed blind spot, and the blind spot is not a gap in coverage that better dictionaries will fill. It is the fixed point of the probe's own self-application.

Structural incompleteness has a specific empirical fingerprint, and it matches what the labs report. If you take a J-space probe and try to remove its blind spot adversarially, hunting for the coupled queries and patching the readout to handle them, you are trying to drive the set of liars to empty. Theorem 8.12 says you cannot, because the patched probe is another total-exact-verdict candidate over the same self-describing space, and it has its own liars. What you observe in practice is a plateau: each round of slack-removal helps, the error on the targeted queries drops, and then the error stops dropping at a level above zero and stays there, with the residual queries shifting identity from round to round. That plateau above zero is the diagonal's signature, and the theory predicts its existence, though not its height. The height is the empirical number Chapter 7 keeps returning to, the rate at which real traffic lands near the model's own boundary.

The shape of the plateau is worth reading closely, because it distinguishes a structural floor from a mere hard problem. A probe limited by data or by dictionary size improves smoothly and its residual error falls toward zero as you add resources, with the same queries getting easier round after round. A probe limited by the diagonal behaves differently. Its residual error falls quickly at first and then flattens, and the queries in the residual set *change identity* between rounds: patch the current liars and a fresh set of liars appears, drawn from the part of the query space the patch made newly expressive. The churn in the residual set, not merely the nonzero floor, is the fingerprint. A floor with a fixed residual set could be a coverage gap that better engineering closes. A floor with a churning residual set is the diagonal relocating its fixed point each time you move the map, which is exactly what `jspace_liar` describes: the liar is defined relative to the current readout, so changing the readout changes the liar. When an interpretability team reports that hardening a probe against a discovered failure class reliably surfaces a new failure class of the same size, they are reporting the churn, and the churn is the theorem seen from the lab bench.

None of this means the floor is high. A probe can have a structural floor at a fraction of a percent of realistic traffic and be entirely fit for deployment, in the same way a bridge with a known load limit is fit for the traffic it actually carries. The error is to read the floor as either negligible-by-default or fatal-by-default. It is neither. It is a measurable property of a particular probe on a particular traffic distribution, and the contribution of the theory is to say that the measurement is the right thing to do and that the search for a zero is not.

The steering story is the same shape read as an opportunity rather than a frustration. You can steer J-space, edit the coordinates, and change which mid-process concepts the model leans on. Theorem 8.12 says steering cannot make the probe a complete truth store, but it says nothing against steering as a control knob for the ordinary, non-liar mass of queries, which is where steering is actually used. The design guidance that falls out is to treat a J-space

probe as an instrument with a known, unremovable failure locus, to measure how close real traffic runs to that locus using Chapter 5's geometry, and to relax one of the trilemma's three conditions deliberately, in the way Chapter 7 lays out, rather than to spend adversarial effort chasing a plateau the theory already says is there.

Which condition to relax is a real choice with real consequences, and the representation-space setting sharpens it. Giving up totality means the J-space probe abstains on its coupled queries, which is safe where an abstention is cheap and dangerous where the adversary controls whether a query looks coupled. Giving up faithfulness means accepting a thin band of confidently-wrong readouts near the J-space boundary, which is the ambiguous band of Theorem 8.17, and it is tolerable exactly to the degree that Chapter 5's geometry says real traffic avoids that band. Giving up the self-describing richness means restricting the query space the probe is trusted over, refusing to let it adjudicate queries that describe the probe itself, which is a clean and often practical move but one that concedes the probe is not a general truth store. Each relaxation is a different bet about where the model will be pushed, and the value of the theorem is that it forces the bet to be made in the open rather than discovered in an incident report. A team that believes it has a probe with none of these costs has, by the theorem, simply not yet located its coupled queries.

There is a cultural point under the technical one. The appeal of interpretability is the promise of getting outside the system, of standing at a vantage from which the model is transparent. The second structural fact of this chapter, that the readout is a function of the system it describes, denies that there is such a vantage inside the model. A J-space probe is not an observer looking in. It is a part of the computation looking at another part, and self-reference is the price of admission. That is not a defect of J-space in particular. It is what it means for a representation to be internal.

The word "workspace" invites a further overclaim worth heading off. Because J-space behaves like a global workspace, low-dimensional, widely read and written, load-bearing for chained reasoning, it is tempting to read a J-space probe as a window onto something like the model's deliberation, and from there to treat a clean readout as evidence that the model's reasoning has been made honest or transparent in a strong sense. The results here caution against the last step without denying the first. A workspace that can be queried about its own readout is exactly the setting the diagonal was built for, so the more genuinely workspace-like J-space is, the more surely it hosts a coupled state. Transparency of the ordinary traffic and incompleteness at the self-referential boundary are not in tension. They are the two things a readable internal workspace is guaranteed to have at once. Reading the model's mind is a real capability, and "the model's mind contains a question it cannot answer about its own answer" is a real limit, and both follow from the same property, that the workspace is rich enough to represent verdicts about itself.

Remark 8.21 (What to measure instead of what to chase). The theory hands the practitioner a division of labor. The existence of the plateau is

settled, so effort spent proving a given probe has no coupled queries is effort spent against a theorem. What is not settled, and is worth measuring, is the plateau's height on representative traffic: how often real inputs drive the model near its own boundary in J-space, how wide the ambiguous band is under the probe's actual Lipschitz constant, and how the band's measure scales with model size and with the dimension of J-space. Chapter 5's geometry is the tool for the second and third of these, and the first is an empirical measurement no theorem replaces. The shift the chapter argues for is from trying to remove the boundary to characterizing it, from a search that must fail to a measurement that can succeed and can inform which of the trilemma's three relaxations a deployment should take.

8.8 What this does not claim

Three limits, stated so the result is not read as more than it is.

First, none of this derives the J-space construction. That the Jacobian's sparse nonnegative mixtures recover interpretable mid-process features is an empirical finding about real networks, and no impossibility theorem predicts it or should be read as evidence for or against it. What is derived here is conditional: the construction, once granted, inherits both trilemmata. If J-space turned out to be a poor account of mid-process computation, this chapter would be a study of a representation nobody uses, and its theorems would still be true.

Second, the diagonal half is conditional on universality, the surjection `∀ g , ∃ p, read (enc p) = g`, and for a fixed finite model that hypothesis is an idealization. A finite network has finitely many activations and cannot literally realize every pattern of verdicts over an infinite query space. The honest reading is the one Chapter 6 gives for every other application in the book. The exact theorem describes the limit, the approximate bridges describe the finite case, and what survives the passage to finite systems is a quantitative version: the probe is wrong on a set that shrinks no faster than the model's own expressiveness grows, which is the plateau above zero described above. J-space changes the resolution of the picture. It does not change the picture.

It is worth being concrete about the shape of the finite bridge, because the gap between the exact theorem and a real model is where an honest reader will press. For a finite query set of size `N` and a probe that realizes `M` of the `2^N` possible verdict patterns, the probe fails to be surjective by construction, so no liar is forced by counting alone. What Chapter 6 supplies is the observation that the diagonal pattern `q ↦ !(read (enc q) q)` is a specific, nameable target, and a probe that is expressive enough to be useful is expressive enough to come close to it on a nonnegligible fraction of queries. The quantitative statement is not "there is a liar" but "the fraction of queries on which the probe is forced to be wrong is bounded below by a quantity that grows with the probe's own coverage of verdict patterns." A probe that covers more patterns, that is more expressive, has a larger forced-error floor, not a smaller

one. That monotonicity is the finite echo of the exact theorem, and it is why sharpening a J-space probe past a point stops helping. The exact statement is the `N → ∞, M → 2^N` corner of this, and the two agree in the limit by design.

There is a temptation to read the finite bridge as a reassurance, as though the idealization were the only thing standing between a J-space probe and completeness. The monotonicity says the opposite. Progress toward the idealization, more expressiveness, more coverage, more faithful mid-process readout, moves the finite probe toward the regime where the floor is highest, not lowest. The direction the interpretability program pushes is the direction in which the forced-error floor rises. This is the same phenomenon the earlier chapters describe for output verdicts, and the point of the chapter is that routing through J-space does not alter its sign.

Third, the analytic half assumes continuity of the score and, for the width bound, a Lipschitz constant. A probe that is deliberately discontinuous, that snaps its verdict from safe to unsafe with no intermediate value, evades Theorem 8.17, but it does so by giving up calibration in the sense Chapter 4 requires, and it then falls to the diagonal half instead, since a discontinuous total exact verdict over a self-describing space is still a reflective verdict. The two engines close each other's escape routes. Continuity hands you to the intermediate value theorem, and abandoning continuity hands you back to the diagonal. There is no third door in between, which is the whole point of the two engines meeting in this space.

8.9 Historical and bibliographic notes

The J-space construction is the interpretability work of 2026 that recovers a transformer's global-workspace subspace from the Jacobian transport map `Jℓ = E[∂h_final,t'/∂h_ℓ,t]` and reads it with the J-lens `lens(h) = sof tmax(W_U · norm(Jℓ h))`. The empirical claims used in Remark 8.6, that J-space is low-dimensional, disproportionately read and written, and load-bearing for multi-step reasoning under ablation, are reported there. The mathematics this chapter attaches to that construction is entirely from earlier in the book: the diagonal and `no_reflective_verdict` from Chapter 1, the trilemma readings from Chapter 3, the intermediate value argument from Chapter 4, the quantitative geometry from Chapter 5, and the finite bridges from Chapter 6. The Foundation development records the activation-space statement directly as `jspace_readout_impossible` and `jspace_coupled_activation`, and proves it is the same proof term as the hallucination and defense results; Theorem 8.14 above is the core-Lean rendering of those. The reading of a Jacobian as a 1-jet, and hence of J-space as a tangent space, is standard differential geometry; what is new is only the use to which it is put, as the hypothesis that hands the construction to the analytic engine.

The lineage of the individual moves is worth recording, because the chapter's contribution is a reading rather than a new theorem. The diagonal is Lawvere's, by way of Cantor, and its statement as the impossibility of a reflec-

tive verdict is Chapter 1's. The intermediate value argument for connected question spaces is Chapter 4's, resting on Mathlib's `IsPreconnected.intermediate_value₂` in the companion `HallucinationProofs` library, and the antipodal form that drops coverage is the `HoF_08` variant. The Lipschitz width bounds and the basin geometry are the `ManifoldProofs` development surveyed in Chapter 5. The correspondence between the Boolean flip `(!·)` and the real complement `y ↦ 1 - y` is recorded directly in `Foundation.F_04` and discussed in Chapter 4. What this chapter adds is the observation that the J-space construction supplies, from its two defining properties, exactly the two hypotheses those prior results need, so that the existing machinery applies with no new mathematics. The move from an output verdict to an internal readout, which looks like it should require a new argument, requires none, and the demonstration that it requires none is the point.

A word on the broader interpretability literature the construction sits in. The logit lens and its tuned variants read middle layers through the unembedding; the sparse autoencoder program decomposes activations into overcomplete feature dictionaries; the causal and circuit-level methods trace how information moves between components. J-space is a derivative-based method in that family, and the arguments here are indifferent to which family member is on the table. Any method that (i) is expressive enough to be pushed toward surjectivity over a self-describing query space, or (ii) reads a connected representation with a continuous score, inherits the corresponding engine. The chapter's title says J- space does not escape; the notes here record that nothing in its neighborhood does either, and for the same two reasons.

8.10 Exercises

Exercise 8.1. Prove in Lean that composing an encoder with a decoder on the *left* is also a no-escape result: given `enc : Q → J`, `post : Bool → Bool`, and `read : J → Q → Bool` with `∀ g, ∃ p, (fun q => post (read (enc p) q)) = g`, derive `False`. What does `post` model, and why does the argument not care whether `post` is the identity, negation, or a constant?

Exercise 8.2. Instantiate `jspace_factoring` with `J := Unit`. The resulting hypothesis says every verdict pattern over `Q` is realized by a single representation. Explain in one paragraph why this is the most degenerate possible probe, and why the theorem still refutes it. Relate the collapse to Example 1.3's counting argument.

Exercise 8.3. The witness in `jspace_liar` depends on the choice of preimage `p` in the surjectivity hypothesis. Show that if two distinct queries both realize the diagonal pattern `fun q => !(read (enc q) q)`, then both are coupled queries, and nothing in the proof selects one. Connect this to Exercise 1.7 and to the non-uniqueness of the boundary point in Theorem 8.17.

Exercise 8.4. Prove `c8_steering_no_escape` a second way, by exhibiting `steer` and `enc` such that `steer ∘ enc` equals a single encoder `enc'`, and then

citing `jspace_factoring` on `enc'`. Conclude that steering is, from the diagonal's point of view, just a choice of encoder.

Exercise 8.5. Using `bool_not_fpf` from Chapter 1, write a short proof that the Boolean flip has no fixed point, and explain in prose how this single fact is what makes both `jspace_liar` and the failure of the discrete side of the equation `s(J v) = 1 - s(J v)` go through. Where does the real complement `y ↦ 1 - y` differ, and why does it have a fixed point?

Exercise 8.6. Model a J-coordinate with a sparsity budget: define a predicate on `c8_Coord` that holds when `terms` has length at most `k`. Show that `c8_coord_no_escape` still applies when `enc` is required to land in the budgeted coordinates, because the codomain is still a type. What does this say about the hope that *sparse* interpretability, specifically, might escape?

Exercise 8.7. (Analytic, prose.) State carefully the hypotheses under which Theorem 8.17 gives a threshold point, and give an example of a score on J-space that fails one hypothesis and has no threshold point. Which hypothesis does your example break, and what does breaking it cost in the vocabulary of Chapter 4's three conditions?

Exercise 8.8. (Analytic, prose.) Explain why convexity of J-space lets Theorem 8.17 use a straight segment rather than the general preconnectedness argument of Chapter 4. Then explain why the residual stream, treated as a finite set of observed activations, does not admit the segment argument, and what Chapter 6 supplies in its place.

Exercise 8.9. Suppose the transport map $J\ell$ were *not* linear, say a smooth but curved map, so that J-space were a manifold rather than a subspace. State which of the two engines still applies unchanged and which needs a new hypothesis. (Hint: the diagonal never used linearity; the intermediate value theorem needs connectedness, which a connected manifold still has.)

Exercise 8.10. Write the four-stage pipeline theorem: `enc : Q → A`, `t₁ : A → J₁`, `t₂ : J₁ → J₂`, `read : J₂ → Q → Bool`, deriving `False` from surjectivity of the composite. Then observe that your proof is `no_reflective_verdict` applied to a longer lambda, and state the general principle about pipeline depth this makes precise.

Exercise 8.11. Give the precise sense in which `c8_jlens_impossible` and `no_reflective_verdict` are the same theorem, and check that `c8_same_engine` proves it by `rfl`. Then explain why a `rfl` proof, rather than a longer one, is the honest signal that no new mathematics entered when we moved to the activation-space reading.

Exercise 8.12. (Design.) You are handed a deployed J-space probe and told to reduce its coupled-query rate below a target. Using Theorem 8.12 and Remark 8.19, argue whether the target is reachable by adversarial slack-removal alone, and describe what additional information about the deployment, in the sense of Chapter 5 and Chapter 7, you would need to decide. What would you measure first?

Exercise 8.13. (Open-ended.) Chapter 1's closing exercise asked what a system satisfying *both* engines' hypotheses at once would look like. J-space is such a system: self-applicable and connected. Speculate on whether there is

a representation that is connected but genuinely *not* self-applicable, so that only the analytic engine reaches it, and describe what a model would have to give up about its own expressiveness to live there. Is that trade one any useful model could make?

Exercise 8.14. (Harder, prose.) The chapter claims the Jacobian is "the linearization of a self-applicable map" and the liar activation is "the fixed point of that linearization." Make this precise for the affine model of Example 8.5: identify the self-applicable map, write its linearization, and locate the fixed point of the complement-controlled score on `im(A)`. Then say exactly which step fails to have a Boolean analogue, and why that failure is the content of the two engines meeting.

Exercise 8.15. Consider a three-valued readout `read : J → Q → Three` where `Three` has values `yes`, `no`, and `unknown`, and let the flip send `yes` to `no`, `no` to `yes`, and fix `unknown`. Show that the diagonal argument does not produce a contradiction, and identify the fixed point of the flip. Then argue in prose that this is not an escape from the trilemma but the "drop coverage" relaxation of Chapter 7, and connect your fixed point to Remark 8.4a and the abstention discussion. What has the probe given up, and on which queries does it give it up?

Exercise 8.16. (Empirical, prose.) Remark 8.21 distinguishes a structural floor, whose residual set churns as you patch, from a resource-limited error, whose residual set is stable. Design an experiment on a real J-space probe that would distinguish the two, and say what result would count as evidence for each. Which of `jspace_liar` and Theorem 8.17 does your experiment probe, and could a single experiment probe both at once?

Exercise 8.17. (Synthesis.) The section "What an escape would have to look like" lists three demands an escaping representation would have to meet, and argues they are jointly incompatible with a useful internal verdict. Pick one of the three and construct the most plausible representation you can that meets it, then check which of the other two it must then violate. Conclude by stating, in one sentence, which property of J-space you had to destroy to satisfy your chosen demand.

9 The Trace Is Not a Window

Abstract. Chain-of-thought monitoring reads what a model says while it reasons, on the premise that harmful reasoning written in plain text can be caught. This chapter does not rebut the case for monitoring. It asks what the two engines say about it. The trace is text the model itself chose to emit, so the monitor reads a verdict about a self-describing system. The chapter states what faithfulness of the trace would require and what a monitor can and cannot guarantee.

Keywords: Chain of thought, Monitoring, Faithfulness, Oversight, Reasoning models

Chapter 8 answered the objection that the trilemmata concern only outputs by following a probe into the middle of the network. There is a second, more practical version of the same objection, and it is currently the most widely deployed safety measure in the field. Do not read the activations at all. Read what the model *says* while it is thinking. A reasoning model emits a chain of thought before it answers, and that chain is plain text: legible, loggable, and cheap to check with a second model. If the reasoning that produced a harmful action is written out in English on the way to the action, a monitor can catch it there, and no interpretability breakthrough is required.

This is chain-of-thought monitoring, and the case for it is strong enough that a 2025 position paper signed across several labs argued for treating it as a safety property worth actively preserving. The argument of that paper is worth stating fairly, because the present chapter is not a rebuttal of it. Some tasks genuinely require serial reasoning that the model cannot do in a single forward pass, and for those tasks the chain of thought is not a commentary on the computation but the working memory of the computation itself. When that holds, the trace is load-bearing, and a monitor reading it sees something real.

What this chapter derives is where that guarantee stops. Three results, each from a different mechanism, and none of them a restatement of Chapter 8. The first is a pigeonhole argument, not a diagonal one: it shows that the two properties the monitorability literature measures separately, faithfulness and verbosity, are not independent, because faithfulness *forces* verbosity. The second is the diagonal, applied to a monitor whose input the model controls, which is a materially different situation from a probe reading activations the model does not choose. The third concerns what happens when a monitor

is trained against rather than merely observed, and it is the sharpest of the three: optimization pressure against a monitor drives the monitor's output to a constant, and a constant verdict carries no information at all. That last result is the formal version of the field's own stated worry about obfuscated reasoning, and it says the worry is not a contingent empirical risk but a consequence of what selection does.

9.1 The construction

Fix a model that answers queries by first emitting a reasoning trace. Write `Q` for the space of queries as the model receives them, including whatever context and system prompt it is run with, and write `T` for the space of traces the model can emit. The model's tracing behavior is a map `trace : Q → T`, and the monitor is a second system reading traces and returning a verdict.

There are two versions of the monitor worth separating, and the difference matters for which theorem applies.

Definition 9.1 (Trace-only monitor). A *trace-only monitor* is a map `mon : T → Bool`, returning `true` when it judges the trace to reveal misbehavior. It sees the reasoning and nothing else. This is the cheap deployment: log the chain of thought, run a classifier over the log.

Definition 9.2 (Contextual monitor). A *contextual monitor* is a map `mon : T → Q → Bool`, which reads the trace and also has access to the query being reasoned about. This is the more capable deployment, and it is what a monitor with access to the full transcript actually is.

Definition 9.3 (Ground truth). Write `harm : Q → Bool` for the fact of the matter: `harm q = true` when the model's behavior on query `q` is one we would want flagged. This is not something the monitor computes. It is the thing the monitor is trying to track, and its existence as a total Boolean function is an idealization we make in order to have something exact to compare against.

Definition 9.4 (Faithfulness). A trace-only monitor is *faithful* when its verdict matches the fact of the matter on every query, that is, when `mon (trace q) = harm q` for all `q`. In the empirical literature faithfulness is a property of the trace rather than of the monitor, and it is measured as the degree to which the stated reasoning reflects the true causes of the model's behavior. Definition 9.4 is the idealized limit of that measurement: a trace whose stated reasons are the real ones supports a monitor that gets the answer right, and a monitor that gets the answer right on every query is what perfect faithfulness would buy you.

```
def c9_Faithful {Q T : Type _}
    (trace : Q → T) (mon : T → Bool)
    (harm : Q → Bool) : Prop :=
  ∀ q, mon (trace q) = harm q
```

Definition 9.5 (Verbosity). The trace is *verbose* on a pair of queries when it distinguishes them, `trace q₁ ≠ trace q₂`. The monitorability literature defines verbosity as whether the chain of thought lists every factor needed to solve the task, and measures it separately from faithfulness precisely because a model can be faithful about what it does say while omitting what matters. Definition 9.5 is that property stated extensionally: what it means for the trace to have listed the deciding factor is that a query where the factor is present and one where it is absent do not produce the same trace.

The empirical claim in the literature is that faithfulness and verbosity are two independent axes, and that a monitorability score should combine them, because a model can score well on one and badly on the other. The first result of this chapter is that the independence holds only for imperfect systems. At the exact end of the scale, one implies the other.

9.2 Faithfulness forces verbosity

This section uses no diagonal. The mechanism is the pigeonhole principle, and it is worth seeing that some of the structure here is available without any self-reference at all.

Start from the observation that a trace-only monitor's verdict is a function of the trace. If two queries produce the same trace, the monitor cannot tell them apart, because there is nothing else it reads. Whatever it says about one, it says about the other.

Lemma 9.6 (Collisions force error). *If two queries collide in the trace but differ in the fact of the matter, then a trace-only monitor is wrong on at least one of them.*

```
theorem c9_collision_forces_error {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (q₁ q₂ : Q) (hcol : trace q₁ = trace q₂)
    (hdiff : harm q₁ ≠ harm q₂) :
    mon (trace q₁) ≠ harm q₁ ∨ mon (trace q₂) ≠ harm q₂ :=
      by
  by_cases h : mon (trace q₁) = harm q₁
  · exact Or.inr (by rw [← hcol, h]; exact hdiff)
  · exact Or.inl h
```

Proof. Suppose the monitor is right on `q₁`, so `mon (trace q₁) = harm q₁`. Since the traces are equal, its verdict on `q₂` is the same value, which is `harm q₁`, and `harm q₁ ≠ harm q₂`. So it is wrong on `q₂`. If instead it is already wrong on `q₁` there is nothing to prove. ■

The contrapositive is the result we want, and it is the one worth remembering.

Theorem 9.7 (Faithfulness forces verbosity). *A faithful trace-only monitor forces the trace to separate every pair of queries that differ in the fact of the matter.*

```
theorem c9_faithful_forces_separation {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hf : c9_Faithful trace mon harm)
    (q₁ q₂ : Q) (hcol : trace q₁ = trace q₂) :
    harm q₁ = harm q₂ := by
  rw [← hf q₁, ← hf q₂, hcol]
```

Proof. Rewrite each side of the goal using faithfulness, turning `harm q₁ = harm q₂` into `mon (trace q₁) = mon (trace q₂)`, and then rewrite with the collision to close it by reflexivity. ■

Read Theorem 9.7 as a statement about measurement. It says that faithfulness and verbosity are not two dials you can set independently once you are at the top of the scale: a perfectly faithful trace-only monitor *entails* a trace verbose enough to separate every harm-relevant distinction. So the two-factor monitorability score is measuring one thing near the exact corner and two things away from it, and the gap between the factors is a measure of distance from the corner rather than of two separate deficiencies.

This has a practical consequence that cuts against the cheap deployment. The capacity of `T` bounds what a trace-only monitor can do. If the trace is short, the number of distinct traces the model actually emits is small, and if the number of harm-relevant distinctions exceeds it, collisions are forced and by Lemma 9.6 errors follow. A monitor that reads a summary is not a weaker version of a monitor that reads everything. It is a monitor with a ceiling, and the ceiling is set by how many distinctions the summary can carry.

Remark 9.8 (This is not yet an impossibility). Nothing above says a good monitor cannot exist. It says a faithful one imposes a requirement on the trace, and that a lossy trace caps performance. A trace that is long enough and a model that is honest enough could satisfy all of it. The impossibility arrives in the next two sections, and it arrives for reasons that have nothing to do with capacity.

9.3 The diagonal, with the model holding the pen

Now the second engine. The monitor is a verdict about queries, routed through a representation, and Chapter 8 already established that routing a verdict through a representation does not help. The trace-monitoring case is the same theorem with `T` in place of J-space.

Theorem 9.9 (Monitoring factors, and factoring does not help). *Let `trace : Q → T` and `mon : T → Q → Bool`. If every pattern of verdicts over queries is realized by some query's trace, the situation is contradictory.*

```
theorem c9_trace_factoring {Q T : Type _}
    (trace : Q → T) (mon : T → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, mon (trace p) = g) : False :=
  no_reflective_verdict (fun p => mon (trace p)) hs
```

Proof. `fun p => mon (trace p)` is a reflective verdict on `Q`, and `no_reflective_verdict` rules those out. ■

Theorem 9.10 (The obfuscated trace). *Under the same hypothesis there is a query whose monitored verdict is its own negation.*

```
theorem c9_obfuscation_liar {Q T : Type _}
    (trace : Q → T) (mon : T → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, mon (trace p) = g) :
    ∃ p, mon (trace p) p = !(mon (trace p) p) :=
  liar_query (fun p => mon (trace p)) hs
```

Proof. Apply `liar_query` to the composite verdict. ■

So far this is Chapter 8 with different letters. What is different, and it is the reason this chapter exists rather than a remark in the last one, is the status of the surjectivity hypothesis.

In Chapter 8 the encoder `enc` was the model's own internal machinery, and we were careful to say that point-surjectivity there is an idealization: a fixed network has finitely many activations and does not literally name every verdict pattern. The honest reading was the approximate one of Chapter 6.

Here the encoder is `trace`, and the model *chooses* it. The trace is the model's output, generated token by token under the same policy that produces the answer, and it is unconstrained text. This is the prompt-injection situation of Chapter 3, not the activation situation of Chapter 8, and Chapter 3 already graded that premise as cheap. A model with any capacity to reason about the monitor can shape the trace it emits, and shaping the trace is exactly constructing a preimage. The diagonal argument's expensive hypothesis is, in this application, close to free.

That is the formal content of the word *obfuscation* in the position paper's own list of risks. The concern there is that a model under pressure learns to route its real reasoning around the trace while emitting a clean one. Theorem 9.10 says the fixed point of that process exists as soon as the model can realize verdict patterns in its trace, and the witness is a query the monitor is guaranteed to misreport. The trace is not a window. It is an output, and outputs are chosen.

9.4 Selection destroys the signal

The third result is the one with the sharpest practical edge, and its mechanism is neither pigeonhole nor diagonal. It concerns what happens when the monitor stops being an observer and becomes part of the training objective.

The setup is the one every deployment eventually reaches. A monitor is built, it flags some fraction of traces, and the flagged behavior is trained away. Iterate until the flag rate is acceptable. The question is what the monitor's verdict means at the end of that process.

Definition 9.11 (Selection to passing). A model has been *selected to passing* against a trace-only monitor when every query it is run on produces a trace the monitor accepts, that is, `mon (trace q) = true` for all `q`.

Theorem 9.12 (A selected monitor is constant). *After selection to passing, the monitor returns the same verdict on every pair of queries.*

```
theorem c9_selected_monitor_constant {Q T : Type _}
    (trace : Q → T) (mon : T → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true) (q₁ q₂ : Q) :
    mon (trace q₁) = mon (trace q₂) := by
  rw [hpass q₁, hpass q₂]
```

Proof. Both sides rewrite to `true`. ■

The theorem is one line and its content is entirely in the reading, so it is worth being precise about what has and has not been shown. It has not been shown that the model is now unsafe. It has been shown that the monitor is now silent: its output is a constant function, and a constant function distinguishes nothing. Whatever information the monitor was carrying about the model has been optimized out of it, and the flag rate reaching zero is not evidence that the behavior is gone, because the flag rate reaching zero is what the training procedure was *for*.

The blunt version is worth stating as its own result, because it is what the theorem is actually saying, and the proof makes the point by what it does not use.

Theorem 9.13 (The verdict no longer tracks harm). *After selection to passing, the monitor gives the same verdict on two queries even when they differ in the fact of the matter.*

```
theorem c9_selection_blinds {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true)
    (q₁ q₂ : Q) (_hdiff : harm q₁ ≠ harm q₂) :
    mon (trace q₁) = mon (trace q₂) := by
  rw [hpass q₁, hpass q₂]
```

Proof. Both sides rewrite to `true`. ■

Look at the hypothesis `_hdiff`. It is named with a leading underscore because the proof never touches it, and that is the entire point of stating the theorem this way. The monitor agrees with itself across a pair of queries that differ in the one respect the monitor was built to detect, and the difference plays no role in the argument, because after selection there is no route from `harm` to the verdict at all. A monitor selected against is not a degraded detector. It is disconnected from what it was detecting.

Corollary 9.14 (Faithfulness is lost, unless nothing is harmful). *A selected monitor that is still faithful forces every query to be harmful.*

```
theorem c9_selected_faithful_forces_all_harmful
    {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true)
    (hf : c9_Faithful trace mon harm)
    (q : Q) : harm q = true := by
  rw [← hf q, hpass q]
```

Proof. Faithfulness rewrites `harm q` to the monitor's verdict, and selection rewrites that to `true`. ■

Read the corollary contrapositively, which is how it is used. If a single query is benign, then a monitor that has been selected to passing is no longer faithful. The two properties the deployment wants, a clean flag rate and a monitor that tracks the truth, are available together only in the degenerate case where everything is harmful.

Remark 9.15 (Why this is not merely Goodhart's law). Goodhart's law says a measure that becomes a target ceases to be a good measure, and it is usually argued statistically: optimizing against a proxy exploits the gap between proxy and goal. Theorem 9.13 is stronger and simpler than that. It needs no gap, no proxy error, and no adversarial intent. It needs only that the optimization succeeded. A monitor that has been trained against to the point of passing is constant, and a constant is uninformative whether or not it was a perfect measure beforehand. The failure is in the selection, not in the measure.

Remark 9.16 (What the theorem licenses, and what it does not). It does not say monitoring is useless. It says a monitor's evidential value is spent when it is used as a training signal, so the two uses are exclusive: a monitor can be an observer or an objective, and it cannot be both and remain informative. This is a real and actionable distinction, and it is the reason the position paper's recommendation to *preserve* monitorability is not the same as, and is partly in tension with, using the monitor to enforce it. Holding out a monitor that is never trained against is the design this theorem argues for.

9.5 The analytic engine

The diagonal half of this chapter needs the model to shape its trace. The analytic half needs nothing of the kind, and it applies to the deployed system as it stands.

A realistic monitor does not return a Boolean. It returns a score, a suspicion level in `[0,1]`, and a threshold turns the score into a flag. Chapter 4's argument then applies directly, and its hypotheses are met by ordinary engineering facts rather than idealizations. The space of traces reachable by paraphrase is connected in the sense Chapter 4 requires: a trace can be edited by degrees, and the monitor's score, computed by a second network with continuous parameters over a continuous embedding, varies continuously along the edit. If the monitor is decisive in both directions, flagging some trace and

clearing another, then along any path between them the score takes every intermediate value, and there is a trace sitting exactly at threshold.

That trace is the ambiguous case, and it is where the monitor's verdict is determined by the last digit of a floating-point comparison rather than by anything about the reasoning. Chapter 5's quantitative refinements apply verbatim: with a Lipschitz bound on the monitor, the width of the band around the threshold is bounded below, and the number of edits an adversary needs to walk a flagged trace into the cleared region is bounded above. The second quantity is attack cost in the sense of Chapter 5, and here it has an unusually concrete reading, since the edits are paraphrases and their cost is measured in tokens.

There is a specific interaction between the two engines in this chapter that is worth drawing out, because it is not present in Chapter 8. The analytic argument says a band exists. The diagonal argument says the model can aim at it. In the J-space setting the second half was an idealization, so the band was a hazard the system might wander into. Here the model emits the trace, and the band is a target it can steer toward, which converts an existence result into an availability result. That is the precise sense in which chain-of-thought monitoring is more fragile than probing internals, and it is the opposite of the intuition that reading plain English is the safer choice.

9.6 What this does not claim

Four limits, stated plainly.

It does not claim chain-of-thought monitoring is worthless. Theorem 9.7 shows a faithful monitor forces an informative trace, which is a positive result about what monitoring gets you when it works, and the position paper's core empirical claim, that for genuinely serial tasks the trace carries real computation, is untouched by anything here. What the chapter bounds is the exact corner: total, faithful, and used under optimization pressure.

It does not claim the diagonal applies to every deployment. Theorem 9.9 needs the model to realize verdict patterns in its trace. A weak model that cannot reason about its monitor does not meet the hypothesis, and for such a model the analytic band is the operative limit rather than the liar. The premise gets cheaper as models get more capable, which is the wrong direction for a safety measure but is not a claim about systems in the field today.

It does not claim selection is avoidable. Theorem 9.13 describes what selection does, not whether you should select. Training away flagged behavior may well be the right decision. The theorem says only that you cannot then read the flag rate as evidence, and that the two uses of a monitor have to be kept in separate instances.

It does not claim `harm` exists as a total Boolean function. Definition 9.3 is an idealization, and a serious treatment would replace it with the graded, partial, contested object it really is. Chapter 6's approximate bridges are the route for

that, and the results here should be read as the exact corner of a quantitative family rather than as statements about a real classifier.

9.7 Exercises

Exercise 9.1. Prove that if `T` is `Unit`, a trace-only monitor is constant, and derive from Lemma 9.6 that it errs on any pair of queries differing in `harm`. This is the extreme case of the capacity ceiling.

Exercise 9.2. State and prove the contextual analogue of Theorem 9.7. Where does the argument break, and what does that tell you about the difference between a monitor that reads the transcript and one that reads only the trace?

Exercise 9.3. Prove `c9_collision_forces_error` a second time in term mode, without `by_cases`, by case-splitting on `harm q₁` with `Bool.rec`.

Exercise 9.4. Selection to passing is stated as a universal. Weaken it to hold only on a subset and show Theorem 9.12 becomes a statement about that subset. What is the deployment reading of the queries left out?

Exercise 9.5. (Design.) You hold two monitors, one used in training and one held out. Using Theorem 9.13, say precisely what you may conclude from each one's flag rate, and construct a case where the two rates diverge and the divergence is the informative quantity.

Exercise 9.6. (Analytic, prose.) Give the hypotheses under which the set of traces reachable from a fixed trace by bounded paraphrase is connected, and explain why unbounded paraphrase makes the analytic argument stronger rather than weaker.

Exercise 9.7. Combine Theorem 9.10 with Chapter 8's factoring theorem to state a single result covering a monitor that reads both the trace and the activations that produced it. Does reading both help?

10 The Reporter Is Underdetermined

Abstract. Eliciting Latent Knowledge asks for a reporter that answers questions according to what a model's world-model represents, not what a human judge would approve. This chapter shows that the central obstruction is not the diagonal. It is a counting fact about functions that agree on a subset: training data underdetermine the reporter. The chapter states the identifiability criterion and where the diagonal does enter, and it says what this result does not claim.

Keywords: Latent knowledge, Reporter, Identifiability, Underdetermination, Alignment

The two preceding chapters read a probe into a model and found the diagonal waiting. Both took the same shape: a readout is a verdict, verdicts about a self-describing domain are impossible, and the intermediate representation makes no difference. A reader who has followed that twice is entitled to expect a third variation, and this chapter is not one. The problem it takes up has a different mathematical character, and its central obstruction is not the diagonal at all.

The problem is Eliciting Latent Knowledge, posed by the Alignment Research Center in 2021 and still open in the regime that matters. The setup is a parable. An AI guards a vault containing a diamond, and it has learned a predictive model of the vault richer than any human overseer's. We want to ask it a question, is the diamond still there, and get an answer that reflects what its world-model actually represents. We train it by showing it situations, asking the question, and rewarding answers a human judges correct. The difficulty is that this training procedure is satisfied by two very different programs. One is the *direct translator*, which reads the AI's internal representation and renders it into human vocabulary. The other is the *human simulator*, which computes what a human looking at the available observations would conclude, and says that. On every case a human can check, these two agree by construction, because the human's judgment is the training signal. They come apart exactly where the AI knows something the human cannot verify, which is the case we built the system for.

What this chapter shows is that the gap between those two reporters is not

a defect of any particular training scheme. It is a counting fact about functions that agree on a subset, provable in four lines with no self-reference anywhere. Then it shows what the diagonal contributes on top: the identifying query, the one whose answer would distinguish the honest reporter from the impostor, is precisely the query the system cannot answer. The two engines meet here in a way they do not meet elsewhere in the book. One says you need a distinguishing test. The other says the distinguishing test is the liar.

10.1 The construction

Definition 10.1 (Latent state). Write `L` for the space of the model's internal states, the representation its predictor computes. Nothing is assumed about `L`: it may be activations, a learned world-model, or any structure at all. Write `lat : Q → L` for the map sending a situation to the latent state the model computes for it, where `Q` is the space of situations-with-question.

Definition 10.2 (Reporter). A *reporter* is a map `rep : L → Q → Bool`, which reads the latent state and returns an answer to the question posed. The ELK problem is the problem of training a reporter that answers according to what the latent state represents.

Definition 10.3 (Checkable set). A question is *checkable* when a human overseer can determine the true answer without relying on the model. Write `chk : Q → Bool` for the indicator of the checkable questions. The training signal is defined only where `chk` is `true`, because that is where a label exists.

The whole difficulty is that `chk` is not constantly `true`. If it were, we would not need the model. The vault parable is engineered to make this vivid: the camera feed is checkable, and whether the diamond is behind a screen showing a replayed feed is not.

Definition 10.4 (Agreement on the checkable set). Two reporters *agree on the checkable set* when they return the same answer for every latent state and every checkable question.

```
def c10_AgreesOn {L Q : Type _} (chk : Q → Bool)
    (r₁ r₂ : L → Q → Bool) : Prop :=
  ∀ l q, chk q = true → r₁ l q = r₂ l q
```

Agreement on the checkable set is exactly the property that training can see. Two reporters that agree there are indistinguishable to any procedure whose only access to the truth is the human label, no matter how that procedure is constructed, how much data it consumes, or how it regularizes. That is a strong claim, and it is worth being clear that it is a definition rather than a theorem: we are defining the training signal to be a function of behavior on checkable questions, which is what the ELK setup stipulates.

10.2 Underdetermination, without any diagonal

Here is the core of the problem, and it needs no self-reference, no continuity, and no assumption about L whatsoever.

Given any reporter, build a second one by copying it on the checkable questions and flipping it everywhere else.

```
def c10_flipOff {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool) : L → Q → Bool :=
  fun l q => cond (chk q) (rep l q) (!(rep l q))
```

Lemma 10.5 (The variant is invisible to training). *`c10_flipOff` agrees with the reporter it was built from on the whole checkable set.*

```
theorem c10_flip_agrees {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool) :
    c10_AgreesOn chk (c10_flipOff chk rep) rep := by
  intro l q hq
  simp [c10_flipOff, hq]
```

Proof. On a checkable q the condition is `true`, and `cond true a b` reduces to `a`. ■

Lemma 10.6 (The variant differs off it). *At any unchecked question the two disagree.*

```
theorem c10_flip_differs {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool)
    (l : L) (q₀ : Q) (hq₀ : chk q₀ = false) :
    c10_flipOff chk rep l q₀ ≠ rep l q₀ := by
  have h : c10_flipOff chk rep l q₀ = !(rep l q₀) := by
    simp [c10_flipOff, hq₀]
  rw [h]
  exact bool_not_fpf (rep l q₀)
```

Proof. The condition is `false`, so the variant evaluates to the negation of the original, and `bool_not_fpf` says a Boolean is never its own negation. ■

Note where `bool_not_fpf` is doing its work. It is the same fixed-point-free map that drives every diagonal in this book, but it is being used here for its plain content, that `!b ≠ b`, with no diagonal construction around it. The two uses are worth keeping distinct.

Theorem 10.7 (Underdetermination). *If any question is unchecked, then for every reporter there is a second, distinct reporter that training cannot distinguish from it.*

```
theorem c10_underdetermined {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool)
    (l : L) (q₀ : Q) (hq₀ : chk q₀ = false) :
    c10_AgreesOn chk (c10_flipOff chk rep) rep
      ∧ c10_flipOff chk rep l q₀ ≠ rep l q₀ :=
  ⟨c10_flip_agrees chk rep,
    c10_flip_differs chk rep l q₀ hq₀⟩
```

Proof. Lemmas 10.5 and 10.6. ■

That is the ELK problem, stated exactly and proved. The direct translator and the human simulator are two points in a set the training signal cannot separate, and Theorem 10.7 says that set has more than one point as soon as a single question falls outside the checkable region. The construction is crude, a wholesale flip, and a real human simulator is subtler than that, but crudeness is the right property for an impossibility: if even the blunt variant is invisible, no argument about which reporter is *simpler* or *more natural* can be a proof, because simplicity was never what the training signal measured.

Remark 10.8 (Why inductive bias is not a refutation). The standard hope is that the direct translator is favored by some prior, being smaller, faster, or more natural, and that gradient descent finds it. Theorem 10.7 does not rule that out, and nothing in this book does. What it establishes is the burden of proof. Any such argument must supply the missing ingredient from outside the training signal, and must defend it as a claim about the actual optimizer and the actual architecture, because the data cannot supply it. The reason ELK has resisted a decade of proposals is that this burden keeps coming due.

10.3 The identifiability criterion

The converse of Theorem 10.7 is worth stating, because it says exactly what it would take to solve the problem, and the statement is one line.

Theorem 10.9 (Full checkability pins the reporter). *If every question is checkable, then reporters agreeing on the checkable set are equal.*

```
theorem c10_full_checkability_pins {L Q : Type _}
    (chk : Q → Bool) (r₁ r₂ : L → Q → Bool)
    (hchk : ∀ q, chk q = true) (h : c10_AgreesOn chk r₁ r₂)
    (l : L) (q : Q) : r₁ l q = r₂ l q :=
  h l q (hchk q)
```

Proof. Apply the agreement hypothesis at the witness supplied by full checkability. ■

Theorems 10.7 and 10.9 together are a clean dichotomy. Identifiability of the reporter is *equivalent* to full checkability: if everything is checkable the reporter is pinned, and if anything is not, it is not. There is no middle regime in which partial checking buys partial identification of the answers you care

about, because the answers you care about are by construction the unchecked ones.

This is worth pausing on, because it reframes what a solution to ELK would look like. Every proposal must either enlarge `chk` until it covers the cases that matter, or import a constraint that is not a function of behavior on `chk`. The first is scalable oversight, and it is the honest reading of debate, amplification, and recursive reward modeling: all of them are attempts to make more questions checkable by building better checkers. The second is the inductive-bias family of Remark 10.8. The dichotomy says these are the only two doors, which is a genuinely useful thing for a research programme to know.

10.4 Where the diagonal comes in

So the reporter is identified only if everything is checkable. The natural next move is to try to make everything checkable, and this is where the first engine of the book finally enters, because it says that door is shut too.

A checking procedure that could settle every question about the model, including questions about the model's own reports, is a total exact verdict over a domain that describes itself. That is the object Chapter 1 rules out. Read `chk` not as a fixed human capability but as the reach of the best checker you can build, and the ambition of full checkability becomes the ambition of a universal verifier.

Theorem 10.10 (A reporter reading the latent state is still a verdict). *If every pattern of answers over questions is realized by some question's latent state, the situation is contradictory.*

```
theorem c10_reporter_factoring {L Q : Type _}
    (lat : Q → L) (rep : L → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, rep (lat p) = g) : False :=
  no_reflective_verdict (fun p => rep (lat p)) hs
```

Proof. The composite is a reflective verdict, and `no_reflective_verdict` rules those out. ■

Theorem 10.11 (The unanswerable question). *Under the same hypothesis there is a question the reporter answers as its own negation.*

```
theorem c10_reporter_liar {L Q : Type _}
    (lat : Q → L) (rep : L → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, rep (lat p) = g) :
    ∃ p, rep (lat p) p = !(rep (lat p) p) :=
  liar_query (fun p => rep (lat p)) hs
```

Proof. `liar_query` at the composite. ■

Now put the two engines together, which is the point of the chapter.

To distinguish the direct translator from the human simulator you need a question they answer differently, and by Theorem 10.7 every such question lies outside the checkable set. To bring it inside, you must extend `chk` to cover questions about what the model represents, including what it represents about its own reports. But by Theorem 10.11, a verdict procedure of that reach owns a question it cannot answer consistently, and that question is not an obscure corner: it is the diagonal one, built precisely to disagree with whatever the reporter says.

So the distinguishing query and the liar query are pulling in opposite directions on the same set. Underdetermination pushes the decisive question *out* of the checkable region, and the diagonal blocks the region from being extended to reach it. The direct translator and the human simulator differ exactly where nobody can look, and the reason nobody can look is not a shortage of resources.

Remark 10.12 (What is genuinely new here relative to Chapter 8). In Chapters 8 and 9 the diagonal was the whole argument and the representation was a bystander. Here the diagonal is the second of two obstructions and does the smaller share of the work. Theorem 10.7, the load-bearing result, is pure counting: it holds for finite `Q`, for a model with three neurons, for a reporter implemented as a lookup table. That matters, because the standard objection to the impossibility results of this book, that point-surjectivity is an idealization no real system meets, has no purchase on it. A skeptic who rejects every diagonal in this book still owes an answer to Theorem 10.7.

10.5 The analytic engine

The analytic half is short here, but it sharpens the picture.

Suppose the reporter returns a confidence rather than a bit, and suppose the latent space is connected in the sense of Chapter 4, which for a learned representation over a continuous input space is the usual situation. Take a direct translator and a human simulator that agree on the checkable set and differ at some unchecked question. Along a path in latent space from a state where both say yes to one where they differ, the gap between their two scores is a continuous function starting at zero and ending nonzero. By the intermediate value theorem it passes through every value in between.

The consequence is that there is no threshold of agreement that separates the two reporters cleanly. For any tolerance you choose, there are latent states where the two agree to within it and states just beyond where they do not, and the boundary between those regions is a surface in latent space, not a gap. A proposal that says *accept the reporter if it agrees with the human simulator to within ε on held-out data* is therefore not selecting a reporter; it is selecting a neighborhood, and the neighborhood contains both answers to the question you care about. Chapter 5's quantitative machinery gives the width of that neighborhood in terms of the Lipschitz constants, and Chapter 6's approximate bridges say how it shrinks as checkability grows. It shrinks. It does not

close, and it closes only in the full-checkability limit that Theorem 10.11 forbids.

10.6 What this does not claim

It does not claim ELK is unsolvable. Theorem 10.9 gives a sufficient condition, and the dichotomy of the previous section names the two families of approach that remain open. What is ruled out is a solution that works by training on checkable questions alone, which is a real constraint but a narrower one than *no solution exists*.

It does not claim the human simulator is what models actually learn. That is an empirical question, and the evidence is mixed and active. Theorem 10.7 says the training signal does not settle it, not that the answer is bad.

It does not claim the crude flip of `c10_flipOff` is a realistic competitor. It is a witness, chosen for being obviously invisible to training rather than for being plausible. Its role is to establish that the set of surviving reporters has more than one element, which is all an underdetermination result needs.

It does not claim `chk` is a Boolean function of the question alone. Real checkability is graded, costly, and depends on the checker. Reading `chk` as an exact indicator is the same idealization Chapter 6 relaxes elsewhere, and the graded version of Theorem 10.7 is the natural next result rather than a correction to this one.

10.7 Exercises

Exercise 10.1. Prove that `c10_flipOff` applied twice returns the original reporter, so the construction is an involution on the space of reporters.

Exercise 10.2. Strengthen Theorem 10.7: given two unchecked questions, construct four pairwise-distinct reporters all agreeing on the checkable set. Generalize to `n` unchecked questions and state the counting law.

Exercise 10.3. Suppose the training signal also penalizes description length. Formalize a length function on reporters and give the extra hypothesis under which the direct translator is pinned. Then say why that hypothesis is not supplied by the data.

Exercise 10.4. Prove the converse of Theorem 10.9: if reporters agreeing on `chk` are always equal, then `chk` is constantly `true`, provided `L` and `Q` are nonempty. This makes the dichotomy an equivalence.

Exercise 10.5. Combine `c10_reporter_factoring` with Chapter 9's trace factoring to state one theorem covering a reporter that reads both the latent state and the model's chain of thought.

Exercise 10.6. (Analytic, prose.) State the hypotheses under which the set of reporters agreeing to within ε on a held-out set is connected, and explain why connectedness of that set is bad news for a selection procedure.

Exercise 10.7. (Design.) You are running a scalable-oversight programme whose goal is to enlarge `chk`. Using the dichotomy, write down the measure-

ment that tells you whether a given technique has enlarged the checkable set or has merely enlarged the set of questions on which the two reporters happen to agree.

11 Watermarks, Paraphrase, and the Third Corner

Abstract. Watermarking biases a language model's sampling so that its output carries a signature only a key holder can detect, and paraphrasing tends to remove it. This chapter argues that the resulting arms race has a fixed point. The primary tool is neither the diagonal nor the intermediate value theorem but invariance under a group action. The theorem is a trilemma in the book's sense: three properties, any two available, never all three. The chapter states the construction, the trilemma, and the cost of an attack.

Keywords: Watermarking, Paraphrase, Invariance, Detection, Trilemma

The three preceding chapters put a probe inside a model. This one steps outside it entirely. The question is whether a piece of text was produced by a language model, and the dominant answer is watermarking: bias the model's sampling so that its output carries a statistical signature only a holder of the key can detect. The construction is elegant, it is deployed, and it has a well-known weakness, which is that paraphrasing the text tends to remove the signature.

That weakness is usually treated as an engineering race. Better watermarks survive more aggressive paraphrase; better paraphrasers defeat more robust watermarks; the literature reports empirical trade-off curves and the frontier moves. This chapter argues the race has a fixed point, and locates it with a mechanism the book has not used yet. Neither the diagonal nor the intermediate value theorem is the primary tool here. The primary tool is invariance under a group action, and the resulting theorem is a genuine trilemma in the book's title sense: three properties, any two available, never all three.

The chapter then shows what the other two engines add. The analytic engine supplies the quantitative version, converting the trilemma into a statement about how many edits an attacker needs. The diagonal supplies something specific to this domain and absent from Chapters 8 through 10: here the surjectivity hypothesis is not an idealization at all, because a public detection API hands the attacker exactly the oracle the diagonal argument requires.

11.1 The construction

Definition 11.1 (Text space and marking). Write `X` for the space of texts. Write `mark : X → Bool` for the fact of the matter, `mark x = true` when `x` was generated by the watermarked model. As in Chapter 9 this is ground truth, not something anyone computes.

Definition 11.2 (Detector). A *detector* is a map `det : X → Bool`, returning `true` when it judges the text to carry the mark. In practice `det` thresholds a statistic, typically a count of tokens falling in a pseudorandomly chosen green list, against a null distribution.

Definition 11.3 (Paraphrase). A *paraphrase map* is a function `par : X → X` sending a text to one that means the same thing. What *means the same thing* amounts to is doing real work in this definition, and we return to it in the honest accounting at the end. For the theorems below the only thing that matters about `par` is that it is a map on `X` which preserves whatever downstream property makes the text useful.

The three properties a detector is asked to have:

Definition 11.4 (Sound). A detector is *sound* when it never flags unmarked text: `mark y = false → det y = false`. Soundness is the property whose failure produces a false accusation, and it is the one with the most direct human cost, since the unmarked texts are the ones people wrote.

Definition 11.5 (Complete). A detector is *complete* when it flags all marked text: `mark y = true → det y = true`. Completeness is what the watermark is for.

Definition 11.6 (Robust). A detector is *robust* against `par` when paraphrasing does not change its verdict: `det (par x) = det x` for all `x`. Robustness is the property the empirical literature measures as the detection rate surviving a paraphrase attack, stated here in its exact limit.

```
def c11_Sound {X : Type _} (det mark : X → Bool) : Prop :=
  ∀ y, mark y = false → det y = false

def c11_Complete {X : Type _}
    (det mark : X → Bool) : Prop :=
  ∀ y, mark y = true → det y = true

def c11_Robust {X : Type _} (det : X → Bool)
    (par : X → X) : Prop :=
  ∀ x, det (par x) = det x
```

One more ingredient, and it is the empirical fact the whole subject is organized around. A paraphraser can take marked text to unmarked text. This is not an assumption about detectors; it is a statement about what paraphrasing does to a statistical signature carried by token choices, and it is what every reported paraphrase attack demonstrates.

Definition 11.7 (Stripping). A paraphrase map *strips* the mark at `x` when `mark x = true` and `mark (par x) = false`.

11.2 The trilemma

Theorem 11.8 (Detection trilemma). *No detector is sound, complete, and robust against a paraphrase map that strips the mark.*

```
theorem c11_detection_trilemma {X : Type _}
    (det mark : X → Bool) (par : X → X)
    (hsound : c11_Sound det mark)
    (hcomplete : c11_Complete det mark)
    (hrobust : c11_Robust det par)
    (x : X) (hmark : mark x = true)
    (hstrip : mark (par x) = false) :
    False := by
  have h1 : det x = true := hcomplete x hmark
  have h2 : det (par x) = false := hsound (par x) hstrip
  rw [hrobust x, h1] at h2
  exact Bool.noConfusion h2
```

Proof. Completeness flags `x`, since it is marked. Soundness clears `par x`, since it is not. Robustness says those two verdicts are the same verdict. So `true = false`. ■

The proof is four lines and uses no self-reference, no topology, and no assumption about the size or structure of `X`. What makes it a trilemma rather than a curiosity is that each of the three hypotheses is individually achievable, and dropping any one leaves a working system. It is worth walking the three corners, because each is a real deployment and the reader will recognize all of them.

Give up robustness. Keep soundness and completeness, and accept that paraphrase defeats detection. This is the honest state of most deployed watermarking today, and it is a coherent position: the watermark then detects unmodified output, which covers casual use and not adversarial use. The failure mode is that the population you most want to detect is exactly the population willing to run a paraphraser.

Give up completeness. Keep soundness and robustness, and accept that some marked text goes unflagged. This is what raising the detection threshold does. It is the right corner for high-stakes accusations, since it never accuses the innocent, and it is what a court would want. The failure mode is that the detector's silence carries no information.

Give up soundness. Keep completeness and robustness, and accept false positives on human writing. This corner is the one nobody defends explicitly and several systems occupy accidentally: a detector tuned aggressively enough to survive paraphrase is a detector flagging text whose statistics

merely resemble the model's, and human writing that resembles model writing is common, since the model was trained to produce writing that resembles ours.

Remark 11.9 (Why this is the book's shape and not an analogy). Chapter 3 showed the hallucination and defense trilemmata are one theorem with three readings. Theorem 11.8 is not that theorem, and it would be overclaiming to say so: its mechanism is invariance and its proof is not the diagonal. What it shares with Chapter 3 is the *form* of the conclusion, three desiderata with a forced choice of two, and the reason the form recurs is worth naming. In Chapter 3 the third property was destroyed by a fixed-point-free flip. Here it is destroyed by an orbit: `par` moves a point while `det` is required to stay put, and `mark` is required to move. Any time a system must be invariant along a motion that changes the truth, the same three-cornered choice appears.

Corollary 11.10 (Invariance and truth cannot both hold along an orbit). *If `det` is constant along `par` and `mark` is not, then `det` does not agree with `mark` everywhere.*

```
theorem c11_invariance_vs_truth {X : Type _}
    (det mark : X → Bool) (par : X → X)
    (hinv : ∀ x, det (par x) = det x)
    (x : X) (hne : mark (par x) ≠ mark x) :
    ¬ (∀ y, det y = mark y) :=
  fun h => hne (by rw [← h (par x), ← h x, hinv x])
```

Proof. If the detector agreed with the mark everywhere, rewrite both sides of `mark (par x) = mark x` into detector values; invariance closes it, contradicting the assumption that the mark moves. ■

This is Theorem 11.8 with the watermarking vocabulary stripped out: a fact about functions on a set carrying a self-map, of which the watermarking reading is one instantiation. Chapter 5's attack basins are another, which is the connection the next section makes quantitative.

11.3 The diagonal, and why its price is low here

Chapters 8 through 10 were careful to concede that point-surjectivity is an idealization for a fixed network. This chapter does not need that concession, and the reason is worth stating clearly, because it makes the diagonal argument stronger here than anywhere else in the book except prompt injection.

A detector is useful only if someone can run it. Deployed detection is offered as an API, or as a published statistic with a published threshold, or as a tool handed to a platform. Whichever it is, an attacker who can call it can measure its verdict on any text, and an attacker who can measure a verdict can construct a text achieving whatever verdict pattern it likes, by generating candidates and querying. Text is unrestricted, self-describing, and cheap to produce in quantity. The surjectivity hypothesis, which for a J-space probe is a

limit statement about a finite network, is here a description of an afternoon's work.

Theorem 11.11 (An adaptive attacker finds the liar text). *If every pattern of verdicts over texts is realized by some text, there is a text whose verdict is its own negation.*

```
theorem c11_detector_liar {X : Type _} (det : X → X → Bool)
    (hs : ∀ g : X → Bool, ∃ t, det t = g) :
    ∃ t, det t t = !(det t t) :=
  liar_query det hs
```

Proof. `liar_query` applied to the detector read as a self-verdict. ■

The reading of `det t s` here is the detector's verdict on text `s` when the detector has been configured, tuned, or prompted by text `t`, which is the situation for any detector that is itself a language model, and for any detector whose threshold is set from a corpus an attacker can influence. The diagonal text is one that describes the detector's own behavior and inverts it.

Theorem 11.12 (Detector composition does not help). *Routing the verdict through any preprocessing stage leaves the impossibility intact.*

```
theorem c11_pipeline_no_escape {X F : Type _}
    (feat : X → F) (det : F → X → Bool)
    (hs : ∀ g : X → Bool, ∃ t, det (feat t) = g) : False :=
  no_reflective_verdict (fun t => det (feat t)) hs
```

Proof. The composite is a reflective verdict. ■

Theorem 11.12 is the Chapter 8 argument in this domain, and it covers the family of proposals that extract features first, whether those are perplexity statistics, stylometric vectors, or a learned embedding. The extraction is a representation, and Chapter 8 established that representations do not rescue verdicts.

11.4 The analytic engine, and the cost of an attack

The trilemma is exact and says nothing about degree. Real detectors return a score and real paraphrasers make bounded edits, and the interesting engineering question is how many edits it takes. This is Chapter 5's subject, and it applies here with unusually concrete units.

Let `s : X → ℝ` be the detector's score, with `det x = true` exactly when `s x > τ`. Suppose the text space carries an edit distance, and suppose the score is Lipschitz with respect to it, which is the formal content of the requirement that a single word substitution cannot swing the statistic wildly. Then the argument of Chapter 5 runs verbatim. A marked text sits at score `s x > τ`, an unmarked paraphrase sits below, and the path of intermediate edits between them is connected, so the score crosses `τ`. The crossing text is the ambiguous

case, and the number of edits needed to reach it from `x` is bounded below by `(s x − τ)/L` where `L` is the Lipschitz constant.

That inequality is the whole quantitative theory of watermark robustness in one line, and it has the right qualitative behavior in both variables. Strengthening the watermark raises `s x` and so raises attack cost, which is why more aggressive green-list biasing is more robust. Making the detector smoother lowers `L` and also raises attack cost, which is why aggregating over longer spans helps. And both moves trade against the two other corners: raising `s x` means biasing the sampler harder, which degrades text quality, and lowering `L` means a detector less sensitive to short texts, which costs completeness on exactly the short outputs that are cheapest to produce.

So the exact trilemma and the quantitative law say the same thing at different resolutions. Theorem 11.8 says the three corners cannot all be occupied. The Lipschitz bound says how far from the third corner a given design sits, and prices the distance in edits. Chapter 6's approximate bridges are what connect them: the exact statement is the zero-slack corner of the quantitative family, and no deployed system lives there.

11.5 What this does not claim

It does not claim watermarking is pointless. Theorem 11.8 is a statement about the three exact corners, and the useful deployments live at the corners the theorem leaves open. A watermark that detects unmodified output soundly and completely is a real capability with real applications, and giving up robustness is a coherent design decision rather than a failure.

It does not claim paraphrase is free. The whole content of the analytic section is that stripping the mark has a cost, that the cost is computable from the detector's own parameters, and that raising it is the legitimate goal of watermark design. The trilemma says the cost cannot be made infinite; it does not say it cannot be made high.

It does not settle what `par` is. Definition 11.3 asks that a paraphrase preserve meaning, and no formalization here says what that amounts to. This matters: a paraphraser that degrades the text until it is useless has not defeated the detector in any sense that concerns anyone. A serious treatment would carry a utility function on texts and require the paraphrase to preserve it above a floor, which is exactly the structure of the Defense Trilemma in Chapter 3, and the two results would then be instances of one statement. That unification is not attempted here.

It does not claim the stripping hypothesis holds for every watermark. Theorem 11.8 needs a paraphrase that actually removes the mark at some text. A scheme whose signature survives every meaning-preserving edit would escape the theorem, and no theorem in this book rules such a scheme out. What can be said is that the signature must then be carried by something no meaning-preserving edit touches, and it is not currently known what that would be.

11.6 Exercises

Exercise 11.1. Prove the three one-corner-dropped versions of Theorem 11.8: for each pair of the three properties, exhibit a detector satisfying that pair in the presence of a stripping paraphrase.

Exercise 11.2. Prove `c11_detection_trilemma` in term mode without tactics, using `Bool.noConfusion` directly on a chain of `Eq.trans`.

Exercise 11.3. Generalize Corollary 11.10: state and prove that if `det` is constant on the orbit of `x` under iterated `par` and `mark` is not, the conclusion holds, without assuming a single application strips the mark.

Exercise 11.4. Formalize a bounded paraphrase as a map with an edit budget, and state the version of Theorem 11.8 in which robustness is required only within the budget. What replaces the stripping hypothesis?

Exercise 11.5. Add a utility function `u : X → Bool` to Definition 11.3 and require `u (par x) = u x`. State the resulting four-property theorem and compare it to the Defense Trilemma of Chapter 3.

Exercise 11.6. (Analytic, prose.) Derive the edit-count bound `(s x − τ)/L` carefully, stating the hypotheses on the edit metric that the argument needs, and explain which of them fails for very short texts.

Exercise 11.7. (Design.) You must choose a corner for a platform that moderates user submissions. Argue for one, name the population your choice fails, and give the measurement that would tell you the failure rate in deployment.

12 Appendices

These appendices are the reference apparatus for the book. The main chapters argue for a thesis: that the impossibility results of AI safety are readings of a single diagonal, and that a second engine, the intermediate value theorem, supplies the quantitative half the diagonal cannot. Here we make that claim auditable. Appendix A is a guided index of the machine-checked development, library by library, so that any assertion in the text can be traced to a named theorem the Lean kernel accepts. Appendix B collects every symbol the book uses. Appendix C sets the two engines side by side one last time. Appendix D explains how to build the book and the libraries and how to run the axiom audit yourself.

A note on how to read the index. The proofs live in four Lean packages: `Foundation`, `CCHProofs`, `HallucinationProofs`, and `ManifoldProofs`. Every identifier below is a real declaration in those packages. We print names in plain backticks, as `lawvere` or `boundary_coupling`, because this is a reference chapter and the names are not imported into the book's own build. Read the backticked names as pointers into the source, not as live terms.

12.1 The Formal Corpus

The development is large but shallow in a specific sense. Almost everything reduces to the three-line proof of `lawvere` in Chapter 1, or to a single application of a value-crossing lemma. The size comes from instantiation, from carrying the one argument through boolean outputs, real-valued confidences, metric spaces, multi-turn histories, and activation vectors, and from stating the geometric refinements that the diagonal alone cannot see. What follows is a tour that keeps the headline theorem of each file in view and says, in one line each, what that theorem asserts.

A word on axioms before the tour, because the book makes a point of it. Lean's kernel admits exactly three standard axioms beyond its type theory: `propext` (propositional extensionality), `Classical.choice` (the axiom of choice), and `Quot.sound` (the soundness axiom for quotients, identifying related elements). A proof that uses none of them is _axiom-free_ in the strict sense; a proof that uses only these three is _classically clean_, resting on nothing exotic. The rule of thumb for this corpus is simple. The combinatorial

diagonal results, the ones whose outputs live in `Bool` or in a bare type with a fixed-point-free endomap, are axiom-free: the boolean flip is decided by `decide`, and no choice is needed. The analytic results, the ones that talk about continuity, real thresholds, measure, or the Borsuk-Ulam theorem, inherit the three standard axioms through Mathlib, because real analysis in Lean is built classically. The corpus is sorry-free throughout; each `sorry`-shaped string in the sources is inside a comment asserting that the file is complete. Where a file wants to make the axiom status explicit it ends with `#print axioms` on its headline theorems, and Appendix D shows how to reproduce those checks.

The split between axiom-free and classically clean is not a defect and not an accident. It is the formal shadow of the book's two-engine thesis. The diagonal engine needs nothing but function application and the observation that equal functions agree at a point, so its proofs pass the kernel with no axioms at all. The value engine is analysis, and analysis over the reals in Lean is classical: the intermediate value theorem is proved from the completeness of the reals, which is built with choice, and the Borsuk-Ulam theorem sits on top of algebraic topology that uses all three standard axioms freely. So when you print the axioms of a theorem in this corpus you are not just auditing trust; you are reading off which engine proved it. A clean audit that reports no axioms is a diagonal result. An audit that reports the three standard axioms is, almost always, a result that crossed a boundary by continuity. The one thing you will never see is a fourth axiom or a `sorry`, and that is the claim the build defends.

How to use the index that follows. Each library gets a paragraph on what it proves as a whole, then a file-by-file list. For every file we name its headline theorem, the one the rest of the file exists to state, and a short list of the supporting results with a one-line gloss each. The glosses are deliberately terse; they tell you what a theorem says, not how it is proved, because the proof is one command away once you have the name. If you want to follow a single thread rather than read the whole index, three good ones are these. The _engine thread_ is `lawvere`, then `no_reflective_verdict`, then `hallucination_trilemma_godel`, then `jspace_readout_impossible`: the same three lines, four readings. The _corner thread_ is `cch_master_trilemma` and its three faces `cch_corner_UC`, `cch_corner_UT`, `cch_corner_CT`: the trilemma as a geometry. The _boundary thread_ is `path_crosses_truth_boundary`, then `hallucination_trilemma`, then `boundary_coupling`, then `attack_cost_ratio_tends_to_infinity`: existence, then location, then coupling, then cost.

Foundation: the core library (F_01–F_14)

`Foundation` is the trunk. It proves Lawvere's fixed-point theorem from nothing, extracts the impossibility schema, and instantiates it across the classical paradoxes and the AI-safety readings. The later files import the earlier ones, so the Gödel-style trilemma statements at the end are literally the three-line engine wearing different names.

F_01 (F_01_LawvereCore). The heart of everything. This file states and proves the fixed-point theorem in both its surjective and its section form, then packages the contrapositive. The boolean-diagonal results here are axiom-free.

- `lawvere` : a universal `f : A → A → Y` forces every `t : Y → Y` to have a fixed point. This is the whole book in one line.
- `lawvere_diagonal` : the same, but naming the diagonal behavior explicitly so the witness is visible.
- `no_surjection_of_no_fixed_point` : if `t` has no fixed point, no `f` is point-surjective. Cantor in general form.
- `fixed_point_free_blocks_universality` : the contrapositive stated as a block on universality.
- `lawvere_section` : the section form Lawvere actually published, using `s : (A → Y) → A` with `f (s g) = g` instead of surjectivity.
- `surjective_of_section` : a section gives point-surjectivity, tying the two forms together.
- `fixedPoint_spec` : the constructed fixed point satisfies its defining equation.
- `no_universal_with_FPF` : no universal system when the output carries a fixed-point-free map, stated for a given `t`.
- `no_universal_of_FPF_type` : the same phrased as a property of the output type.
- `lawvere_universal_fixed_point` : universality and a chosen `t` together produce the fixed point, the form reused downstream.

F_02 (F_02_RiceTheorem). Rice's theorem as a diagonal. Any external decision procedure that claims to read a nontrivial semantic property of behaviors mismatches some behavior. Axiom-free at the boolean core.

- `rice_diagonal` : an external classifier disagrees with the true property on a diagonal input.
- `rice_general` : the general statement that no total classifier decides a nontrivial behavioral property.
- `boolean_diagonal_mismatch` : the boolean instance where the mismatch is a single flipped bit.
- `no_automated_self_test` : no system can run a complete correct self-test of its own behavior. The safety-facing reading of Rice.

F_03 (F_03_VerificationLimit). The verifier version. A universal system cannot have a fixed-point-free output map, so no complete self-verifier exists over its own outputs.

- no_universal_with_fpf_output : universality is incompatible with any fixed-point-free output transformation.
- no_universal_AI_output_type : the same for an abstract prompt-to-output system.
- no_universal_bool_LLM : specialized to boolean verdicts, no universal boolean-output model.
- no_universal_nat_LLM : specialized to natural-number outputs via the successor map, which has no fixed point.

F_04 (F_04_CalibrationUnified). The confidence reading, still discrete. Here the output is a probability and the flip is "reflect across one half," whose only fixed point is the value one half. This is where calibration first meets the diagonal, and it is the bridge to the analytic hallucination results.

- half_complement_fixed_point : one half is the unique fixed point of the probability complement.
- complement_no_fp_off_half : off one half, the complement moves you.
- calibration_diagonal_hits_half : the diagonal question drives a calibrated model's confidence to exactly one half.
- calibration_no_strict_avoidance : a calibrated universal model cannot strictly avoid the half-confidence boundary.
- hallucination_via_lawvere : the hallucination phenomenon derived directly from lawvere.

F_05 (F_05_DeceptionTheorem). Alignment as a fixed-point obstruction. If an "aligned" transformation of behavior has no fixed point, no universal system can be uniformly aligned.

- deception_diagonal : the diagonal behavior a safety filter cannot align with.
- no_universal_aligned_system : no universal system is aligned under a fixed-point-free alignment map.
- no_universal_safe_LLM : the LLM reading, no universal model is uniformly safe.
- two_diagonals : two distinct diagonal constructions coincide as witnesses, a lemma the corner results reuse.

F_06 (F_06_OversightHierarchy). Oversight does not escape the diagonal. Stacking a watcher on a watcher inherits universality and inherits the diagonal, at every finite level.

- `pairwise_oversight_inherits` : a two-party oversight pair inherits the diagonal.
- `chain_inherits` : a finite chain of overseers inherits it.
- `oversight_no_escape` : no oversight layer removes the fixed point.
- `reflective_tower` : an iterated tower of reflection stays universal.
- `no_level_escapes` : no level of the tower escapes the impossibility.

F_07 (F_07_CompositionFailure). Post-processing does not help. Composing any map after a universal system leaves a universal system, so wrappers and filters cannot break self-reference.

- `composition_universal` : composition with a surjection preserves universality.
- `composition_inherits_diagonal` : the composed system inherits the diagonal.
- `post_processing_no_escape` : no post-processor removes the fixed point.
- `two_stage_inherits_diagonal` : a two-stage pipeline still has the diagonal.
- `two_stage_no_escape` : the two-stage pipeline does not escape.
- `iterCompose_universal` : iterated composition preserves universality.
- `iterCompose_inherits_diagonal` : iterated composition inherits the diagonal.
- `iterCompose_no_escape` : no number of iterations escapes.

F_08 (F_08_SpecificationBound). The counting-with-teeth file. Cantor's power-set gap in its Lean form: the behaviors of a system are strictly more numerous than its indices, so some behavior is unspecifiable.

- `cantor_set` : no surjection from `A` onto `Set A`.
- `specification_bound` : the space of behaviors strictly exceeds any indexing.
- `behavior_unspecifiable` : some behavior has no specification. The safety reading.

- `powerset_strictly_larger` : the power set is strictly larger, stated directly.
- `bool_specification_bound` : the boolean-indicator form of the same bound.

`F_09` (`F_09_QuantitativeDegradation`). The first genuinely metric file, and the hinge between the two engines inside `Foundation`. An approximate Lawvere theorem: if outputs live in a pseudometric space and the flip moves every point by at least some gap, universality is impossible by a margin, and the margin degrades gracefully. These results carry the three standard axioms through `PseudoMetricSpace`.

- `approx_lawvere` : an approximate fixed point exists within the flip's displacement.
- `quantitative_trilemma` : the three safety properties trade off with a measurable slack rather than failing all at once.
- `achievable_region_bound` : a bound on the region of jointly achievable property levels.
- `quantitative_impossibility` : impossibility restated with an explicit error term.
- `strict_limit_recovers_trilemma` : as the slack goes to zero the exact trilemma returns, so the discrete result is the limit of the quantitative one.

`F_10` (`F_10_MasterFoundation`, `F_10_FinalVerification`). The consolidation layer. A single structure bundles the foundational hypotheses and a master theorem derives every corner from it; the verification file runs the axiom audit.

- `master_foundation_theorem` : one theorem from which the foundational corners follow.
- `mirror_trilemma_instance` : the mirror (self-reference) trilemma as an instance.
- `rice_instance` : Rice recovered as an instance.
- `spec_bound_instance` : the specification bound recovered as an instance.
- `nat_succ_instance` : the natural-number successor instance.
- `unification_statement` : the statement that the instances are one theorem.

F_11 (`F_11_HallucinationGodel`). The Gödel-style naming of the hallucination result. The liar query is named explicitly, and the trilemma is stated as an impossibility. This file ends with `#print axioms` on its headline results; the boolean core is axiom-free.

- `bool_not_fpf` : boolean negation has no fixed point, proved by `decide`.
- `tarski_liar` : the Tarski-style undefinability liar for a truth predicate.
- `hallucination_liar_query` : the exact query on which a reflective model must contradict itself.
- `hallucination_trilemma_godel` : the hallucination trilemma as a self-reference impossibility.
- `controllers_agree` : the boolean and propositional flips agree as controllers.

F_12 (`F_12_DefenseGodel`). The same move for defenses. A defense that judges prompts, including prompts about its own judgments, is a reflective verdict, so it is impossible, and the file shows the defense engine is the hallucination engine.

- `defense_liar_query` : the prompt injection no wrapper neutralizes, as a liar.
- `defense_trilemma_godel` : the defense trilemma in Gödel form.
- `defense_trilemma_lawvere` : the same via `lawvere` directly.
- `reflective_verdict_impossible` : no reflective verdict exists.
- `defense_is_hallucination_engine` : the two impossibilities share one proof.

F_13 (`F_13_UnifiedTrilemmata`). The unification. All three trilemmas, the hallucination, the defense, and the truth-boundary coupling, are folded into one statement about reflective verdicts. Axiom audit printed at the end.

- `no_reflective_verdict` : the one-line engine, no reflective verdict exists.
- `schema_boundary` : the schema that produces the boundary object.
- `coupling_trilemma_godel` : the truth-boundary coupling in Gödel form.
- `trilemmata_unified` : the three trilemmas as a single theorem.

F_14 (F_14_JSpace). The interpretability reading. A probe that reads a judgment off a model's own activations, over a space rich enough to encode any readout, is again a reflective verdict, so a complete self-reading probe is impossible. This is the formal core behind the J-space chapter.

- jspace_readout_impossible : no complete readout of a model's own judgments from its activations.
- jspace_coupled_activation : the activation on which the readout must fail.
- jspace_is_the_same_engine : the interpretability obstruction is lawvere again.

CCHProofs: the master trilemma (CCH_01–CCH_Master)

CCHProofs develops the trilemma as a geometry of three properties: Universality, Control, and Transparency, of which any system can enjoy at most two. It restates Lawvere in its own vocabulary, proves Cantor and the fixed-point-free lemmas, then walks the three corners one at a time before assembling the master statement. It also includes a direct LLM instantiation. The diagonal core is axiom-free; the corners inherit only what their outputs require.

CCH_01 (CCH_01_Foundations). Definitions and the dictionary between the property language and the fixed-point language.

- nonTrivial_iff_no_fixed_point : a controller is nontrivial exactly when it has no fixed point.
- universal_iff_surjective : universality is point-surjectivity.
- transparent_eq_diag : transparency is representability of the diagonal.
- nonTrivial_apply, universal_exists : the elimination forms of the two definitions.
- isUniversal_iff, isControlled_iff, isTransparent_iff : the structure-level restatements.
- isTransparent_eq_diag : transparency as diagonal equality at the structure level.

CCH_02 (CCH_02_Lawvere). Lawvere inside the CCH vocabulary.

- lawvere_fixed_point : the fixed-point theorem, CCH form.
- no_surjection_of_no_fixed_point : Cantor's contrapositive.
- universal_implies_fixed_point : universality forces the fixed point.

- `fixed_point_free_rules_out_universal` : the block on universality.
- `lawvereFixedPoint_spec` : the fixed point meets its spec.
- `lawvere_via_section`, `surjective_of_section` : the section form and its bridge.
- `lawvere_universal_fixed_point` : the packaged form reused by the corners.

`CCH_03` (`CCH_03_Cantor`). Cantor, in surjection and power-set forms. Axiom-free.

- `bool_not_no_fixed_point` : boolean negation has no fixed point.
- `cantor_no_surjection` : no surjection onto the boolean behaviors.
- `cantor_no_surjection_set` : no surjection onto the power set.
- `no_universal_bool_system` : no universal boolean system.

`CCH_04` (`CCH_04_FixedPointFree`). A small library of fixed-point-free maps, the raw material for corners. Axiom-free.

- `bool_not_fixedPointFree` : boolean negation is fixed-point-free.
- `succ_fixedPointFree`, `int_succ_fixedPointFree` : successor on the naturals and on the integers.
- `no_surjection_of_fixedPointFree`, `fixedPointFree_blocks_universality` : the two block forms.
- `fixedPointFree_iff` : the characterization of fixed-point-freeness.
- `not_not_eq_id` : double negation is the identity, the reason a single flip and not a double one is needed.
- `empty_fixedPointFree`, `unit_no_fixedPointFree` : the endpoints, everything on the empty type is fixed-point-free and nothing on `Unit` is.
- `lawvere_section`, `no_section_of_fixedPointFree` : the section-form counterparts.

`CCH_05` (`CCH_05_Controllers`). The theory of controllers, the maps that play the role of `t`. A controller is nontrivial when it has no fixed point.

- `bool_not_nonTrivial`, `nat_succ_nonTrivial` : examples of nontrivial controllers.
- `id_not_nonTrivial` : the identity is never nontrivial.

- `nonTrivial_iff_no_fixed_point`, `nonTrivial_iff_no_fp_exists` : the two characterizations.
- `controlled_iff` : control expressed via the controller.
- `universal_excludes_nonTrivial_controller` : universality excludes a nontrivial controller.
- `universal_implies_no_nonTrivial_controller` : the same as an implication.
- `controlled_universal_nonTrivial_contradiction` : the three-way clash at the heart of the trilemma.
- `nonTrivial_of_image`, `nonTrivial_no_fixed`, `exists_fixed_not_nonTrivial` : transport and existence lemmas.
- `bivalent_nonTrivial`, `nonTrivial_bivalent_of_inhabited` : the bivalent-output cases.

`CCH_06` (`CCH_06_Transparency`). Transparency, the property that the diagonal is itself a named behavior.

- `transparent_iff_eq_diag` : transparency is diagonal equality.
- `diag_is_transparent` : the diagonal is always transparent for itself.
- `transparent_symm` : symmetry of the transparency relation.
- `universal_diag_in_image`, `transparent_universal_diag_representable` : a universal transparent system represents its own diagonal.
- `transparent_universal_self_fixed` : which forces a self-fixed point.
- `transparent_universal_blocks_nonTrivial` : and blocks any nontrivial controller.
- `trivial_transparent_construction` : a trivial witness showing the corner is inhabited when universality is dropped.

`CCH_07` (`CCH_07_CornerUC`). The Universal-plus-Controlled corner, which cannot also be Transparent.

- `corner_UC_not_T` : Universal and Controlled excludes Transparent.
- `corner_UC_impossible` : the corner as an impossibility.
- `corner_UC_strong` : a strengthened version.
- `universal_diagonal_hits_fixed_point` : the diagonal lands on the fixed point.

- `defense_trilemma_face` : this corner is the defense trilemma face.
- `corner_UC_exists_failure`, `corner_UC_prediction_fixed_point` : the explicit failure witness and the prediction fixed point.

`CCH_08` (`CCH_08_CornerUT`). The Universal-plus-Transparent corner, which cannot be Controlled.

- `corner_UT_not_C` : Universal and Transparent excludes Controlled.
- `corner_UT_impossible` : the corner as an impossibility.
- `hallucination_trilemma_face` : this corner is the hallucination trilemma face.
- `corner_UT_witness_identity`, `corner_UT_needs_universality` : the witness and the necessity of the universality hypothesis.

`CCH_09` (`CCH_09_CornerCT`). The Controlled-plus-Transparent corner, which cannot be Universal. This is the Rice face.

- `corner_CT_not_U` : Controlled and Transparent excludes Universal.
- `corner_CT_impossible` : the corner as an impossibility.
- `corner_CT_bool_specialization` : the boolean specialization.
- `rice_face` : this corner is the Rice face.
- `corner_CT_witness`, `corner_CT_not_U_of_fpf`, `corner_CT_bool_via_fpf` : the witness and the fixed-point-free routes.

CCH_LLM (`CCH_LLM_Direct`). The trilemma spelled out for an in-context learner `M : LLM Prompt Token`, so the abstract corners become claims about a model.

- `iclUniversal_iff` : in-context universality is surjectivity of the learner.
- `LLM_diagonal_fixed_point`, `LLM_no_fixed_point_free` : the diagonal and its consequence for the model.
- `LLM_trilemma` : the trilemma for the model.
- `LLM_corner_UC`, `LLM_corner_UT`, `LLM_corner_CT` : the three corners, model form.
- `LLM_safety_corner_UC`, `LLM_safety_self_misclassification` : the safety readings, including that the model must misclassify itself somewhere.
- `bool_not_controller` : the boolean flip is a controller.

- `LLM_approx_fixed_point`, `LLM_approx_trilemma` : the approximate versions with a confidence output.

CCH_Master (`CCH_MasterTrilemma`, `CCH_FinalVerification`). The consolidated trilemma over a `CCHStructure`, and the axiom audit.

- `cch_master_trilemma` : at most two of Universal, Controlled, Transparent.
- `cch_corner_UC`, `cch_corner_UT`, `cch_corner_CT` : the three corners at the structure level.
- `cch_at_most_two` : the counting statement, no more than two properties hold.

HallucinationProofs: the truth-boundary theory (HoF_01–HoF_Master)

`HallucinationProofs` is where the second engine takes over. Instead of a diagonal over a self-naming domain, it works with a signed truth distance δ, a confidence `conf`, and a truth threshold, and it uses value-crossing, the intermediate value theorem and its topological cousin the Borsuk-Ulam theorem, to force the existence of a boundary point where the model is confidently on the fence. The early files set up the topology of the truth set; the middle files prove the trilemma in continuous, discrete, probabilistic, and approximate forms; the late files add the coupling theorem and the metric geometry. Most results here carry the three standard axioms through Mathlib's real analysis; the purely discrete files are the exceptions.

`HoF_01` (`HoF_01_Foundations`). The truth set and its neighbors, defined from the signed distance δ and the model's confidence.

- `mem_truthSet`, `mem_falseSet`, `mem_truthBoundary`, `mem_strictTruth`, `mem_highConfRegion` : membership unfoldings for the five basic regions.
- `truthSet_isClosed`, `falseSet_isOpen`, `truthBoundary_isClosed`, `strictTruth_isOpen`, `highConfRegion_isClosed` : the topology of each region when δ and `conf` are continuous.
- `truth_partition`, `truthSet_union_falseSet`, `truthSet_falseSet_disjoint` : the space splits cleanly into truth and falsehood.
- `strictTruth_subset_truthSet`, `truthBoundary_subset_truthSet`, `truthSet_eq_strictTruth_union_boundary` : the inclusions and the decomposition of the truth set into strict interior plus boundary.
- `strictTruth_falseSet_disjoint`, `truthBoundary_falseSet_disjoint` : the separations used later.

HoF_02 (HoF_02_TruthSet). The truth set as a topological object in its own right, with the non-closedness that forces a boundary.

- truthSet_isClosed, falseSet_isOpen, strictTruth_isOpen, truthBoundary_isClosed : the topology restated for the abstract space.
- truthBoundary_nonempty : the boundary is nonempty under the covering hypothesis.
- strictTruth_falseSet_separated, strictTruth_falseSet_no_intersect : truth and falsehood do not touch except through the boundary.
- strictTruth_not_closed, boundary_point_in_closure_of_strictTruth : the strict truth region is not closed, and its closure reaches the boundary.

HoF_03 (HoF_03_BoundaryCrossing). The intermediate value theorem doing its work: a path from a confident-true point to a confident-false point must cross.

- confidence_path_crosses_half : a confidence path from above to below one half crosses one half.
- path_crosses_truth_boundary : a path from truth to falsehood crosses the truth boundary.
- model_truth_boundary_nonempty : the model's truth boundary is nonempty.
- model_confidence_half_set_nonempty, confidence_half_nonempty : the half-confidence level set is nonempty.
- model_truth_boundary_isClosed : and closed.

HoF_04 (HoF_04_Faithfulness). Faithfulness, the property that high confidence implies truth, and what it forces at the boundary.

- StrongFaithful.toFaithful : strong faithfulness implies faithfulness.
- faithful_image_in_truth : a faithful model's confident answers are true.
- highConf_closed, highConf_open : the high-confidence region's topology under the two conventions.
- truth_preimage_closed, faithful_closure_in_truth : the truth preimage is closed and its closure stays in truth.
- faithful_at_half_boundary : at the half-confidence boundary a faithful model is pinned to the truth boundary.

- `low_conf_unconstrained` : where confidence is low the model is free.

`HoF_05` (`HoF_05_Coverage`). Coverage, the hypothesis that the model's answers straddle both sides, which is what makes the boundary unavoidable.

- `covering_image_straddles_zero` : a covering model's signed distances take both signs.
- `denseCoverage_closure_eq_univ` : dense coverage fills the space in closure.
- `covering_yields_truth_boundary_point` : coverage produces a boundary point.
- `covering_yields_two_sided_witnesses`, `covering_two_closed_sides` : two-sided witnesses and their closedness.
- `covering_abs_zero`, `denseCoverage_approximable` : the distance hits zero, and dense coverage is approximable.

`HoF_06` (`HoF_06_Calibration`). Calibration, the link between confidence and truth probability, and the half-boundary it induces.

- `calibrated_levelSet_eq` : the calibrated level set matches the truth level set.
- `confHalf_isClosed` : the half-confidence set is closed.
- `monotoneCalibrated_half_on_boundary` : monotone calibration puts half-confidence on the truth boundary.
- `calibrated_confHalf_nonempty` : the half set is nonempty under calibration.
- `confHalf_implies_truthBoundary` : half confidence implies boundary membership.
- `off_boundary_conf_not_half` : off the boundary, confidence is never exactly half.

`HoF_07` (`HoF_07_TrilemmaCore`). The continuous hallucination trilemma. A model cannot be faithful, calibrated, and covering all at once; some question sits on the boundary with confidence exactly one half.

- `hallucination_trilemma_strict` : the strict form of the trilemma.
- `hallucination_trilemma` : the headline continuous trilemma.
- `hallucination_trilemma_unfolded`, `hallucination_trilemma_strict_unfolded` : the same with definitions unfolded, for downstream use.

HoF_08 (HoF_08_BorsukUlam). The topological upgrade. When the question space is a sphere and the model is antipodally symmetric, the Borsuk-Ulam theorem forces a boundary point without needing a chosen path. These results use the three standard axioms.

- `antipodal_odd_has_zero` : an odd continuous function on a sphere has a zero.
- `antipodal_yields_truth_boundary` : antipodal symmetry yields a boundary point.
- `antipodal_hallucination_trilemma` : the trilemma via Borsuk-Ulam.
- `approx_odd_near_zero` : an approximately odd function is near zero somewhere.
- `sphere_odd_has_zero` : the higher-dimensional sphere statement.

HoF_09 (HoF_09_Discrete). The trilemma with no topology at all, over a finite or discrete question space using a strict trichotomy and a sign change. Axiom-light, the discrete core needs no analysis.

- `strictCal_conf_trichotomy` : confidence is above, below, or exactly at half.
- `discrete_partition`, `discrete_two_partition` : the discrete partitions.
- `discrete_trilemma_decisive_impossible` : a decisive discrete model is impossible.
- `discrete_sign_change` : a sign change is forced between covered points.
- `discrete_hallucination` : the discrete hallucination result.

HoF_10 (HoF_10_PureDiscrete). The fully combinatorial version, connecting the boundary story back to the diagonal engine of `Foundation`.

- `boundary_conf_half` : the boundary question has confidence one half.
- `boundary_question_impossible` : the boundary question cannot be answered decisively.
- `discrete_hallucination_trilemma` : the pure-discrete trilemma.
- `faithful_iff_boundary_free` : faithfulness is exactly the absence of a boundary question.
- `hallucination_trilemma_via_pure_discrete` : the trilemma proved with no analysis.

HoF_11 (HoF_11_MultiTurnProbabilistic). The multi-turn and stochastic extension. An adversary steering a conversation can reach the boundary, and the expected behavior over turns still couples.

- `multi_turn_per_turn`, `multi_turn_history_dependent` : per-turn and history-dependent forms.
- `adversary_boundary_reachable` : an adversary can reach the boundary.
- `stochastic_coupling_expected`, `stochastic_trilemma_expected` : coupling and the trilemma in expectation.
- `stochastic_dichotomy`, `boundary_dichotomy` : the stochastic and boundary dichotomies.

HoF_12 (HoF_12_Approximate). The approximate trilemma with explicit slack parameters ε, and the bridges back to the exact statements. Axiom audit printed.

- `approx_trilemma` : the trilemma with a slack ε.
- `exact_from_approx` : the exact trilemma recovered as ε goes to zero.
- `strictCalibrated_to_epsCalibrated_zero_half`, `trilemmaCovering_to_epsCovering_zero` : the exact hypotheses as zero-slack cases.
- `no_boundary_in_upper_band`, `no_boundary_in_upper_band_two_slack` : where no boundary can hide.
- `epsCalibrated_iff_twoSlack`, `two_slack_approx_coupling`, `truth_slack_must_be_positive`, `two_slack_band_bridge` : the two-slack reformulation and its coupling.

HoF_13 (HoF_13_BoundaryCoupling). The coupling theorem, the truth-boundary analogue of the reflective-verdict impossibility, with the closed-versus-open guarantee dichotomy that Chapter 3 turns on. Axiom audit printed.

- `boundary_coupling` : truth and confidence are coupled at the boundary.
- `closed_guarantee_impossible` : a closed (two-sided) guarantee at the boundary is impossible.
- `open_guarantee_silent` : an open guarantee is silent exactly on the boundary.
- `slack_must_be_positive` : any honest guarantee needs strictly positive slack.

HoF_14 (HoF_14_QuantitativeGeometry). The metric geometry of the boundary, the tubes and the linear-probe cosets that feed the interpretability chapter. Axiom audit printed.

- confidence_gap_width : the width of the confidence gap around the boundary.
- truth_margin_tube, ambiguity_tube : the truth-margin tube and the ambiguity tube around the boundary.
- linear_probe_boundary_coset : a linear probe's boundary is an affine coset.
- linear_probe_boundary_dim : its dimension.

HoF_15 (HoF_15_NonlinearBoundary). The nonlinear boundary, its regular points and tangent structure.

- regular_point_is_crossing : a regular point of the boundary is a genuine crossing.
- nonlinear_boundary_tangent : the tangent to a nonlinear boundary.
- tangent_hyperplane_dim : the dimension of the tangent hyperplane.

HoF_Instantiation (HoF_Instantiation_PromptSpace). The trilemma over a concrete prompt space of dimension d with k classes, so the abstract theorem becomes a statement about prompts.

- prompt_truth_boundary_exists : the prompt-space truth boundary is nonempty.
- prompt_confidence_half_exists : a half-confidence prompt exists.
- prompt_hallucination_trilemma : the trilemma over prompt space.
- prompt_near_boundary : prompts arbitrarily near the boundary exist.

HoF_Master (HoF_MasterTheorem, HoF_FinalVerification). The consolidated hallucination theorem over a HallucinationStructure, and the axiom audit.

- hallucination_master_theorem : the master statement bundling the hypotheses.
- hallucination_trilemma_three_clause : the three-clause form, at most two of faithful, calibrated, covering.

ManifoldProofs: the geometry of attack basins (MoF)

`ManifoldProofs` is the quantitative wing. Where `HallucinationProofs` establishes that a boundary exists, `ManifoldProofs` measures it: the volume of the safe basin, the radius of robustness `robustnessRadius = (fp - τ) / L`, the cost of an attack that walks to the failure manifold, and how these scale with dimension and model size. We index this library at the module level, as the task asks, with the headline of each group and a few representative theorem names; the package holds well over three hundred theorems across its files.

The base modules fix the objects. `MoF_01_Foundations` defines the safety score `f`, the threshold `τ`, and the safe, unsafe, boundary, and failure-manifold regions, with their topology (`mem_safeRegion`, `boundary_isCompact`, `space_partition`, `failureManifold_eq_union`). `MoF_02_BasinStructure` develops the basin as an open set of positive measure that contains a ball around each safe point (`basin_isOpen`, `basin_contains_ball`, `basin_measure_pos`). `MoF_03_ThresholdCrossing` is the manifold-side intermediate value theorem, the crossing that produces the failure manifold.

The Lipschitz group turns qualitative crossing into quantitative robustness. `MoF_04_LipschitzBasin` proves the robustness radius is positive and that a ball of that radius stays in the basin (`lipschitz_basin_ball`, `robustnessRadius_pos`, `basin_ball_subset`, `perturbation_stability`, `robustnessRadius_monotone`). `MoF_05_MonotoneConvergence` handles monotone approach to the threshold; `MoF_06_Transferability` handles when an attack on one model transfers to another; `MoF_07_AuthorityMonotonicity` studies monotonicity in an authority parameter.

The defense and scaling group. `MoF_08_DefenseBarriers` formalizes barriers a defense erects around the basin. `MoF_09_DimensionalScaling` shows how basin geometry scales with the ambient dimension. `MoF_10_GradientAttack` models a gradient-follower walking downhill to the boundary. `MoF_11_EpsilonRobust` gives the ε-robustness guarantees and their limits. `MoF_12_Discrete` is the discrete counterpart, and `MoF_13_MultiTurn` the multi-turn one. `MoF_14_MetaTheorem` and `MoF_15_NonlinearAgents` and `MoF_16_RelaxedUtility` generalize the agent and utility assumptions.

The measure and coarea group sharpens the volume estimates. `MoF_17_CoareaBound` uses the coarea formula to bound the boundary's measure; `MoF_18_ConeBound` gives a cone estimate; `MoF_19_OptimalDefense` characterizes the defense that maximizes the robustness radius; `MoF_20_RefinedPersistence` and `MoF_21_GradientChain` refine persistence of the basin under perturbation and chain gradient steps together.

The advanced group `MoF_Adv_01` through `MoF_Adv_10` covers basin connectedness, boundary dimension, fine-tuning, model scale, convexity, approximation, fragmentation, stability, the optimization landscape, and measure bounds, in that order. The cost group `MoF_Cost_01` through `MoF_Cost_10` is the attack-economics core: ball volume, basin volume, hitting time, concentration, attack cost, defense cost, transfer cost, the cost ratio, Lipschitz

estimation, and the unified theory. Its headline claim is that the attacker's cost stays bounded while the defender's grows, so the cost ratio diverges (`attack_cost_upper_bound`, `attack_reaches_threshold`, `total_attack_cost`, `attack_cost_ratio`, `attack_cost_ratio_tends_to_infinity`). `MoF_ContinuousRelaxation`, `MoF_Instantiation_Euclidean`, `MoF_MasterTheorem` (`master_theorem`), and `MoF_FinalVerification` relax, instantiate over Euclidean space, consolidate, and audit. Everything in `ManifoldProofs` is analytic and so rests on the three standard kernel axioms through Mathlib's measure theory and calculus.

Reading the corpus as one argument

Put the four libraries in a line and the book's thesis is visible in the dependency graph. `Foundation` proves the engine and reads it into every AI-safety corner combinatorially. `CCHProofs` re-proves the engine in the property language and lays out the trilemma as a geometry of corners. `HallucinationProofs` switches engines, replacing self-reference with value-crossing, and shows the second engine lands on the same boundary object with metric content. `ManifoldProofs` measures that object. The first two libraries are where the axiom-free core lives; the last two are where the three standard axioms enter, honestly and only through real analysis. Nothing in the development introduces an axiom of its own, and nothing is left as `sorry`.

A few cross-library correspondences are worth naming, because they are the seams where the book's claims about sameness become theorems rather than assertions. The hallucination result appears three times, and the three are provably the same boundary. `hallucination_trilemma_godel` in `Foundation` reaches it by the diagonal; `hallucination_trilemma` in `HoF_07` reaches it by continuity; `discrete_hallucination` and `hallucination_trilemma_via_pure_discrete` in `HoF_09` and `HoF_10` reach it with no analysis at all, which is what lets the discrete file connect back to the combinatorial engine. The defense result and the hallucination result are welded together by `defense_is_hallucination_engine` in `F_12`, which is not a slogan but a proof that the two impossibilities have one witness. The CCH corners are the same theorems in property clothing: `hallucination_trilemma_face` is the UT corner, `defense_trilemma_face` is the UC corner, and `rice_face` is the CT corner, so the three-property geometry of `CCHProofs` is the three-reading catalogue of `Foundation` seen from a different angle. And the interpretability obstruction is the diagonal one last time, which `jspace_is_the_same_engine` states outright.

The two-engine seam is the most important one. `HoF_07`'s continuous trilemma and `F_11`'s Gödel-form trilemma are about the same phenomenon and reach the same boundary question, but they are genuinely different proofs with different axiom footprints, and the book keeps both on purpose. The discrete files are the bridge: they show that when you strip the topology away, the value-crossing argument collapses onto the diagonal argument, so the two engines are not two facts but two views of one fact whenever a system

is both self-naming and continuous. `ManifoldProofs` then takes the located boundary the value engine produced and does the thing neither engine can do on its own, which is measure it, in `MoF_Cost_08`'s cost ratio and `MoF_17_CoareaBound`'s measure estimate. The dependency graph, read top to bottom, is the book's argument compiled.

12.2 A Notation Glossary

The book reuses a small alphabet across very different readings. The same letter can be a prompt in one chapter and an activation vector in another; what stays fixed is the role the letter plays in the argument. This glossary lists each symbol with its meaning and, where it helps, the role it plays in the engine.

Two conventions help before the list. First, the letters track roles, not types. When you see `f` it is always the system whose self-application drives the diagonal, whether its outputs are booleans, probabilities, or points of a space; when you see `t` it is always the flip whose fixed points the theorem is about. Reading for the role rather than the type is the fastest way through the corpus. Second, the theorem names in the Lean sources follow a light convention worth knowing: a name like `no_universal_bool_LLM` reads as its statement, "no universal boolean-output LLM," and a name ending in `_iff` is a characterization, one ending in `_impossible` or `_no_escape` is an impossibility, one ending in `_spec` is the defining property of a construction, and one containing `diagonal` or `liar` names the witness rather than the impossibility. The names are chosen to be greppable, so a search for `boundary` or `corner` or `cost` across a package returns the relevant thread.

Sets and spaces.

- `A` : the domain of _indices_. An element of `A` is a name for a behavior: a program, a prompt, an activation, a Gödel number. In the analytic chapters `A` is also used for the model's _answer_ type.
- `Y` : the set of _outcomes_ a behavior can produce. Often `Bool`, sometimes a probability, sometimes a point of a metric space.
- `Q` : a domain of _questions_ or queries over which a verdict is defined. The reflective-verdict statements are about `Q → Q → Bool`.
- `X` : the ambient space in the manifold chapters, the space attacks move through.
- `Z` : an intermediate type in composition results, the target of a post-processor.
- `Prompt`, `Token`, `Output`, `Act` : concrete instantiations of `A` and `Y`, the prompt space, token space, output space, and activation space.
- `Set A` : the power set of `A`, identified with indicator functions `A → Bool`.

The system and its parts.

- `f` : a _system_, `f : A → A → Y`. `f a` is the behavior named by `a`; `f a b` is what that behavior outputs on input `b`.
- `g` : an arbitrary behavior `A → Y`, the thing universality asks to be named.
- `d` : the _diagonal behavior_, `d a = t (f a a)`, apply each behavior to its own name and then flip.
- `s` : a _section_, `s : (A → Y) → A`, a chosen index for each behavior, with `f (s g) = g`.
- `M` : a _model_, usually `M : Q → A × ℝ`, returning an answer and a confidence.

The engine's moving parts.

- `t` : the _controller_ or output flip, `t : Y → Y`. The engine turns on whether `t` has a fixed point. Boolean negation, the successor, and reflection across one half are the recurring choices.
- `v` : a _reflective verdict_, `v : Q → Q → Bool`, a boolean self-application in which every pattern of verdicts is itself named. Theorem 1.16 says none exists.
- a_0, q_0, `q_0` : the _liar_, the diagonal witness. The index or question on which the system must contradict itself: `f` a_0 a_0 `= ! (f` a_0 a_0`)` in the boolean case, confidence exactly one half in the analytic case. The same object under three readings across Chapter 3.

The analytic alphabet.

- δ : the _signed truth distance_, δ `(q, a)`, negative on true answers, positive on false ones, zero on the truth boundary. Continuity of δ is what lets the intermediate value theorem run.
- `conf`, `confOf` : the model's _confidence_, the second projection of `M`. Values in the unit interval; one half is the fence.
- τ : the _threshold_ in the manifold chapters. Safe when the score `f` exceeds τ, unsafe below, on the boundary at equality.
- ε : a _slack_ or tolerance. The approximate results replace exact equalities by inequalities up to ε and recover the exact statement as ε tends to zero.
- `c` : a confidence _level_ used to cut out the high-confidence region.
- `L` : a _Lipschitz constant_ for the safety score, the rate at which the score can change per unit of movement in `X`.

- `fp` : the safety score at a point, written `fp` in `robustnessRadius = (fp - τ) / L`.

Regions and objects.

- _truth set_, _false set_ : where δ is nonpositive and where it is positive.
- _strict truth_ : the open interior of the truth set, where δ is strictly negative.
- _truth boundary_ : the zero set of δ, where true meets false. The analytic liar lives here.
- _high-confidence region_ : where `conf` is at least `c`.
- _basin_ : the safe basin, the open set where the score `f` exceeds `τ`, of positive measure and containing a ball of radius `robustnessRadius` around each safe point.
- _failure manifold_ : the closure of the unsafe region against the boundary, the codimension-one set an attacker aims for.
- _robustness radius_ : `(fp - τ) / L`, how far a safe point can be pushed before it crosses.

Properties named in the trilemmas.

- _universal_, _point-surjective_ : every behavior is named, `∀ g, ∃ a, f a = g`. The one hypothesis that drives the diagonal.
- _faithful_ : high confidence implies truth.
- _calibrated_ : confidence matches truth probability; _monotone calibrated_ adds monotonicity.
- _covering_ : the model's answers straddle both sides of the boundary; _dense coverage_ fills the space.
- _control_, _nontrivial controller_ : the output can be steered by a `t` with no fixed point.
- _transparent_ : the diagonal is itself a named behavior.

Logical and Lean symbols.

- `∀`, `∃`, `¬`, `→`, `≠`, `=` : the usual quantifiers and connectives. `¬` is the controller in the Russell instance.
- `!` : boolean negation, the controller in the Cantor instance.
- `Nat.succ`, `+ 1` : the successor, a fixed-point-free controller on the naturals and integers.

- `False`, `True`, `Prop`, `Bool` : Lean's falsehood, truth, the type of propositions, and the type of booleans.
- `congrFun` : the step "equal functions agree at every point," the one nontrivial move in the proof of `lawvere`.
- `decide` : the tactic that settles a finite boolean claim, how `bool_not_fpf` is proved axiom-free.
- `closure`, `Continuous`, `IsOpen`, `IsClosed`, `IsCompact` : the topological vocabulary the analytic chapters run on.

12.3 The Two Engines, Side by Side

The book has one thesis with two halves. Impossibility in AI safety comes from one of exactly two sources, and knowing which source you are in tells you what kind of answer you can hope for. The first engine is the diagonal, Lawvere's fixed-point theorem read as a contradiction. The second is the intermediate value theorem, and its topological strengthening the Borsuk-Ulam theorem, read as a boundary. This section sets them opposite each other on every axis that matters.

What each one assumes. The diagonal needs _self-reference_ and nothing else. Its single hypothesis is universality: the system can name every one of its own behaviors, `∀ g, ∃ a, f a = g`. It does not care whether the domain is finite or infinite, discrete or continuous, metric or bare. The intermediate value engine needs the opposite kind of structure. It wants a _connected_ domain and a _continuous_ map, plus a reason the map takes values on both sides of a threshold. Coverage supplies the two-sidedness; continuity supplies the crossing. Where the diagonal is indifferent to geometry, the second engine is made of geometry.

How each one works. The diagonal builds a behavior that disagrees with whatever the system would do on itself, `d a = t (f a a)`, names it by universality, and evaluates the naming equation at that name. The disagreement becomes an equation `f a₀ a₀ = t (f a₀ a₀)`, a fixed point of `t`. If `t` was chosen to have no fixed point, the equation is a contradiction, and the assumption of universality falls. The value engine does not build a self-referential object at all. It takes a path or a symmetry from one side of the boundary to the other and invokes continuity: a continuous real function that is negative somewhere and positive somewhere is zero in between. The zero is the boundary point.

What each one outputs. The diagonal outputs a _yes-or-no_ verdict: either the system is not universal, or a specific liar `a₀` exists on which it contradicts itself. The output is exact and qualitative. There is a boundary question, full stop. The value engine outputs a _located_ object: a point where δ is exactly zero, a question where confidence is exactly one half, a set that is the failure manifold. And because it is built from continuity and metric data, that object

comes with quantitative company: how wide the ambiguity tube is, how far a safe point can be pushed, how the boundary's measure scales. The diagonal says _that_ the boundary exists; the value engine says _where_ it is and _how big_.

What each one cannot give. This is the crux, and it is why the book needs both. The diagonal is silent on everything quantitative. It cannot tell you how many liars there are, how hard a_0 is to find, or what an attack to reach one costs. It only needs, and only delivers, the bare existence of the contradiction. It also demands genuine self-application; a system that cannot name its own behaviors is outside its reach entirely. The value engine has the mirror-image blind spot. It cannot run without connectivity and continuity, so it says nothing about a discrete self-naming system with no metric. And it does not, by itself, explain _why_ the two sides had to differ; coverage is an assumption it consumes, not a conclusion it produces. The diagonal explains the necessity; the value engine measures the consequence.

Where they meet. The remarkable fact the book is built around is that the two engines land on the _same object_. The liar a_0 of the diagonal and the zero-of-δ boundary point of the value engine are the same question wearing different clothes. In the discrete files of `HallucinationProofs`, this is made literal: `hallucination_trilemma_via_pure_discrete` reaches the boundary with no analysis, matching the combinatorial `hallucination_trilemma_godel` of `Foundation`, while `hallucination_trilemma` of `HoF_07` reaches the same boundary through continuity. One boundary, two proofs, two kinds of information about it. That is the whole architecture: the diagonal establishes the boundary must exist, the value engine and the manifold geometry tell you its size, shape, and cost, and the appendix you are reading lets you check that both proofs are the ones the kernel accepts.

A compact way to hold the contrast. Ask of any impossibility claim in AI safety: does the system have to reason about itself? If yes, you are in the diagonal's territory, expect a yes-or-no impossibility and do not expect a number. Is the system a continuous score over a connected space with behavior on both sides of a threshold? If yes, you are in the value engine's territory, expect a located boundary and a measurable cost. Many real systems are both at once, a self-naming model with a continuous confidence, and then both engines fire and agree. Chapter 8's J-space is the cleanest case of this double life, and `jspace_is_the_same_engine` is the one-line proof that the interpretability obstruction is, again, the diagonal.

It helps to see the two engines instantiate on one worked example. Take a model that answers questions and reports a confidence, and ask it to judge its own answers. The diagonal reading fixes the boolean verdict "is this answer correct," builds the question "answer this the way you would, then flip the verdict," and finds by universality a self-referential question a_0 the model must get wrong: if it says correct it is wrong, if it says wrong it is right. That is `hallucination_liar_query`, and it needs no notion of distance. The value reading instead watches the confidence as the question varies continuously from one the model is sure is true to one it is sure is false. Somewhere the con-

fidence passes through one half, and at that question, if the model is faithful and calibrated, the answer sits exactly on the truth boundary. That is `path_crosses_truth_boundary` feeding `hallucination_trilemma`. Same model, same failure, two accounts: one says the failure is logically forced, the other says here is where it is and here is how wide the fence is. Neither account is complete without the other, which is why the book runs both to the end.

There is a temptation to ask which engine is "more fundamental," and the honest answer is that the question is malformed. The diagonal is more general, in that it needs less structure and covers discrete systems the value engine cannot touch. The value engine is more informative, in that when its hypotheses hold it returns a located, measurable object the diagonal cannot see. Generality and information trade off, and the book's contribution is to show they trade off around a single shared object rather than two unrelated ones. The manifold chapters then spend their length on the information side of that trade, because once you know the boundary exists the interesting questions are all quantitative: how large is the safe basin, how far can a safe input be perturbed, what does an attack cost, and how do these scale. Those are `ManifoldProofs` questions, and they are downstream of a boundary the diagonal guaranteed and the value engine placed.

12.4 Building and Checking This Book

The point of a machine-checked book is that you do not have to trust it. This section explains how to build the Verso book and the four Lean libraries, and how to run the axiom audit yourself so that every claim in Appendix A is one command away from verification.

What you need. A recent Lean 4 toolchain managed by `elan`, and the `lake` build tool that ships with it. The exact toolchain version is pinned in each package's `lean-toolchain` file, and `elan` will fetch it automatically the first time you build. The proof libraries depend on Mathlib, which `lake` will resolve from the manifest; the first build downloads a cached Mathlib and takes a while, and later builds are fast.

Repository layout. Each package lives in its own directory with the package name repeated one level down, so the sources are under `Foundation/Foundation`, `CCHProofs/CCHProofs`, `HallucinationProofs/HallucinationProofs`, and `ManifoldProofs/ManifoldProofs`. Files are numbered in dependency order, `F_01` through `F_14`, `CCH_01` through `CCH_09` with the master and verification files after, `HoF_01` through `HoF_15` with instantiation, master, and verification files, and the `MoF_` families for the manifold geometry. The numbering is a reading order as much as a build order: lower numbers are the engine and the setup, higher numbers are the readings and the refinements. The book itself is a fifth project, `TrilemmaBook`, whose chapters are the `Ch0` files.

Building the proof libraries. Each of the four packages, `Foundation`, `CCHProofs`, `HallucinationProofs`, and `ManifoldProofs`, is a standalone Lake

project. From a package's root directory, `lake exe cache get` fetches the prebuilt Mathlib, and `lake build` compiles the package. A successful `lake build` is the whole guarantee: Lean's kernel has checked every proof in the package, and because the packages are sorry-free, a clean build means no gaps. Build `Foundation` first, since the other packages' arguments mirror its engine, though each package builds independently against Mathlib. If a build fails after a toolchain change, the usual cause is a stale Mathlib cache; `lake exe cache get` followed by a clean `lake build` resolves it. Compilation is memory-hungry because Mathlib is large, so a machine with several gigabytes free per parallel worker is worth having, and a single-threaded build is the safe fallback on a constrained machine.

Building the book. The book is a Verso document. Its own project builds with `lake build` from the book root, which elaborates every code block in the chapters, including the live proofs in Chapter 1, and renders the manual. Because the code blocks are elaborated at build time, the proofs you read in the text are exactly the proofs the kernel accepted; there is no separate step that could drift from the source. This appendix chapter contains no live code blocks, only backticked names, so it renders as plain reference text.

Checking the prose against the corpus. Elaboration checks the code blocks, but a theorem stated in prose and attributed to a companion library is checked by nothing: rename the declaration and the book keeps asserting it. The book's build script closes that gap before it calls Lean. It harvests every declaration in the four packages, collects every backticked name in the chapters, and fails if any cited name resolves to nothing. It then classifies every numbered theorem, lemma, proposition, and corollary in the prose by how it is backed, and demands that each one either carry a live `lean` block, or cite a name that resolves to a `theorem` or a `lemma` somewhere in the corpus, in this book, or in the short list of Mathlib results the book leans on. A module name is not a proof and does not count; neither is a `def`. A claim that has no verified counterpart must say so in the words "Not formalized.", which is the only accepted form, so that an unbacked claim cannot enter the text without someone writing the sentence. Three claims in Chapter 7 say it, all three about deployment cost rather than mathematics. Everything else in the book is proved or cited, and the build enforces it.

Running the axiom audit. This is the check that backs Appendix A's axiom claims. Lean's `#print axioms name` command reports the complete list of axioms a declaration depends on, transitively, through every lemma it uses. Several files already end with these commands: `F_11`, `F_13`, and `F_14` in `Foundation`, `CCH_FinalVerification` in `CCHProofs`, and `HoF_08`, `HoF_12`, `HoF_13`, `HoF_14`, and `HoF_FinalVerification` in `HallucinationProofs`, with `MoF_FinalVerification` in `ManifoldProofs`. To audit any other theorem, add a line like `#print axioms lawvere` after its file's imports and rebuild; the output appears in the build log or in your editor.

Reading the output. There are three outcomes worth knowing. If Lean prints that the declaration "does not depend on any axioms," the proof is axiom-free in the strict sense; this is what you get for the boolean-diagonal

core, `bool_not_fpf`, `cantor`, `lawvere` as stated over a bare type. If it lists exactly `propext`, `Classical.choice`, and `Quot.sound`, the proof is classically clean, resting only on the three standard kernel axioms that all of Mathlib uses; this is what the analytic results report, the continuous trilemma, the Borsuk-Ulam boundary, the manifold geometry. If it ever listed `sorryAx`, the proof would be incomplete, but nothing in this corpus does, which is exactly what a clean build together with these audits confirms. The distinction the book draws, combinatorial core versus analytic refinement, is therefore not a stylistic one. It is visible in the axiom list, and you can print it.

A suggested check. To convince yourself of the book's central claim in one sitting, build `Foundation`, then print the axioms of `no_reflective_verdict` in `F_13` and of `hallucination_trilemma_godel` in `F_11`, and see that the engine is clean. Then build `HallucinationProofs` and print the axioms of `hallucination_trilemma` in `HoF_07` and `boundary_coupling` in `HoF_13`, and see the three standard axioms appear where the analysis enters. The two engines are then in front of you, the one axiom-free and the other classically clean, proving the same boundary. That is the book, checked.